



EMV[®]

Contactless Specifications for Payment Systems

Book C-2

Kernel 2 Specification

Version 2.12
May 2026

Legal Notice

Unless the user has an applicable separate agreement with EMVCo or with the applicable payment system, any and all uses of these Specifications is subject to the terms and conditions of the EMVCo Terms of Use agreement available at www.emvco.com and the following supplemental terms and conditions.

The license granted in the EMVCo Terms of Use specifically excludes (a) the right to disclose, distribute or publicly display these Specifications or otherwise make these Specifications available to any third party, and (b) the right to make, use, sell, offer for sale, or import any software or hardware that practices, in whole or in part, these Specifications. Further, EMVCo does not grant any right to use the Kernel Specifications to develop contactless payment applications designed for use on a Card (or components of such applications). As used in these supplemental terms and conditions, the term “Card” means a proximity integrated circuit card or other device containing an integrated circuit chip designed to facilitate contactless payment transactions. Additionally, a Card may include a contact interface and/or magnetic stripe used to facilitate payment transactions. To use the Specifications to develop contactless payment applications designed for use on a Card (or components of such applications), please contact the applicable payment system. To use the Specifications to develop or manufacture products, or in any other manner not provided in the EMVCo Terms of Use, please contact EMVCo.

These Specifications are provided "AS IS" without warranties of any kind, and EMVCo neither assumes nor accepts any liability for any errors or omissions contained in these Specifications. EMVCO DISCLAIMS ALL REPRESENTATIONS AND WARRANTIES, EXPRESS OR IMPLIED, INCLUDING WITHOUT LIMITATION IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AS TO THESE SPECIFICATIONS.

EMVCo makes no representations or warranties with respect to intellectual property rights of any third parties in or in relation to the Specifications. EMVCo undertakes no responsibility to determine whether any implementation of these Specifications may violate, infringe, or otherwise exercise the patent, copyright, trademark, trade secret, know-how, or other intellectual property rights of third parties, and thus any person who implements any part of these Specifications should consult an intellectual property attorney before any such implementation.

Without limiting the foregoing, the Specifications may provide for the use of public key encryption and other technology, which may be the subject matter of patents in several countries. Any party seeking to implement these Specifications is solely responsible for determining whether its activities require a license to any such technology, including for patents on public key encryption technology. EMVCo shall not be liable under any theory for any party's infringement of any intellectual property rights in connection with these Specifications.

Revision Log – Version 2.12

The following changes have been made to Book C-2 since the publication of Version 2.11.

Incorporated changes described in the following Specification Bulletin:

- Specification Bulletin No. 317: Mag-stripe Mode Implementation Option Deprecated
- Specification Bulletin No. 318: ERRD Time Measurement by PCD for Kernel 2
- Specification Bulletin No. 319: Update to Terminal Risk Management Data of Kernel 2

Other changes:

- Replaced On-device Cardholder Verification Method (OD-CVM) with Consumer Device Cardholder Verification Method (CDCVM)
- Marked unused bits in Terminal Verification Results as RFU in the data dictionary

Contents

Revision Log – Version 2.12	iii
Contents	iv
Figures	xiii
Tables	xiv
1 Using This Manual	16
1.1 Purpose	16
1.2 Audience	16
1.3 Related Information	16
1.4 Terminology	18
1.5 Notations	19
1.5.1 State Machine	19
1.5.2 Data Object Notation	21
1.5.3 Other Notational Conventions	22
2 General Architecture	25
2.1 Introduction	25
2.2 Reader Processes	26
2.2.1 Process P	27
2.2.2 Process D	27
2.2.3 Process S	28
2.2.4 Process K	28
2.2.5 Process M	30
2.3 The Reader Database	32
3 Reader Process K — Kernel Processing	35
3.1 Implementation Option	35
3.2 The Kernel Database	35
3.3 Transaction Flow	37
3.4 Data Exchange	37
3.4.1 Introduction	37
3.4.2 Sending Data to the Terminal	38
3.4.3 Requesting Data from the Terminal	38
3.4.4 Synchronization	39
3.5 Data Storage	40
3.5.1 Introduction	40
3.5.2 Standalone Data Storage	40
3.5.3 Integrated Data Storage	41
3.6 Consumer Device Cardholder Verification	45
3.7 Relay Resistance Protocol	46

3.7.1	Introduction	46
3.7.2	Protocol	46
3.8	Optimisation For Transactions Without CDA.....	47
3.8.1	Introduction	47
3.8.2	Activation/Deactivation.....	48
3.9	Kernel Configuration Options	48
4	Data Organization.....	50
4.1	TLV Database.....	50
4.1.1	Principles.....	50
4.1.2	Access Conditions.....	51
4.1.3	Services	51
4.1.4	DOL Handling.....	56
4.2	Working Variables.....	57
4.3	List Handling	58
4.4	Configuration Data.....	60
4.4.1	Configuration Data – TLV Database	60
4.4.2	CA Public Key Database	62
4.4.3	Certification Revocation List	63
4.5	Lists of Data Objects in OUT	64
4.5.1	Outcome Parameter Set	64
4.5.2	User Interface Request Data	64
4.5.3	Data Record	64
4.5.4	Discretionary Data.....	66
4.6	Data Object Format	67
4.6.1	Format.....	67
4.6.2	Format Checking.....	67
4.7	Reserved for Future Use (RFU)	68
5	C-APDU Commands.....	69
5.1	Introduction	69
5.2	Exchange Relay Resistance Data	70
5.2.1	Definition and Scope	70
5.2.2	Command Message	70
5.2.3	Data Field Returned in the Response Message	70
5.2.4	Status Bytes	71
5.3	Generate AC.....	72
5.3.1	Definition and Scope	72
5.3.2	Command Message	72
5.3.3	Data Field Returned in the Response Message	73
5.3.4	Status Bytes	75
5.4	Get Data	76

5.4.1	Definition and Scope	76
5.4.2	Command Message	76
5.4.3	Data Field Returned in the Response Message	77
5.4.4	Status Bytes	77
5.5	Get Processing Options	78
5.5.1	Definition and Scope	78
5.5.2	Command Message	78
5.5.3	Data Field Returned in the Response Message	78
5.5.4	Status Bytes	80
5.6	Put Data	81
5.6.1	Definition and Scope	81
5.6.2	Command Message	81
5.6.3	Data Field Returned in the Response Message	81
5.6.4	Status Bytes	82
5.7	Read Record	83
5.7.1	Definition and Scope	83
5.7.2	Command Message	83
5.7.3	Data Field Returned in the Response Message	83
5.7.4	Status Bytes	84
6	Kernel State Machine	85
6.1	Implementation Principles	85
6.2	Kernel Started	86
6.2.1	Local Variables	86
6.2.2	Flow Diagram	87
6.2.3	Processing	88
6.3	State 1 – Idle	90
6.3.1	Local Variables	90
6.3.2	Flow Diagram	91
6.3.3	Processing	95
6.4	State 2 – Waiting for PDOL Data	101
6.4.1	Local Variables	101
6.4.2	Flow Diagram	102
6.4.3	Processing	104
6.5	State 3 – Waiting for GPO Response	106
6.5.1	Local Variables	106
6.5.2	Flow Diagram	107
6.5.3	Processing	112
6.6	State R1 – Waiting for Exchange Relay Resistance Data Response	116
6.6.1	Local Variables	116
6.6.2	Flow Diagram	117

6.6.3	Processing	121
6.7	States 3, R1 – Common Processing	126
6.7.1	Local Variables	126
6.7.2	Flow Diagram	126
6.7.3	Processing	130
6.8	State 4 – Waiting for Read Record Response	134
6.8.1	Local Variables	134
6.8.2	Flow Diagram	135
6.8.3	Processing	140
6.9	State 5 – Waiting for Get Data Response	147
6.9.1	Local Variables	147
6.9.2	Flow Diagram	148
6.9.3	Processing	151
6.10	State 6 – Waiting for First Write Flag	154
6.10.1	Local Variables	154
6.10.2	Flow Diagram	155
6.10.3	Processing	157
6.11	States 4, 5, and 6 – Common Processing	158
6.11.1	Local Variables	158
6.11.2	Flow Diagram	158
6.11.3	Processing	165
6.12	State 9 – Waiting for Generate AC Response	174
6.12.1	Local Variables	174
6.12.2	Flow Diagram	175
6.12.3	Processing	188
6.13	State 12 – Waiting for Put Data Response Before Generate AC	210
6.13.1	Local Variables	210
6.13.2	Flow Diagram	211
6.13.3	Processing	213
6.14	State 15 – Waiting for Put Data Response After Generate AC	215
6.14.1	Local Variables	215
6.14.2	Flow Diagram	216
6.14.3	Processing	218
7	Procedures	220
7.1	Procedure – CVM Selection	220
7.1.1	Local Variables	220
7.1.2	Flow Diagram	221
7.1.3	Processing	225
7.2	Procedure – Prepare Generate AC Command	231
7.2.1	Local Variables	231

7.2.2	Flow Diagram	231
7.2.3	Processing	238
7.3	Procedure – Processing Restrictions	244
7.3.1	Local Variables	244
7.3.2	Flow Diagram	244
7.3.3	Processing	250
7.4	Procedure – Terminal Action Analysis.....	256
7.4.1	Local Variables	256
7.4.2	Flow Diagram	256
7.4.3	Processing	259
8	Security Algorithms	262
8.1	Unpredictable Number Generation	262
8.2	OWHF2	263
8.3	OWHF2AES.....	264
Annex A	Data Dictionary	265
A.1	Data Objects by Name.....	265
A.1.1	Account Type	265
A.1.2	Acquirer Identifier	265
A.1.3	Active AFL	265
A.1.4	Active Tag.....	266
A.1.5	AC Type.....	266
A.1.6	Additional Terminal Capabilities	266
A.1.7	Amount, Authorized (Numeric).....	268
A.1.8	Amount, Other (Numeric).....	268
A.1.9	Application Capabilities Information	269
A.1.10	Application Cryptogram.....	270
A.1.11	Application Currency Code	270
A.1.12	Application Currency Exponent.....	271
A.1.13	Application Effective Date	271
A.1.14	Application Expiration Date	271
A.1.15	Application File Locator.....	272
A.1.16	Application Interchange Profile	273
A.1.17	Application Label.....	273
A.1.18	Application Preferred Name.....	274
A.1.19	Application PAN	274
A.1.20	Application PAN Sequence Number	274
A.1.21	Application Priority Indicator	274
A.1.22	Application Transaction Counter	275
A.1.23	Application Usage Control	275
A.1.24	Application Version Number (Card)	276

A.1.25	Application Version Number (Reader)	276
A.1.26	CA Public Key Index (Card)	276
A.1.27	Card Data Input Capability	277
A.1.28	CDOL1	277
A.1.29	CDOL1 Related Data	277
A.1.30	Cryptogram Information Data	278
A.1.31	CVM Capability – CVM Required	278
A.1.32	CVM Capability – No CVM Required	279
A.1.33	CVM List	279
A.1.34	CVM Results	280
A.1.35	Data Needed	280
A.1.36	Data Record	280
A.1.37	Data To Send	281
A.1.38	Device Estimated Transmission Time For Relay Resistance R-APDU	281
A.1.39	Device Relay Resistance Entropy	281
A.1.40	DF Name	282
A.1.41	Discretionary Data	282
A.1.42	DS AC Type	282
A.1.43	DS Digest H	283
A.1.44	DSDOL	283
A.1.45	DS ID	283
A.1.46	DS Input (Card)	284
A.1.47	DS Input (Term)	284
A.1.48	DS ODS Card	284
A.1.49	DS ODS Info	285
A.1.50	DS ODS Info For Reader	285
A.1.51	DS ODS Term	286
A.1.52	DS Requested Operator ID	286
A.1.53	DS Slot Availability	287
A.1.54	DS Slot Management Control	287
A.1.55	DS Summary 1	288
A.1.56	DS Summary 2	288
A.1.57	DS Summary 3	288
A.1.58	DS Summary Status	289
A.1.59	DS Unpredictable Number	289
A.1.60	DSVN Term	290
A.1.61	Error Indication	290
A.1.62	File Control Information Issuer Discretionary Data	292
A.1.63	File Control Information Proprietary Template	292
A.1.64	File Control Information Template	293

A.1.65	Hold Time Value	294
A.1.66	ICC Dynamic Number	294
A.1.67	ICC Public Key Certificate	294
A.1.68	ICC Public Key Exponent	294
A.1.69	ICC Public Key Remainder	295
A.1.70	IDS Status	295
A.1.71	Interface Device Serial Number	295
A.1.72	Issuer Action Code – Default	296
A.1.73	Issuer Action Code – Denial	296
A.1.74	Issuer Action Code – Online	296
A.1.75	Issuer Application Data	296
A.1.76	Issuer Code Table Index	297
A.1.77	Issuer Country Code	297
A.1.78	Issuer Public Key Certificate	297
A.1.79	Issuer Public Key Exponent	297
A.1.80	Issuer Public Key Remainder	298
A.1.81	Kernel Configuration	298
A.1.82	Kernel ID	298
A.1.83	Language Preference	299
A.1.84	Log Entry	299
A.1.85	Maximum Relay Resistance Grace Period	299
A.1.86	Max Time For Processing Relay Resistance APDU	300
A.1.87	Measured Relay Resistance Processing Time	300
A.1.88	Merchant Category Code	300
A.1.89	Merchant Custom Data	301
A.1.90	Merchant Identifier	301
A.1.91	Merchant Name and Location	301
A.1.92	Message Hold Time	301
A.1.93	Minimum Relay Resistance Grace Period	302
A.1.94	Min Time For Processing Relay Resistance APDU	302
A.1.95	Next Cmd	303
A.1.96	ODA Status	303
A.1.97	Outcome Parameter Set	304
A.1.98	Payment Account Reference	306
A.1.99	PDOL	306
A.1.100	PDOL Related Data	306
A.1.101	PDOL Values	307
A.1.102	Phone Message Table	307
A.1.103	POS Cardholder Interaction Information	308
A.1.104	Post-Gen AC Put Data Status	309

A.1.105	Pre-Gen AC Put Data Status	309
A.1.106	Proceed To First Write Flag.....	310
A.1.107	Protected Data Envelope 1	310
A.1.108	Protected Data Envelope 2	310
A.1.109	Protected Data Envelope 3	311
A.1.110	Protected Data Envelope 4	311
A.1.111	Protected Data Envelope 5	311
A.1.112	Reader Contactless Floor Limit.....	311
A.1.113	Reader Contactless Transaction Limit	312
A.1.114	Reader Contactless Transaction Limit (No CDCVM)	312
A.1.115	Reader Contactless Transaction Limit (CDCVM)	312
A.1.116	Reader CVM Required Limit	313
A.1.117	Read Record Response Message Template	313
A.1.118	Reference Control Parameter	314
A.1.119	Relay Resistance Accuracy Threshold	314
A.1.120	Relay Resistance Transmission Time Mismatch Threshold	315
A.1.121	Response Message Template Format 1	315
A.1.122	Response Message Template Format 2	315
A.1.123	RRP Counter	316
A.1.124	Security Capability	316
A.1.125	Signed Dynamic Application Data	316
A.1.126	Static Data Authentication Tag List	317
A.1.127	Static Data To Be Authenticated	317
A.1.128	Tags To Read	317
A.1.129	Tags To Read Yet	318
A.1.130	Tags To Write After Gen AC	318
A.1.131	Tags To Write Before Gen AC	319
A.1.132	Tags To Write Yet After Gen AC	319
A.1.133	Tags To Write Yet Before Gen AC	319
A.1.134	Terminal Action Code – Default.....	320
A.1.135	Terminal Action Code – Denial.....	320
A.1.136	Terminal Action Code – Online.....	320
A.1.137	Terminal Capabilities	321
A.1.138	Terminal Country Code	321
A.1.139	Terminal Expected Transmission Time For Relay Resistance C-APDU.....	322
A.1.140	Terminal Expected Transmission Time For Relay Resistance R-APDU.....	322
A.1.141	Terminal Identification	322
A.1.142	Terminal Relay Resistance Entropy	323
A.1.143	Terminal Risk Management Data	323
A.1.144	Terminal Type	325

A.1.145	Terminal Verification Results	325
A.1.146	Token Requestor ID	326
A.1.147	Third Party Data	327
A.1.148	Time Out Value.....	327
A.1.149	Track 1 Discretionary Data.....	327
A.1.150	Track 2 Discretionary Data.....	328
A.1.151	Track 2 Equivalent Data	328
A.1.152	Transaction Category Code	329
A.1.153	Transaction Currency Code	329
A.1.154	Transaction Currency Exponent.....	329
A.1.155	Transaction Date	329
A.1.156	Transaction Time	330
A.1.157	Transaction Type	330
A.1.158	Unpredictable Number	330
A.1.159	Unprotected Data Envelope 1	331
A.1.160	Unprotected Data Envelope 2	331
A.1.161	Unprotected Data Envelope 3	331
A.1.162	Unprotected Data Envelope 4	331
A.1.163	Unprotected Data Envelope 5	332
A.1.164	User Interface Request Data	332
A.2	Data Objects by Tag	334
Annex B	Kernel State Machine	340
B.1	Application States	340
B.2	State Machine	341
Annex C	Glossary	342

Figures

Figure 1.1: Symbols Used in Transaction Flow Diagrams	20
Figure 2.1: General Architecture	25
Figure 2.2: Reader Logical Architecture.....	26
Figure 2.3: Reader Database – Persistent Datasets	32
Figure 3.1: Summaries – Basic Principle	44
Figure 6.1: Kernel Started Flow Diagram.....	87
Figure 6.2: State 1 Flow Diagram	91
Figure 6.3: State 2 Flow Diagram	102
Figure 6.4: State 3 Flow Diagram	107
Figure 6.5: State R1 Flow Diagram.....	117
Figure 6.6: States 3 and R1 – Common Processing – Flow Diagram	126
Figure 6.7: State 4 Flow Diagram	135
Figure 6.8: State 5 Flow Diagram	148
Figure 6.9: State 6 Flow Diagram	155
Figure 6.10: States 4, 5, and 6 – Common Processing – Flow Diagram.....	158
Figure 6.11: State 9 Flow Diagram.....	175
Figure 6.12: State 12 Flow Diagram	211
Figure 6.13: State 15 Flow Diagram	216
Figure 7.1: CVM Selection Flow Diagram.....	221
Figure 7.2: Prepare Generate AC Command Flow Diagram	231
Figure 7.3: Processing Restrictions Flow Diagram	244
Figure 7.4: Terminal Action Analysis Flow Diagram	256

Tables

Table 1.1: Terminology	18
Table 1.2: Other Notational Conventions	22
Table 2.1: Reader Processes	26
Table 2.2: Process P Signals	27
Table 2.3: Services from Process K	29
Table 2.4: Responses from Process K	30
Table 2.5: Reader Databases	33
Table 2.6: Persistent Dataset Kernel 2	34
Table 3.1: Kernel Database Categories	36
Table 3.2: Kernel Configuration Options	48
Table 4.1: Access Conditions	51
Table 4.2: Configuration Data in TLV Database that Require Default Value	60
Table 4.3: Phone Message Table – Default Value	61
Table 4.4: CA Public Key Related Data	62
Table 4.5: Certification Revocation List Related Data	63
Table 4.6: Data Record Detail	64
Table 4.7: Discretionary Data	66
Table 5.1: Coding of the Instruction Byte	69
Table 5.2: Generic Status Bytes	69
Table 5.3: Exchange Relay Resistance Data Command Message	70
Table 5.4: Exchange Relay Resistance Data Response Message Data Field	70
Table 5.5: Status Bytes for Exchange Relay Resistance Data Command	71
Table 5.6: Generate AC Command Message	72
Table 5.7: Generate AC Reference Control Parameter	73
Table 5.8: Generate AC Response Message Data Field (Format 1)	73
Table 5.9: Generate AC Response Message Data Field (Format 2) – No CDA	74
Table 5.10: Generate AC Response Message Data Field (Format 2) – CDA	74
Table 5.11: Status Bytes for Generate AC Command	75
Table 5.12: Get Data Command Message	76
Table 5.13: Supported P1 P2 Values for Get Data Command	76
Table 5.14: Status Bytes for Get Data Command	77
Table 5.15: Get Processing Options Command Message	78
Table 5.16: Get Processing Options Response Message Data Field (Format 1)	79
Table 5.17: Get Processing Options Response Message Data Field (Format 2)	79
Table 5.18: Status Bytes for Get Processing Options Command	80
Table 5.19: Put Data Command Message	81
Table 5.20: Supported P1 P2 values for Put Data Command	81
Table 5.21: Status Bytes for Put Data Command	82
Table 5.22: Read Record Command Message	83
Table 5.23: P2 of Read Record Command	83
Table 5.24: Read Record Response Message Data Field	83
Table 5.25: Status Bytes for Read Record Command	84
Table 6.1: Response Message Data Field	152
Table 6.2: Mandatory Data Objects	168

Table 6.3: Mandatory Card CDA Data Objects	171
Table 6.4: ICC Dynamic Data (IDS)	194
Table 6.5: ICC Dynamic Data (IDS + RRP)	195
Table 6.6: ICC Dynamic Data (No IDS).....	196
Table 6.7: ICC Dynamic Data (RRP).....	197

1 Using This Manual

1.1 Purpose

This document, *EMV Contactless Specifications for Payment Systems, Book C-2 – Kernel 2 Specification*, should be read in conjunction with:

- *EMV Contactless Specifications for Payment Systems, Book A – Architecture and General Requirements*, hereafter referred to as [EMV Book A], and
- *EMV Contactless Specifications for Payment Systems, Book B – Entry Point Specification*, hereafter referred to as [EMV Book B].

This document defines the behaviour of the Kernel used in combination with cards supporting a Mastercard brand or cards having a Kernel Identifier indicating Kernel 2, as defined in [EMV Book B].

1.2 Audience

This specification is intended for use by manufacturers of contactless readers and terminals. It may also be of interest to manufacturers of contactless cards and to financial institution staff responsible for implementing financial applications in contactless cards.

1.3 Related Information

The following references are used in this document. The latest version applies unless a publication date is explicitly stated.

Reference	Document Title
[EMV Book 1]	<i>Integrated Circuit Card Specifications for Payment Systems – Book 1, Application Independent ICC to Terminal Interface Requirements</i> , Version 4.4, October 2022
[EMV Book 2]	<i>Integrated Circuit Card Specifications for Payment Systems – Book 2, Security and Key Management</i> , Version 4.4, October 2022
[EMV Book 3]	<i>Integrated Circuit Card Specifications for Payment Systems – Book 3, Application Specification</i> , Version 4.4, October 2022
[EMV Book 4]	<i>Integrated Circuit Card Specifications for Payment Systems – Book 4, Cardholder, Attendant, and Acquirer Interface Requirements</i> , Version 4.4, October 2022

Reference	Document Title
[EMV Book A]	<i>EMV Contactless Specifications for Payment Systems, Book A – Architecture and General Requirements</i> , Version 2.11
[EMV Book B]	<i>EMV Contactless Specifications for Payment Systems, Book B – Entry Point Specification</i> , Version 2.11
[EMV CL L1]	<i>EMV Level 1 Specifications for Payment Systems, EMV Contactless Interface Specification</i> , Version 3.2
[EMV Token]	EMV Payment Tokenisation Specification, Technical Framework, Version 2.3, October 2021
[ISO 639-1]	Codes for the representation of names of languages – Part 1: Alpha-2 Code
[ISO 3166-1]	Codes for the representation of names of countries and their subdivisions – Part 1: Country codes
[ISO 4217]	Codes for the representation of currencies and funds
[ISO/IEC 7813]	Information technology — Identification cards — Financial transaction cards
[ISO/IEC 7816-4]	Identification cards — Integrated circuit(s) cards with contacts — Part 4: Organization, security and commands for interchange
[ISO/IEC 7816-5]	Registration of application providers
[ISO 8583:1987]	Financial transaction card originated messages – Interchange message specifications
[ISO 8583:1993]	Financial transaction card originated messages – Interchange message specifications
[ISO/IEC 8825-1]	Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER)
[ISO/IEC 8859]	Information technology – 8-bit single-byte coded graphic character sets
[ISO 18031:2005]	Information technology – Security techniques – Random bit generation
[NIST SP800-22A]	A statistical test suite for random and pseudorandom number generators for cryptographic algorithms

1.4 Terminology

The following terms are used in this document, carrying specialized meanings as indicated.

Table 1.1: Terminology

Term	Description
Card	Card, as used in these specifications, is a consumer device supporting contactless transactions.
Combination	Combination is the combination of an AID and a Kernel ID.
Configuration Option	A Configuration Option allows activation or deactivation of the Kernel software behind this option. The Configuration Option may change the execution path of the software but it does not change the software itself. A Configuration Option is set in the Kernel database per AID and transaction type.
EMV Mode	Operating mode that indicates that the contactless payment transaction is validated based on EMV minimum data.
Implementation Option	An Implementation Option allows the vendor to select whether the functionality behind the option will be implemented in a particular installation.
Kernel	The Kernel contains the interface routines, security and control functions, and logic to manage a set of commands and responses to retrieve all the necessary data from the Card to complete a transaction. The Kernel processing covers the interaction with the Card between the selection of the card application (excluded) and the processing of the transaction's outcome (excluded).
Mag-stripe Mode	Mag-stripe Mode describes an operating mode of the POS System that indicates that this particular acceptance environment and acceptance rules support magnetic stripe infrastructure. Mag-stripe Mode transactions are no longer supported by this version of the specification.
POS System	The POS System is the collective term given to the payment infrastructure present at the merchant. It is made up of the Terminal and Reader.

Term	Description
Process	A Process is a logical component within a Reader that has one or more Queues to receive Signals. The processing of Signals, in combination with the carried data, may then generate other Signals to be sent. Processing can continue until all the Queues of a Process are empty, or until the Process terminates.
Queue	A Queue is a buffer that stores events to be processed. The events are stored in the order received.
Reader	The Reader is the device that supports the Kernel(s) and provides the contactless interface used by the Card. In this specification, the Reader is considered as a separate logical entity, although it can be an integral part of the POS System.
Signal	A Signal is an asynchronous event that is placed in a Queue. A Signal can convey data as parameters, and the data provided in this way is used in the processing of the Signal.
Terminal	The Terminal is the device that connects to the authorization and/or clearing network and that together with the Reader makes up the POS System. The Terminal and the Reader may exist in a single integrated device. However, in this specification, they are considered separate logical entities.

1.5 Notations

This section lists notational conventions used in this specification:

1.5.1 State Machine

This document specifies the Kernel processing as a state machine that is triggered by Signals that cause state transitions. The application states of the Kernel are written in a specific format to distinguish them from the text:

`state`

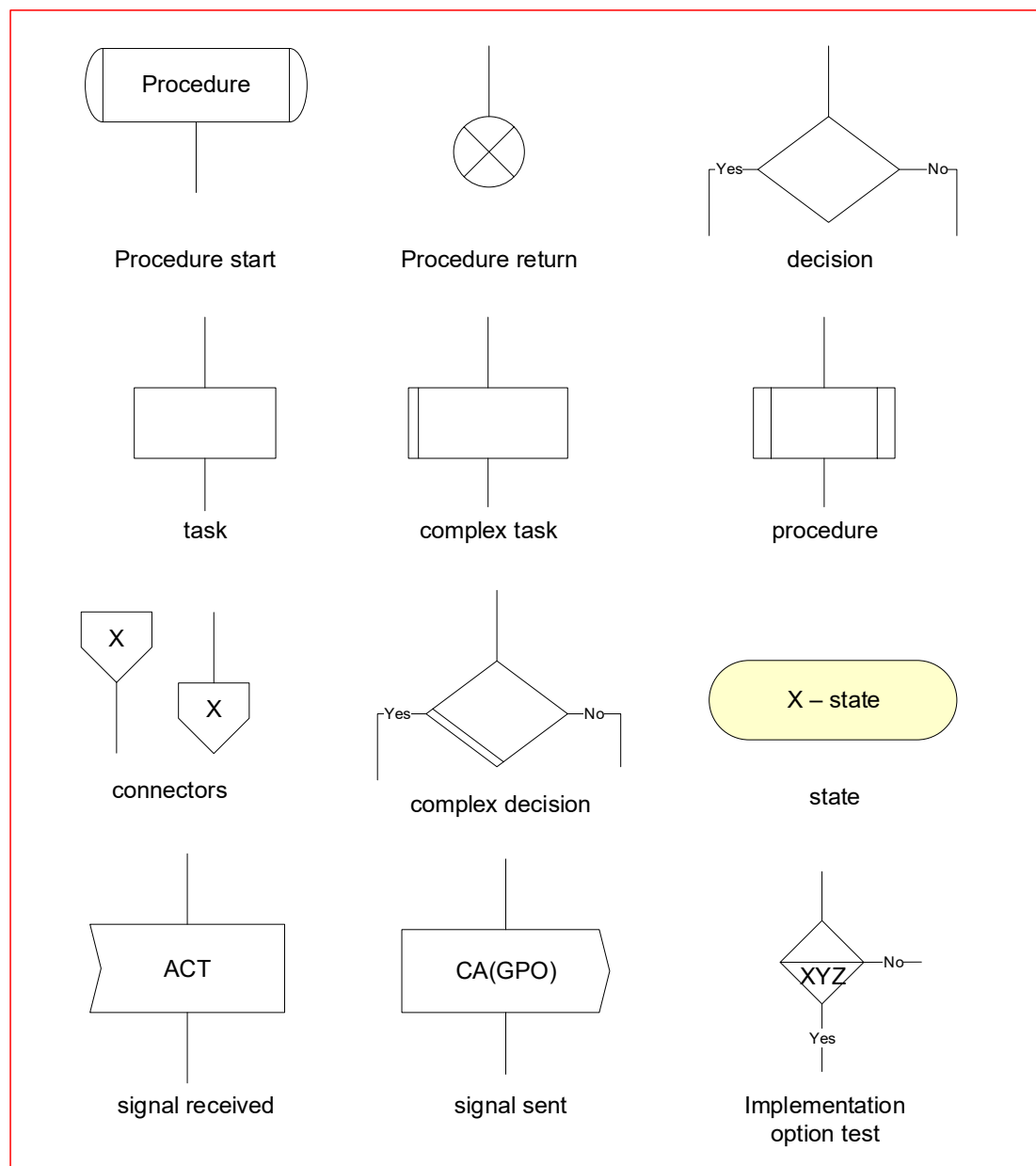
Example:

`GOTO s4 - waiting for read record response`

The state machine of the Kernel is represented by means of a state diagram (as shown in Figure B.1).

This document uses a combination of flow diagrams and textual description in order to describe the state transitions in the state machine of the Kernel. Figure 1.1 shows the symbols used in the flow diagrams.

Figure 1.1: Symbols Used in Transaction Flow Diagrams



The combination of the flow diagrams and the corresponding textual descriptions constitute the requirements on the Kernel behaviour:

- Each diagram in this specification has a unique label.
- Each symbol in a diagram has a unique identifier that is the concatenation of the diagram label with the symbol number.

- The textual description corresponding to the symbol in a diagram starts with the identifier of the symbol.

The flow diagrams are read from top to bottom and define the order of execution of the processing steps. The textual description specifies the behaviour of the individual steps but bears no information on the order of execution.

The requirements relate to the behaviour of the Kernel but leave flexibility in the actual implementation. The implementation must behave in a way that is indistinguishable from the behaviour specified in this document. Indistinguishable means that it creates the output as predicted by this specification for a given input. There is no requirement that the implementation realize the behaviour through a state machine as described in this document.

1.5.2 Data Object Notation

Data object names are capitalized to distinguish them from the text:

Data Object Name

To refer to a sub-element of a data object (i.e. a specific bit, set of bits, or byte of a multi-byte data object), the following notational convention is used:

- If the sub-element is defined in the data dictionary (Annex A), with each possible value of the sub-element having a name, then the following conventions apply:
 - The reference to the sub-element is 'Name of Sub-element' in Data Object Name.
 - The reference to the value is VALUE OF SUB-ELEMENT.

Examples:

- 'CDCVM successful' in POS Cardholder Interaction Information refers to bit 5 of byte 2 in POS Cardholder Interaction Information.
- 'CVM' in Outcome Parameter Set := ONLINE PIN means the same as “bits 4 to 1 of byte 4 of Outcome Parameter Set are set to 0010b”.
- Alternatively, an index may be used to identify a sub-element of a data object. In this case the following notational conventions apply:
 - To refer to a specific byte of a multi-byte data object, a byte index is used within brackets (i.e. []).

For example, Terminal Verification Results[2] represents byte 2 of Terminal Verification Results. The first byte (leftmost or most significant) of a data object has index 1.

- To refer to a specific bit of a single byte multi-bit data object, a bit index is used within brackets [].

For example, Cryptogram Information Data[7] represents the 7th bit of the Cryptogram Information Data. The first bit (rightmost or least significant) of a data object has index 1.

- To refer to a specific bit of a multi-byte data object, a byte index and a bit index are used within brackets (i.e. [][]).

For example, Terminal Verification Results[2][4] represents bit 4 of byte 2 of the Terminal Verification Results.

- Ranges of bytes are expressed with the x:y notational convention:

For example, Terminal Verification Results[1:4] represents bytes 1, 2, 3, and 4 of the Terminal Verification Results.

- Ranges of bits are expressed with the y:x notational convention:

For example, Cryptogram Information Data[5:1] represents bits 5, 4, 3, 2, and 1 of the Cryptogram Information Data.

1.5.3 Other Notational Conventions

Notations for processing data and managing memory are described in Table 1.2.

Table 1.2: Other Notational Conventions

Symbol	Meaning	Example
'0' to '9' and 'A' to 'F'	Hexadecimal notation. Values expressed in hexadecimal form are enclosed in straight single quotes.	27509 decimal is expressed in hexadecimal as '6B75'.
1001b	Binary notation. Values expressed in binary form are followed by the letter b.	'08' hexadecimal is expressed in binary as 00001000b.
340	Decimal notation. Values expressed in decimal form are not enclosed in single quotes.	'0C' hexadecimal is expressed in decimal as 12.
SET	A specific bit in a data object is set to the value 1b	SET 'CDA failed' in Terminal Verification Results
CLEAR	A specific bit in a data object is set to the value 0b	CLEAR 'Cardholder verification was not successful' in Terminal Verification Results

Symbol	Meaning	Example
:=	A specific value is assigned to a data object or to a sub-element of a data object	'Status' in Outcome Parameter Set := END APPLICATION
OR	This notation is used for both the logical and bitwise OR operation. Its meaning is therefore context-specific.	Bitwise AND and OR: IF [((Terminal Action Code – Online OR Issuer Action Code – Online) AND Terminal Verification Results) = '0000000000']
AND	This notation is used for both the logical and bitwise AND operation. Its meaning is therefore context-specific.	Logical AND: IF [IsEmptyList(Data To Send) AND IsEmptyList(Tags To Read Yet)]
NOT	This notation is used for the logical negation operation.	IF [NOT ParseAndStoreCardResponse(TLV)]
	Two binary data objects are concatenated.	A := 'AB34' B := A 'FFFF' means that B is assigned the value 'AB34FFFF'
IF THEN ELSE ENDIF	This textual description is used to specify decision logic, using the following syntax: IF T THEN GOTO X ELSE GOTO Y ENDIF where T is a statement resulting in true or false and X and Y are symbol identifiers.	IF Amount, Authorized (Numeric) > Reader CVM Required Limit THEN GOTO S456.E25 ELSE GOTO S456.E26 ENDIF

Symbol	Meaning	Example
GOTO	A GOTO statement is used to indicate the next step in the following two instances: • A decision diamond containing a test whose outcome determines subsequent processing • An off-page reference to another flow diagram	
$A \bmod n$	The reduction of the integer A modulo the integer n, that is, the unique integer r, $0 \leq r < n$, for which there exists an integer d such that $A = dn + r$	$54 \bmod 16 = 6$
$A \text{ div } n$	The integer division of A by n, that is, the unique integer d for which there exists an integer r, $0 \leq r < n$, such that $A = dn + r$	$54 \text{ div } 16 = 3$
$X \oplus Y$	The bit-wise exclusive-OR of the data blocks X and Y. If one data block is shorter than the other then it is first padded to the left with sufficient binary zeros to make it the same length as the other.	$11001100b \oplus 10101010b = 01100110b$ $1110b \oplus 101010b = 001110b$ $001110b \oplus 101010b = 100100b$
$A := \text{ALG}(K)[X]$	Encryption of a data block X with a block cipher (ALG) using a secret key K. Typical values for ALG are AES, DES, TDES, AES^{-1}, DES^{-1}, and TDES^{-1}.	$T := \text{AES}(K)[M]$
C-APDU	C-APDUs are written in all caps to distinguish them from the text	GET PROCESSING OPTIONS

2 General Architecture

2.1 Introduction

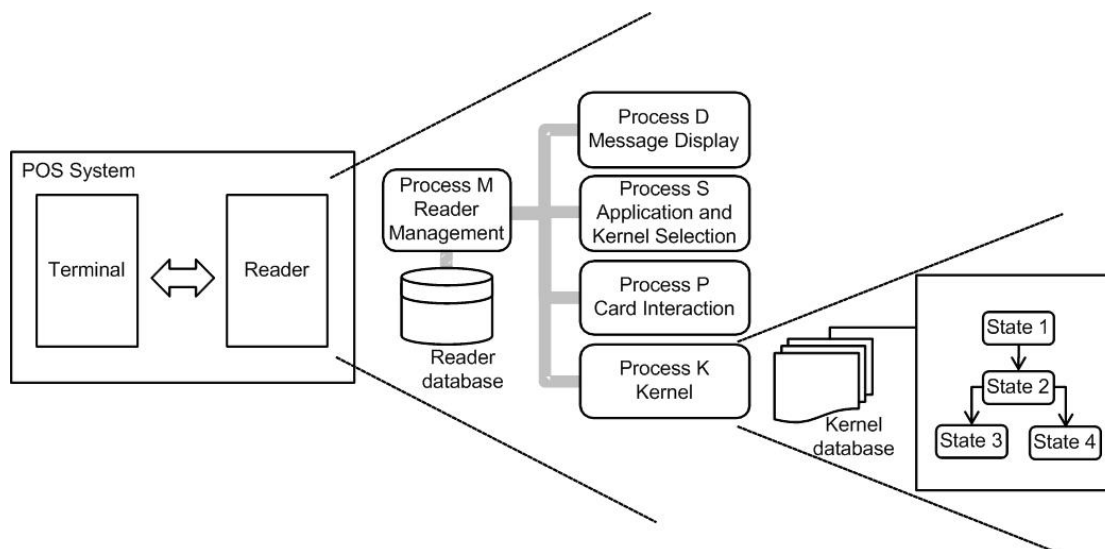
As described in [EMV Book A], the general architecture of a POS System consists of a Terminal and a Reader, where the terms Terminal and Reader refer to a separation in responsibility and functionality between two logical entities.

This document starts from this general architecture, as illustrated in the left hand side of Figure 2.1, then zooms in on the Reader. Figure 2.1 shows how the Reader functionality is allocated to different processes: Process M(ain), Process D(isplay), Process S(elect), Process P(CD), and Process K(ernel).

Communication between different processes happens through Signals stored on Queues. A Signal is an asynchronous event that is placed on a Queue waiting to be processed. A Signal can convey data as parameters, and the data provided in this way is used in the processing of the Signal.

Zooming in further on Process K, Figure 2.1 illustrates the two components of the Kernel: the Kernel software, modelled as a state machine, and the Kernel database, consisting of a number of separate datasets.

Figure 2.1: General Architecture



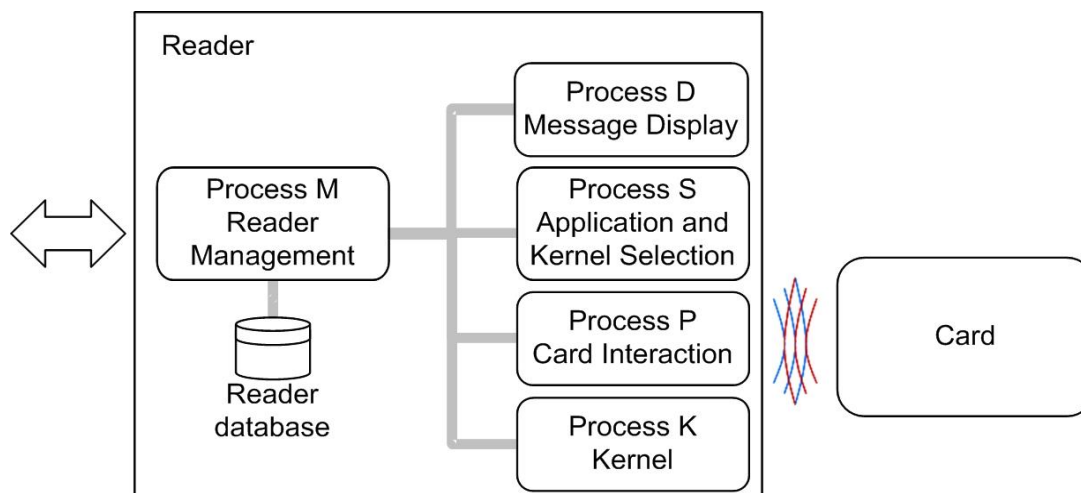
There is no requirement to create devices that use the architecture and the partitioning as laid out in this document, as equally there is no requirement in [EMV Book A] on the partitioning.

The only requirements in this document apply to the Kernel and these requirements define the externally-observable behaviour, independent of the internal organization of the Reader.

2.2 Reader Processes

As illustrated in Figure 2.2, the Reader is modelled as a set of Processes and each Process runs independently of the other processes. The role of the Reader database is explained in section 2.3.

Figure 2.2: Reader Logical Architecture



The different processes are listed in Table 2.1.

Table 2.1: Reader Processes

Process	Responsibility
Process P(CD)	Management of the contactless interface
Process D(isplay)	Management of the user interface
Process S(election)	Selection of the Card Application and Kernel
Process K(ernel)	Interaction with the Card once the application has been selected, covering the payment and data storage transaction flow specific to Kernel 2
Process M(ain)	Overall control and sequencing of the different processes. As part of this role, it is also responsible for the configuration and activation of the Kernel and the processing of its outcome.

The remainder of this section introduces the functionality and configuration of the different processes.

2.2.1 Process P

Process P implements the functionality described in [EMV CL L1] and [ISO/IEC 7816-4] and manages the access to the Card.

Process K communicates with Process P through the CA, RA and L1RSP Signals as shown in Table 2.2.

Table 2.2: Process P Signals

Signal In	Signal Out	Comment
CA	RA(R-APDU)	If there is no L1 error, the RA Signal contains the R-APDU sent back in response to a C-APDU.
	L1RSP(code)	If there is an L1 error, L1RSP is returned with code as one of the following: • Error – Timeout: An L1 timeout has occurred • Error – Protocol: An L1 protocol error has occurred • Error – Transmission: Any other error

Process P sends the C-APDU included in the CA Signal to the Card and responds with either:

- An RA Signal containing the R-APDU and status bytes returned by the Card, or
- An L1RSP Signal that includes an L1 event such as a timeout, transmission error, or protocol error.

2.2.2 Process D

Process D manages the User Interface Requests as defined in [EMV Book A] and displays a message and/or a status.

A MSG Signal is used as a carrier of the User Interface Request Data. Process D may receive MSG Signals from any other Process.

The STOP Signal clears the display immediately and flushes all pending messages.

The MSG and STOP Signals are not acknowledged.

The User Interface Request Data can include a message identifier, a status, a hold time, a language preference, and an amount to be displayed.

For more information on the User Interface Request Data, please refer to section 7.1 of [EMV Book A].

For displaying messages and/or indicating status, Process D needs the following configuration data:

- Default language
- The currency symbol to display for each currency code and the number of minor units for that currency code
- A number of message strings in the default language and potentially other languages
- A number of status identifiers (and the corresponding audio and LED Signals)

The status identifiers and message identifiers are defined in section 9.2 and section 9.4 respectively of [EMV Book A].

2.2.3 Process S

Process S manages the application and Kernel selection. It runs constantly, and it is waiting for an ACT Signal from Process M. After processing the ACT Signal, it returns the selected Combination of application and Kernel (AID and Kernel ID) and the File Control Information Template and status bytes returned by the Card in response to the final SELECT command in an OUT Signal.

Refer to [EMV Book B] for the specification of an implementation of Process S that supports a multi-kernel architecture.

2.2.4 Process K

The Reader may support multiple Kernels but only one Kernel will execute at a time. The Kernel that is activated depends on the information returned by Process S, which may in turn depend on data retrieved from the Card.

For each transaction, Process K is initialized with a Kernel-specific dataset. Within the different available datasets, the value of the data objects may vary depending on the selected AID and the transaction type. More information on the initialization of the Kernel-specific dataset is provided in section 2.3.

The database for each Kernel can be different and the data items are specific to the Kernel; a payment system or private tag can have a different meaning for different Kernels.

Once the Kernel is selected and configured, it executes as Process K. Using the services of Process P as an intermediary, Process K manages the interaction with the Card application beyond application selection. Upon completion, Process K sends its results to Process M in an OUT Signal and then terminates.

For the remainder of the document, it is assumed that Kernel 2 is selected. More detail on the configuration and initialization of Kernel 2 is provided in section 3.1.

As part of its interaction with the Card, Kernel 2:

- Checks the compatibility between the Kernel settings and the Card settings; these checks include both business (for example transaction type, domestic or international acceptance) and technical (for example versioning) aspects,
- Reads and writes the necessary payment and non-payment related data,
- Determines the need for cardholder verification and the method to be used,
- Performs risk management, resulting in the decision to approve/decline the transaction offline or seek online authorization,
- Requests messages to be displayed depending on the details of the transaction,
- Authenticates data, if and when relevant,
- Informs Process M of the transaction outcome through the OUT Signal.

From the viewpoint of the Reader and depending on the configuration options chosen, Kernel 2 provides three services:

- Through its interaction with the Card, it creates a transaction record for authorization and/or clearing.
- It can interact with the Terminal directly for Data Exchange.
- It allows reading and writing data from and to the Card.

The different services are listed in Table 2.3, with the corresponding Signal to call the service indicated in the right column. Only Process M or the Terminal request these services from Process K.

Table 2.3: Services from Process K

Services	Corresponding Signal
Return an authorization or clearing record.	ACT(Data) As a minimum, “Data” includes the File Control Information Template received from the Card in the response to the SELECT command.
Return data from the Kernel database or from the Card. Write data to the Kernel database or to the Card.	DET(Data)

Process K responds to the incoming service request with an outgoing Signal as described in Table 2.4.

Table 2.4: Responses from Process K

Signal In	Signal Out	Comment
ACT	OUT	The OUT Signal includes • Outcome Parameter Set • Data Record – if any • Discretionary Data • User Interface Request Data – if any
DET	DEK or n/a	The DEK Signal can be used to request additional data to be provided in a subsequent DET Signal, as well as to provide data that was requested via a configuration setting or a previous DET Signal. The DEK Signal contains • The Data Needed data object, which is the list of tags of data items that the Kernel needs from the Terminal • The Data To Send data object, which is the list of tags with data values that the Terminal has requested
n/a	DEK	

Within a transaction, the Terminal can only send one or more DET Signals after receiving a DEK Signal indicating that Process K is active.

The DEK Signal is sent only if the Kernel has data for the Terminal or needs data from the Terminal. The DEK and DET Signals are exchanged as part of the Data Exchange mechanism.

2.2.5 Process M

Process M is responsible for coordinating the other processes to perform a transaction. The overall process is as follows:

1. Process M receives the ACT Signal from the Terminal.
2. Process M starts Process P by sending it an ACT Signal to start polling for cards as described in [EMV CL L1].
3. Process M requests Process D to display the READY message (through a MSG Signal).
4. When Process P indicates to Process M that a Card is detected, Process M activates Process S. When Process S completes successfully, it responds with an OUT Signal with the selected Combination {AID – Kernel ID}, the File Control Information Template of the selected Card application, and the status bytes returned by the Card.

5. Based on this information, Process M then configures Process K for the specific Transaction Type and AID, using a Kernel-specific dataset, and sends it an ACT Signal containing transactional data such as the Amount, Authorized (Numeric) and the File Control Information Template. When Process K has completed the transaction, it returns an OUT Signal to Process M, including the Outcome Parameter Set, Discretionary Data, Data Record (if any) and optionally a User Interface Request Data.
6. Process M analyzes the 'Status' in Outcome Parameter Set and executes the instructions encoded in the other fields of the Outcome Parameter Set. As required, Process M instructs Process P to perform the removal sequence. Alternatively it may instruct Process S to select the next application on the Card.
7. Process M passes a subset of the Outcome Parameter Set, the Data Record, and the Discretionary Data to the Terminal in the OUT Signal.
8. If the transaction is processed online, the Reader should receive a MSG Signal from the Terminal to indicate whether the transaction was approved or declined.

2.3 The Reader Database

The Reader maintains a database that is divided into different datasets hold in persistent memory. An overview of the different persistent datasets is given in Figure 2.3, with additional details in Table 2.5.

Figure 2.3: Reader Database – Persistent Datasets

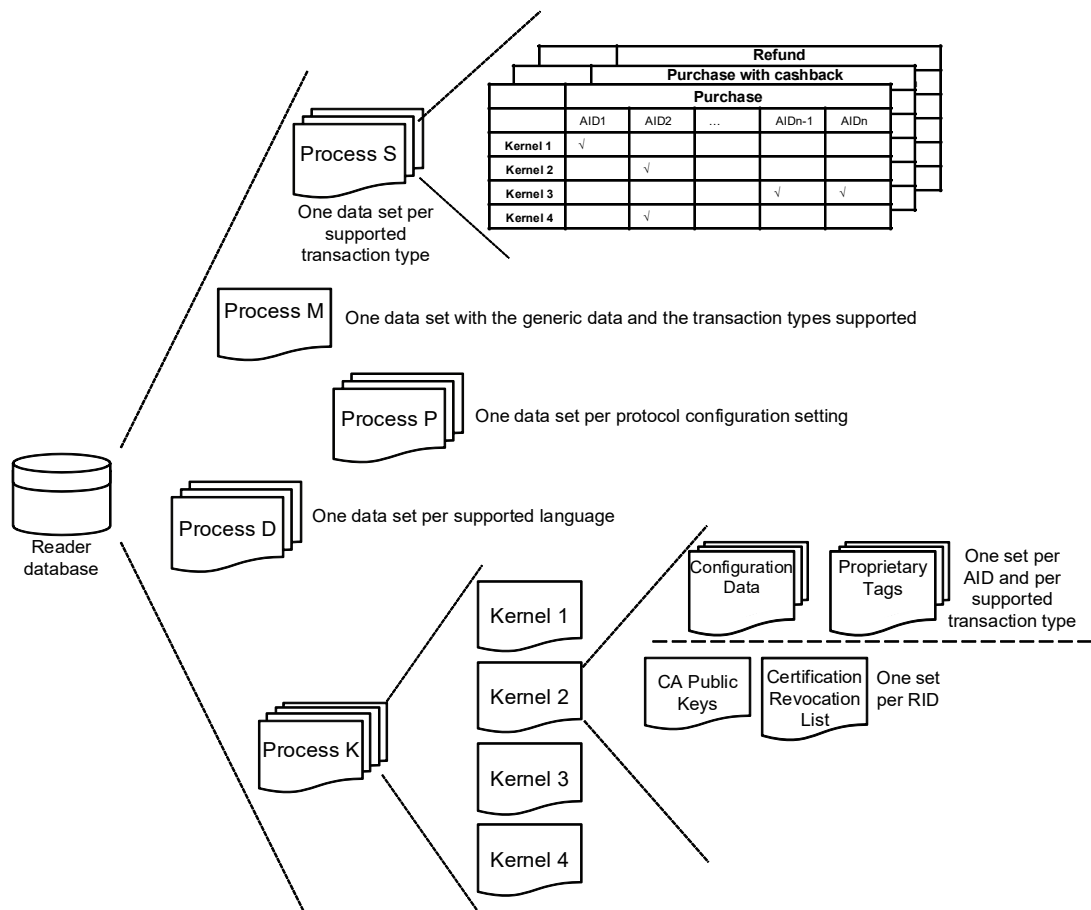


Table 2.5: Reader Databases

Process	Database
Process M	One dataset, including generic data and the different transaction types supported. Examples of generic data are Interface Device Serial Number, Terminal Country Code, Transaction Currency Code, and Transaction Currency Exponent. Examples of transaction types are purchase, purchase with cashback, and refund.
Process P	One or more datasets, one for each protocol configuration setting. Each dataset contains (part of) the configuration settings as defined in Annex A of [EMV CL L1].
Process D	Multiple datasets for Process D, one for each supported language. Each dataset contains the message strings behind the message identifiers.
Process S	Multiple datasets for Process S, one dataset per transaction type. Each dataset contains a list of Combinations {AID – Kernel ID}.
Process K	Multiple Kernel-specific datasets for Process K. Each Kernel-specific dataset includes different subsets. For Kernel 2, see Table 2.6.

If the transaction type has not been indicated by the Terminal in the ACT Signal then a configurable default transaction type is used.

Table 2.6: Persistent Dataset Kernel 2

Subset	Purpose
Configuration Data	Contains the TLV-encoded persistent data objects relevant to a specific AID (used by Process S to identify the application during application selection) and Transaction Type. The values of the TLV-encoded data objects do not vary per transaction.
Proprietary Tags	Data objects with proprietary tags (i.e. data objects with tags not listed in Table A.1). A proprietary tag may have a zero length for data returned from the Card or a length different from zero for terminal sourced data.
The list of CA public keys	Information linked to the CA public keys, including the index, modulus, and exponent. CA public keys can be shared between AIDs that have the same RID and sharing can be done across Kernels. The Reader should be able to store the information for at least six keys per RID.
The Certification Revocation List	A list of Issuer Public Key Certificates that payment systems have revoked for each RID supported by the Kernel. Note that as for the list of CA public keys, entries in the Certification Revocation List may be shared between Kernels where Kernels support the same RID.

3 Reader Process K — Kernel Processing

3.1 Implementation Option

Kernel 2 supports the Data Exchange and Data Storage Implementation Option. This Implementation Option gives the implementer the choice to build Kernel 2 with or without support for Data Exchange, IDS and SDS. This Implementation Option is further on referenced as DE/DS.

3.2 The Kernel Database

When the Kernel process starts, the Kernel database, as introduced in section 2.3, is already initiated with the persistent dataset of Kernel 2 for a specific AID (or RID) and Transaction Type. This includes the Configuration Data, Proprietary Tags, list of CA Public Keys and the Certification Revocation List (see Table 2.6).

At the start of the Kernel, the Kernel instantiates a TLV Database including the tags defined in Table A.1. The TLV Database is extended with the Proprietary Tags (if any). The Kernel copies the Configuration Data in the TLV Database at the start of the transaction. The TLV Database is stored in transient memory, being used as a working database which is cleared after the Kernel exits. Within the TLV Database, entries may exist with zero length.

In addition to the Kernel database, the Kernel receives transaction data items in the ACT Signal. These data items originate from the (Terminal) ACT Signal and from the OUT Signal of the application and Kernel selection process (Process S). These data items with their volatile values are added to the database as well.

During transaction processing, the Kernel may receive events from Process M, the Card, and the Terminal. This input, together with the Kernel's progression through the transaction processing, causes further updates to the Kernel database.

While performing a transaction, the Kernel ensures that updates to the TLV Database are done only by the authorized 'source' (origin) of the data item. For this purpose, data items are put in different categories and the category determines the Signal – and therefore source – that can update data objects within a category.

The different categories and corresponding Signals are illustrated in Table 3.1.

Table 3.1: Kernel Database Categories

Data Category	Signal
Terminal sourced data object – configuration data	n/a
Terminal sourced data object – transaction data	DET, ACT
Kernel defined value or internal data object	n/a Value can only be changed as part of Kernel processing
Card sourced data object	RA ¹

¹ The File Control Information Template is received in an ACT Signal but is treated as an RA as that is how it was delivered to Process S.

3.3 Transaction Flow

Upon receipt of an ACT Signal, the Kernel initiates the transaction on the Card through a GET PROCESSING OPTIONS command.

After the GET PROCESSING OPTIONS command, the Kernel continues with the following steps:

1. It determines which form of Offline Data Authentication to perform.
2. It reads the data records of the Card (using READ RECORD commands).
3. It performs Terminal Risk Management and Terminal Action Analysis, and selects a cardholder verification method for the transaction.
4. It requests an Application Cryptogram from the Card by issuing a GENERATE AC command.
5. It performs Offline Data Authentication as appropriate.

Once all the data from the Card, including the cryptogram, are retrieved, the Kernel indicates that the Card can be removed.

The Kernel completes the transaction by preparing the remainder of the Data Record, the Outcome Parameter Set information, and Discretionary Data (as defined in [EMV Book A]). The Kernel returns the above data to the main process (Process M) and this concludes the transaction for the Kernel, which then terminates execution.

3.4 Data Exchange

3.4.1 Introduction

Terminal and Kernel can communicate through the Data Exchange mechanism if the DE/DS Implementation Option is implemented.

The Kernel can send tagged data to and request data from the Terminal through the DEK Signal.

The Terminal can control the Kernel through the DET Signal by virtue of its ability to:

- Update the current transaction database of the Kernel
- Request tagged data from the Kernel or from the Card
- Manage the transaction flow pace by withholding necessary data (so that the Kernel asks for it) or providing these data earlier than needed to avoid delays.

3.4.2 Sending Data to the Terminal

As part of its configuration or through an ACT or DET Signal, the Kernel has a data object (Tags To Read) containing the tags of the data objects to be sent to the Terminal. If a tag refers to Card data, this data is retrieved through READ RECORD commands – as part of reading the records listed in the Application File Locator – or through a GET DATA command. The Kernel has a list of data objects that are read using GET DATA (see Table 5.13). All other data objects are read using READ RECORD commands.

Note that Tags To Read excludes the IDS data which is sent automatically if IDS is activated in the Kernel.

When the Kernel has completed the (currently outstanding) requests from the Terminal, it sends the data to the Terminal via a DEK Signal.

The information in the DEK Signal may trigger the Terminal to send another list of data to read (DET Signal). This list is then appended to the original list and may result in another set of GET DATA commands if the request includes tags referring to card data.

The Kernel uses a buffer, called Tags To Read Yet, to accumulate the different read requests included in Tags To Read.

Data To Send is another buffer, accumulating the multiple data that the Kernel has for the Terminal. It is populated with TLV data retrieved in response to Tags To Read Yet processing.

The process continues until all records have been read and there are no more data objects in the list that need to be read using a GET DATA command.

3.4.3 Requesting Data from the Terminal

If one of the following data objects is present in the Kernel database with the length of the value field set to zero, then the Kernel sends a DEK Signal to request the value of the data object:

- Tags To Read
- Tags To Write Before Gen AC
- Tags To Write After Gen AC
- Proceed To First Write Flag

In more general terms, the Kernel applies the following rules for Terminal sourced data objects (as opposed to Kernel and Card sourced data objects):

1. If the Kernel database contains a Terminal sourced data object that has length of zero and if this data object is needed during the transaction, then the Kernel requests this data object in a DEK Signal by including its tag in Data Needed.

The data object can be needed during the transaction for two reasons:

- The Kernel needs it for its own processing, e.g. Amount, Authorized (Numeric).
 - The Card requests it in a DOL, e.g. Merchant Custom Data.
2. If the data object is not present in the Kernel database, it is not requested from the Terminal. This condition applies only if the Kernel does not need this data object for its own processing. When this data object is requested by the Card in a DOL, it is zero filled in the data of the corresponding command.
 3. If the data object is present with length different from zero, it is not requested from the Terminal. It is sent to the Card when requested in a DOL and normal padding and truncation rules apply.

By putting a Terminal sourced data object or one or more of the data objects listed above in the database with a zero length, the Terminal deliberately withholds the data so that the Kernel specifically asks for it, thereby giving the Terminal the ability to pace the transaction flow and change the value of transaction data, based on information received during the transaction flow.

As indicated above, the Kernel uses a buffer, called Data Needed, to accumulate tags that the Kernel needs from the Terminal. It is populated with a list of tags. In a similar manner, if IDS is being used, the Kernel uses DEK Signals to request the data that it needs to complete the transaction.

The Terminal may send multiple DET Signals, each DET Signal containing a Tags To Write Before Gen AC or Tags To Write After Gen AC data object. The Kernel manages these DET Signals through two buffers: Tags To Write Yet Before Gen AC and Tags To Write Yet After Gen AC. These buffers are used to accumulate the TLV data objects included in Tags To Write Before Gen AC tag and Tags To Write After Gen AC tag respectively.

3.4.4 Synchronization

All data are read from the Card before any data are written. To make sure the reading process is completed and that the Terminal has received all required data, the Kernel checks whether it can move to the writing stage.

This check uses the Proceed To First Write Flag data object, as introduced in section 3.4.3. The Proceed To First Write Flag may take one of the following values:

- When Proceed To First Write Flag is absent, the Kernel can move to the writing phase of the transaction.
- When Proceed To First Write Flag has length zero, the Kernel requests a value for the Proceed To First Write Flag from the Terminal. It waits until the Terminal provides this value before moving to the writing phase.
- When Proceed To First Write Flag has value zero, the Kernel waits until the Terminal provides a value different from zero before moving to the writing phase.

- When Proceed To First Write Flag has a value different from zero, the Kernel can move to the writing phase of the transaction.

3.5 Data Storage

3.5.1 Introduction

Data storage is an extension of the regular transaction flow such that the Card can be used as a scratch pad or mini data store with simple write and read functionality. Data storage is only supported if the DE/DS Implementation Option is implemented.

Two types of data storage are possible: Standalone Data Storage (SDS) or Integrated Data Storage (IDS). They rely on the Data Exchange mechanism as described in section 3.4.

The Kernel may support one or both data storage methods and is configured accordingly. However, the use of data storage by the Kernel in a given transaction is conditional on the Card's indication of support for data storage. The Card support for SDS and IDS is indicated in the response to the SELECT AID command. The File Control Information Template may contain the Application Capabilities Information data object which, if present, indicates the support provided for SDS and IDS.

3.5.2 Standalone Data Storage

SDS uses dedicated commands (GET DATA, PUT DATA) for explicit reading and writing of data. It introduces a range of payment system tags ('9F70' to '9F79') for the reading and writing of non-payment data, so that they can be included in Tags To Read, Tags To Write Before Gen AC, or Tags To Write After Gen AC. The whole range is freely readable using the GET DATA command.

Writing is done using a PUT DATA command without secure messaging, for tags '9F75' to '9F79'. Writing to the tags '9F70' to '9F74' requires secure messaging and is outside the scope of this specification.

The length of the data is variable. The maximum length is implementation specific, and is between 32 and 192 bytes. If present, the Application Capabilities Information from the Card indicates the configuration of the SDS tags. The relevant coding is described in the data dictionary (Annex A).

Writing can be done before and after the GENERATE AC, hence two lists to distinguish between data objects to be written to the Card before and those to be written afterwards. This distinction is indicated by the list names: Tags To Write Before Gen AC and Tags To Write After Gen AC.

Each list is TLV coded, containing Tag, Length as well as Value of the data to write. The lists may be part of the Kernel configuration, or may be communicated to the Kernel during the transaction using Data Exchange, via a DET Signal.

Once the Kernel has the go-ahead to move to writing, it may send one or more PUT DATA commands to the Card, each command containing one data object from the first list (Tags To Write Before Gen AC) and in the order as they are in this list. Once all data from this first list are sent to the Card, the Kernel sends the GENERATE AC command.

After the GENERATE AC command, the Kernel then repeats this process for the second list (Tags To Write After Gen AC).

3.5.3 Integrated Data Storage

IDS builds the reading and writing functions into existing payment commands (GET PROCESSING OPTIONS and GENERATE AC). The command-response sequence exchanged between the Card and Kernel is therefore unchanged from a normal purchase transaction. It also addresses the security mechanisms of those exchanges.

This section describes the overall transaction flow and the security design.

IDS: Overall Transaction Flow

Support for IDS in the transaction flow can be summarized as follows:

1. Process S selects the application. If the Card supports IDS, this is indicated in the Card's response and the PDOL includes the tag of the operator identifier. The Card's response is included in the ACT Signal activating the Kernel, and is therefore part of the current transaction database of the Kernel.
2. The operator's slot is selected through the inclusion of the operator identifier in the GET PROCESSING OPTIONS command data as part of the PDOL Related Data.
3. If a slot is currently present for this identifier, the Card returns the contents of the slot in its response to the GET PROCESSING OPTIONS command together with slot management data. If it is not present, the Card indicates whether a new slot is available for allocation to this identifier. As well as the normal Application Interchange Profile and Application File Locator data objects, the GET PROCESSING OPTIONS response (using Format 2) returns², if available, the following:
 - The non-payment data (DS ODS Card)
 - The type of data (DS Slot Management Control)
 - A hash of the transaction context calculated by the Card when data was written to the Card in a previous transaction (DS Summary 1)
 - An indication of which type of data (volatile or permanent) may be stored (DS Slot Availability)

² Although not relevant to the reading of the data, note that the GET PROCESSING OPTIONS response also includes a card challenge (*DS Unpredictable Number*). This is part of the IDS security mechanism.

4. The information on the slot data is passed to the Terminal (DEK Signal), which can then decide to update the data or allocate a new slot, as appropriate for the particular transaction. The Terminal passes this information to the Kernel (DET Signal) and the Kernel sends the new data to the Card appended to the end of the CDOL1 data in the GENERATE AC command.

For this purpose, the Card supports a (single) DSDOL, applicable for the GENERATE AC command. DSDOL is read through the READ RECORD command, in a record present in the Application File Locator. The Kernel appends the (non-payment data) data objects listed in the DSDOL in the order as indicated in the DSDOL and with the lengths as indicated in DSDOL (except for the last element which may be shorter). Except for the last tag in DSDOL, all tags are handled according to the rules specified in section 5.4 of [EMV Book 3]. The last tag indicated in DSDOL is appended with the length defined in the TLV Database and no padding is applied if the length specified in the DSDOL entry is greater than the actual length of the data object in the Kernel database.

The data objects that are included in the DSDOL tags list are:

- The type of data (DS ODS Info)
 - The result of a one-way function, to set a new access control (DS Digest H)
 - The input to a one-way function, to get access control (DS Input (Card))
 - The non-payment data envelope (DS ODS Term)
5. Including the additional data in the GENERATE AC command may influence the outcome of the transaction and does not automatically result in a data update or a slot allocation. Whether data will be written to the Card and the outcome of the transaction depends on four elements:
 - The type of application cryptogram (i.e. TC, ARQC, or AAC) proposed by the Terminal in the DS AC Type
 - The type of application cryptogram resulting from the Kernel (Terminal) risk management and action analysis, indicated in AC Type
 - The settings in DS ODS Info For Reader sent by the Terminal
 - The type of Application Cryptogram generated by the Card, as reported in Cryptogram Information Data – see step 6

The algorithm is described below and assumes there is an order amongst the different application cryptograms TC, ARQC, and AAC, with TC being the highest and AAC being the lowest (i.e. $TC > ARQC > AAC$). The algorithm is as follows:

The Kernel compares its AC Type to the Terminal's DS AC Type.

- If the Kernel AC Type is higher, then the Kernel sets its AC Type equal to the DS AC Type, and the Kernel includes the IDS data in the GENERATE AC command data. For example, if the Terminal requests an ARQC in

DS AC Type and the Kernel's risk management decision results in a TC in AC Type, then the Kernel sets its AC Type to ARQC, which is lower.

- If the Kernel AC Type is lower, then:
 - If DS ODS Info For Reader indicates that the IDS data can be used for AC Type, then the Kernel includes the IDS data in the GENERATE AC command data.
 - Otherwise:
 - If DS ODS Info For Reader indicates that the transaction may continue without IDS data in the GENERATE AC command data then the Kernel sends the GENERATE AC without IDS data.
 - Otherwise, the Kernel terminates the transaction and returns an OUT Signal.
- 6. If the IDS data are included in GENERATE AC command data, then the Card may or may not write the data. If the data is written, then the Card confirms to the Kernel that the slot has been allocated and that the new data has been updated. If there is an error with the data or if the type of Application Cryptogram generated by the Card is different from that requested by the Kernel, then the Card does not store the data. In any case, the Card response includes an authenticated hash of the transaction context of the initial data read (DS Summary 2) as well as a hash of the transaction context of the resulting data (DS Summary 3).
- 7. If the response to the GENERATE AC command indicates that the data were not written, the Kernel checks DS ODS Info For Reader on whether the transaction should be continued or not.

IDS: Security Design

The security design is based on the following assumptions and mechanisms.

Assumptions

The service is provided based on the data read (DS ODS Card) and is conditional on the data being authentic. If the data cannot be authenticated, then the service will not be provided.

The Terminal has a cryptographic method to add a MAC to the data that it stores in the data written to the Card to ensure that a third party has not tampered with the data.

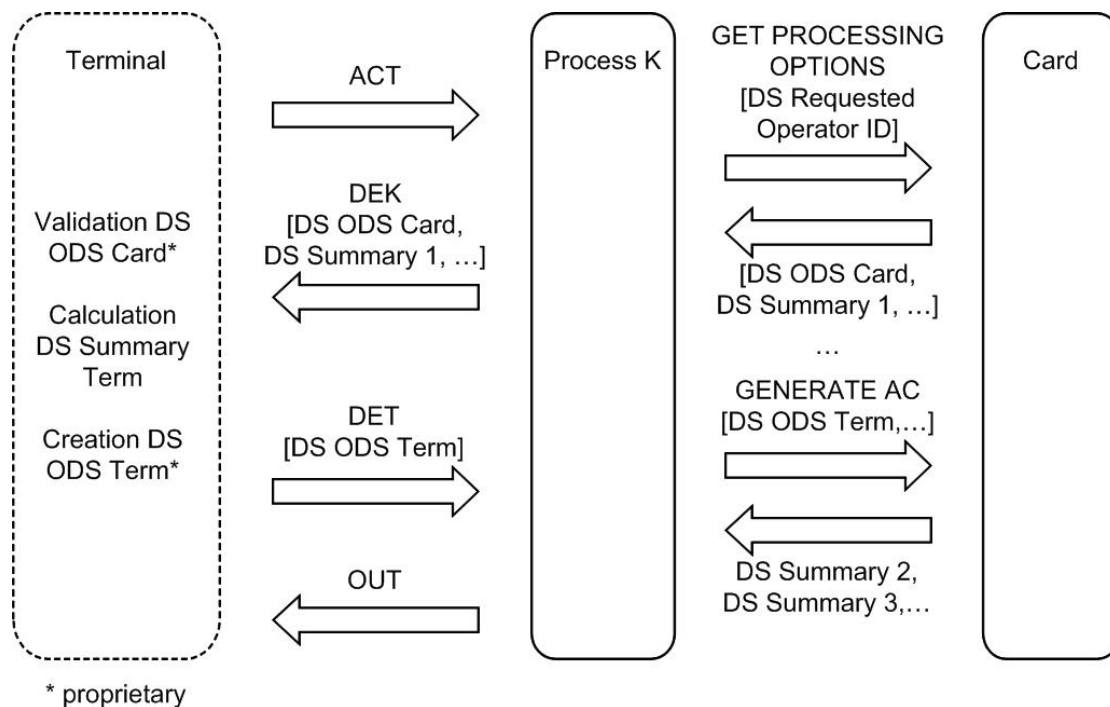
If the Terminal wants to protect the data against skimming and replay, the operator uses the security mechanisms as proposed in this specification.

Mechanisms

The security is built on a combination of the proprietary mechanisms implemented in the Terminal, hashes over the transaction data – called Summaries – and strong offline card authentication using public key cryptography.

The basic principle behind the Summaries is illustrated in Figure 3.1.

Figure 3.1: Summaries – Basic Principle



There is a Summary for the data read and for the data written.

The Summary is a data item that is:

- Computed independently by both the Card (= DS Summary 3) and the Terminal at each write operation
- Computed as a one-way function on the identity of the Card and transaction critical data
- Used by the Terminal as input into the (proprietary) security mechanism for protecting its data (= DS ODS Term)
- Returned by the Card to the Terminal next time the data is read (= DS ODS Card, DS Summary 1)
- Included in the CDA signature of the transaction to authenticate the Summaries

Because DS Summary 1 is returned outside of the CDA signature (and therefore not authenticated), the Card returns the data object in the CDA signature as well, where it is then referred to as DS Summary 2.

DS ODS Card and DS Summary 1 (as well as other data) are returned by the Card and passed to the Terminal. The Terminal validates the authenticity and integrity of DS ODS Card, using a proprietary mechanism in combination with DS Summary 1.

Assuming that DS Summary 1 is authentic (which will be confirmed through DS Summary 2), the Terminal calculates a Summary over the new transaction data and updates DS ODS Card, which then becomes DS ODS Term. DS ODS Term is sent to the Kernel, which passes it on to the Card.

If the Card updates the slot data with DS ODS Term, it calculates a new Summary, taking the existing Summary as input, and stores this new Summary with the slot data. If for some reason the slot data are not updated, no new Summary is calculated and the Summary stored with the slot data does not change.

The Summary stored with the slot data is returned by the Card as DS Summary 3.

For the Kernel it is simple to see whether the slot update was successful or not: If the value of DS Summary 3 is different from the value of DS Summary 1 (and hence DS Summary 2), then the slot data has been updated.

Wedge attacks are detected as both the Card and Reader independently hash critical data into these Summaries. Both of the Summaries calculated by the Card (DS Summary 2 and DS Summary 3) are included in the CDA signature as part of the ICC Dynamic Data. The Kernel will detect tampering with the communication between Terminal and Card when it compares DS Summary 2 with DS Summary 1 and DS Summary 3 with DS Summary 2.

Copying and cloning is prevented through inclusion of an authenticated Card identifier (DS ID) and a Card challenge (DS Unpredictable Number) in the Summary, in combination with the operator's proprietary mechanism for generating a MAC from the data.

For write control, the security is built on a one-way function. At personalization, the Card stores the result of a one-way function over the DS Input (Card) data item, which must match the digest that protects the write access to the slot in the Card. Together with the new data, the Terminal provides a new digest (DS Digest H) to fit the newly written data (DS ODS Term).

3.6 Consumer Device Cardholder Verification

The Kernel is able to distinguish between a consumer device that delegates the CVM processing to the Terminal and a consumer device that can perform cardholder verification itself. The distinction between the two types of devices is independent of the form factor and is based on the Application Interchange Profile.

If the Kernel is configured not to support CDCVM or if the consumer device does not indicate support for CDCVM, then the Kernel performs CVM processing based on the CVM List.

The Kernel checks the Application Interchange Profile and Kernel Configuration data objects and sets the Reader Contactless Transaction Limit either equal to the Reader Contactless Transaction Limit (CDCVM) if CDCVM is supported or to the value Reader Contactless Transaction Limit (No CDCVM) if CDCVM is not supported.

If a device identifies itself as one that supports CDCVM, then CDA is to be used in the GENERATE AC command.

The Kernel checks the Amount, Authorized (Numeric) against the Reader Contactless Transaction Limit and returns an OUT Signal if the transaction amount is greater than this limit. The OUT Signal includes a Status value of Select Next, to request that the next AID from the candidate list should be selected.

The Kernel then checks the transaction amount against the Reader CVM Required Limit.

If the transaction amount is equal to or below the Reader CVM Required Limit, then cardholder verification is not required. If the transaction amount is greater than the limit, then the Kernel sets the CVM Results to indicate that CDCVM was performed successfully.

The CVM Results are included in the GENERATE AC command, as part of the data requested by CDOL1.

Once the response to the GENERATE AC command has been received, the Kernel performs offline card authentication.

The response to the GENERATE AC may include the POS Cardholder Interaction Information. The Kernel uses this in the case of a decline, to inform the customer to take corrective action.

3.7 Relay Resistance Protocol

3.7.1 Introduction

A relay attack is where a fraudulent terminal is used to mislead an unsuspecting cardholder into transacting, where the actual transaction is relayed via a fraudulent Card (or simulator) to the authentic terminal of an unsuspecting merchant. It may also be that a fraudulent reader is used without the cardholder being aware of the transaction.

3.7.2 Protocol

The relay resistance protocol works as follows:

1. A bit in Application Interchange Profile is used to tell the Reader that the Card supports the relay resistance protocol. A bit in Kernel Configuration is used to configure the support of the relay resistance protocol by the Reader.
2. The Reader invokes the relay resistance protocol if both the Card and Reader support it. In this case it sends a timed C-APDU (EXCHANGE RELAY RESISTANCE DATA) to the Card with a random number (Terminal Relay Resistance Entropy). The Card responds with a random number (Device Relay Resistance Entropy) and timing estimates (Min Time For Processing Relay Resistance APDU, Max Time For Processing Relay Resistance APDU and Device Estimated Transmission Time For Relay Resistance R-APDU).

3. If the timings determined by the Reader exceed the maximum limit computed, the Reader will try again in case there was a communication error or in case other processing on the device interrupted the EXCHANGE RELAY RESISTANCE DATA command processing. The Reader will execute up to two retries.
4. Terminal Verification Results are used to permit the Reader to be configured through the Terminal Action Codes to decline or send transactions online in the event that timings are outside the limits computed.
5. The relay resistance protocol relies on CDA. The timings returned by the Card in the EXCHANGE RELAY RESISTANCE DATA are also included in the Signed Dynamic Application Data returned in the response to the GENERATE AC command.
If a transaction is completed without CDA, then the Reader cannot trust the outcome of the relay resistance protocol. In case the transaction is not declined offline, additional data is included in the online message (in Track 2 Equivalent Data) so that the issuer host may perform the checks.
6. The Terminal Relay Resistance Entropy is the same as the Unpredictable Number. In the event of retries, new values for the Unpredictable Number are computed.
7. The Reader considers a transaction OK if the processing time determined from the measured time is within the window stated by the Card (i.e. Max Time For Processing Relay Resistance APDU and Min Time For Processing Relay Resistance APDU) In addition some tolerance is given by the reader in the form of a grace period below and above the window defined by the Card (Minimum Relay Resistance Grace Period and Maximum Relay Resistance Grace Period).
The Reader has an accuracy threshold (Relay Resistance Accuracy Threshold) that indicates whether the measured time is greater than a reader permitted limit. Another accuracy threshold (Relay Resistance Transmission Time Mismatch Threshold) considers the mismatch of the Card communication time.

3.8 Optimisation For Transactions Without CDA

3.8.1 Introduction

When CDA is not used for a transaction, the transaction may be sped up if the public key certificates are not read from the Card. If CDA is not used for a transaction, the Kernel stops reading records from the Card as soon as the minimum data required for the transaction are retrieved. If the Card data are carefully stored in the records, then reading the public key certificates is avoided, speeding up the transaction.

However, in some circumstances, it may be desirable to always read all the records referenced in the Application File Locator. So, this optimisation can be deactivated by configuration.

3.8.2 Activation/Deactivation

When 'Read all records even when no CDA' is set in Kernel Configuration, the Kernel always reads all the records referenced in the Application File Locator.

When 'Read all records even when no CDA' is not set in Kernel Configuration and if the transaction is without CDA, the Kernel stops reading the records referenced in the Application File Locator as soon as the minimum data required for the transaction are retrieved.

3.9 Kernel Configuration Options

Not all the features have to be activated in each deployment of Kernel 2. Certain features can be deactivated through the Kernel configuration.

The different configuration options are listed in Table 3.2, as well as the method to activate a particular option. If the condition for activation is not satisfied, the option is de-activated.

Table 3.2: Kernel Configuration Options

Configuration Options	Description	Activation
IDS	The Kernel supports IDS.	Through the data object DS Requested Operator ID and DSVN Term If DS Requested Operator ID is present (even with a length of zero) and DSVN Term is present with a length different from zero, then IDS is supported. This Configuration Option is only applicable if the DE/DS Implementation Option is implemented.
CDCVM	The Kernel supports CDCVM	Through the setting of 'CDCVM supported' in Kernel Configuration
Relay resistance protocol	The Kernel supports the relay resistance protocol	Through the setting of 'Relay resistance protocol supported' in Kernel Configuration

Configuration Options	Description	Activation
Deactivate optimisation when no CDA	The Kernel always reads all the records referenced in the Application File Locator for transactions without CDA.	Through the setting of 'Read all records even when no CDA' in Kernel Configuration.

All the above configuration options for the Kernel are set at the level of the AID and the transaction type and are part of the TLV Database in the persistent dataset of Kernel 2.

4 Data Organization

This chapter defines the data organization of the Kernel.

4.1 TLV Database

4.1.1 Principles

The Kernel maintains a TLV Database to store all the TLV encoded data objects. This TLV Database is instantiated by the Kernel as described in section 3.2.. It will be updated during the processing of the transaction. The list of tags included in the TLV Database is specific to the different Implementation Options and is defined in Table A.1.

The TLV Database is updated using information received from a number of sources: at start-up from the Reader, with data from the Card, with data from the Terminal, and with data that results from the Kernel's own processing.

A data object is known by the Kernel if its tag is listed in the data dictionary applicable for the Implementation Option as defined in Table A.1. Other data objects with proprietary tags not listed in Table A.1 may be present in the database at the time of instantiation.

A data object is considered to be present if its tag appears in the TLV Database (length may be zero).

A data object is empty if it is present and its length is zero. A data object is not empty if it is present and its length is greater than zero.

Data objects in the TLV Database have a name, a tag, a length, and a value; for example:

Name:	Amount, Authorized (Numeric)
Tag:	'9F02'
Length:	6
Value:	'000000002345'

The index to access data objects in the TLV Database is the tag. The list of tags known by the Kernel is fixed and is defined by the tags of the TLV encoded data objects in Table A.1.

The name of the TLV encoded data object is also used to represent the value field. The following example initializes the value field of the Terminal Verification Results to zero:

Terminal Verification Results := '0000000000'

4.1.2 Access Conditions

Data objects in the TLV Database are assigned access conditions as described in Table 4.1.

Table 4.1: Access Conditions

Access Condition	Description
ACT/DET	These data objects are transaction related data objects sent to the Kernel by the Terminal with the ACT and DET Signals. They may also be present in the TLV Database when the Kernel is instantiated. Proprietary data objects (i.e. data objects with tags not listed in Table A.1) can be updated with the ACT and DET Signals if, and only if, their length at instantiation is different from zero.
RA	These data objects are transaction related data objects sent to the Kernel by the Card with the RA Signal. Proprietary data objects can be updated with the RA Signal if, and only if, their length at instantiation is equal to zero. An exception is data objects contained in the File Control Information Template which are passed to the Kernel with the ACT Signal, but which have the RA access condition assigned.
K	All data objects in the TLV Database can be updated by the Kernel. Every data object has the K (Kernel) access condition assigned.

All data objects can be read by the Card (via a DOL) and by the Terminal (via Tags To Read).

4.1.3 Services

Services available to interrogate and manipulate the TLV Database are the following:

Boolean IsKnown(T)

Returns TRUE if tag T is defined in the data dictionary of the Kernel applicable for the Implementation Option. Table A.1 defines the content of the data dictionary for each Implementation Option.

Boolean IsPresent(T)

Returns TRUE if the TLV Database includes a data object with tag T.

Note that the length of the data object may be zero.

Note also that proprietary data objects that are not known can be present if they have been provided in the TLV Database at Kernel instantiation. In this case the IsKnown() service returns FALSE and the IsPresent() service returns TRUE.

Boolean IsNotPresent(T)

Returns TRUE if the TLV Database does not include a data object with tag T.

Boolean IsNotEmpty(T)

Returns TRUE if all of the following are true:

- The TLV Database includes a data object with tag T.
- The length of the data object is different from zero.

Boolean IsEmpty(T)

Returns TRUE if all the following are true:

- The TLV Database includes a data object with tag T.
- The length of the data object is zero.

T TagOf(DataObjectName)

Returns the tag of the data object with name DataObjectName.

Initialize(T)

Initializes the data object with tag T with a zero length. After initialization the data object is present in the TLV Database.

DataObject GetTLV(T)

Retrieves the TLV encoded data object with tag T from the TLV Database. Returns NULL if the TLV Database does not include a data object with tag T.

Length GetLength(T)

Retrieves from the TLV Database the length in bytes of the data object with tag T.
Returns NULL if the TLV Database does not include a data object with tag T.

Boolean ParseAndStoreCardResponse(TLV String)

TLV Encoding Error := FALSE

Parse TLV String according the Basic Encoding Rules in [ISO/IEC 8825-1] and set TLV Encoding Error to TRUE if parsing error.

If TLV String is not a single constructed or primitive data object then set TLV Encoding Error to TRUE.

IF [TLV Encoding Error]

THEN

Return FALSE

ELSE

FOR every primitive TLV in TLV String

{

IF [NOT (IsKnown(T) AND

class of T is Private class³ AND

NOT update conditions of T include RA Signal)]

THEN

IF [IsKnown(T)]

THEN

IF [(IsNotPresent(T) OR IsEmpty(T))

AND

update conditions of T include RA Signal

AND

L is within the range specified by Length field of the data object with tag T in the data dictionary in Annex A

AND

TLV is included in the correct template (if any) within TLV String⁴

]

THEN

Store LV in the TLV Database for tag T

ELSE

Return FALSE

ENDIF

ELSE

³ As defined in Annex B of [EMV Book 3], the tag is Private class if bits b7 and b8 of the first byte of the tag are both set to 1b.

⁴ Data objects must be included in the correct template as indicated in the data dictionary in Annex A. Data objects for which no template is indicated ("–") must not be returned in a template from the card.

```
        IF      [IsPresent(T)]
        THEN
            IF      [IsEmpty(T) AND update conditions of T include RA Signal]
            THEN
                Store LV in the TLV Database for tag T
            ELSE
                Return FALSE
            ENDIF
        ENDIF
    ENDIF
    ENDIF
}
Return TRUE
ENDIF
```

UpdateWithDetData(Terminal Sent Data)⁵

Copies all incoming data (Terminal Sent Data) to the Kernel TLV Database if update conditions allow.

Note that individual data objects contained within lists in Terminal Sent Data are not stored in the database.

FOR every TLV in Terminal Sent Data

```
{
  IF    [(IsKnown(T) OR IsPresent(T)) AND
         update conditions of T include DET Signal]
  THEN
    Store LV in the TLV Database for tag T
  ENDIF
}
IF    [Terminal Sent Data includes Tags To Read]
THEN
  AddListToList(Tags To Read, Tags To Read Yet)
ENDIF
IF    [Terminal Sent Data includes Tags To Write Before Gen AC]
THEN
  AddListToList(Tags To Write Before Gen AC, Tags To Write Yet Before Gen AC)
ENDIF
IF    [Terminal Sent Data includes Tags To Write After Gen AC]
THEN
  AddListToList(Tags To Write After Gen AC, Tags To Write Yet After Gen AC)
ENDIF
```

⁵ Only implemented for the DE/DS Implementation Option

4.1.4 DOL Handling

TLV encoded data objects moved from the Kernel to the Card are identified by a DOL sent to the Kernel by the Card.

DOLs used in this specification are processed as follows:

- CDOL1 and PDOL
DOL handling must be performed according to the rules specified in section 5.4 of [EMV Book 3].
- DSDOL⁶
All entries except the last must be handled according to the rules specified in section 5.4 of [EMV Book 3].
The last entry in DSDOL must be handled according to the rules specified in section 5.4 of [EMV Book 3], unless the length specified in this entry is greater than the actual length of the data object in the TLV Database. In this case, no padding must be applied and the value must be appended with the length defined in the TLV Database.

Note that tags in a DOL that exist in the TLV Database with zero length are still handled according to the rules specified in section 5.4 of [EMV Book 3], but in addition in case the DE/DS Implementation Option is implemented, any such data objects get requested from the Terminal so that the Terminal is afforded the opportunity to furnish a value for these data objects.

⁶ Only when DE/DS is implemented.

4.2 Working Variables

The Kernel makes use of a number of working variables that are not stored in the TLV Database. They are managed by the Kernel in an implementation specific way.

Working variables can be:

- Local

The lifetime of local working variables is limited to the state transition process or procedure in which they are defined. These data objects do not appear in the data dictionary.

- Global

The lifetime of global working variables is the same as the lifetime of the Kernel process. Global working variables are listed in the data dictionary without a tag.

These data objects are managed by the Kernel itself.

Global working variables can only be read and written by internal processing of the Kernel.

4.3 List Handling

Data is passed between the Kernel and other entities within Signals. The data within the Signals contain a list of tags, in order to request data, or a list of data objects in response to a request.

Each list has a unique name, and acts as a container for a collection of ListItems. A ListItem is a single element in a List. A ListItem is a tag in a list of tags or a data object in a list of data objects.

The following lists of tags are only supported if DE/DS is implemented:

- Tags To Read
- Tags To Read Yet
- Data Needed

The following lists of TLV encoded data objects are supported:

- Tags To Write After Gen AC⁷
- Tags To Write Before Gen AC⁷
- Tags To Write Yet After Gen AC⁷
- Tags To Write Yet Before Gen AC⁷
- Data To Send⁷
- Data Record
- Discretionary Data

The following methods are used to manipulate lists.

Initialize(List)

Initializes a List. This creates the List structure if it does not exist, and initializes its contents to be empty, i.e. the List contains no ListItems. This method can be called at any time during the operation of the Kernel in order to clear and reset a list.

AddToList(ListItem, List)

If ListItem is not included in List, then adds ListItem to the end of List.

Updates ListItem if it is already included in the List.

RemoveFromList(ListItem, List)

Removes ListItem from the List if ListItem is present in List. Ignores otherwise.

⁷ Only if DE/DS is implemented.

AddListToList(List1, List2)

Adds the ListItems in List1 that are not yet included in List2 to the end of List2.

Updates ListItems that are already included in List2.

ListItem GetAndRemoveFromList(List)

Removes and returns the first ListItem from List. Returns NULL if List is empty.

T GetNextGetDataTagFromList(List)⁸

Removes and returns the first tag from a list of tags that is categorized as being available from the Card using a GET DATA command.

If no tag is found, NULL is returned.

Boolean IsEmptyList(List)

Returns TRUE if List contains no ListItems.

Boolean IsNotEmptyList(List)

Returns TRUE if List contains ListItems.

⁸ Only if DE/DS is implemented.

4.4 Configuration Data

At the time of instantiation of the Kernel the data objects listed in this section are initialized.

4.4.1 Configuration Data – TLV Database

Configuration data objects in the TLV Database should receive a value at instantiation of the Kernel.

The data objects listed in Table 4.2 are the configuration data objects that must be present for the Kernel to work properly. If these data objects are not present at instantiation, a default value must be stored in the TLV Database.

Dependent on the chosen Implementation Option some data objects listed in Table 4.2 may not be present in the Configuration Data as specified in Table A.1.

Table 4.2: Configuration Data in TLV Database that Require Default Value

Data Object	Default Value
Additional Terminal Capabilities	'0000000000'
Application Version Number (Reader)	'0002'
Card Data Input Capability	'00'
CVM Capability – CVM Required	'00'
CVM Capability – No CVM Required	'00'
Hold Time Value	'0D'
Kernel Configuration	'00'
Kernel ID	'02'
Message Hold Time	'000013'
Phone Message Table	See Table 4.3
Reader Contactless Floor Limit	'000000000000'
Reader Contactless Transaction Limit (No CDCVM)	'000000000000'
Reader Contactless Transaction Limit (CDCVM)	'000000000000'
Reader CVM Required Limit	'000000000000'
Security Capability	'00'
Terminal Action Code – Default	'840000000C'
Terminal Action Code – Denial	'840000000C'
Terminal Action Code – Online	'840000000C'
Terminal Country Code	'0000'

Data Object	Default Value
Terminal Expected Transmission Time For Relay Resistance C-APDU	'0012'
Terminal Expected Transmission Time For Relay Resistance R-APDU	'0018'
Terminal Type	'00'
Time Out Value	'01F4'
Transaction Type	'00'

Table 4.3 gives the default value of the Phone Message Table for the current definition of the POS Cardholder Interaction Information.

Table 4.3: Phone Message Table – Default Value

PCII Mask	PCII Value	Message Identifier	Status
'000001'	'000001'	SEE PHONE	NOT READY
'000800'	'000800'	SEE PHONE	NOT READY
'000400'	'000400'	SEE PHONE	NOT READY
'000100'	'000100'	SEE PHONE	NOT READY
'000200'	'000200'	SEE PHONE	NOT READY
'000000'	'000000'	DECLINED	NOT READY

4.4.2 CA Public Key Database

The Kernel has access to a CA Public Key Database containing the CA Public Keys applicable for the RID of the selected AID. This CA Public Key Database is made available to the Kernel and is read-only.

The CA Public Key Index uniquely identifies the CA Public Key in the CA Public Key Database.

Table 4.4 lists the set of data objects that must be available in the CA Public Key Database for each CA Public Key.

Table 4.4: CA Public Key Related Data

Field Name	Length	Description	Format
CA Public Key Index	1	Identifies the CA Public Key in conjunction with the RID	b
CA Hash Algorithm Indicator	1	Identifies the hash algorithm used to produce the Hash Result in the digital signature scheme	b
CA Public Key Algorithm Indicator	1	Identifies the digital signature algorithm to be used with the CA Public Key	b
CA Public Key Modulus	var. (max 248)	Value of the modulus part of the CA Public Key	b
CA Public Key Exponent	1 or 3	Value of the exponent part of the CA Public Key, equal to 3 or $2^{16} + 1$	b
CA Public Key Check Sum (Only necessary if used to verify the integrity of the CA Public Key)	20	A check value calculated on the concatenation of all parts of the CA Public Key (RID, CA Public Key Index, CA Public Key Modulus, CA Public Key Exponent) using SHA-1	b

4.4.3 Certification Revocation List

The Kernel has access to a CRL applicable for the RID of the selected AID. This CRL is made available to the Kernel and is read-only.

Table 4.5 lists the set of data objects that must be available in the CRL for each revoked certificate. If, during CDA, a concatenation of the CA Public Key Index (Card) and the Certificate Serial Number recovered from the Issuer Public Key Certificate is on this list, then CDA fails.

Table 4.5: Certification Revocation List Related Data

Field Name	Length	Description	Format
CA Public Key Index	1	Identifies the CA Public Key in conjunction with the RID	b
Certificate Serial Number	3	Number unique to this certificate assigned by the certification authority	b
Additional Data	var.	Optional Terminal proprietary data, such as the date the certificate was added to the revocation list	b

4.5 Lists of Data Objects in OUT

This section specifies the lists of data objects included in the OUT Signal.

4.5.1 Outcome Parameter Set

This data object is used to indicate to the Terminal the outcome of the transaction processing by the Kernel. Its content is described in section A.1.97.

4.5.2 User Interface Request Data

Depending on the outcome of the transaction, the User Interface Request Data may be included in the OUT Signal to communicate a message to be shown on the Terminal display. The User Interface Request Data may also be sent by the Kernel in a MSG Signal. Its format is described in section A.1.164.

4.5.3 Data Record

Depending on the outcome of the transaction, the Kernel may provide the Terminal with an OUT Signal including a Data Record that contains the necessary data objects for authorization and clearing. The Data Record is a list of data objects as shown in Table 4.6.

Table 4.6: Data Record Detail

Data Object
Amount, Authorized (Numeric)
Amount, Other (Numeric)
Application Cryptogram
Application Expiration Date
Application Interchange Profile
Application Label
Application PAN
Application PAN Sequence Number
Application Preferred Name
Application Transaction Counter
Application Usage Control
Application Version Number (Reader)
Cryptogram Information Data
CVM Results

Data Object
DF Name
Interface Device Serial Number
Issuer Application Data
Issuer Code Table Index
Payment Account Reference
Terminal Capabilities
Terminal Country Code
Terminal Type
Terminal Verification Results
Token Requestor ID
Track 2 Equivalent Data
Transaction Category Code
Transaction Currency Code
Transaction Date
Transaction Type
Unpredictable Number

The following method is used to create the Data Record:

CreateDataRecord ()

 Initialize(Data Record)

 FOR every Data Object in Table 4.6

 {

 IF [IsPresent(TagOf(Data Object))]

 THEN

 AddToList(GetTLV(TagOf(Data Object)), Data Record)

 ENDIF

 }

4.5.4 Discretionary Data

The Kernel always includes Discretionary Data in the OUT Signal. The Discretionary Data is a list of data objects as shown in Table 4.7.

Table 4.7: Discretionary Data

Data Object
Application Capabilities Information
Application Currency Code
DS Summary 3 ⁹
DS Summary Status ⁹
Error Indication
Post-Gen AC Put Data Status ⁹
Pre-Gen AC Put Data Status ⁹
Third Party Data

The following method is used to create the Discretionary Data:

```
CreateDiscretionaryData ()
    Initialize(Discretionary Data)
    FOR every Data Object in Table 4.7
    {
        IF [IsPresent(TagOf(Data Object))]
        THEN
            AddToList(GetTLV(TagOf(Data Object)), Discretionary Data)
        ENDIF
    }
```

⁹ Only if DE/DS is implemented.

4.6 Data Object Format

4.6.1 Format

All data objects known to the Kernel (other than local working variables) are listed in the data dictionary in Annex A. All the length indications in the data dictionary are given in number of bytes. Data object formats are binary (b), numeric (n), compressed numeric (cn), alphanumeric (an), or alphanumeric special (ans).

Data objects that have the numeric (n) format are BCD encoded, right justified with leading hexadecimal zeros. Data objects that have the compressed numeric (cn) format are BCD encoded, left justified, and padded with trailing 'F's. Note that the length indicator in the numeric and compressed numeric format notations (e.g. n 4) specifies the number of digits and not the number of bytes.

Data objects that have the alphanumeric (an) or alphanumeric special (ans) format are ASCII encoded, left justified, and padded with trailing hexadecimal zeros.

Data objects that have the binary (b) format consist of either unsigned binary numbers or bit combinations that are defined in the specification.

When moving data from one entity to another (for example Card to Reader) or when concatenating data, the data must always be passed in decreasing order, regardless of how it is stored internally. The leftmost byte (byte 1) is the most significant byte.

Data objects are TLV encoded in the following cases:

- Data objects sent from the Card to the Kernel (RA Signal)
- Data objects sent to the Kernel at instantiation or with the ACT and DET Signals
- Data objects sent to the Terminal included in Data To Send
- Data objects included in the MSG and OUT Signals

4.6.2 Format Checking

It is the responsibility of the issuer to ensure that data in the Card is of the correct format. No format checking other than that specifically defined is mandated for the Kernel.

However, if during normal processing it is recognized that data read from the Card or provided by the Terminal is incorrectly formatted, the Kernel must perform the processing described in this section.

Other than exceptions specifically defined in this document, data object formatting that does not comply with the requirements in section 12.2.4 of [EMV Book 1] and sections 7.5 and 10.5 of [EMV Book 3] can be considered as a format error.

If a format error is detected in data received from the Card, the Kernel must update the Error Indication data object as follows:

'L2' in Error Indication := CARD DATA ERROR

If a format error is detected in data received from the Terminal, the Kernel must update the Error Indication data object as follows:

'L2' in Error Indication := TERMINAL DATA ERROR

The Kernel must then process the exception according to the state in which it occurs, as described here.

States 1, 2, 3, R1, 4, 5 and 6

The Kernel must prepare the User Interface Request Data, the Discretionary Data and the Outcome Parameter Set and send an OUT Signal (as shown here):

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

'Hold Time' in User Interface Request Data := Message Hold Time

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

Initialize(Discretionary Data)

AddToList(GetTLV(TagOf(Error Indication)), Discretionary Data)

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)), GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request Data))) Signal

The Kernel must then exit.

State 9

The Kernel must process the error as “Invalid Response – 1”, as described under connector C in Figure 6.11.

4.7 Reserved for Future Use (RFU)

A bit specified as Reserved for Future Use (RFU) must be set as specified, or to 0b if no indication is given. An entity receiving a bit specified as RFU must ignore such a bit and must not change its behaviour, unless explicitly stated otherwise.

A data field having a value coded on multiple bits or bytes must not be set to a value specified as RFU. An entity receiving a data field having a value specified as RFU behaves as defined by a requirement that specifically addresses the situation.

5 C-APDU Commands

This chapter defines the commands and responses supported by the Kernel.

5.1 Introduction

The INS byte of the C-APDU is structured according to [EMV Book 1]. The coding of INS and its relationship to CLA are shown in Table 5.1.

Table 5.1: Coding of the Instruction Byte

CLA	INS	Meaning
'80'	'EA'	EXCHANGE RELAY RESISTANCE DATA
'80'	'AE'	GENERATE AC
'80'	'CA'	GET DATA ¹⁰
'80'	'A8'	GET PROCESSING OPTIONS
'80'	'DA'	PUT DATA ¹⁰
'00'	'B2'	READ RECORD

The status bytes returned by the Card are coded as specified in section 6.3.5 of [EMV Book 3]. In addition to the status bytes specific to each command, the Card may return the status bytes shown in Table 5.2.

Table 5.2: Generic Status Bytes

SW1	SW2	Meaning
'6D'	'00'	Instruction code not supported or invalid
'6E'	'00'	Class not supported
'6F'	'00'	No precise diagnosis

¹⁰ Only implemented for the DE/DS Implementation Option

5.2 Exchange Relay Resistance Data

5.2.1 Definition and Scope

The EXCHANGE RELAY RESISTANCE DATA command exchanges relay resistance related data with the Card.

5.2.2 Command Message

The EXCHANGE RELAY RESISTANCE DATA command message is coded according to Table 5.3.

Table 5.3: Exchange Relay Resistance Data Command Message

Code	Value
CLA	'80'
INS	'EA'
P1	'00'
P2	'00'
Lc	'04'
Data	Terminal Relay Resistance Entropy
Le	'00'

5.2.3 Data Field Returned in the Response Message

The data object returned in the response message is a primitive data object with tag '80' and length '0A'. The value field consists of the concatenation without delimiters (tag and length) of the value fields of the data objects specified in Table 5.4.

Table 5.4: Exchange Relay Resistance Data Response Message Data Field

Tag	Length	Byte	Value	Presence
'80'	'0A'	1-4	Device Relay Resistance Entropy	M
		5-6	Min Time For Processing Relay Resistance APDU	M
		7-8	Max Time For Processing Relay Resistance APDU	M
		9-10	Device Estimated Transmission Time For Relay Resistance R-APDU	M

5.2.4 Status Bytes

The status bytes that may be sent in response to the EXCHANGE RELAY RESISTANCE DATA command are listed in Table 5.5.

Table 5.5: Status Bytes for Exchange Relay Resistance Data Command

SW1	SW2	Meaning
'67'	'00'	Wrong length
'69'	'85'	Conditions of use not satisfied
'6A'	'86'	Incorrect parameters P1-P2
'90'	'00'	Normal processing

5.3 Generate AC

5.3.1 Definition and Scope

The GENERATE AC command sends transaction-related data to the Card, which then computes and returns an Application Cryptogram. Depending on the risk management in the Card, the cryptogram returned by the Card may differ from that requested in the command message. The Card may return an AAC (transaction declined), an ARQC (online authorization request), or a TC (transaction approved).

5.3.2 Command Message

The GENERATE AC command message is coded according to Table 5.6.

Table 5.6: Generate AC Command Message

Code	Value
CLA	'80'
INS	'AE'
P1	Reference Control Parameter (see Table 5.7)
P2	'00'
Lc	var.
Data	CDOL1 Related Data DSDOL related data (conditional (if IDS write performed))
Le	'00'

Table 5.7: Generate AC Reference Control Parameter

b8	b7	b6	b5	b4	b3	b2	b1	Meaning
0	0							AAC
0	1							TC
1	0							ARQC
1	1							RFU
		x						RFU
		0						Other values RFU
			0					CDA not requested
			1					CDA requested
				x	x	x	x	RFU
				0	0	0	0	Other values RFU

The data field of the command message contains CDOL1 Related Data coded according to CDOL1 following the rules defined in section 4.1.4.

In the case of IDS data writing, the data field of the command message is a concatenation of CDOL1 Related Data followed by DSDOL related data coded according to DSDOL following the rules defined in section 4.1.4.

5.3.3 Data Field Returned in the Response Message

The data field in the response message to the GENERATE AC command is coded according to either format 1 or format 2, as follows.

Format 1

In the case of format 1, the data object returned in the response message is a primitive data object Response Message Template Format 1 with tag equal to '80'. The value field consists of the concatenation without delimiters (tag and length) of the value fields of the data objects specified in Table 5.8.

Format 1 is not used if CDA is performed.

Table 5.8: Generate AC Response Message Data Field (Format 1)

Tag	Length	Value	Presence
'80'	Var.	Cryptogram Information Data	M
		Application Transaction Counter	M
		Application Cryptogram	M
		Issuer Application Data	O

Format 2

In the case of format 2, the data object returned in the response message varies depending on whether CDA was performed or not.

CDA Not Performed

If CDA is not performed, the data object returned in the response message is a constructed data object with tag equal to '77' (Response Message Template Format 2), as specified in Table 5.9.

Data objects in Response Message Template Format 2 may appear in any order.

Table 5.9: Generate AC Response Message Data Field (Format 2) – No CDA

Tag	Value		Presence
'77'	Response Message Template Format 2		M
	'9F27'	Cryptogram Information Data	M
	'9F36'	Application Transaction Counter	M
	'9F26'	Application Cryptogram	M
	'9F10'	Issuer Application Data	O
	'DF4B'	POS Cardholder Interaction Information	O

CDA Performed

If CDA is performed, the data object returned in the response message is a constructed data object with tag equal to '77' (Response Message Template Format 2). It contains at least the three mandatory data objects specified in Table 5.10, and optionally the Issuer Application Data.

Data objects in Response Message Template Format 2 may appear in any order.

Table 5.10: Generate AC Response Message Data Field (Format 2) – CDA

Tag	Value		Presence
'77'	Response Message Template Format 2		M
	'9F27'	Cryptogram Information Data	M
	'9F36'	Application Transaction Counter	M
	'9F4B'	Signed Dynamic Application Data	M
	'9F10'	Issuer Application Data	O
	'DF4B'	POS Cardholder Interaction Information	O

5.3.4 Status Bytes

The status bytes that may be sent in response to the GENERATE AC command are listed in Table 5.11.

Table 5.11: Status Bytes for Generate AC Command

SW1	SW2	Meaning
'67'	'00'	Wrong length
'69'	'85'	Conditions of use not satisfied
'6A'	'86'	Incorrect parameters P1-P2
'90'	'00'	Normal processing

5.4 Get Data

5.4.1 Definition and Scope

The GET DATA command is used to retrieve a primitive data object from the Card not encapsulated in a record.

5.4.2 Command Message

The GET DATA command message is coded according to Table 5.12.

Table 5.12: Get Data Command Message

Code	Value
CLA	'80'
INS	'CA'
P1 P2	Tag
Lc	Not present
Data	Not present
Le	'00'

Table 5.13 lists the tag values supported for the GET DATA command.

Table 5.13: Supported P1 || P2 Values for Get Data Command

P1 P2	Data Object
'9F70'	Protected Data Envelope 1
'9F71'	Protected Data Envelope 2
'9F72'	Protected Data Envelope 3
'9F73'	Protected Data Envelope 4
'9F74'	Protected Data Envelope 5
'9F75'	Unprotected Data Envelope 1
'9F76'	Unprotected Data Envelope 2
'9F77'	Unprotected Data Envelope 3
'9F78'	Unprotected Data Envelope 4
'9F79'	Unprotected Data Envelope 5

5.4.3 Data Field Returned in the Response Message

The data field of the response message contains the primitive data object referred to in P1 || P2 of the command message (in other words, including its tag and its length).

5.4.4 Status Bytes

The status bytes that may be sent in response to the GET DATA command are listed in Table 5.14.

Table 5.14: Status Bytes for Get Data Command

SW1	SW2	Meaning
'69'	'85'	Conditions of use not satisfied
'6A'	'81'	Wrong parameter(s) P1 P2; function not supported
'6A'	'88'	Referenced data (data object) not found
'90'	'00'	Normal processing

5.5 Get Processing Options

5.5.1 Definition and Scope

The GET PROCESSING OPTIONS command initiates the transaction within the Card.

5.5.2 Command Message

The GET PROCESSING OPTIONS command message is coded according to Table 5.15.

Table 5.15: Get Processing Options Command Message

Code	Value
CLA	'80'
INS	'A8'
P1	'00'
P2	'00'
Lc	var.
Data	PDOL Related Data
Le	'00'

The data field of the command message consists of PDOL Related Data. PDOL Related Data is the Command Template with tag '83' and with a value field coded according to the PDOL provided by the Card in the response to the SELECT command. If the PDOL is not provided by the Card, the length field of the Command Template is set to zero. Otherwise the length field is the total length of the value fields of the data objects transmitted to the Card. The value fields are concatenated according to the rules defined in section 4.1.4.

5.5.3 Data Field Returned in the Response Message

The data field in the response message to the GET PROCESSING OPTIONS command is coded according to either format 1 or format 2, as follows.

Format 1

In the case of format 1, the data object returned in the response message is a primitive data object with tag equal to '80'. The value field consists of the concatenation without delimiters (tag and length) of the value fields of the Application Interchange Profile and the Application File Locator, as shown in Table 5.16.

Table 5.16: Get Processing Options Response Message Data Field (Format 1)

Tag	Length	Value	Presence
'80'	Var.	Application Interchange Profile	M
		Application File Locator	M

Format 2

In the case of format 2, the data object returned in the response message is a constructed data object with tag '77' (Response Message Template Format 2). The value field may include several TLV coded objects, but always includes the Application Interchange Profile and Application File Locator, as shown in Table 5.17. If IDS is supported by both Card and Kernel, then also the IDS related data objects shown in Table 5.17 may be included in the Response Message Template Format 2.

Data objects in Response Message Template Format 2 may appear in any order.

Table 5.17: Get Processing Options Response Message Data Field (Format 2)

Tag	Value		Presence
'77'	Response Message Template Format 2		M
	'82'	Application Interchange Profile	M
	'94'	Application File Locator	M
	'9F6F'	DS Slot Management Control	O
	'9F5F'	DS Slot Availability	O
	'9F7F'	DS Unpredictable Number	O
	'9F7D'	DS Summary 1	O
	'9F54'	DS ODS Card	O

5.5.4 Status Bytes

The status bytes that may be sent in response to the GET PROCESSING OPTIONS command are listed in Table 5.18.

Table 5.18: Status Bytes for Get Processing Options Command

SW1	SW2	Meaning
'67'	'00'	Wrong length
'69'	'85'	Conditions of use not satisfied
'6A'	'86'	Incorrect parameters P1-P2
'90'	'00'	Normal processing

5.6 Put Data

5.6.1 Definition and Scope

The PUT DATA command is used to store a primitive data object not encapsulated in a record in the Card.

5.6.2 Command Message

The PUT DATA command message is coded according to Table 5.19.

Table 5.19: Put Data Command Message

Code	Value
CLA	'80'
INS	'DA'
P1 P2	Tag
Lc	var.
Data	New data value
Le	Not present

Single byte tags are preceded with a leading '00' byte to fill P1 || P2. Table 5.20 lists the minimum set of tag values that must be supported for the PUT DATA command.

Table 5.20: Supported P1 || P2 values for Put Data Command

P1 P2	Data Object
'9F75'	Unprotected Data Envelope 1
'9F76'	Unprotected Data Envelope 2
'9F77'	Unprotected Data Envelope 3
'9F78'	Unprotected Data Envelope 4
'9F79'	Unprotected Data Envelope 5

5.6.3 Data Field Returned in the Response Message

There is no data field in the response message of the PUT DATA command.

5.6.4 Status Bytes

The status bytes that may be sent in response to the PUT DATA command are listed in Table 5.21.

Table 5.21: Status Bytes for Put Data Command

SW1	SW2	Meaning
'67'	'00'	Wrong length
'6A'	'88'	Referenced data (data object) not found
'90'	'00'	Normal processing

5.7 Read Record

5.7.1 Definition and Scope

The READ RECORD command reads a file record in a linear file. The response of the Card consists of returning the record.

5.7.2 Command Message

The READ RECORD command message is coded according to Table 5.22.

Table 5.22: Read Record Command Message

Code	Value
CLA	'00'
INS	'B2'
P1	Record number
P2	See Table 5.23
Lc	Not present
Data	Not present
Le	'00'

Table 5.23 specifies the coding of P2 of the READ RECORD command.

Table 5.23: P2 of Read Record Command

b8	b7	b6	b5	b4	b3	b2	b1	Meaning
x	x	x	x	x				SFI
					1	0	0	P1 is a record number

5.7.3 Data Field Returned in the Response Message

The data field in the Card response contains the record requested by the command. For SFIs in the range 1-10, the record is a TLV constructed data object with tag '70' as shown in Table 5.24.

Table 5.24: Read Record Response Message Data Field

'70'	Length	Record Template
------	--------	-----------------

5.7.4 Status Bytes

The status bytes that may be sent in response to the READ RECORD command are listed in Table 5.25.

Table 5.25: Status Bytes for Read Record Command

SW1	SW2	Meaning
'69'	'85'	Conditions of use not satisfied
'6A'	'82'	Wrong parameters P1 P2; file not found
'6A'	'83'	Wrong parameters P1 P2; record not found
'6A'	'86'	Incorrect parameters P1 P2
'90'	'00'	Normal processing

6 Kernel State Machine

This chapter describes the transaction processing of the Kernel after it has been initiated by Process M.

6.1 Implementation Principles

The transaction processing is specified as a state machine that is triggered by external Signals that cause state transitions. The state machine is presented in more detail in Annex B.

These principles are used in order to present the application concepts. The same principles do not have to be followed in the actual implementation. However, the implementation must behave in a way that is indistinguishable from the behaviour specified in this chapter.

The Kernel receives incoming Signals on a Queue. The Queue is used by Process M for the ACT Signal and by Process P for the RA and L1RSP Signals.

The Kernel can be in a state for which a state transition is triggered by Signals that are not at the top of a Queue. In this case the state transition is triggered by the first Signal on the Queue that causes a state transition. Signals on the Queue that are not handled in the current state must not be lost and must stay on the Queue for processing in subsequent states.

6.2 Kernel Started

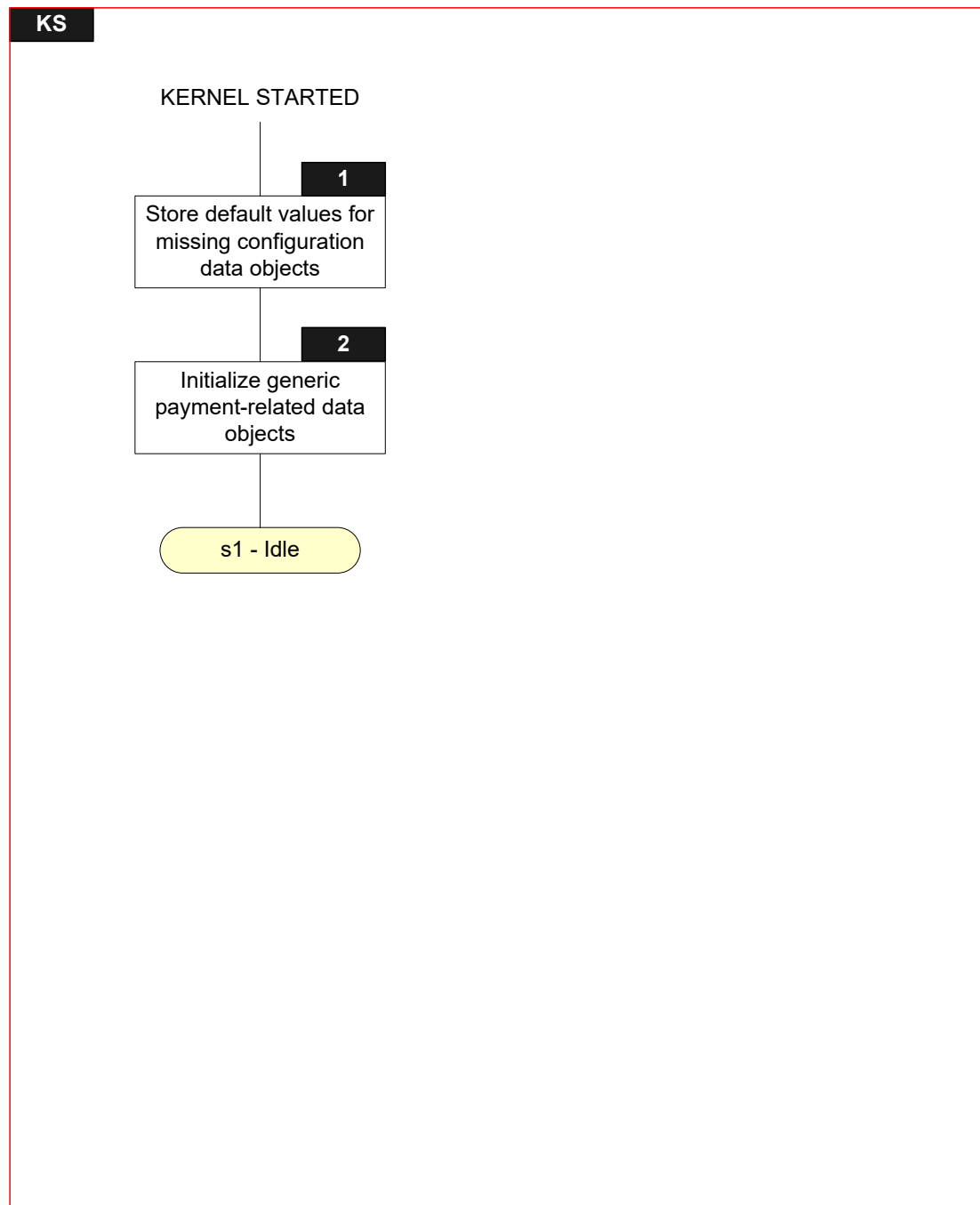
6.2.1 Local Variables

Name	Length	Format	Description
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 253	b	Value of TLV encoded string

6.2.2 Flow Diagram

Figure 6.1 shows the flow diagram of startup of the Kernel. Symbols in this diagram are labelled KS.X.

Figure 6.1: Kernel Started Flow Diagram



6.2.3 Processing

KS.1

Instantiate the TLV Database to be able to store the tags identified in Table A.1. Extend the TLV Database with the Proprietary Tags, if any.

Copy the value of the configuration data objects from the Configuration Data into the TLV Database.

Use the default value as defined in Table 4.2 if a mandatory configuration data object is missing from the Configuration Data:

FOR every T for which a default value is defined in Table 4.2

```
{  
    IF      [IsNotPresent(T)]  
    THEN  
        Store LV as per Table 4.2 in the TLV Database for tag T  
    ENDIF  
}
```

KS.2

Initialize Outcome Parameter Set as follows:

```
Outcome Parameter Set := '0000 ... 00'  
'Status' in Outcome Parameter Set := N/A  
'Start' in Outcome Parameter Set := N/A  
'CVM' in Outcome Parameter Set := N/A  
CLEAR 'UI Request on Outcome Present' in Outcome Parameter Set  
CLEAR 'UI Request on Restart Present' in Outcome Parameter Set  
CLEAR 'Data Record Present' in Outcome Parameter Set  
SET 'Discretionary Data Present' in Outcome Parameter Set  
'Receipt' in Outcome Parameter Set := N/A  
'Alternate Interface Preference' in Outcome Parameter Set := N/A  
'Field Off Request' in Outcome Parameter Set := N/A  
'Removal Timeout' in Outcome Parameter Set := 0  
'Online Response Data' in Outcome Parameter Set := N/A
```

Initialize User Interface Request Data as follows:

```
User Interface Request Data := '0000 ... 00'  
'Message Identifier' in User Interface Request Data := N/A  
'Status' in User Interface Request Data := N/A  
'Hold Time' in User Interface Request Data := Message Hold Time  
'Language Preference' in User Interface Request Data := '000000000000000000'
```


Initialize Error Indication as follows:

Error Indication := '0000 ... 00'

'L1' in Error Indication := OK

'L2' in Error Indication := OK

'L3' in Error Indication := OK

'SW12' in Error Indication := '0000'

'Msg On Error' in Error Indication := N/A

6.3 State 1 – Idle

6.3.1 Local Variables

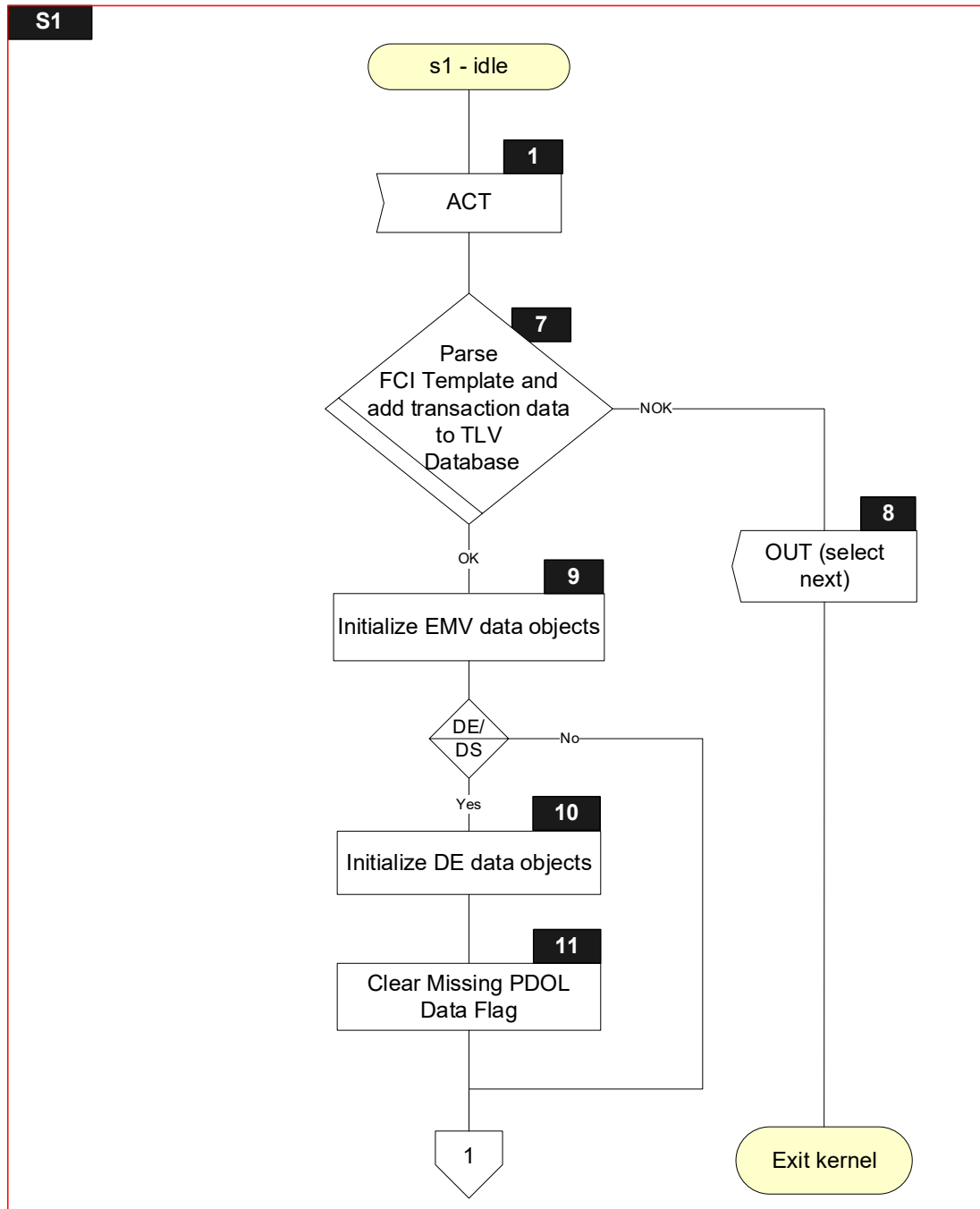
Name	Length	Format	Description
Sync Data	var.	b	List of data objects returned with ACT Signal
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
Missing PDOL Data Flag ⁽¹⁾	1	b	Boolean used to indicate if data referenced in PDOL is not present in the TLV Database.

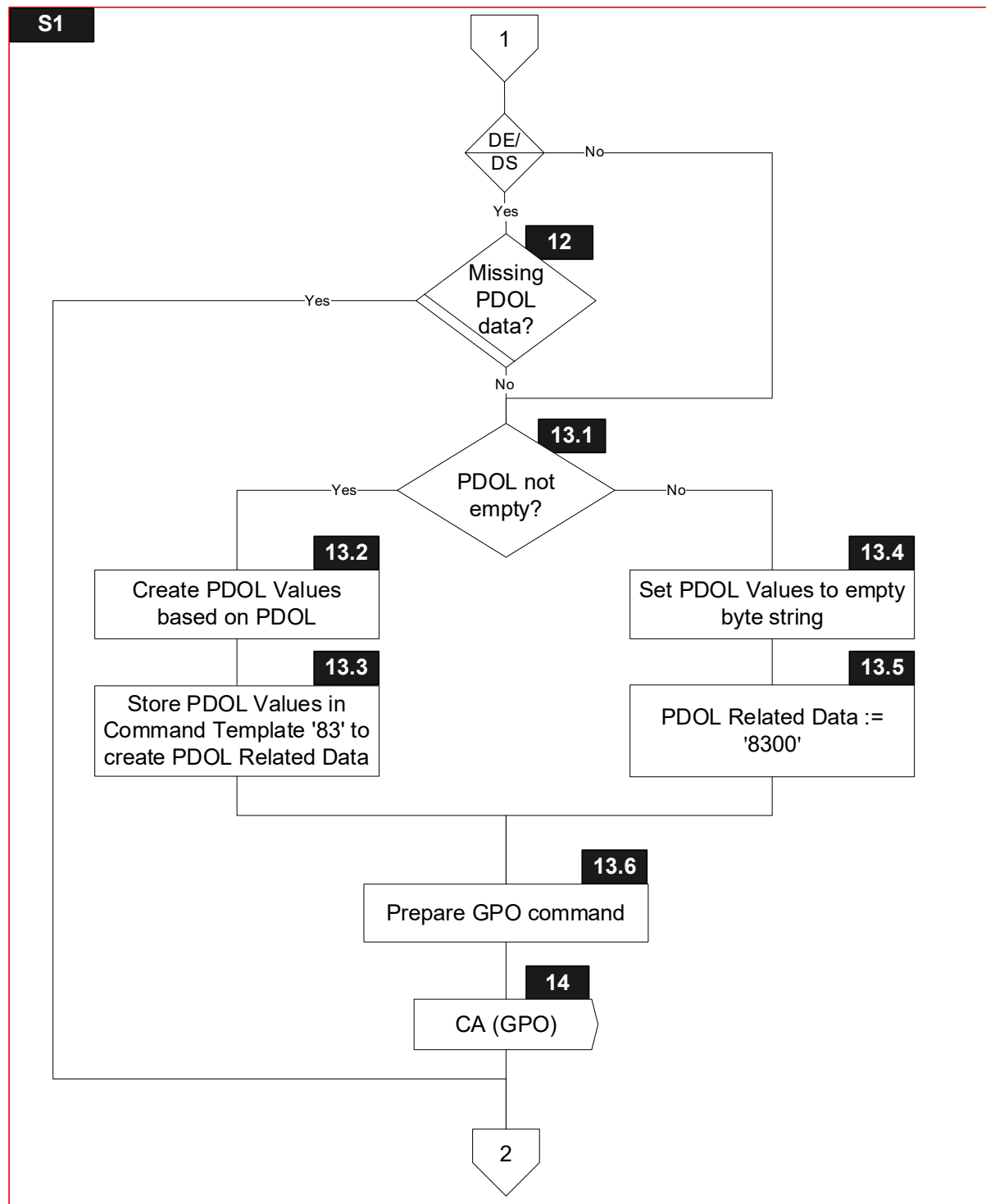
⁽¹⁾ Only for the DE/DS Implementation Option

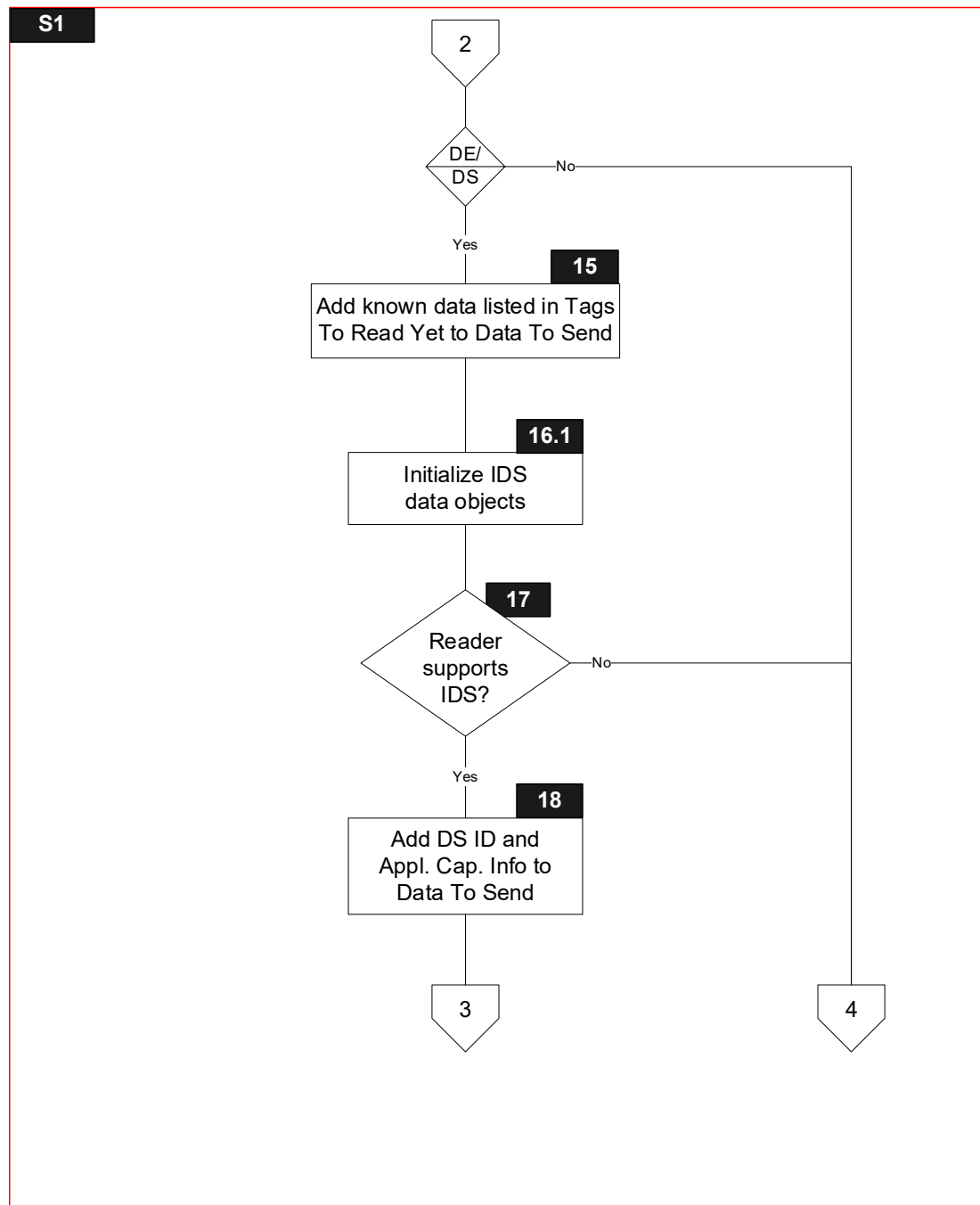
6.3.2 Flow Diagram

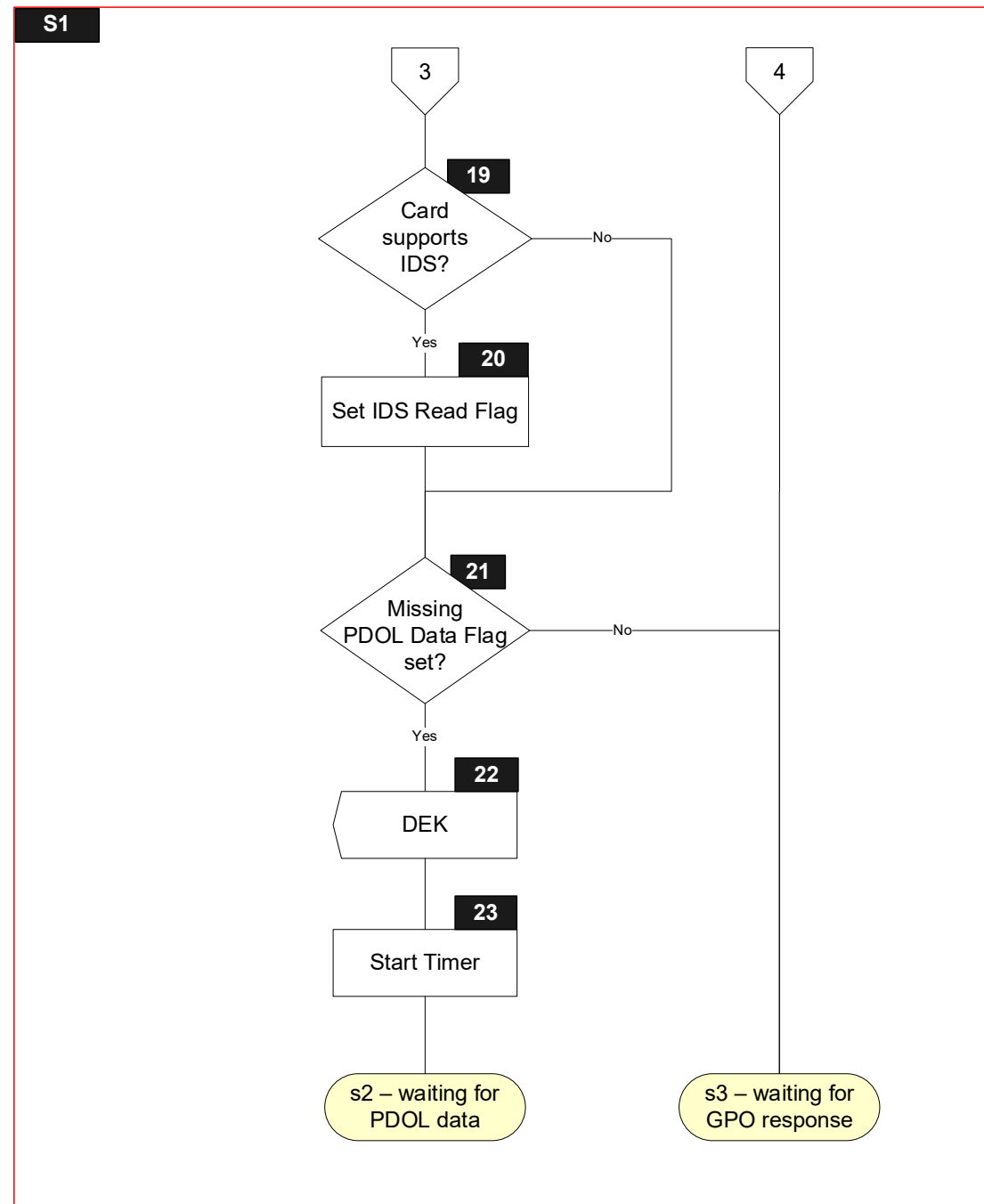
Figure 6.2 shows the flow diagram of `s1 - idle`. Symbols in this diagram are labelled S1.X.

Figure 6.2: State 1 Flow Diagram









6.3.3 Processing

Note that symbols S1.10, S1.11, S1.12, S1.15, S1.16.1, S1.17, S1.18, S1.19, S1.20, S1.21, S1.22 and S1.23 are only implemented for the DE/DS Implementation Option.

S1.1

Receive ACT Signal with Sync Data

S1.7

Add the transaction data provided in the ACT Signal to the TLV Database

Parse and store the File Control Information Template if included in Sync Data

FOR every TLV in Sync Data

```
{
    IF      [T = TagOf(File Control Information Template)]
    THEN
        IF      [NOT ParseAndStoreCardResponse(TLV)]
        THEN
            'L2' in Error Indication := PARSING ERROR
            GOTO S1.8
        ENDIF
    ELSE
        IF      [(IsKnown(T) OR IsPresent(T)) AND
            update conditions of T include ACT Signal]
        THEN
            Store LV in the TLV Database for tag T
        ENDIF
    ENDIF
}
```

If the Language Preference is returned from the Card, then copy it to 'Language Preference' in User Interface Request Data:

```
IF      [IsNotEmpty(TagOf(Language Preference))]
THEN
    'Language Preference' in User Interface Request Data := Language Preference
    If the length of Language Preference is less than 8 bytes, then pad 'Language
    Preference' in User Interface Request Data with trailing hexadecimal zeroes to 8
    bytes.
ENDIF
```

```
IF [IsNotPresent(TagOf(DF Name)) OR IsEmpty(TagOf(DF Name))]  
THEN  
    'L2' in Error Indication := CARD DATA MISSING  
    GOTO S1.8  
ENDIF  
IF [IsNotEmpty(TagOf(Application Capabilities Information))]  
THEN  
    IF ['Support for field off detection' in Application Capabilities Information is set]  
    THEN  
        'Field Off Request' in Outcome Parameter Set := Hold Time Value  
    ENDIF  
ENDIF
```

GOTO S1.9

S1.8

```
'Status' in Outcome Parameter Set := SELECT NEXT  
'Start' in Outcome Parameter Set := C  
Initialize(Discretionary Data)  
AddToList(GetTLV(TagOf(Error Indication)), Discretionary Data)  
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),  
          GetTLV(TagOf(Discretionary Data))) Signal
```

S1.9

```
CVM Results := '000000'  
'AC type' in AC Type := TC  
Terminal Verification Results := '0000000000'  
ODA Status := '00'  
RRP Counter := '00'  
Terminal Capabilities[1] := Card Data Input Capability  
Terminal Capabilities[2] := '00'  
Terminal Capabilities[3] := Security Capability  
Initialize(Static Data To Be Authenticated)  
Generate Unpredictable Number as specified in section 8.1 and store in the TLV Database for  
TagOf(Unpredictable Number)
```


S1.10

Post-Gen AC Put Data Status := '00'

Pre-Gen AC Put Data Status := '00'

Initialize(Data Needed)

Initialize(Data To Send)

Initialize(Tags To Read Yet)

Initialize(Tags To Write Yet After Gen AC)

Initialize(Tags To Write Yet Before Gen AC)

IF [IsEmpty(TagOf(Tags To Read))]

THEN

AddListToList(Tags To Read, Tags To Read Yet)

ENDIF

IF [IsEmpty(TagOf(Tags To Read))]

THEN

AddToList(TagOf(Tags To Read), Data Needed)

ENDIF

IF [IsEmpty(TagOf(Tags To Write Before Gen AC))]

THEN

AddListToList(Tags To Write Before Gen AC, Tags To Write Yet Before Gen AC)

ENDIF

IF [IsEmpty(TagOf(Tags To Write After Gen AC))]

THEN

AddListToList(Tags To Write After Gen AC, Tags To Write Yet After Gen AC)

ENDIF

IF [IsEmpty(TagOf(Tags To Write Before Gen AC))]

THEN

AddToList(TagOf(Tags To Write Before Gen AC), Data Needed)

ENDIF

IF [IsEmpty(TagOf(Tags To Write After Gen AC))]

THEN

AddToList(TagOf(Tags To Write After Gen AC), Data Needed)

ENDIF

S1.11

CLEAR Missing PDOL Data Flag

S1.12

FOR every TL entry in the PDOL

```
{
    IF      [(L > 0) AND IsEmpty(T) AND Update Conditions of T include 'DET']
    THEN
        SET Missing PDOL Data Flag
        AddToList(T, Data Needed)
    ENDIF
}
IF      [Missing PDOL Data Flag]
THEN
    GOTO S1.15
ELSE
    GOTO S1.13.1
ENDIF
```

S1.13.1

```
IF [IsNotEmpty(TagOf(PDOL))]
THEN
    GOTO S1.13.2
ELSE
    GOTO S1.13.4
ENDIF
```

S1.13.2

Use PDOL to create PDOL Values as a concatenated list of data objects without tags or lengths following the rules specified in section 4.1.4.

S1.13.3

Store PDOL Values in Command Template '83' to create PDOL Related Data.

S1.13.4

Store empty byte string in PDOL Values

S1.13.5

PDOL Related Data := '8300'

S1.13.6

Prepare GET PROCESSING OPTIONS command as specified in section 5.5.

S1.14

Send CA(GET PROCESSING OPTIONS) Signal

S1.15

FOR every T in Tags To Read Yet

```
{
    IF      [IsEmpty(T)]
    THEN
        AddToList(GetTLV(T), Data To Send)
        RemoveFromList(T, Tags To Read Yet)
    ENDIF
}
```

S1.16.1

IDS Status := '00'

DS Summary Status := '00'

DS Digest H := '000000000000000000'

S1.17

```
IF      [IsEmpty(TagOf(DSVN Term))
        AND IsPresent(TagOf(DS Requested Operator ID)) ]
```

THEN

GOTO S1.18

ELSE

GOTO S1.21

ENDIF

S1.18

```
IF      [IsPresent(TagOf(DS ID))]
```

THEN

AddToList(GetTLV(TagOf(DS ID)), Data To Send)

ELSE

Add empty DS ID to Data To Send:

AddToList(TagOf(DS ID) || '00', Data To Send)

ENDIF

```
IF      [IsPresent(TagOf(Application Capabilities Information))]
```

THEN

AddToList(GetTLV(TagOf(Application Capabilities Information)), Data To Send)

ELSE

Add empty Application Capabilities Information to Data To Send:

AddToList(TagOf(Application Capabilities Information) || '00', Data To Send)

ENDIF

S1.19

```
IF      [IsEmpty(TagOf (Application Capabilities Information)) AND
        (('Data Storage Version Number' in Application Capabilities Information =
         VERSION 1)
        OR
        ('Data Storage Version Number' in Application Capabilities Information =
         VERSION 2))
        AND IsNotEmpty(TagOf (DS ID)) ]
THEN
    GOTO S1.20
ELSE
    GOTO S1.21
ENDIF
```

S1.20

SET 'Read' in IDS Status

S1.21

```
IF      [Missing PDOL Data Flag is set]
THEN
    GOTO S1.22
ELSE
    GOTO s3 - waiting for GPO response
ENDIF
```

S1.22

Send DEK(Data To Send, Data Needed) Signal
Initialize(Data To Send)
Initialize(Data Needed)

S1.23

Start Timer (Time Out Value)

6.4 State 2 – Waiting for PDOL Data

State 2 is a state specific to Data Exchange processing and is only implemented for the DE/DS Implementation Option.

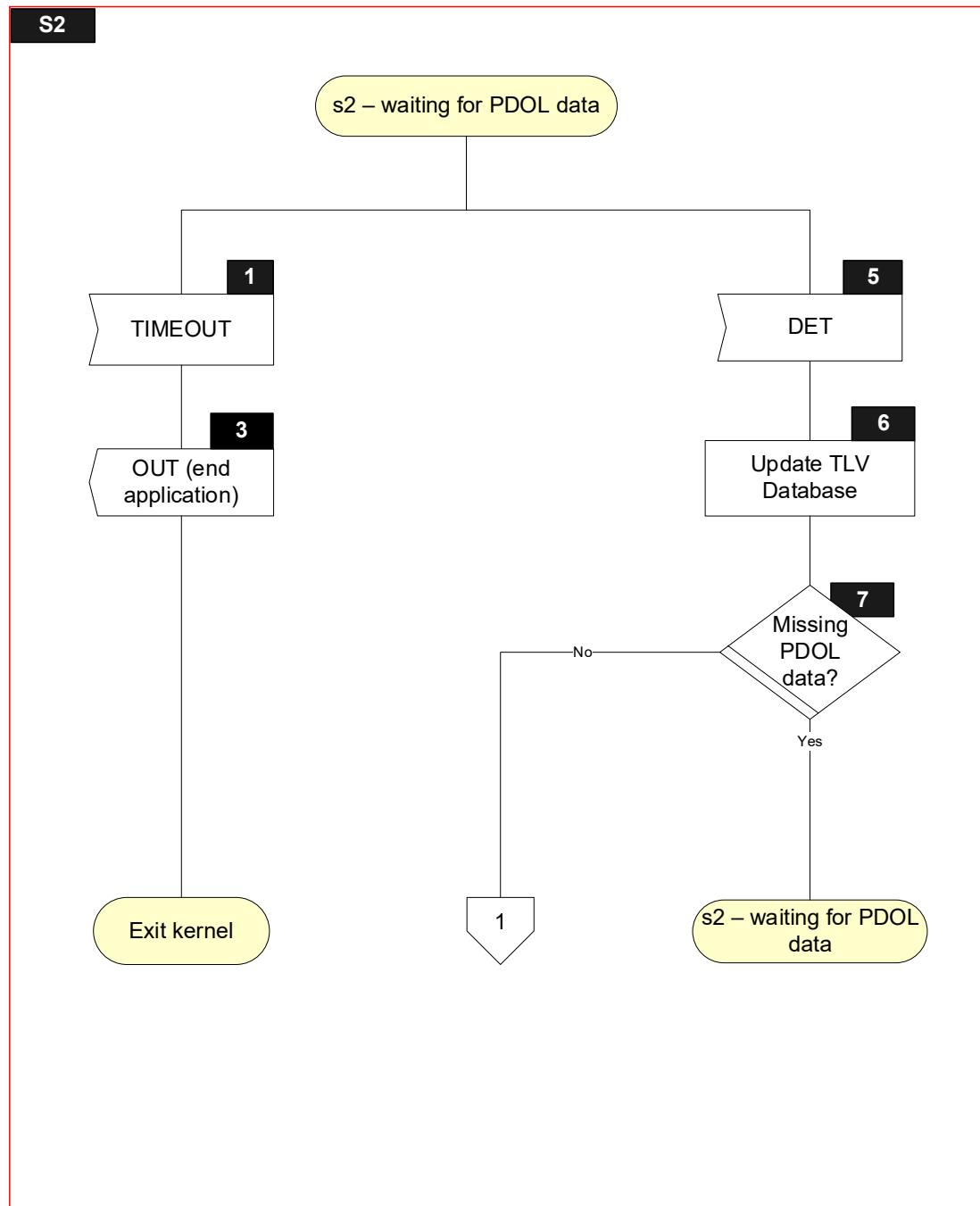
6.4.1 Local Variables

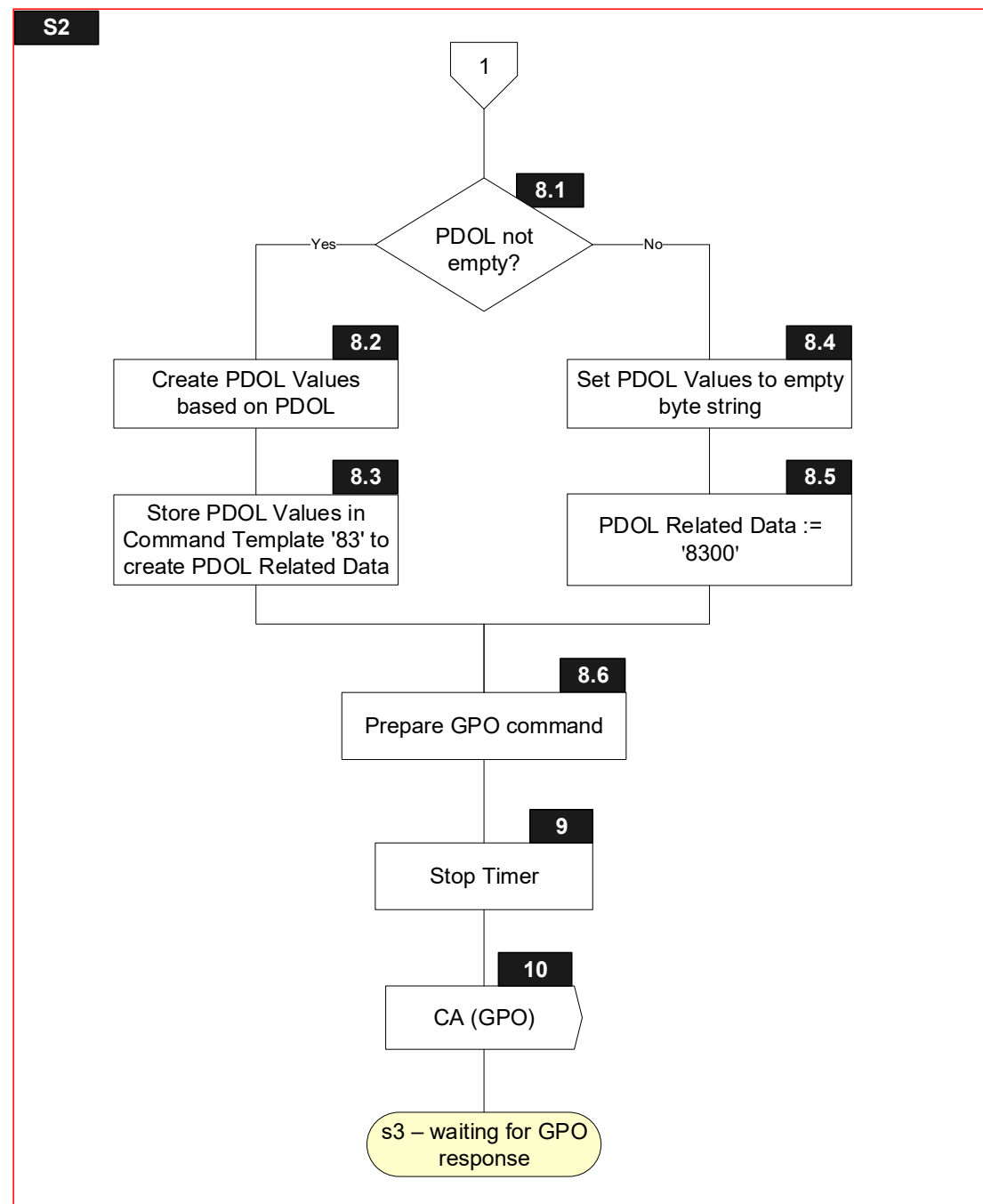
Name	Length	Format	Description
Sync Data	var.	b	List of data objects returned with DET Signal
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
Missing PDOL Data Flag	1	b	Boolean used to indicate if data referenced in PDOL is not present in the TLV Database.

6.4.2 Flow Diagram

Figure 6.3 shows the flow diagram of s2 – waiting for PDOL data. Symbols in this diagram are labelled S2.X.

Figure 6.3: State 2 Flow Diagram





6.4.3 Processing

S2.1

Receive TIMEOUT Signal

S2.3

'Status' in Outcome Parameter Set := END APPLICATION

'L3' in Error Indication := TIME OUT

Initialize(Discretionary Data)

AddToList(GetTLV(TagOf(Error Indication)), Discretionary Data)

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data))) Signal

S2.5

Receive DET Signal with Sync Data

S2.6

UpdateWithDetData(Sync Data)

S2.7

CLEAR Missing PDOL Data Flag

FOR every TL entry in PDOL

```
{
    IF    [IsEmpty(T) AND Update Conditions of T include 'DET']
    THEN
        SET Missing PDOL Data Flag
    ENDIF
}
```

```
IF    [Missing PDOL Data Flag]
THEN
    GOTO s2 - waiting for PDOL data
ELSE
    GOTO S2.8.1
ENDIF
```

S2.8.1

IF [IsEmpty(TagOf(PDOL))]

```
THEN
    GOTO S2.8.2
```

```
ELSE
    GOTO S2.8.4
```

ENDIF

S2.8.2

Use PDOL to create PDOL Values as a concatenated list of data objects without tags or lengths following the rules specified in section 4.1.4.

S2.8.3

Store PDOL Values in Command Template '83' to create PDOL Related Data.

S2.8.4

Store empty byte string in PDOL Values

S2.8.5

PDOL Related Data := '8300'

S2.8.6

Prepare GET PROCESSING OPTIONS command as specified in section 5.5.

S2.9

Stop Timer

S2.10

Send CA(GET PROCESSING OPTIONS) Signal

6.5 State 3 – Waiting for GPO Response

6.5.1 Local Variables

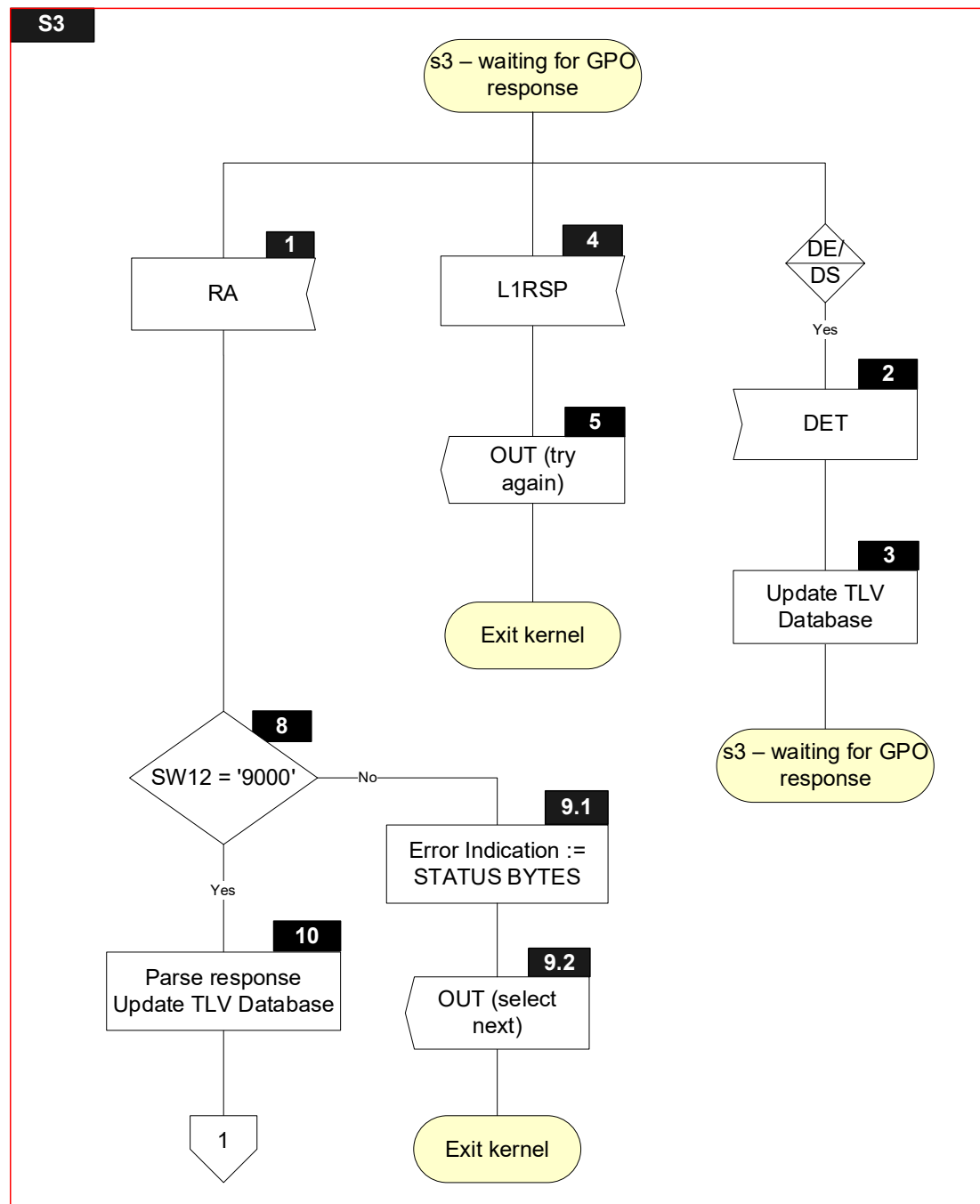
Name	Length	Format	Description
Return Code	1	b	Value returned with L1RSP Signal (TIME OUT ERROR, PROTOCOL ERROR, TRANSMISSION ERROR)
Sync Data ⁽¹⁾	var.	b	List of data objects returned with DET Signal
Parsing Result	1	b	Boolean used to store result of parsing a TLV string
SW12	2	b	Status bytes
Response Message Data Field	var. up to 256	b	TLV encoded string included in R-APDU of GET PROCESSING OPTIONS
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 253	b	Value of TLV encoded string

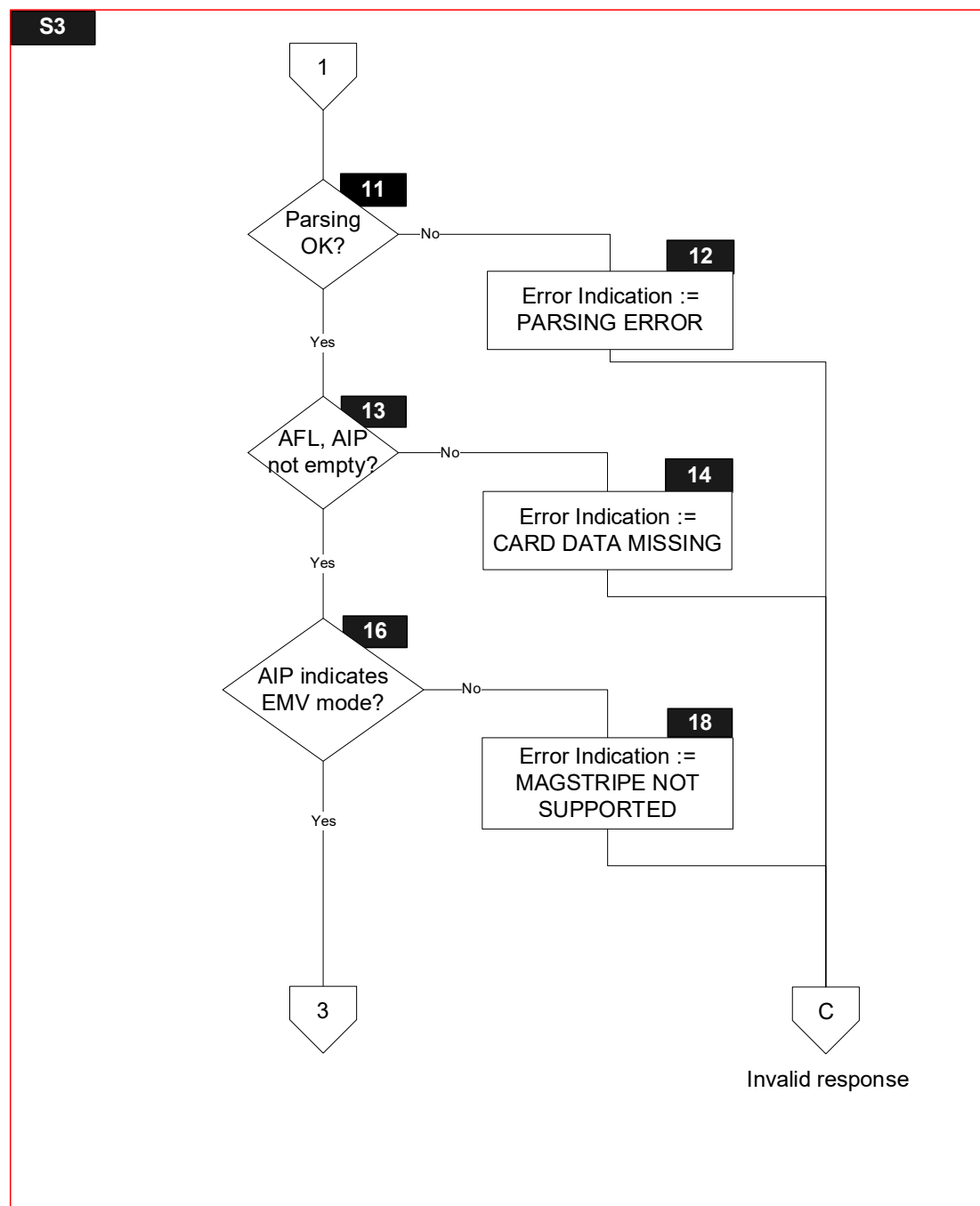
⁽¹⁾ Only for the DE/DS Implementation Option

6.5.2 Flow Diagram

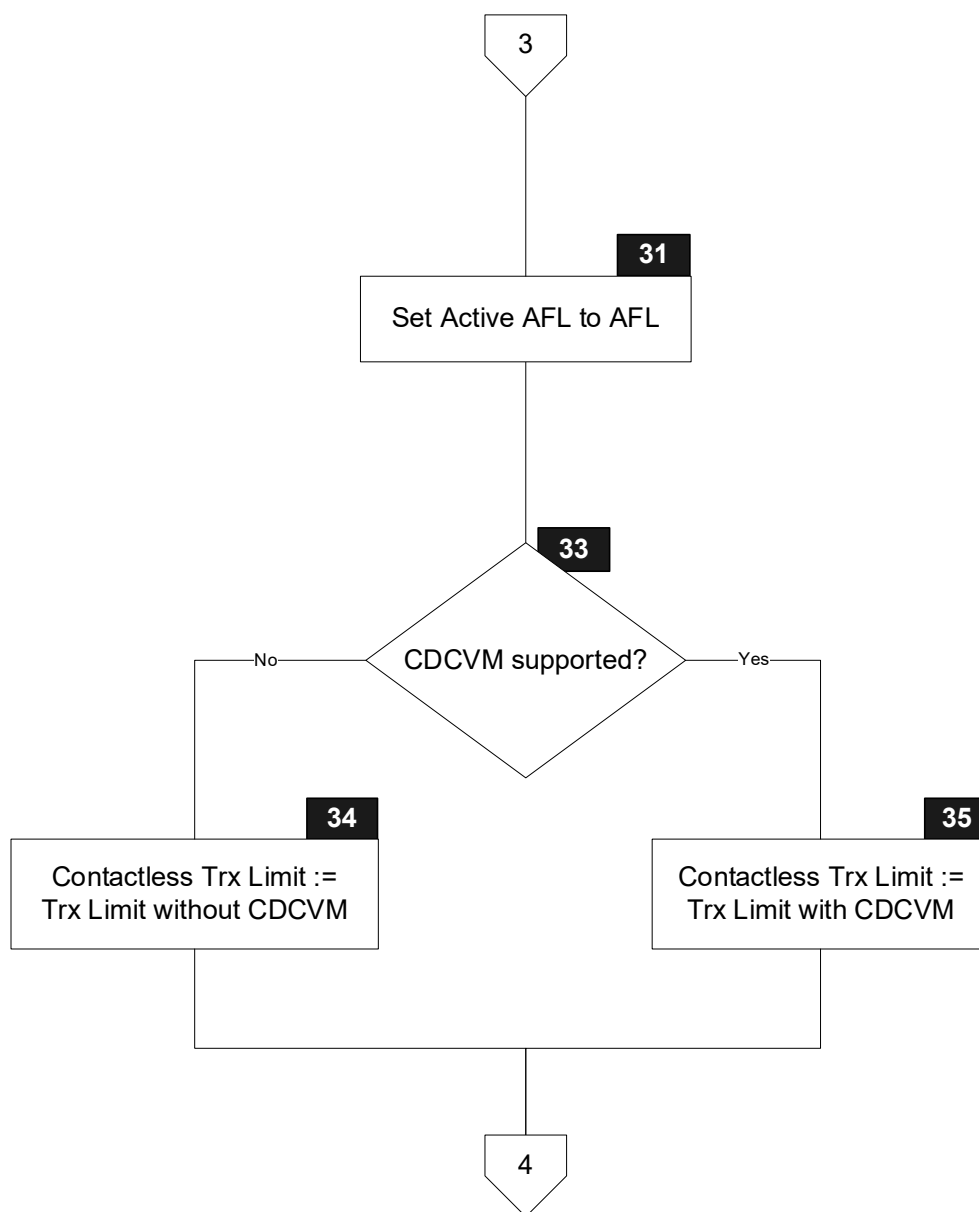
Figure 6.4 shows the flow diagram of s3 – waiting for GPO response. Symbols in this diagram are labelled S3.X.

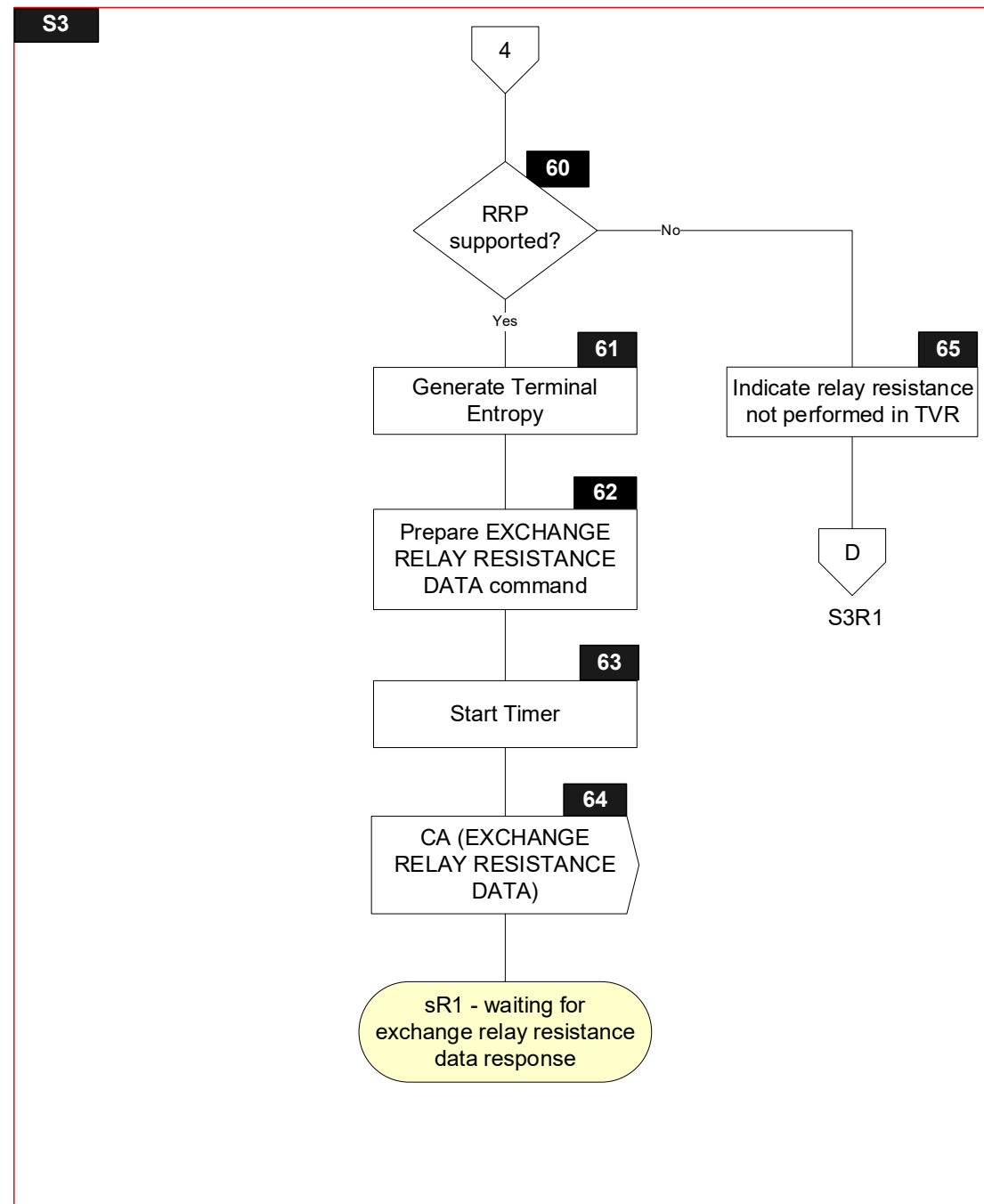
Figure 6.4: State 3 Flow Diagram



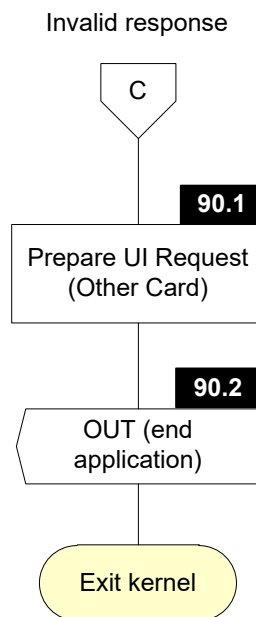


S3





S3



6.5.3 Processing

Note that symbols S3.2 and S3.3 are only implemented for the DE/DS Implementation Option.

S3.1

Receive RA Signal with Response Message Data Field and SW12

S3.2

Receive DET Signal with Sync Data

S3.3

UpdateWithDetData(Sync Data)

S3.4

Receive L1RSP Signal with Return Code

S3.5

'Status' in Outcome Parameter Set := TRY AGAIN

'Start' in Outcome Parameter Set := B

'L1' in Error Indication := Return Code

Initialize(Discretionary Data)

AddToList(GetTLV(TagOf(Error Indication)), Discretionary Data)

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data))) Signal

S3.8

IF [SW12 = '9000']

THEN

GOTO S3.10

ELSE

GOTO S3.9.1

ENDIF

S3.9.1

'L2' in Error Indication := STATUS BYTES

'SW12' in Error Indication := SW12

S3.9.2

'Field Off Request' in Outcome Parameter Set := N/A

'Status' in Outcome Parameter Set := SELECT NEXT

'Start' in Outcome Parameter Set := C

Initialize(Discretionary Data)

AddToList(GetTLV(TagOf(Error Indication)), Discretionary Data)

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data))) Signal

S3.10

Parsing Result := FALSE

IF [(Length of Response Message Data Field > 0) AND
(Response Message Data Field[1] = '77')]

THEN

Parsing Result := ParseAndStoreCardResponse(Response Message Data Field)

ELSE

IF [(Length of Response Message Data Field > 0) AND
(Response Message Data Field[1] = '80')]

THEN

Parse the Response Message Data Field according to section 5.5.3 to retrieve the value field:

IF [Response Message Data Field does not parse correctly OR
The length of the value field of the Response Message Data Field is less than 6 OR
The length of the value field of the Response Message Data Field – 2 is not a multiple of 4 OR
IsEmpty(TagOf(Application Interchange Profile)) OR
IsEmpty(TagOf(Application File Locator))]

THEN

Parsing Result := FALSE

ELSE

Store the first two bytes of the value field of Response Message Data Field in the TLV Database for tag TagOf(Application Interchange Profile).

Store from the third up to the last byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Application File Locator).

Parsing Result := TRUE

ENDIF

ENDIF

ENDIF

S3.11

IF [Parsing Result]

THEN

GOTO S3.13

ELSE

GOTO S3.12

ENDIF

S3.12

'L2' in Error Indication := PARSING ERROR

S3.13

```
IF      [IsEmpty(TagOf(Application File Locator)) AND  
        IsNotEmpty(TagOf(Application Interchange Profile))]  
THEN  
    GOTO S3.16  
ELSE  
    GOTO S3.14  
ENDIF
```

S3.14

'L2' in Error Indication := CARD DATA MISSING

S3.16

```
IF      ['EMV mode is supported' in Application Interchange Profile is set]  
THEN  
    GOTO S3.31  
ELSE  
    GOTO S3.18  
ENDIF
```

S3.18

'L2' in Error Indication := MAGSTRIPE NOT SUPPORTED

S3.31

Active AFL := Application File Locator

S3.33

```
IF      ['CDCVM is supported' in Application Interchange Profile is set AND  
        'CDCVM supported' in Kernel Configuration is set]  
THEN  
    GOTO S3.35  
ELSE  
    GOTO S3.34  
ENDIF
```

S3.34

Reader Contactless Transaction Limit := Reader Contactless Transaction Limit (No CDCVM)

S3.35

Reader Contactless Transaction Limit := Reader Contactless Transaction Limit (CDCVM)

S3.60

```
IF      ['Relay resistance protocol supported' in Kernel Configuration is set AND  
        'Relay resistance protocol is supported' in Application Interchange Profile is set]  
THEN  
    GOTO S3.61  
ELSE  
    GOTO S3.65  
ENDIF
```

S3.61

Generate Unpredictable Number as specified in section 8.1 and store in the TLV Database for TagOf(Unpredictable Number)
Terminal Relay Resistance Entropy := Unpredictable Number

S3.62

Prepare EXCHANGE RELAY RESISTANCE DATA command as specified in section 5.2.

S3.63

A timer is started to measure the time taken by the EXCHANGE RELAY RESISTANCE DATA command

Start Timer ()

This step can be bypassed if the time measurement of the EXCHANGE RELAY RESISTANCE DATA command is performed by the PCD of the terminal or if the start time is provided by the PCD.

S3.64

Send CA(EXCHANGE RELAY RESISTANCE DATA) Signal

S3.65

'Relay resistance performed' in Terminal Verification Results := RRP NOT PERFORMED

S3.90.1

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

S3.90.2

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

Initialize(Discretionary Data)

AddToList(GetTLV(TagOf(Error Indication)), Discretionary Data)

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
 Data))) Signal

6.6 State R1 – Waiting for Exchange Relay Resistance Data Response

6.6.1 Local Variables

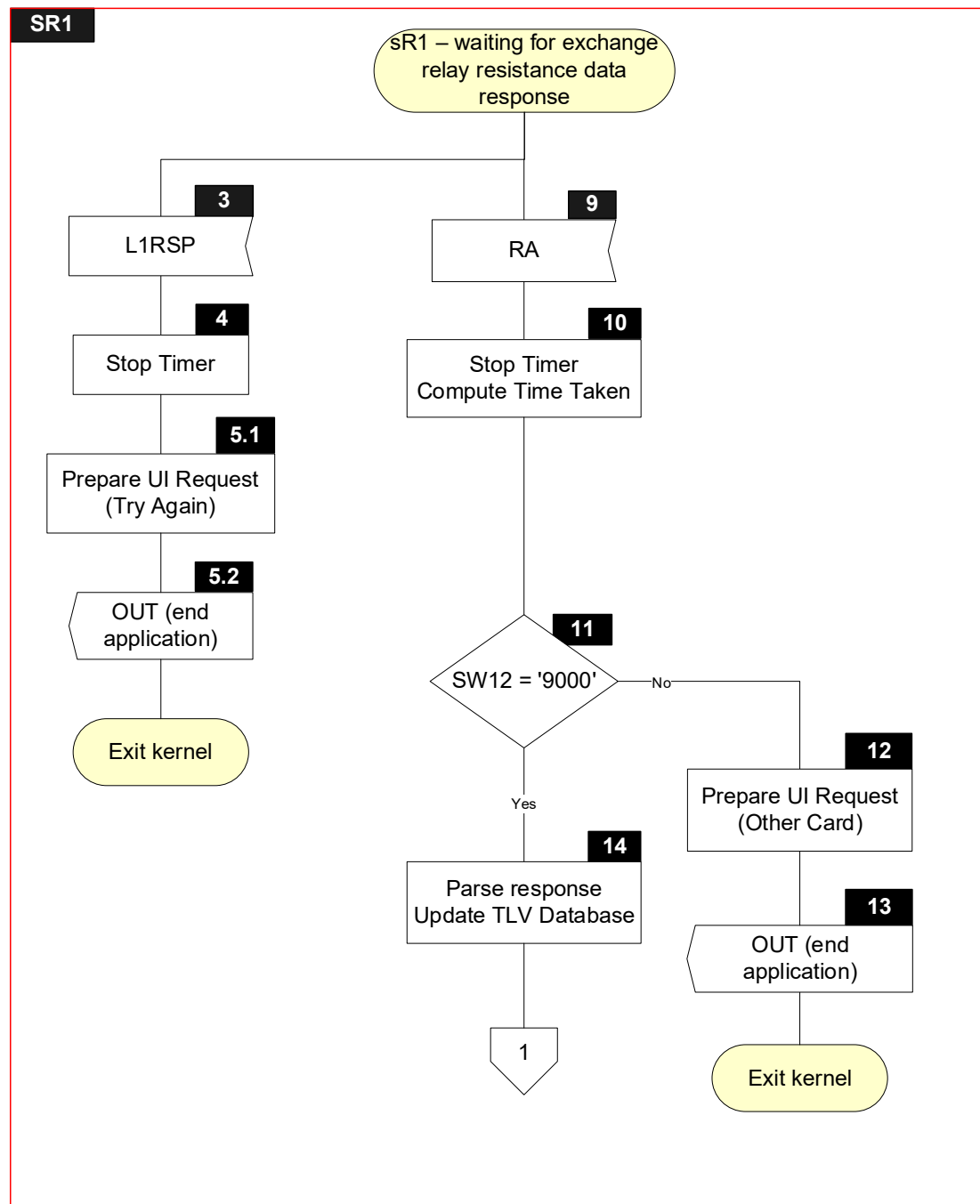
Name	Length	Format	Description
Return Code	1	b	Value returned with L1RSP Signal (TIME OUT ERROR, PROTOCOL ERROR, TRANSMISSION ERROR)
Sync Data ⁽¹⁾	var.	b	List of data objects returned with DET Signal
Parsing Result	1	b	Boolean used to store result of parsing a TLV string
SW12	2	b	Status bytes
Response Message Data Field	var. up to 256	b	TLV encoded string included in R-APDU of EXCHANGE RELAY RESISTANCE DATA
Time Taken	4	b	Time of processing the EXCHANGE RELAY RESISTANCE DATA command measured by Timer. Time Taken is expressed in microseconds.
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 253	b	Value of TLV encoded string

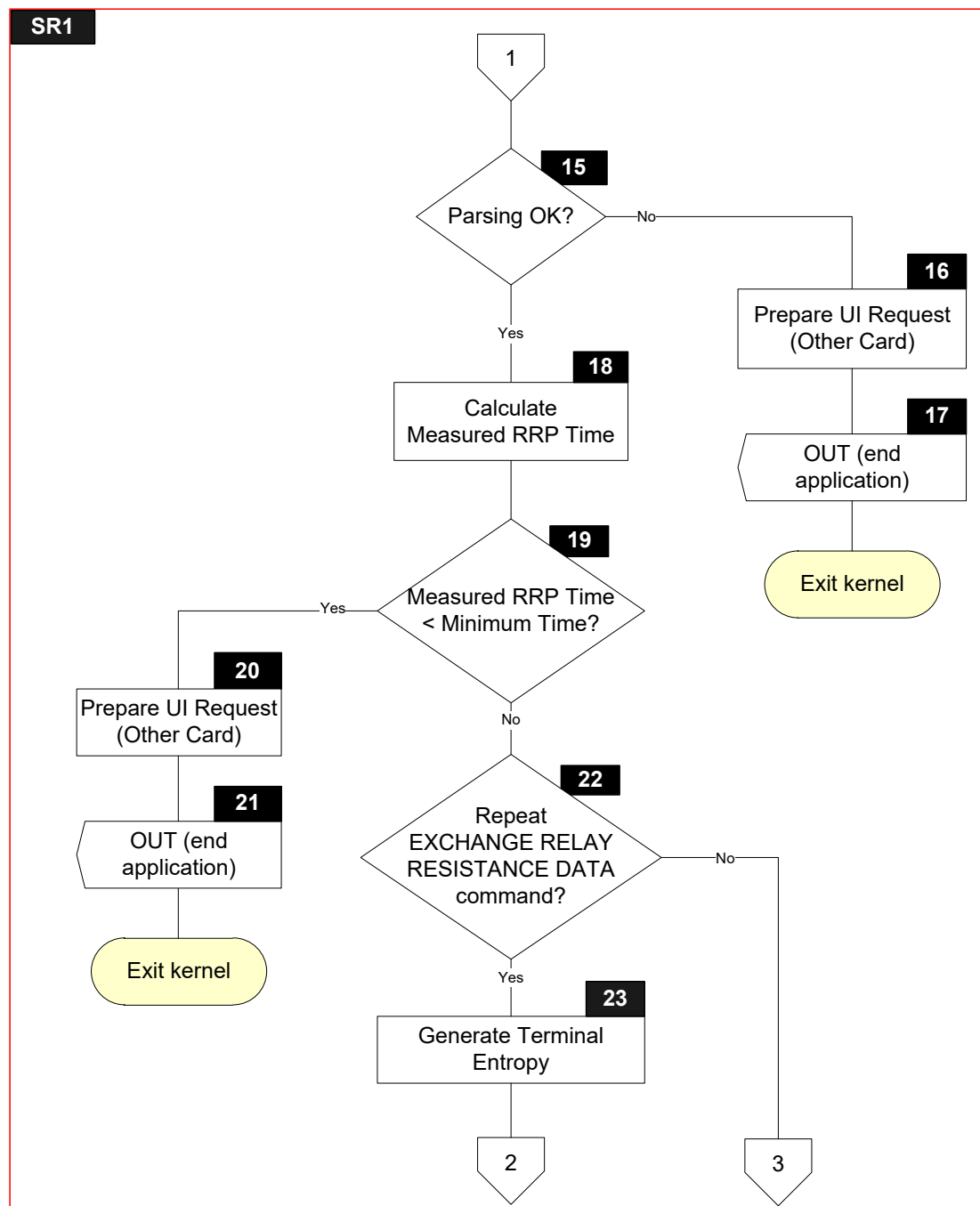
⁽¹⁾ Only for the DE/DS Implementation Option

6.6.2 Flow Diagram

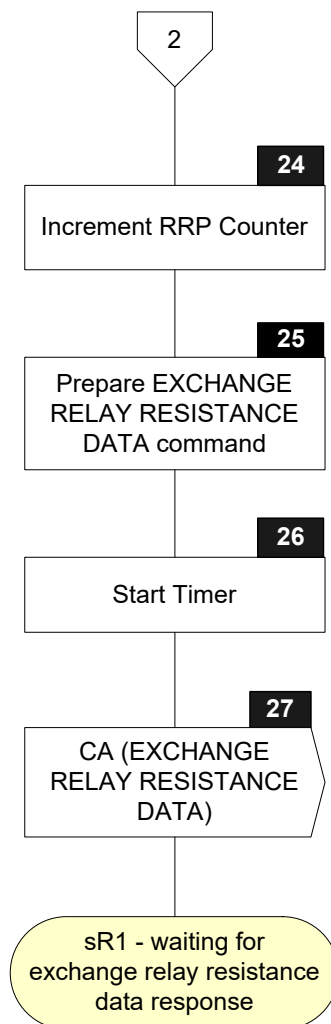
Figure 6.5 shows the flow diagram of SR1 – waiting for exchange relay resistance data response. Symbols in this diagram are labelled SR1.X.

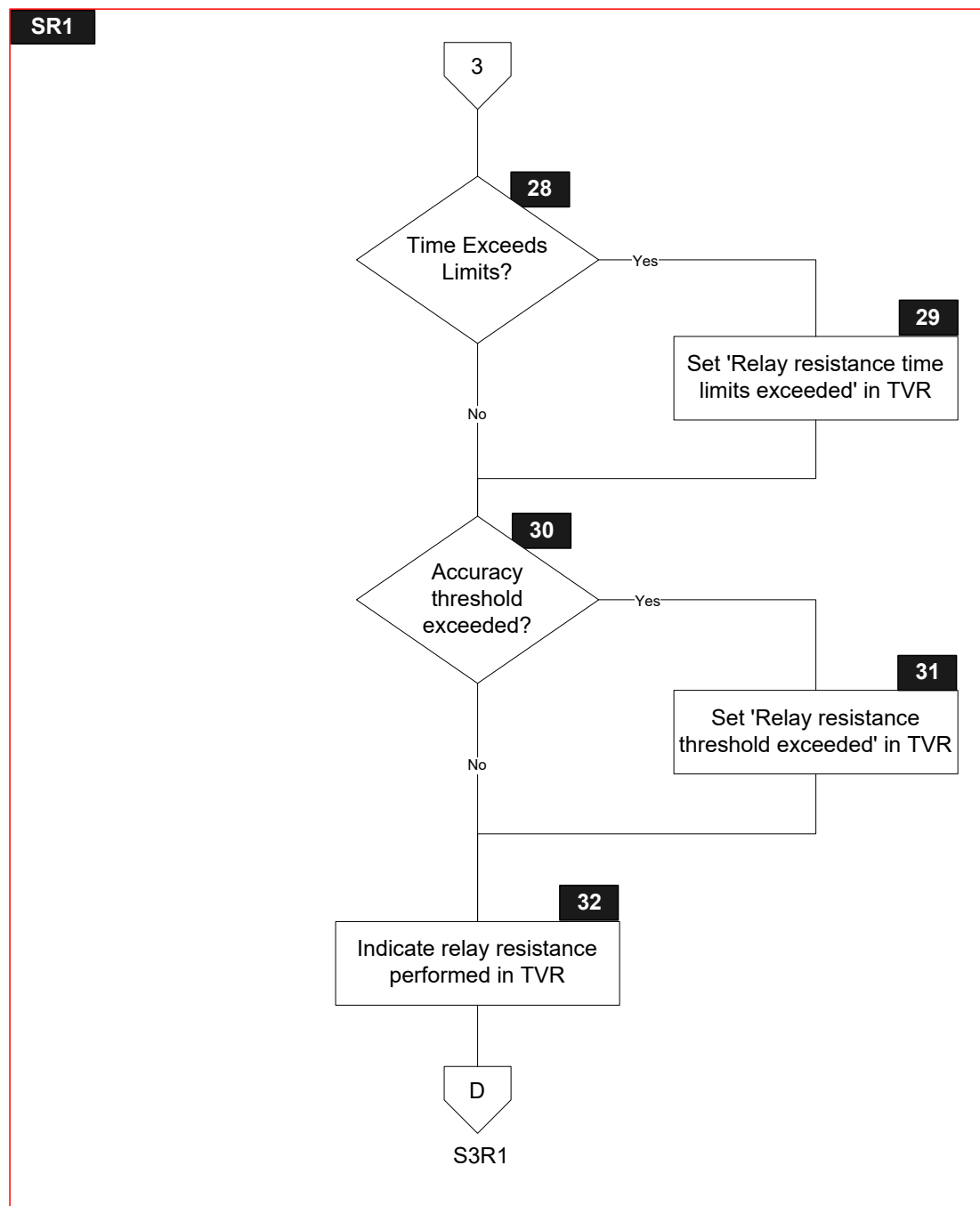
Figure 6.5: State R1 Flow Diagram





SR1





6.6.3 Processing

SR1.3

Receive L1RSP Signal with Return Code

SR1.4

Stop Timer

SR1.5.1

'Message Identifier' in User Interface Request Data := TRY AGAIN

'Status' in User Interface Request Data := READY TO READ

'Hold Time' in User Interface Request Data := '000000'

SR1.5.2

'Status' in Outcome Parameter Set := END APPLICATION

'Start' in Outcome Parameter Set := B

SET 'UI Request on Restart Present' in Outcome Parameter Set

'L1' in Error Indication := Return Code

'Msg On Error' in Error Indication := TRY AGAIN

CreateDiscretionaryData ()

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

SR1.9

Receive RA Signal with Response Message Data Field and SW12

SR1.10

Stop Timer

Compute Time Taken from the start and stop times.

Alternatively, the stop time or the Time Taken may be provided directly by the PCD of the terminal.

The method by which these data could be exchanged between the PCD and the terminal is implementation specific.

SR1.11

IF [SW12 = '9000']

THEN

GOTO SR1.14

ELSE

GOTO SR1.12

ENDIF

SR1.12

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

SR1.13

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

'L2' in Error Indication := STATUS BYTES

'SW12' in Error Indication := SW12

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

SR1.14

Parsing Result := FALSE

IF [(Length of Response Message Data Field > 11) AND
(Response Message Data Field[1] = '80') AND
Length of the value field of the Response Message Data Field = 10]

THEN

Store the first four bytes of the value field of Response Message Data Field in the TLV Database for tag TagOf(Device Relay Resistance Entropy).

Store from the fifth up to the sixth byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Min Time For Processing Relay Resistance APDU).

Store from the seventh up to the eighth byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Max Time For Processing Relay Resistance APDU).

Store from the ninth up to the tenth byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Device Estimated Transmission Time For Relay Resistance R-APDU).

Parsing Result := TRUE

ENDIF

SR1.15

IF [Parsing Result]

THEN

GOTO SR1.18

ELSE

GOTO SR1.16

ENDIF

SR1.16

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

SR1.17

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

'L2' in Error Indication := PARSING ERROR

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

SR1.18

Measured Relay Resistance Processing Time := MAX (0, (Time Taken div 100) – Terminal
Expected Transmission Time For Relay Resistance C-APDU – MIN (Device Estimated
Transmission Time For Relay Resistance R-APDU , Terminal Expected Transmission Time
For Relay Resistance R-APDU))

Note that the implementation should compensate for any known fixed timing latency. All implementations will have some inevitable delay between starting the timer and sending the C-APDU and between receiving the R-APDU and stopping the timer. If this latency is predictable and can be compensated for by the implementation then it does not need to be compensated by increasing the maximum grace period.

SR1.19

IF [Measured Relay Resistance Processing Time < MAX (0, (Min Time For Processing
Relay Resistance APDU – Minimum Relay Resistance Grace Period))]

THEN

GOTO SR1.20

ELSE

GOTO SR1.22

ENDIF

SR1.20

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

SR1.21

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

'L2' in Error Indication := CARD DATA ERROR

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

SR1.22

IF [(RRP Counter < 2) AND
Measured Relay Resistance Processing Time > Max Time For Processing Relay
Resistance APDU + Maximum Relay Resistance Grace Period]
THEN
GOTO SR1.23
ELSE
GOTO SR1.28
ENDIF

SR1.23

Generate Unpredictable Number as specified in section 8.1 and store in the TLV Database for
TagOf(Unpredictable Number)
Terminal Relay Resistance Entropy := Unpredictable Number

SR1.24

RRP Counter := RRP Counter + 1

SR1.25

Prepare EXCHANGE RELAY RESISTANCE DATA command as specified in section 5.2.

SR1.26

A timer is started to measure the time taken by the EXCHANGE RELAY RESISTANCE
DATA command

Start Timer ()

This step can be bypassed if the time measurement of the EXCHANGE RELAY
RESISTANCE DATA command is performed by the PCD of the terminal or if the start time
is provided by the PCD.

SR1.27

Send CA(EXCHANGE RELAY RESISTANCE DATA) Signal

SR1.28

IF [Measured Relay Resistance Processing Time > (Max Time For Processing Relay
Resistance APDU + Maximum Relay Resistance Grace Period)]
THEN
GOTO SR1.29
ELSE
GOTO SR1.30
ENDIF

SR1.29

SET 'Relay resistance time limits exceeded' in Terminal Verification Results

SR1.30

```
IF      [(Device Estimated Transmission Time For Relay Resistance R-APDU ≠ 0)
AND
(Terminal Expected Transmission Time For Relay Resistance R-APDU ≠ 0)]
THEN
    IF      [((( (Device Estimated Transmission Time For Relay Resistance R-APDU *
100) div Terminal Expected Transmission Time For Relay Resistance R-
APDU) < Relay Resistance Transmission Time Mismatch Threshold)
OR
(( (Terminal Expected Transmission Time For Relay Resistance R-APDU *
100) div Device Estimated Transmission Time For Relay Resistance R-
APDU) < Relay Resistance Transmission Time Mismatch Threshold)
OR
(MAX (0, (Measured Relay Resistance Processing Time – Min Time For
Processing Relay Resistance APDU) ) > Relay Resistance Accuracy
Threshold)]
    THEN
        GOTO SR1.31
    ELSE
        GOTO SR1.32
    ENDIF
ELSE
    GOTO SR1.31
ENDIF
```

SR1.31

SET 'Relay resistance threshold exceeded' in Terminal Verification Results

SR1.32

'Relay resistance performed' in Terminal Verification Results := RRP PERFORMED

6.7 States 3, R1 – Common Processing

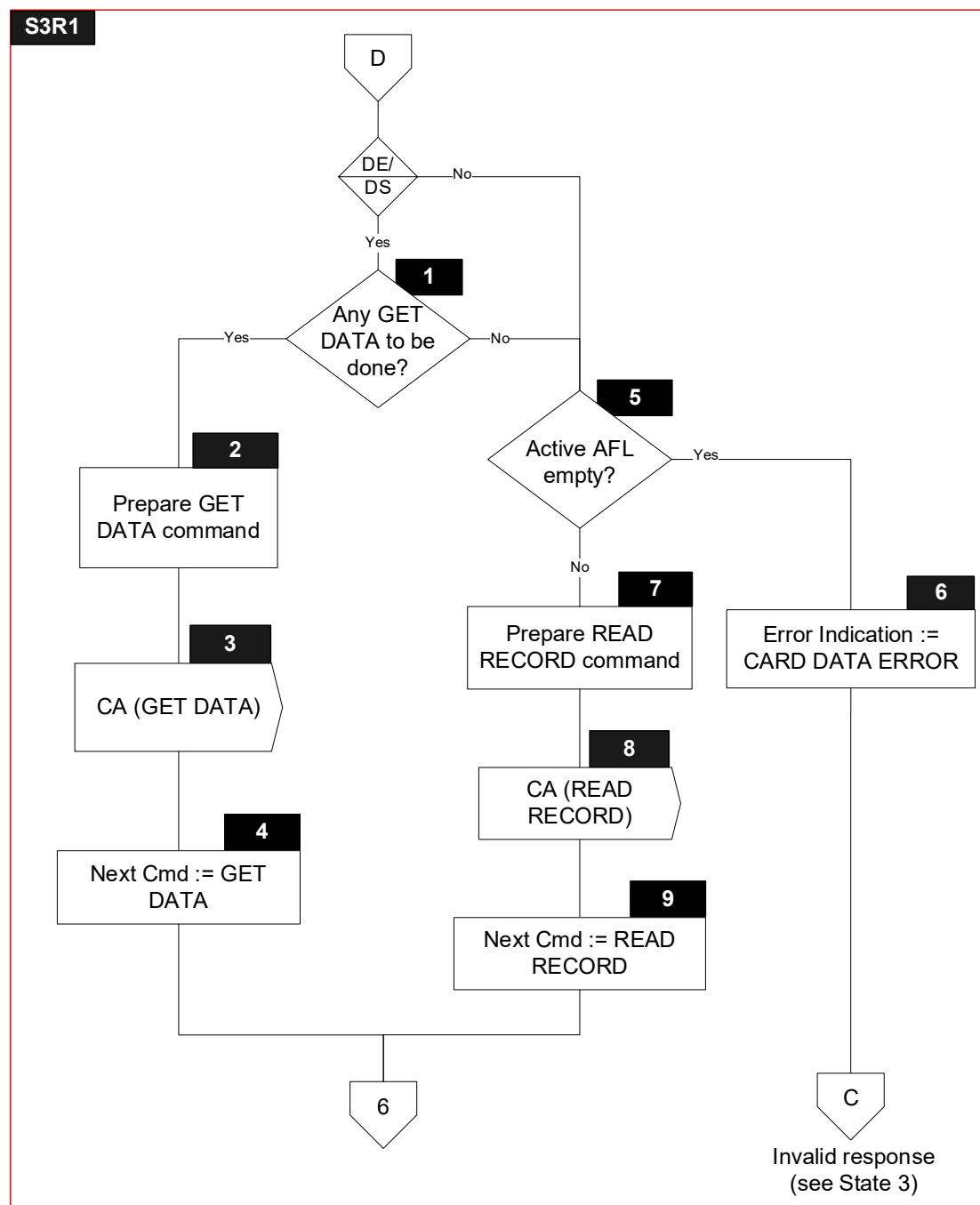
6.7.1 Local Variables

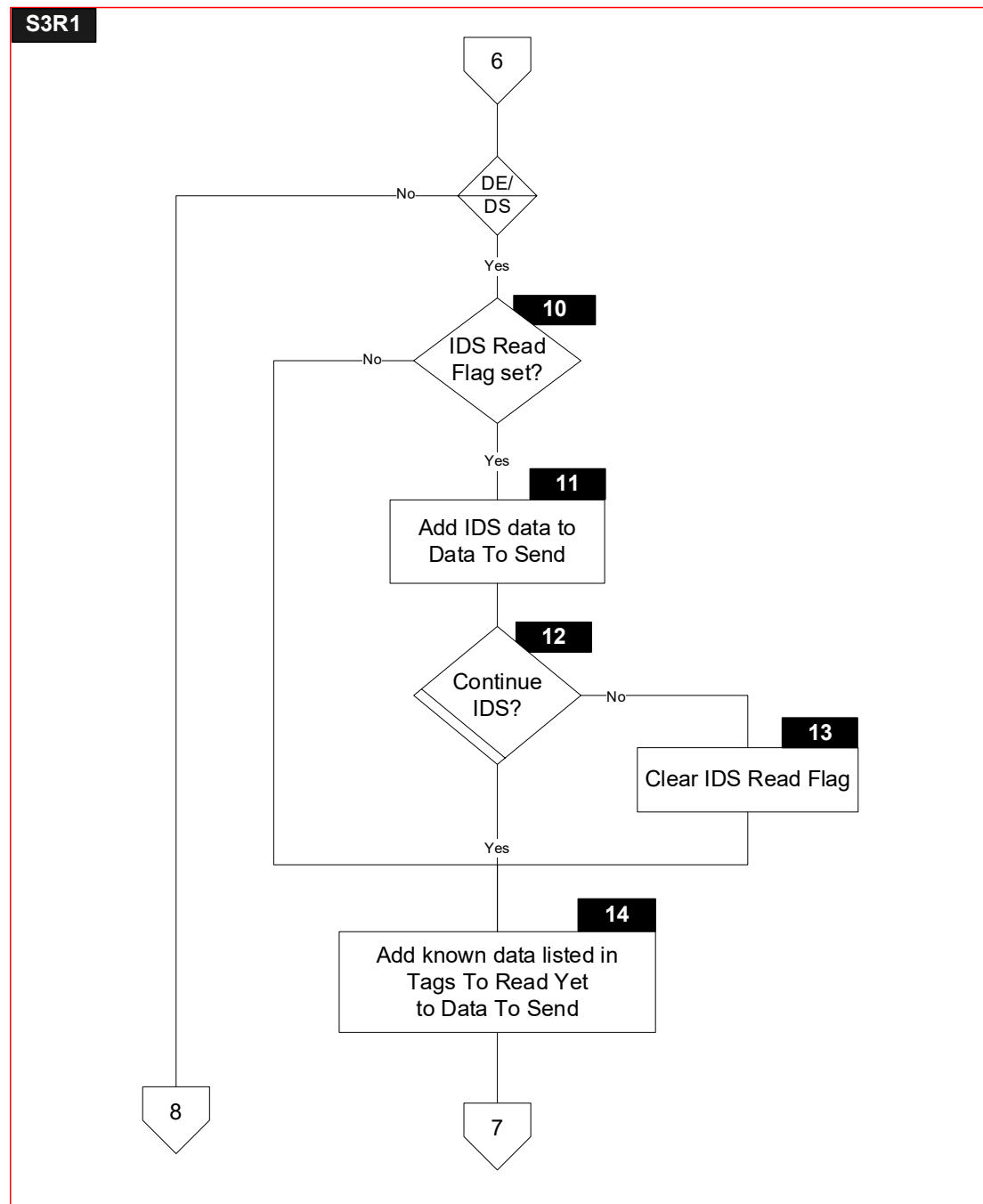
Local variables for common processing are defined in states 3 and R1.

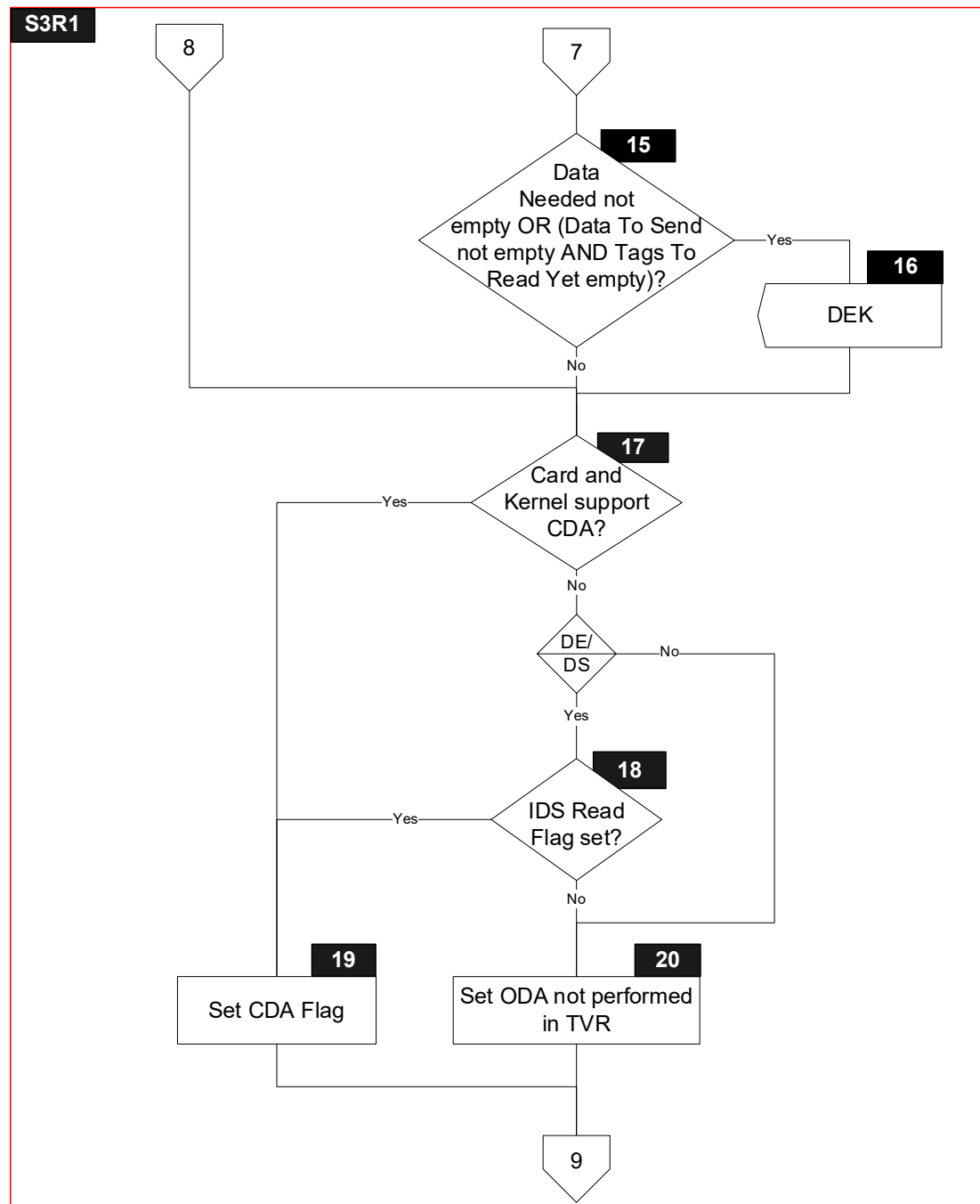
6.7.2 Flow Diagram

Figure 6.6 shows the flow diagram for common processing between states 3 and R1. Symbols in this diagram are labelled S3R1.X.

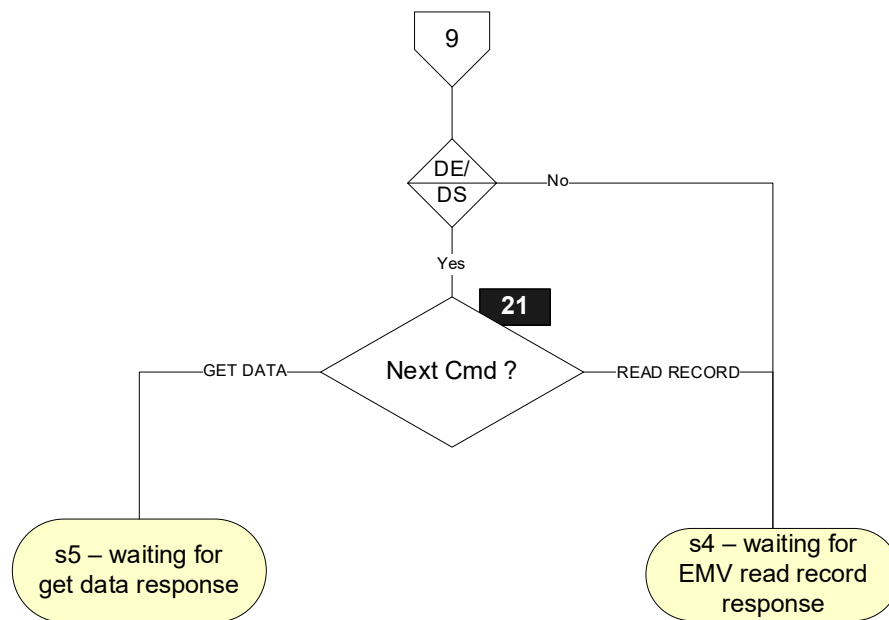
Figure 6.6: States 3 and R1 – Common Processing – Flow Diagram







S3R1



6.7.3 Processing

Note that symbols S3R1.1, S3R1.2, S3R1.3, S3R1.4, S3R1.10, S3R1.11, S3R1.12, S3R1.13, S3R1.14, S3R1.15, S3R1.16, S3R1.18 and S3R1.21 are only implemented for the DE/DS Implementation Option.

S3R1.1

Active Tag := GetNextGetDataTagFromList(Tags To Read Yet)

IF [Active Tag = NULL]

THEN

GOTO S3R1.5

ELSE

GOTO S3R1.2

ENDIF

S3R1.2

Build GET DATA command for Active Tag as defined in section 5.4

S3R1.3

Send CA(GET DATA) Signal

S3R1.4

'Next Cmd' in Next Cmd := GET DATA

S3R1.5

IF [Active AFL is empty]

THEN

GOTO S3R1.6

ELSE

GOTO S3R1.7

ENDIF

S3R1.6

'L2' in Error Indication := CARD DATA ERROR

S3R1.7

Build READ RECORD command for the first record indicated by Active AFL as defined in section 5.7

S3R1.8

Send CA(READ RECORD) Signal

S3R1.9

'Next Cmd' in Next Cmd := READ RECORD

S3R1.10

```
IF      ['Read' in IDS Status is set]
THEN
    GOTO S3R1.11
ELSE
    GOTO S3R1.14
ENDIF
```

S3R1.11

```
IF      [IsEmpty(TagOf(DS Slot Availability))]
THEN
    AddToList(GetTLV(TagOf(DS Slot Availability)), Data To Send)
ENDIF
IF      [IsEmpty(TagOf(DS Summary 1))]
THEN
    AddToList(GetTLV(TagOf(DS Summary 1)), Data To Send)
ENDIF
IF      [IsEmpty(TagOf(DS Unpredictable Number))]
THEN
    AddToList(GetTLV(TagOf(DS Unpredictable Number)), Data To Send)
ENDIF
IF      [IsEmpty(TagOf(DS Slot Management Control))]
THEN
    AddToList(GetTLV(TagOf(DS Slot Management Control)), Data To Send)
ENDIF
IF      [IsPresent(TagOf(DS ODS Card))]
THEN
    AddToList(GetTLV(TagOf(DS ODS Card)), Data To Send)
ENDIF
AddToList(GetTLV(TagOf(Unpredictable Number)), Data To Send)
```

S3R1.12

Continue with IDS when:

- DS Requested Operator ID is not known by the Card, but all the necessary data objects are returned by the Card to perform an IDS write, or
- DS Requested Operator ID is known by the Card

This is done as follows:

```
IF      [(IsNotEmpty(TagOf(DS Slot Availability)) AND
        IsNotEmpty(TagOf(DS Summary 1)) AND
        IsNotEmpty(TagOf(DS Unpredictable Number)) AND
        IsNotPresent(TagOf (DS ODS Card)))
        OR
        (IsNotEmpty(TagOf(DS Summary 1)) AND
        IsPresent(TagOf (DS ODS Card))) ]
THEN
    GOTO S3R1.14
ELSE
    GOTO S3R1.13
ENDIF
```

S3R1.13

CLEAR 'Read' in IDS Status

S3R1.14

FOR every entry T in Tags To Read Yet

```
{
    IF      [IsNotEmpty(T)]
    THEN
        AddToList(GetTLV(T), Data To Send)
        RemoveFromList(T, Tags To Read Yet)
    ENDIF
}
```

S3R1.15

```
IF      [IsNotEmptyList(Data Needed) OR
        (IsNotEmptyList(Data To Send) AND IsEmptyList(Tags To Read Yet))]
THEN
    GOTO S3R1.16
ELSE
    GOTO S3R1.17
ENDIF
```

S3R1.16

Send DEK(Data To Send, Data Needed) Signal

Initialize(Data To Send)

Initialize(Data Needed)

S3R1.17

IF ['CDA supported' in Application Interchange Profile is set
AND 'CDA' in Terminal Capabilities is set]

THEN

GOTO S3R1.19

ELSE

GOTO S3R1.18

ENDIF

S3R1.18

IF ['Read' in IDS Status is set]

THEN

GOTO S3R1.19

ELSE

GOTO S3R1.20

ENDIF

S3R1.19

SET 'CDA' in ODA Status

S3R1.20

SET 'Offline data authentication was not performed' in Terminal Verification Results

S3R1.21

IF ['Next Cmd' in Next Cmd = READ RECORD]

THEN

GOTO s4 - waiting for read record response

ELSE

GOTO s5 - waiting for get data response

ENDIF

6.8 State 4 – Waiting for Read Record Response

6.8.1 Local Variables

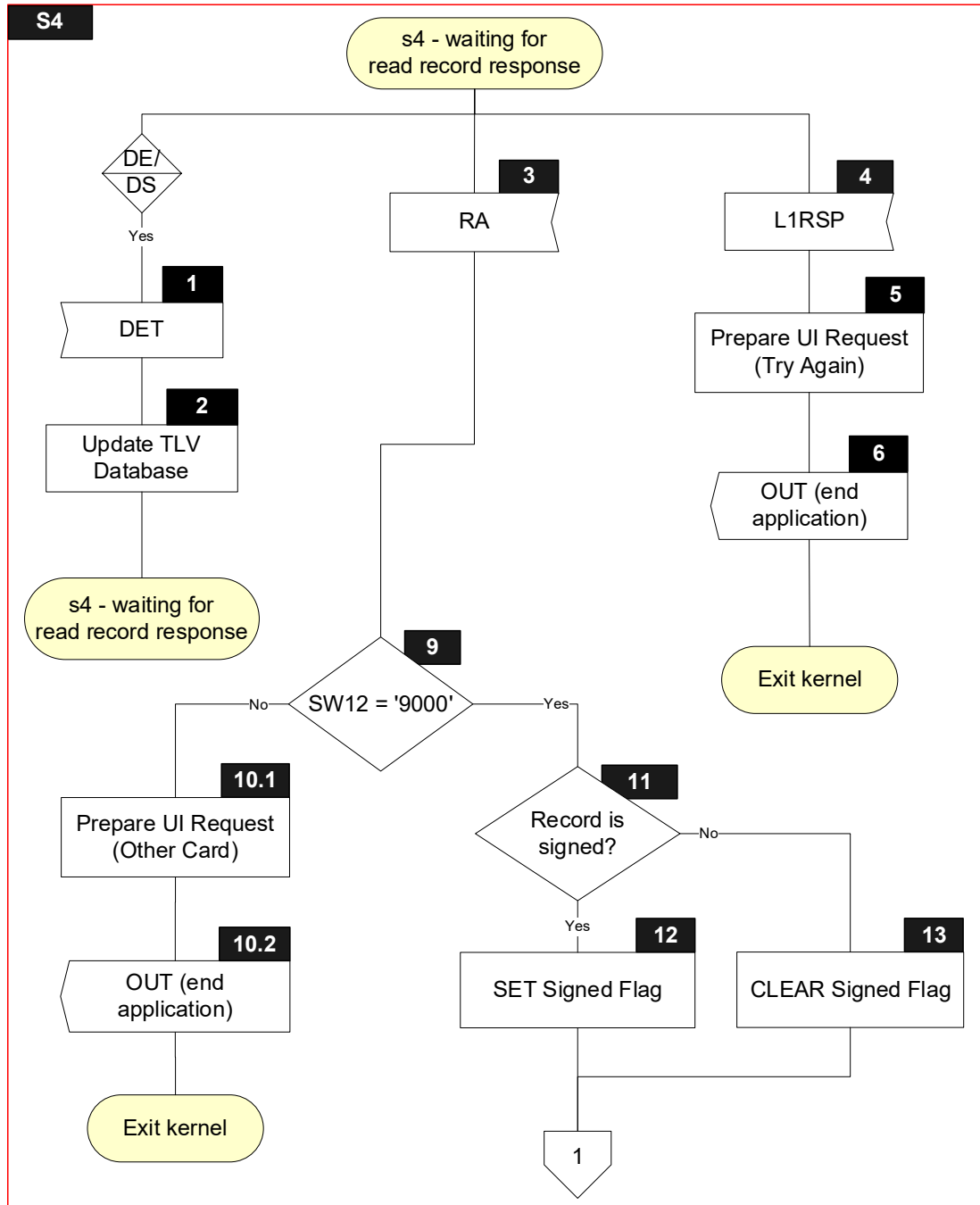
Name	Length	Format	Description
Return Code	1	b	Value returned with L1RSP Signal (TIME OUT ERROR, PROTOCOL ERROR, TRANSMISSION ERROR)
Sync Data ⁽¹⁾	var.	b	List of data objects returned with DET Signal
Parsing Result	1	b	Boolean used to store result of parsing a TLV string
SW12	2	b	Status bytes
Record	var. up to 254	b	Response Message Data Field of the R-APDU of READ RECORD
Signed Flag	1	b	Boolean used to indicate if current record is signed
Sfi	1	b	SFI of current record
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 253	b	Value of TLV encoded string

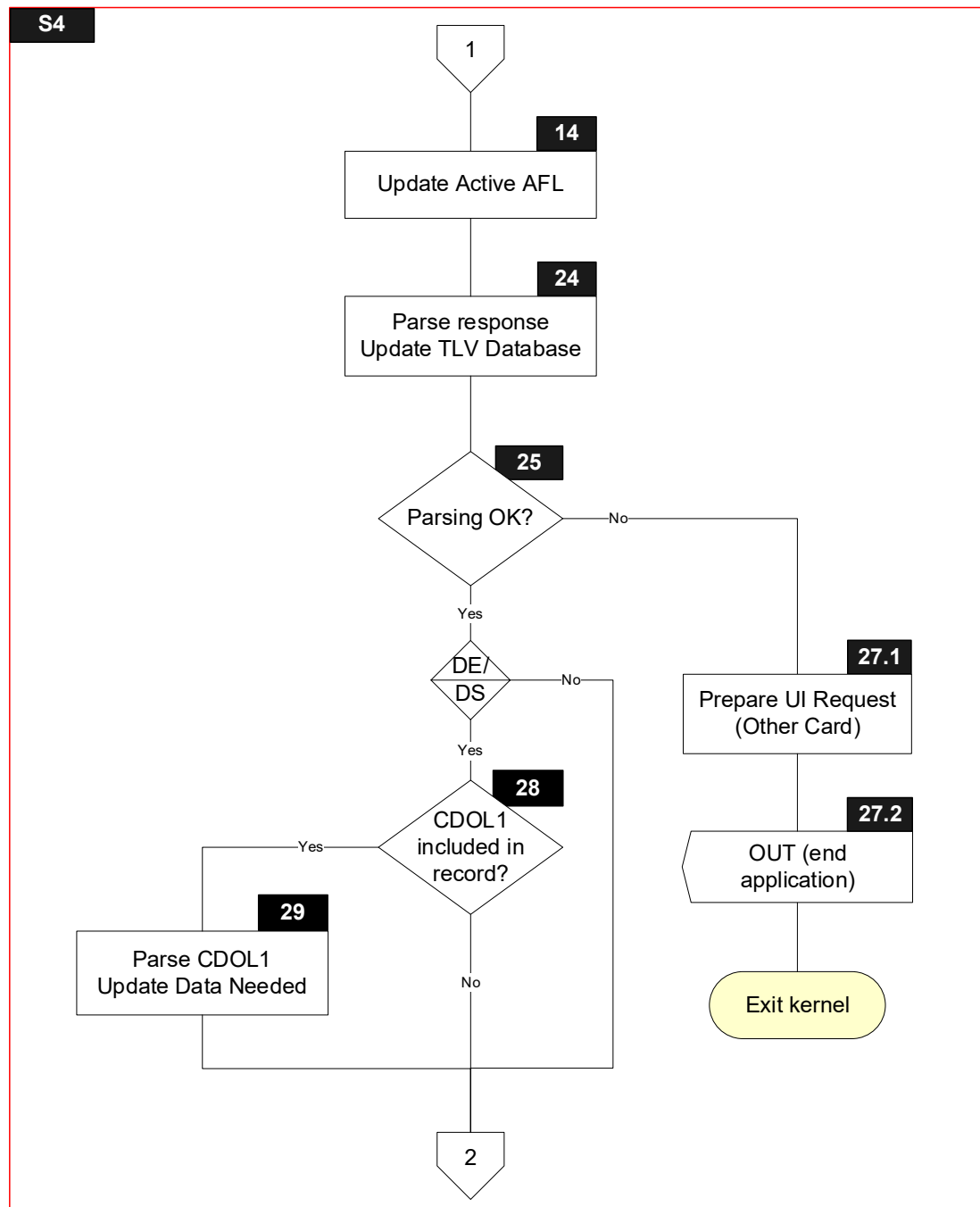
⁽¹⁾ Only for the DE/DS Implementation Option

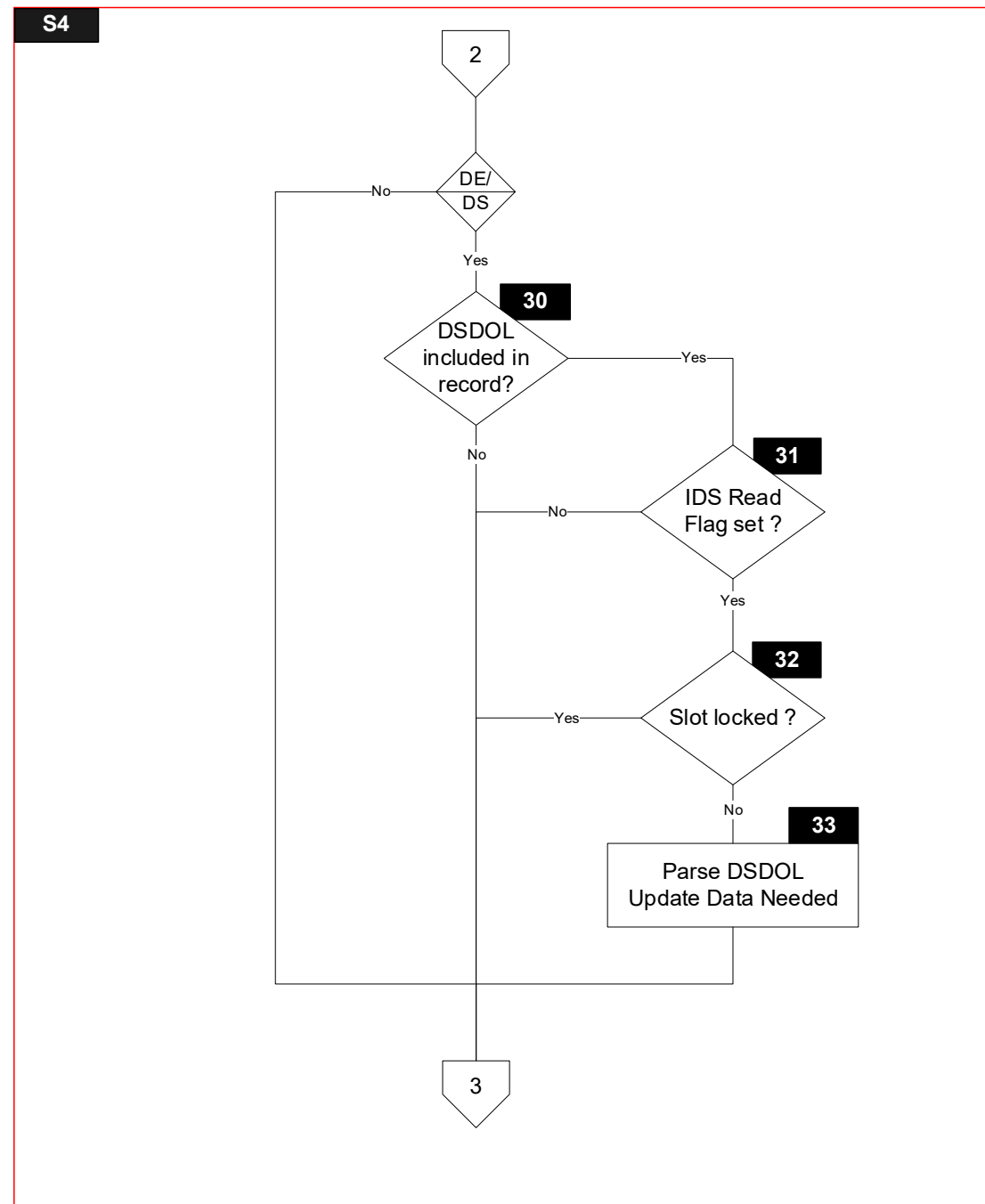
6.8.2 Flow Diagram

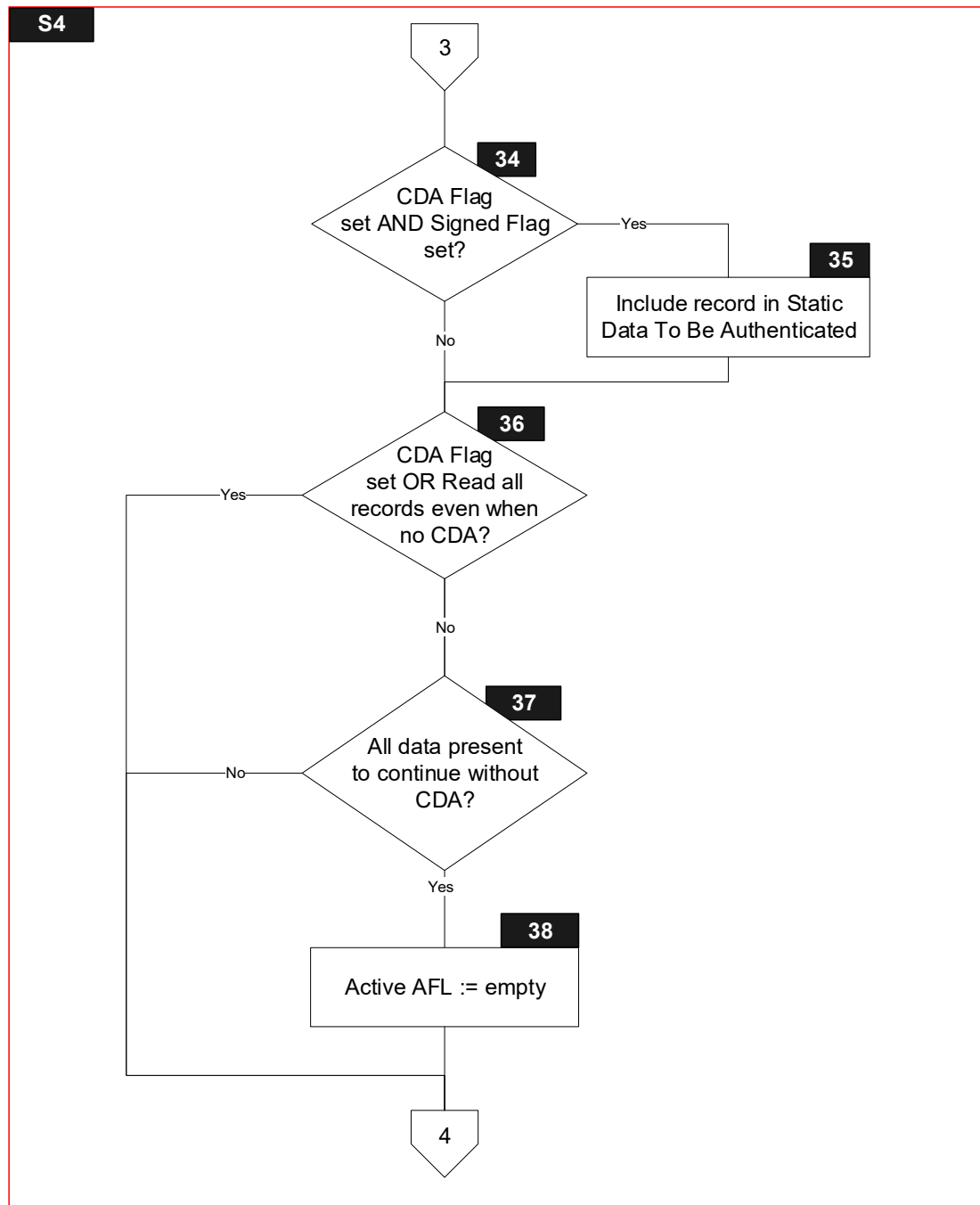
Figure 6.7 shows the flow diagram of s4 – waiting for read record response. Symbols in this diagram are labelled S4.X.

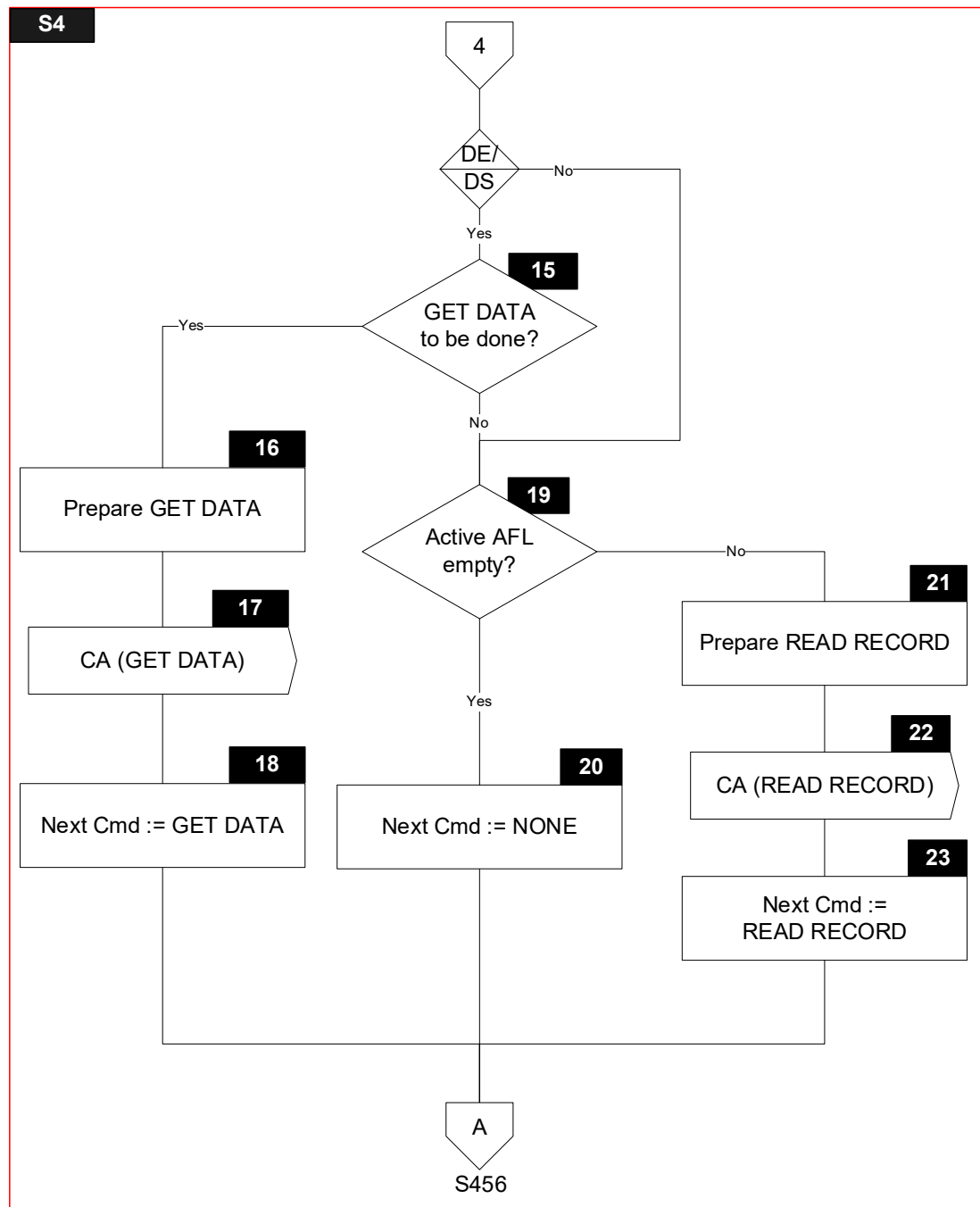
Figure 6.7: State 4 Flow Diagram











6.8.3 Processing

Note that symbols S4.1, S4.2, S4.15, S4.16, S4.17, S4.18, S4.28, S4.29, S4.30, S4.31, S4.32 and S4.33 are only implemented for the DE/DS Implementation Option.

S4.1

Receive DET Signal with Sync Data

S4.2

UpdateWithDetData(Sync Data)

S4.3

Receive RA Signal with Record and SW12

S4.4

Receive L1RSP Signal with Return Code

S4.5

'Message Identifier' in User Interface Request Data := TRY AGAIN

'Status' in User Interface Request Data := READY TO READ

'Hold Time' in User Interface Request Data := '000000'

S4.6

'Status' in Outcome Parameter Set := END APPLICATION

'Start' in Outcome Parameter Set := B

SET 'UI Request on Restart Present' in Outcome Parameter Set

'L1' in Error Indication := Return Code

'Msg On Error' in Error Indication:= TRY AGAIN

CreateDiscretionaryData ()

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

S4.9

IF [SW12 = '9000']

THEN

GOTO S4.11

ELSE

GOTO S4.10.1

ENDIF

S4.10.1

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

S4.10.2

'Status' in Outcome Parameter Set := END APPLICATION
'Msg On Error' in Error Indication := ERROR – OTHER CARD
'L2' in Error Indication := STATUS BYTES
'SW12' in Error Indication := SW12
CreateDiscretionaryData ()
SET 'UI Request on Outcome Present' in Outcome Parameter Set
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
 Data))) Signal

S4.11

IF [Active AFL indicates first record (i.e. current record) is signed]
THEN
 GOTO S4.12
ELSE
 GOTO S4.13
ENDIF

S4.12

SET Signed Flag

S4.13

CLEAR Signed Flag

S4.14

Sfi := SFI of first record in Active AFL
Remove first record from Active AFL

S4.24

IF [Sfi ≤ 10]
THEN
 IF [(Length of Record > 0) AND (Record[1] = '70')]
 THEN
 Parsing Result := ParseAndStoreCardResponse(Record)
 ELSE
 Parsing Result := FALSE
 ENDIF
ELSE
 Processing of records in proprietary files is beyond the scope of this specification
ENDIF

S4.25

```
IF      [Parsing Result]
THEN
    GOTO S4.28 or S4.34
ELSE
    GOTO S4.27.1
ENDIF
```

S4.27.1

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD
'Status' in User Interface Request Data := NOT READY

S4.27.2

'Status' in Outcome Parameter Set := END APPLICATION
'Msg On Error' in Error Indication := ERROR – OTHER CARD
'L2' in Error Indication := PARSING ERROR
CreateDiscretionaryData ()
SET 'UI Request on Outcome Present' in Outcome Parameter Set
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
 Data))) Signal

S4.28

```
IF      [Record includes data object with tag equal to TagOf(CDOL1)]
THEN
    GOTO S4.29
ELSE
    GOTO S4.30
ENDIF
```

S4.29

```
FOR every TL in CDOL1
{
    IF      [IsEmpty(T)]
    THEN
        AddToList(T, Data Needed)
    ENDIF
}
```

S4.30

```
IF      [Record includes data object with tag equal to TagOf(DSDOL)]
THEN
    GOTO S4.31
ELSE
    GOTO S4.34
ENDIF
```

S4.31

```
IF      ['Read' in IDS Status is set]
THEN
    GOTO S4.32
ELSE
    GOTO S4.34
ENDIF
```

S4.32

```
IF      [IsEmpty(TagOf(DS Slot Management Control)) AND
        'Locked slot' in DS Slot Management Control is set]
THEN
    GOTO S4.34
ELSE
    GOTO S4.33
ENDIF
```

S4.33

```
FOR every TL in DSDOL
{
    IF      [IsEmpty(T)]
    THEN
        AddToList(T, Data Needed)
    ENDIF
}
```

S4.34

```
IF      [Signed Flag AND 'CDA' in ODA Status is set]
THEN
    GOTO S4.35
ELSE
    GOTO S4.36
ENDIF
```

S4.35

```
IF      [Sfi ≤ 10]
THEN
    IF      [Enough space left in Static Data To Be Authenticated to append Record
              (without tag '70' and length)]
    THEN
        Append Record (excluding tag '70' and length) at the end of Static Data To Be
        Authenticated
    ELSE
        SET 'CDA failed' in Terminal Verification Results
    ENDIF
ELSE
    IF      [(Record[1] = '70') AND
              Record is TLV encoded AND
              Enough space left in Static Data To Be Authenticated to append
              Record]
    THEN
        Append Record (including tag '70' and length) at the end of Static Data To Be
        Authenticated
    ELSE
        SET 'CDA failed' in Terminal Verification Results
    ENDIF
ENDIF
```

S4.36

```
IF      ['CDA' in ODA Status is set OR 'Read all records even when no CDA' in Kernel
          Configuration is set]
THEN
    GOTO S4.15 or S4.19
ELSE
    GOTO S4.37
ENDIF
```


S4.37

```
IF      [IsEmpty(TagOf(Application Expiration Date)) AND
        IsNotEmpty(TagOf(Application PAN)) AND
        IsNotEmpty(TagOf(Application PAN Sequence Number)) AND
        IsNotEmpty(TagOf(Application Usage Control)) AND
        IsNotEmpty(TagOf(CVM List)) AND
        IsNotEmpty(TagOf(Issuer Action Code – Default)) AND
        IsNotEmpty(TagOf(Issuer Action Code – Denial)) AND
        IsNotEmpty(TagOf(Issuer Action Code – Online)) AND
        IsNotEmpty(TagOf(Issuer Country Code)) AND
        IsNotEmpty(TagOf(Track 2 Equivalent Data)) AND
        IsNotEmpty(TagOf(CDOL1))]
THEN
    GOTO S4.38
ELSE
    GOTO S4.15 or S4.19
ENDIF
```

S4.38

Active AFL := empty

S4.15

Active Tag := GetNextGetDataTagFromList (Tags To Read Yet)

```
IF      [Active Tag is not NULL]
THEN
    GOTO S4.16
ELSE
    GOTO S4.19
ENDIF
```

S4.16

Prepare GET DATA command for Active Tag as specified in section 5.4

S4.17

Send CA(GET DATA command) Signal

S4.18

'Next Cmd' in Next Cmd := GET DATA
GOTO S456.1

S4.19

```
IF      [Active AFL is empty]
THEN
    GOTO S4.20
ELSE
    GOTO S4.21
ENDIF
```

S4.20

'Next Cmd' in Next Cmd := NONE
GOTO S456.1

S4.21

Prepare READ RECORD command for first record in Active AFL as specified in section 5.7

S4.22

Send CA(READ RECORD command) Signal

S4.23

'Next Cmd' in Next Cmd := READ RECORD
GOTO S456.1

6.9 State 5 – Waiting for Get Data Response

State 5 is a state specific to SDS processing and is only implemented for the DE/DS Implementation Option.

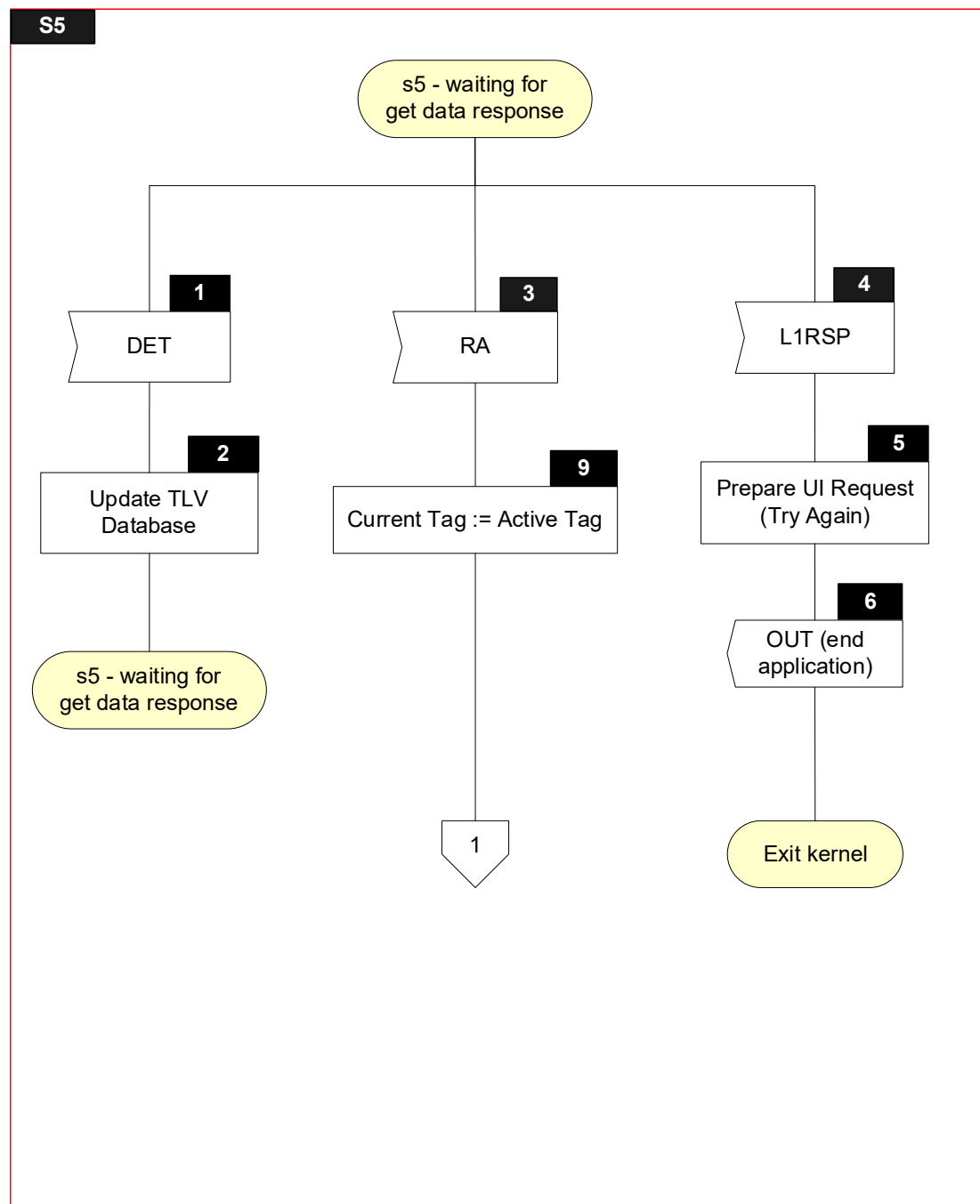
6.9.1 Local Variables

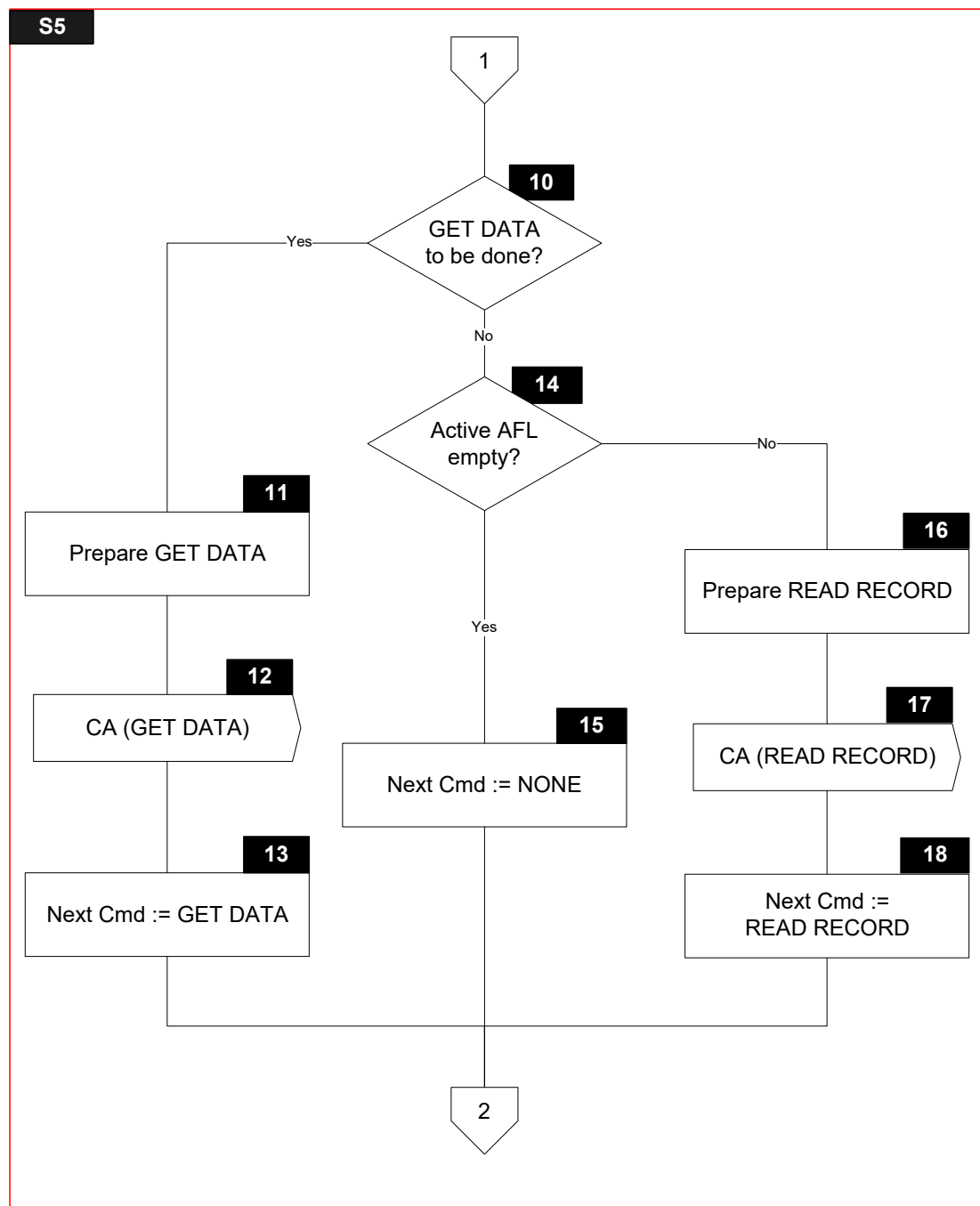
Name	Length	Format	Description
Return Code	1	b	Value returned with L1RSP Signal (TIME OUT ERROR, PROTOCOL ERROR, TRANSMISSION ERROR)
Sync Data	var.	b	List of data objects returned with DET Signal
Parsing Result	1	b	Boolean used to store result of parsing a TLV string
SW12	2	b	Status bytes
Response Message Data Field	var. up to 256	b	TLV encoded string included in R-APDU of GET DATA
Current Tag	var.	b	Tag indicating the tag of the current GET DATA
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 252	b	Value of TLV encoded string

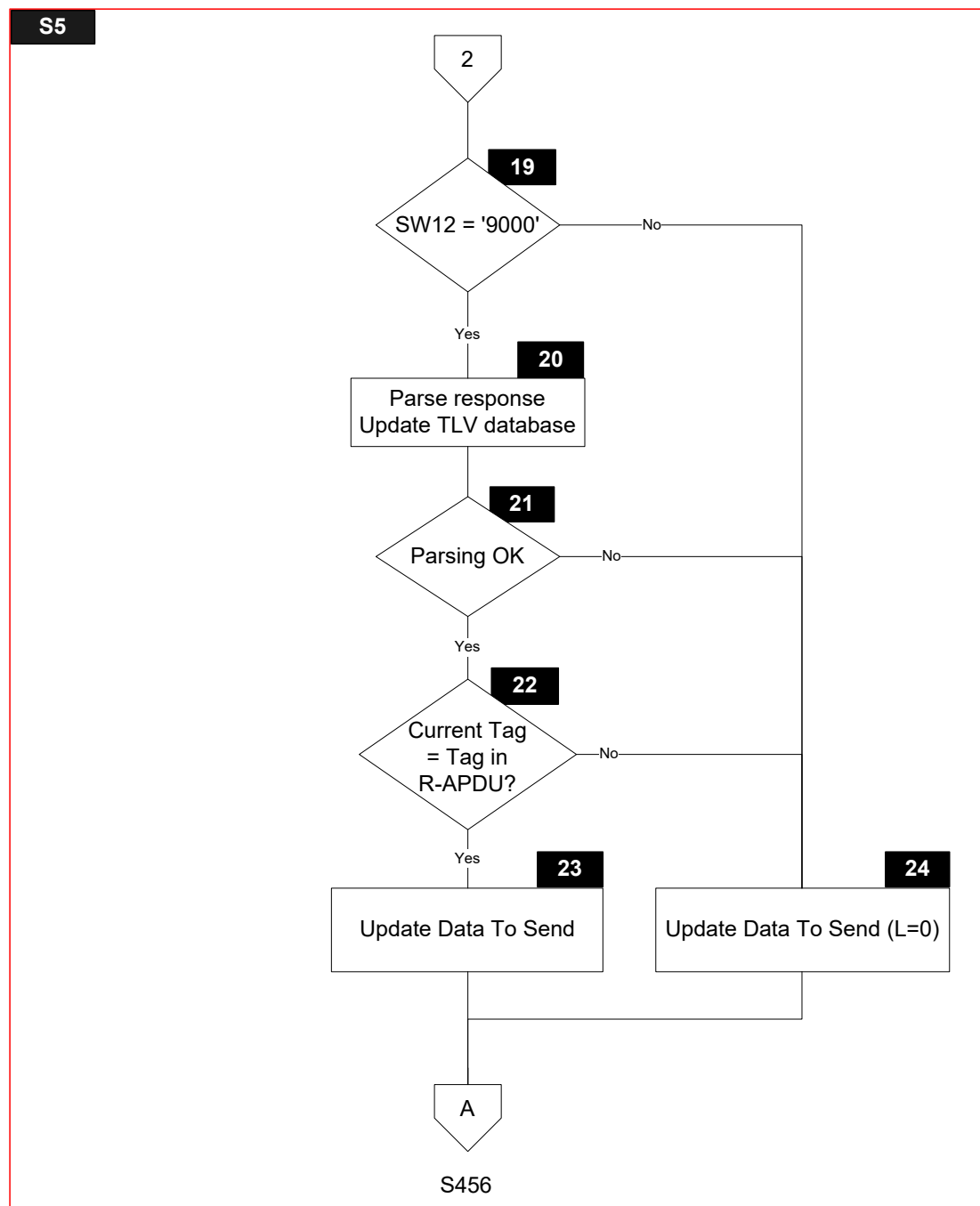
6.9.2 Flow Diagram

Figure 6.8 shows the flow diagram of s5 – waiting for get data response. Symbols in this diagram are labelled S5.X.

Figure 6.8: State 5 Flow Diagram







6.9.3 Processing

S5.1

Receive DET Signal with Sync Data

S5.2

UpdateWithDetData(Sync Data)

S5.3

Receive RA Signal with Response Message Data Field and SW12

S5.4

Receive L1RSP Signal with Return Code

S5.5

'Message Identifier' in User Interface Request Data := TRY AGAIN

'Status' in User Interface Request Data := READY TO READ

'Hold Time' in User Interface Request Data := '000000'

S5.6

'Status' in Outcome Parameter Set := END APPLICATION

'Start' in Outcome Parameter Set := B

SET 'UI Request on Restart Present' in Outcome Parameter Set

'L1' in Error Indication := Return Code

'Msg On Error' in Error Indication:= TRY AGAIN

CreateDiscretionaryData ()

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

S5.9

Current Tag := Active Tag

S5.10

Active Tag := GetNextGetDataTagFromList (Tags To Read Yet)

IF [Active Tag is not NULL]

THEN

 GOTO S5.11

ELSE

 GOTO S5.14

ENDIF

S5.11

Prepare GET DATA command for Active Tag as specified in section 5.4

S5.12

Send CA(GET DATA command) Signal

S5.13

'Next Cmd' in Next Cmd := GET DATA

S5.14

IF [Active AFL is empty]

THEN

GOTO S5.15

ELSE

GOTO S5.16

ENDIF

S5.15

'Next Cmd' in Next Cmd := NONE

S5.16

Prepare READ RECORD command for first record in Active AFL as specified in section 5.7

S5.17

Send CA(READ RECORD command) Signal

S5.18

'Next Cmd' in Next Cmd := READ RECORD

S5.19

IF [SW12 = '9000']

THEN

GOTO S5.20

ELSE

GOTO S5.24

ENDIF

S5.20

Parsing Result := ParseAndStoreCardResponse(Response Message Data Field)

Retrieve T, L and V from Response Message Data Field

Table 6.1: Response Message Data Field

T	L	V
---	---	---

S5.21

```
IF      [Parsing Result]
THEN
    GOTO S5.22
ELSE
    GOTO S5.24
ENDIF
```

S5.22

```
IF      [Current Tag = T]
THEN
    GOTO S5.23
ELSE
    GOTO S5.24
ENDIF
```

S5.23

AddToList(TLV, Data To Send)

S5.24

Add Current Tag with zero length to Data To Send:
AddToList(Current Tag || '00', Data To Send)

6.10 State 6 – Waiting for First Write Flag

State 6 is a state specific to Data Exchange processing and is only implemented for the DE/DS Implementation Option.

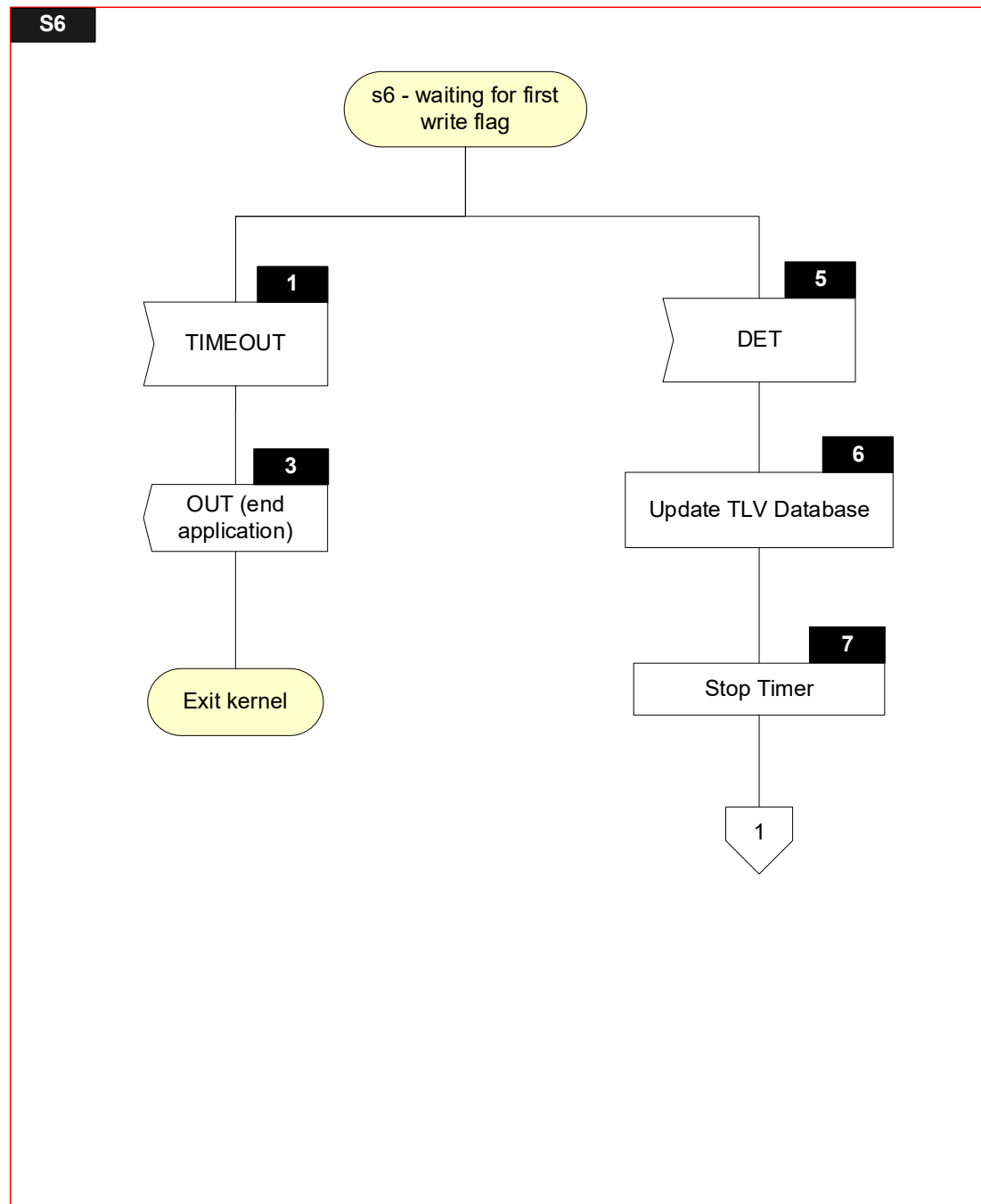
6.10.1 Local Variables

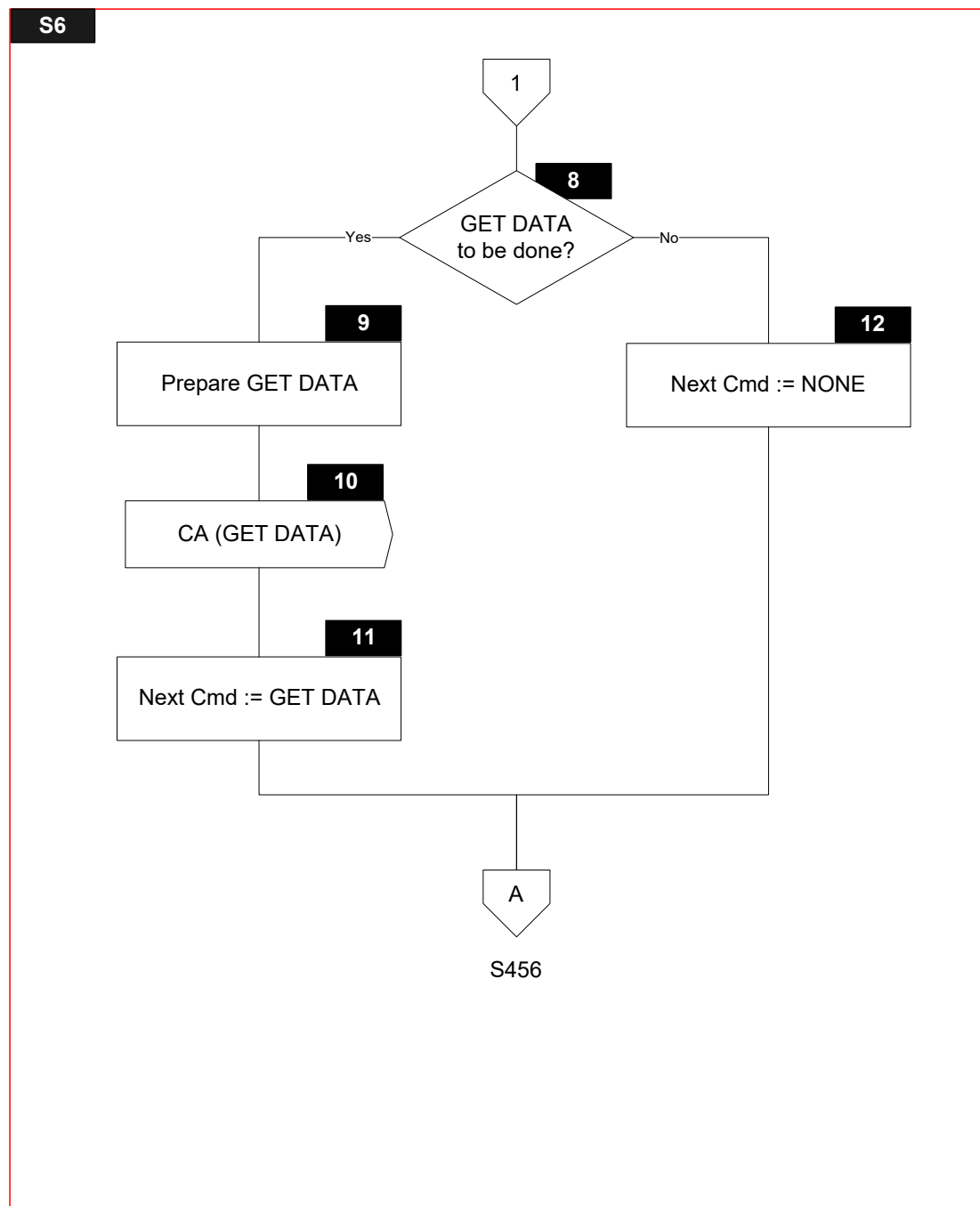
Name	Length	Format	Description
Sync Data	var.	b	List of data objects returned with DET Signal
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 252	b	Value of TLV encoded string

6.10.2 Flow Diagram

Figure 6.9 shows the flow diagram of s6 – waiting for first write flag. Symbols in this diagram are labelled S6.X.

Figure 6.9: State 6 Flow Diagram





6.10.3 Processing

S6.1

Receive TIMEOUT Signal

S6.3

'Status' in Outcome Parameter Set := END APPLICATION

'L3' in Error Indication := TIME OUT

CreateDiscretionaryData ()

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data))) Signal

S6.5

Receive DET Signal with Sync Data

S6.6

UpdateWithDetData(Sync Data)

S6.7

Stop Timer

S6.8

Active Tag := GetNextGetDataTagFromList (Tags To Read Yet)

IF [Active Tag is not NULL]

THEN

GOTO S6.9

ELSE

GOTO S6.12

ENDIF

S6.9

Prepare GET DATA command for Active Tag as specified in section 5.4

S6.10

Send CA(GET DATA command) Signal

S6.11

'Next Cmd' in Next Cmd := GET DATA

S6.12

'Next Cmd' in Next Cmd := NONE

6.11 States 4, 5, and 6 – Common Processing

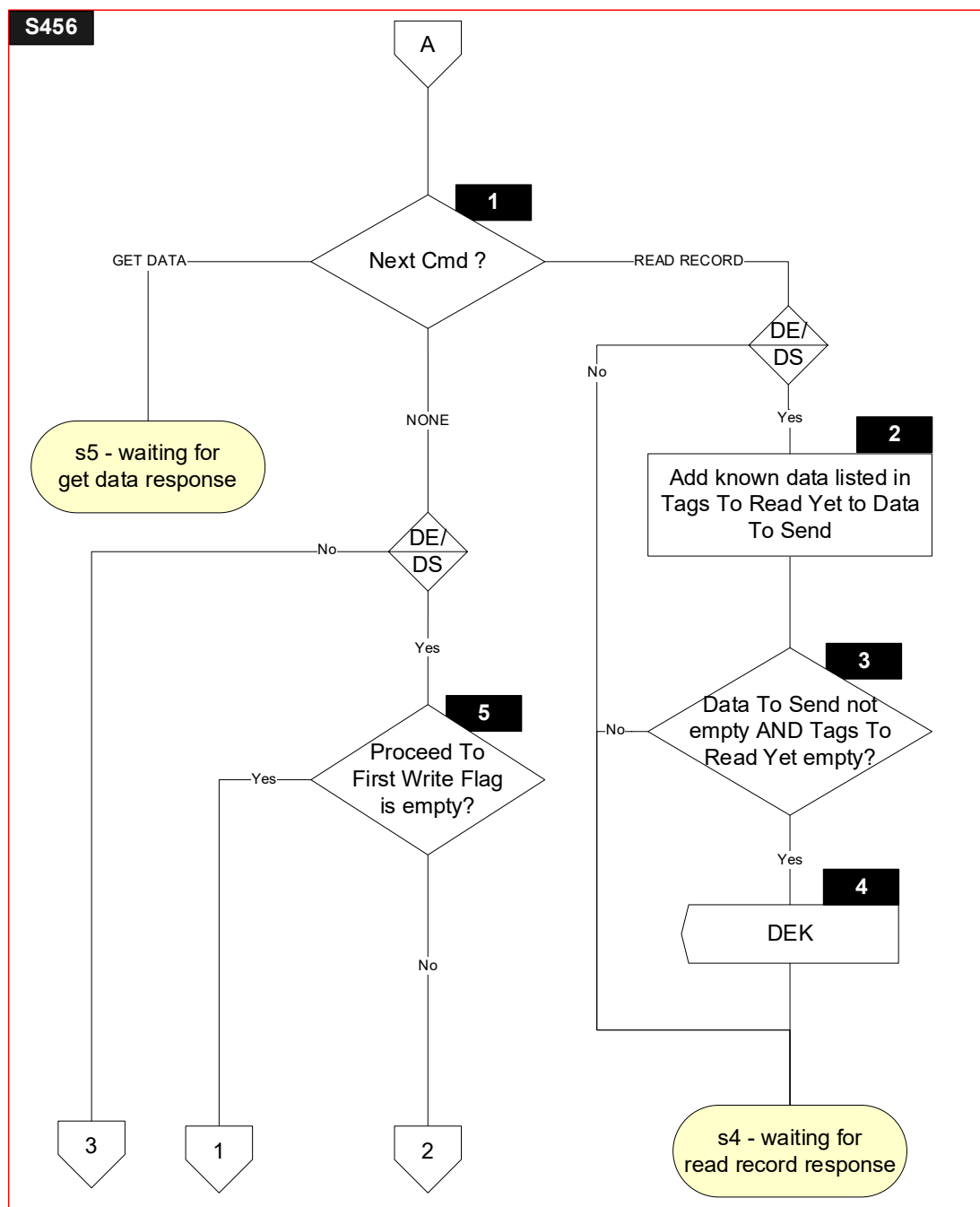
6.11.1 Local Variables

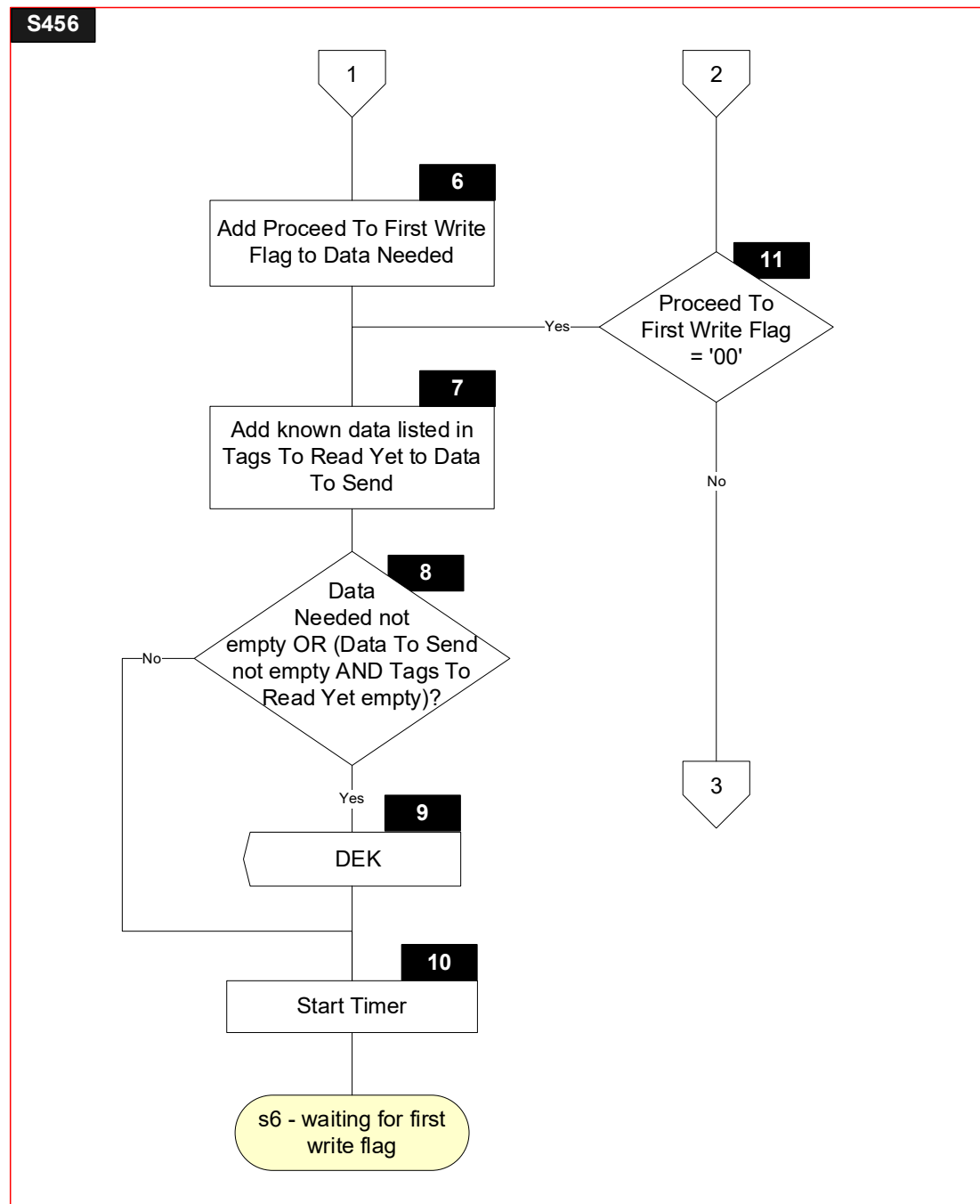
Local variables for common processing are defined in states 4, 5, and 6.

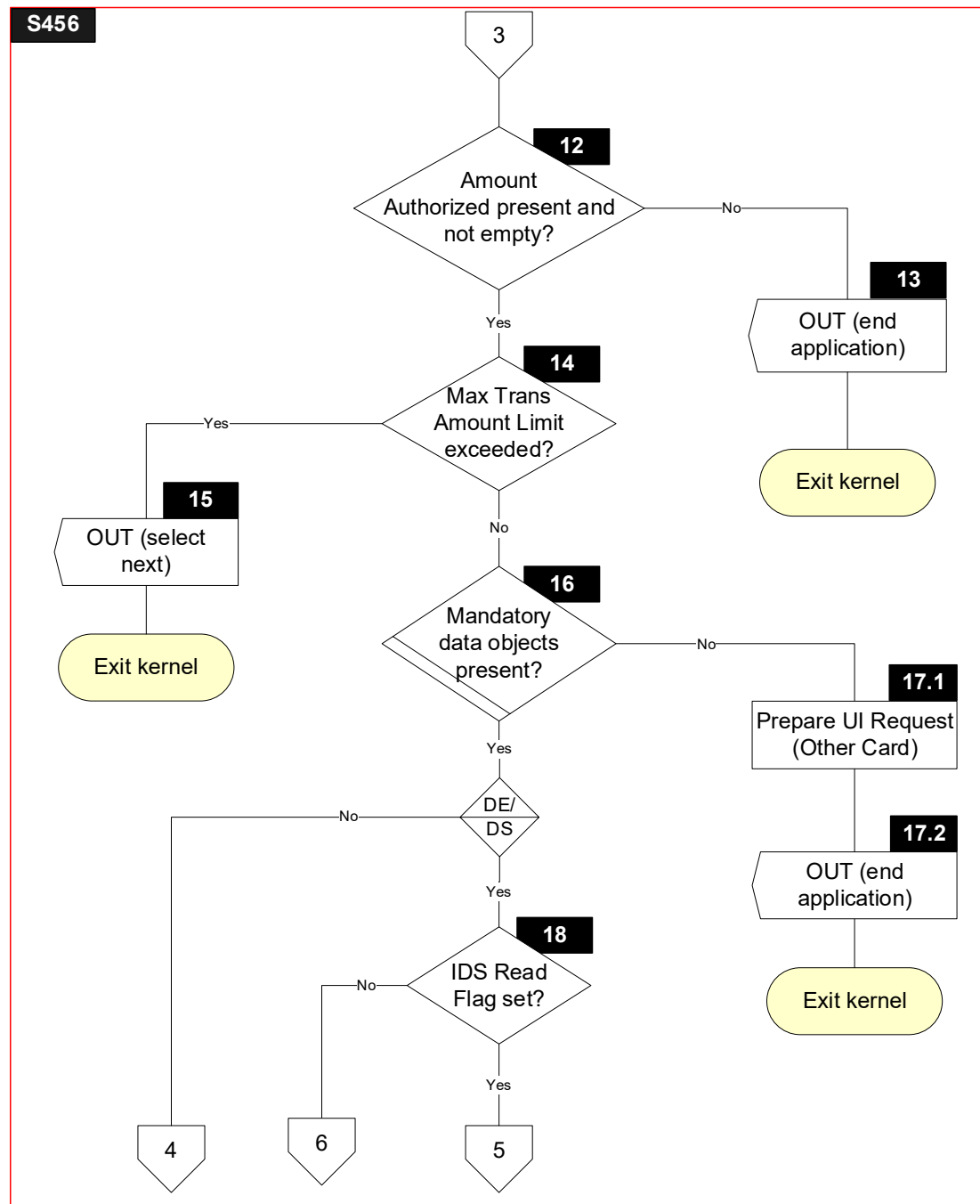
6.11.2 Flow Diagram

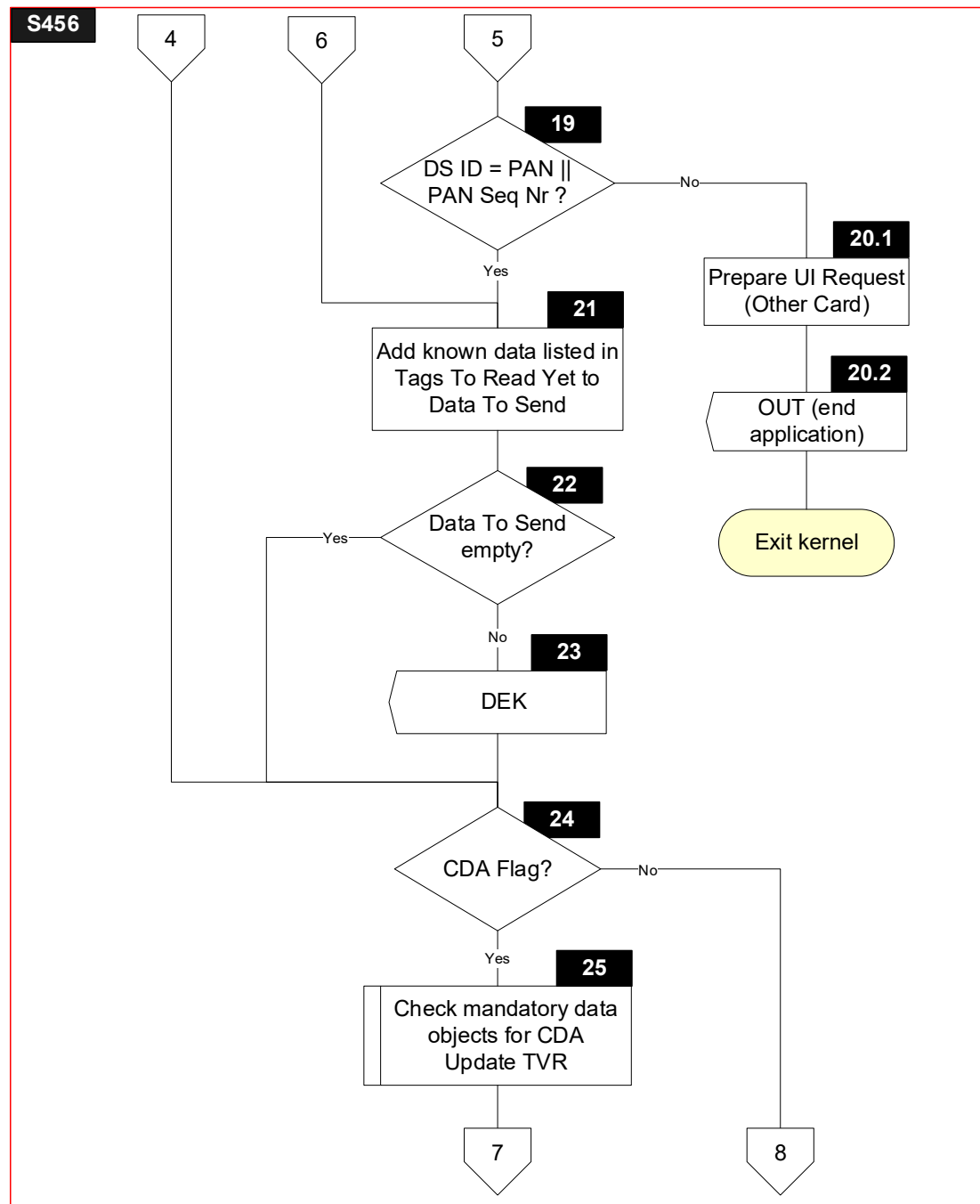
Figure 6.10 shows the flow diagram for common processing between states 4, 5, and 6. Symbols in this diagram are labelled S456.X.

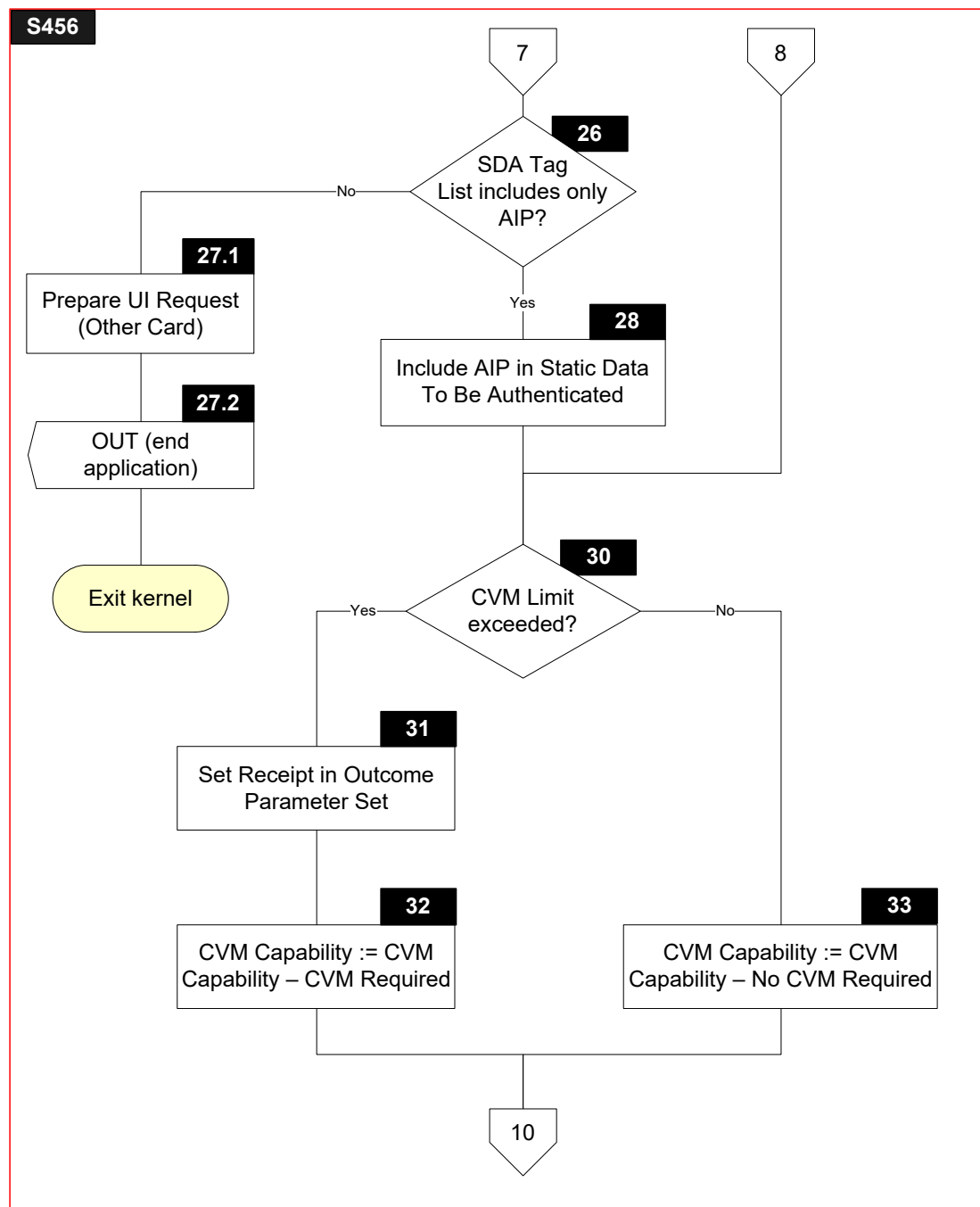
Figure 6.10: States 4, 5, and 6 – Common Processing – Flow Diagram

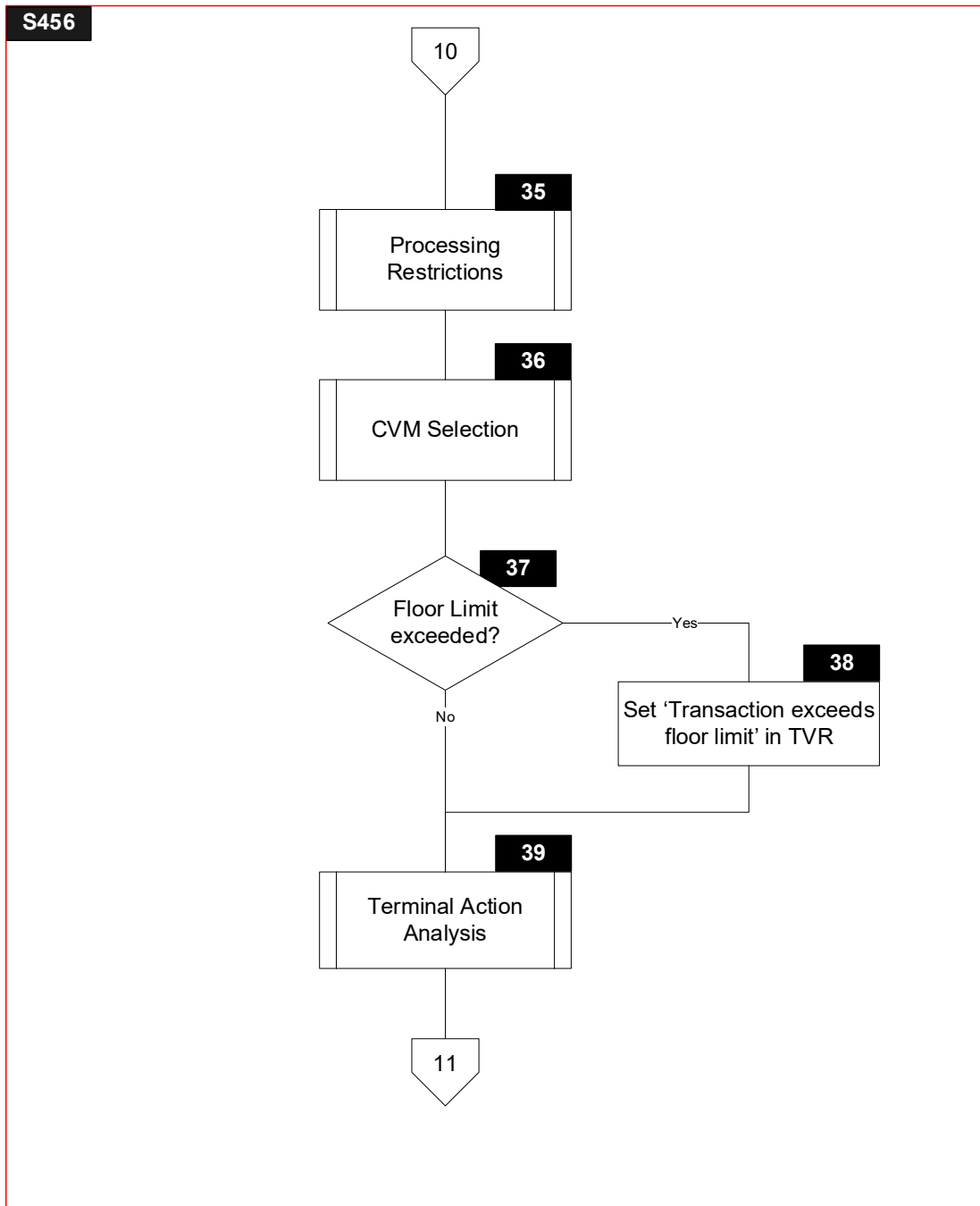


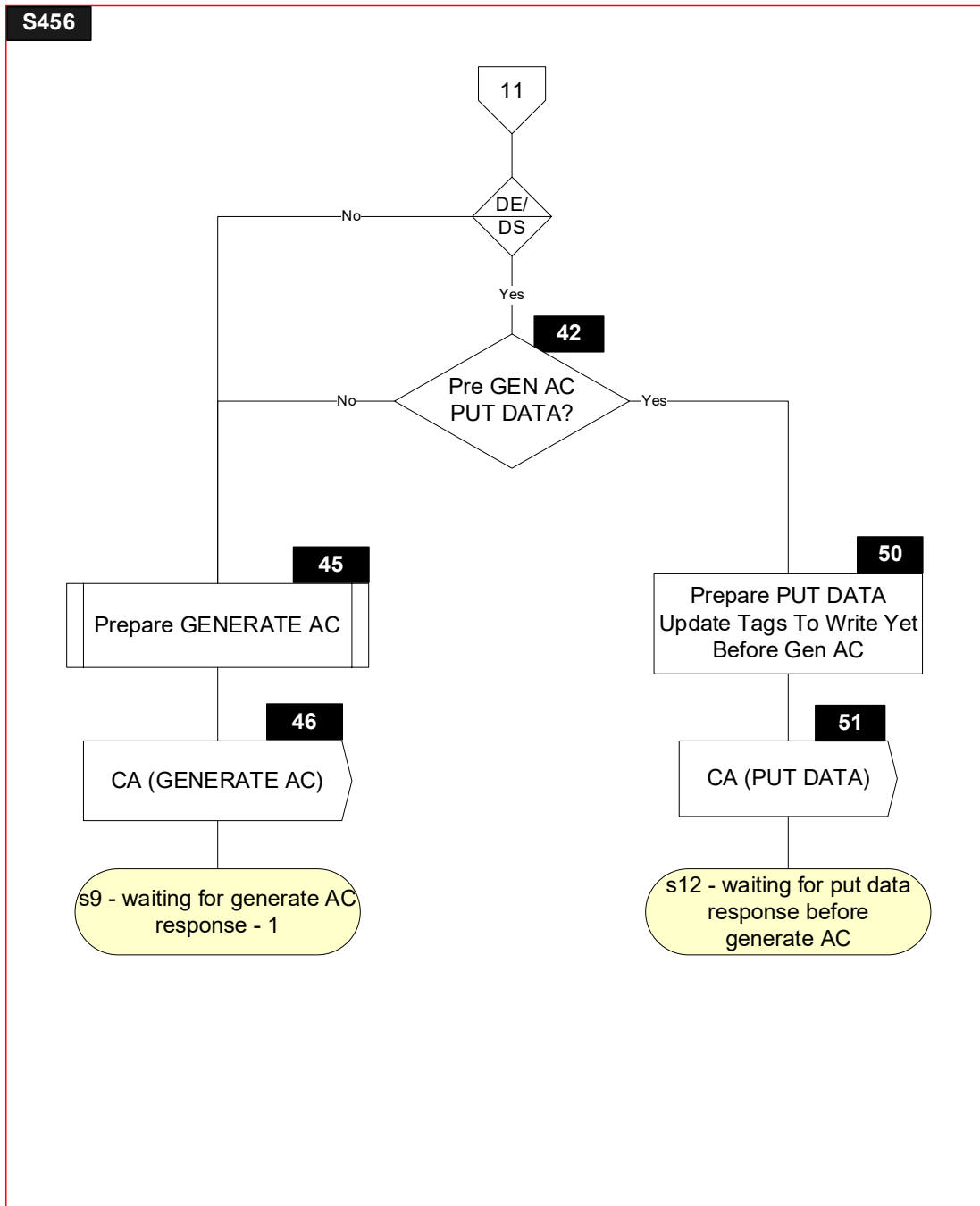












6.11.3 Processing

Note that symbols S456.2, S456.3, S456.4, S456.5, S456.6, S456.7, S456.8, S456.9, S456.10, S456.11, S456.18, S456.19, S456.20.1, S456.20.2, S456.21, S456.22, S456.23, S456.42, S456.50 and S456.51 are only implemented for the DE/DS Implementation Option.

S456.1

If DE/DS implemented:

```
IF      ['Next Cmd' in Next Cmd = READ RECORD]
THEN
    GOTO S456.2
ELSE
    IF ['Next Cmd' in Next Cmd = GET DATA]
    THEN
        GOTO s5 - waiting for get data response
    ELSE
        GOTO S456.5
    ENDIF
ENDIF
```

If DE/DS not implemented:

```
IF      ['Next Cmd' in Next Cmd = READ RECORD]
THEN
    GOTO s4 - waiting for read record response
ELSE
    GOTO S456.12
ENDIF
```

S456.2

FOR every T in Tags To Read Yet

```
{
    IF      [IsEmpty(T)]
    THEN
        AddToList(GetTLV(T), Data To Send)
        RemoveFromList(T, Tags To Read Yet)
    ENDIF
}
```

S456.3

```
IF      [IsEmptyList(Data To Send) AND IsEmptyList(Tags To Read Yet)]
THEN
    GOTO S456.4
ELSE
    GOTO s4 - waiting for read record response
ENDIF
```

S456.4

Send DEK(Data To Send, Data Needed) Signal

Initialize(Data To Send)

Initialize(Data Needed)

S456.5

IF [IsEmpty(TagOf(Proceed To First Write Flag))]

THEN

GOTO S456.6

ELSE

GOTO S456.11

ENDIF

S456.6

AddToList (TagOf (Proceed To First Write Flag), Data Needed)

S456.7

FOR every T in Tags To Read Yet

{

IF [IsNotEmpty(T)]

THEN

AddToList(GetTLV(T), Data To Send)

RemoveFromList(T, Tags To Read Yet)

ENDIF

}

S456.8

IF [IsNotEmptyList(Data Needed) OR
(IsNotEmptyList(Data To Send) AND IsEmptyList(Tags To Read Yet))]

THEN

GOTO S456.9

ELSE

GOTO S456.10

ENDIF

S456.9

Send DEK(Data To Send, Data Needed) Signal

Initialize(Data To Send)

Initialize(Data Needed)

S456.10

Start Timer (Time Out Value)

S456.11

```
IF      [IsPresent(TagOf(Proceed To First Write Flag)) AND  
        (Proceed To First Write Flag = '00')]  
THEN  
    GOTO S456.7  
ELSE  
    GOTO S456.12  
ENDIF
```

S456.12

```
IF      [IsNotEmpty(TagOf(Amount, Authorized (Numeric)))]  
THEN  
    GOTO S456.14  
ELSE  
    GOTO S456.13  
ENDIF
```

S456.13

```
'Status' in Outcome Parameter Set := END APPLICATION  
'L3' in Error Indication := AMOUNT NOT PRESENT  
CreateDiscretionaryData ()  
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),  
         GetTLV(TagOf(Discretionary Data))) Signal
```

S456.14

```
IF      [Amount, Authorized (Numeric) > Reader Contactless Transaction Limit]  
THEN  
    GOTO S456.15  
ELSE  
    GOTO S456.16  
ENDIF
```

S456.15

```
'Field Off Request' in Outcome Parameter Set := N/A  
'Status' in Outcome Parameter Set := SELECT NEXT  
'Start' in Outcome Parameter Set := C  
'L2' in Error Indication := MAX LIMIT EXCEEDED  
CreateDiscretionaryData ()  
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),  
         GetTLV(TagOf(Discretionary Data))) Signal
```

S456.16

Check if all mandatory data objects are present in the TLV Database

Table 6.2: Mandatory Data Objects

Data Object
Application Expiration Date
Application PAN
CDOL1

IF [IsNotEmpty(TagOf(Application Expiration Date)) AND
IsNotEmpty(TagOf(Application PAN)) AND
IsNotEmpty(TagOf(CDOL1))]

THEN

GOTO S456.18 or S456.24

ELSE

GOTO S456.17.1

ENDIF

S456.17.1

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

S456.17.2

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

'L2' in Error Indication := CARD DATA MISSING

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

S456.18

IF ['Read' in IDS Status is set]

THEN

GOTO S456.19

ELSE

GOTO S456.21

ENDIF

S456.19

Concatenate from left to right the Application PAN (without any 'F' padding) with the Application PAN Sequence Number (if the Application PAN Sequence Number is not present, then it is replaced by a '00' byte). The result, Y, must be padded to the left with a hexadecimal zero if necessary to ensure whole bytes. It must also be padded to the left with hexadecimal zeroes if necessary to ensure a minimum length of 8 bytes.

```
IF      [DS ID = Y]
THEN
    GOTO S456.21
ELSE
    GOTO S456.20.1
ENDIF
```

S456.20.1

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

S456.20.2

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

'L2' in Error Indication := CARD DATA ERROR

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

S456.21

FOR every T in Tags To Read Yet

```
{
    IF      [IsPresent(T)]
    THEN
        AddToList(GetTLV(T), Data To Send)
    ELSE
        IF [IsKnown(T)]
        THEN
            Add an empty data object with tag T to Data To Send if the TLV
            Database does not include a data object with tag T:
            AddToList(T || '00', Data To Send)
        ENDIF
    ENDIF
    RemoveFromList(T, Tags To Read Yet)
}
```

S456.22

```
IF      [IsEmptyList(Data To Send)]  
THEN  
    GOTO S456.24  
ELSE  
    GOTO S456.23  
ENDIF
```

S456.23

```
Send DEK(Data To Send) Signal  
Initialize(Data To Send)
```

S456.24

```
IF      ['CDA' in ODA Status is set]  
THEN  
    GOTO S456.25  
ELSE  
    GOTO S456.30  
ENDIF
```

S456.25

Check if all mandatory Card data objects for CDA are present in the TLV Database

Table 6.3: Mandatory Card CDA Data Objects

Data Object
CA Public Key Index (Card)
Issuer Public Key Certificate
Issuer Public Key Exponent
ICC Public Key Certificate
ICC Public Key Exponent
Static Data Authentication Tag List

```

IF      [NOT (
        IsNotEmpty(TagOf(CA Public Key Index (Card))) AND
        IsNotEmpty(TagOf(Issuer Public Key Certificate)) AND
        IsNotEmpty(TagOf(Issuer Public Key Exponent)) AND
        IsNotEmpty(TagOf(ICC Public Key Certificate)) AND
        IsNotEmpty(TagOf(ICC Public Key Exponent)) AND
        IsNotEmpty(TagOf(Static Data Authentication Tag List))
      )]
THEN
    SET 'ICC data missing' in Terminal Verification Results
    SET 'CDA failed' in Terminal Verification Results
ENDIF
IF      [The CA Public Key Index (Card) is not present in the CA Public Key Database]
THEN
    SET 'CDA failed' in Terminal Verification Results
ENDIF

```

S456.26

```

IF      [IsNotEmpty(TagOf(Static Data Authentication Tag List)) AND
        (Static Data Authentication Tag List = '82')]
THEN
    GOTO S456.28
ELSE
    GOTO S456.27.1
ENDIF

```

S456.27.1

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

S456.27.2

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

'L2' in Error Indication := CARD DATA ERROR

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)) , GetTLV(TagOf(User Interface Request
 Data))) Signal

S456.28

IF [Enough space left in Static Data To Be Authenticated to append Application
Interchange Profile]

THEN

Append Application Interchange Profile at the end of Static Data To Be Authenticated

ELSE

SET 'CDA failed' in Terminal Verification Results

ENDIF

S456.30

IF [Amount, Authorized (Numeric) > Reader CVM Required Limit]

THEN

GOTO S456.31

ELSE

GOTO S456.33

ENDIF

S456.31

'Receipt' in Outcome Parameter Set := YES

S456.32

Terminal Capabilities[2] := CVM Capability – CVM Required

S456.33

Terminal Capabilities[2] := CVM Capability – No CVM Required

S456.35

Process Processing Restrictions as specified in section 7.3

S456.36

Process CVM Selection as specified in section 7.1

S456.37

```
IF      [Amount, Authorized (Numeric) > Reader Contactless Floor Limit]
THEN
    GOTO S456.38
ELSE
    GOTO S456.39
ENDIF
```

S456.38

SET 'Transaction exceeds floor limit' in Terminal Verification Results

S456.39

Process Terminal Action Analysis as specified in section 7.4

S456.42

```
IF      [IsEmptyList(Tags To Write Yet Before Gen AC)]
THEN
    GOTO S456.50
ELSE
    GOTO S456.45
ENDIF
```

S456.45

Prepare GENERATE AC command as specified in section 7.2

S456.46

Send CA(GENERATE AC command) Signal

S456.50

TLV := GetAndRemoveFromList(Tags To Write Yet Before Gen AC)
Prepare PUT DATA command for TLV as specified in section 5.6

S456.51

Send CA(PUT DATA command) Signal

6.12 State 9 – Waiting for Generate AC Response

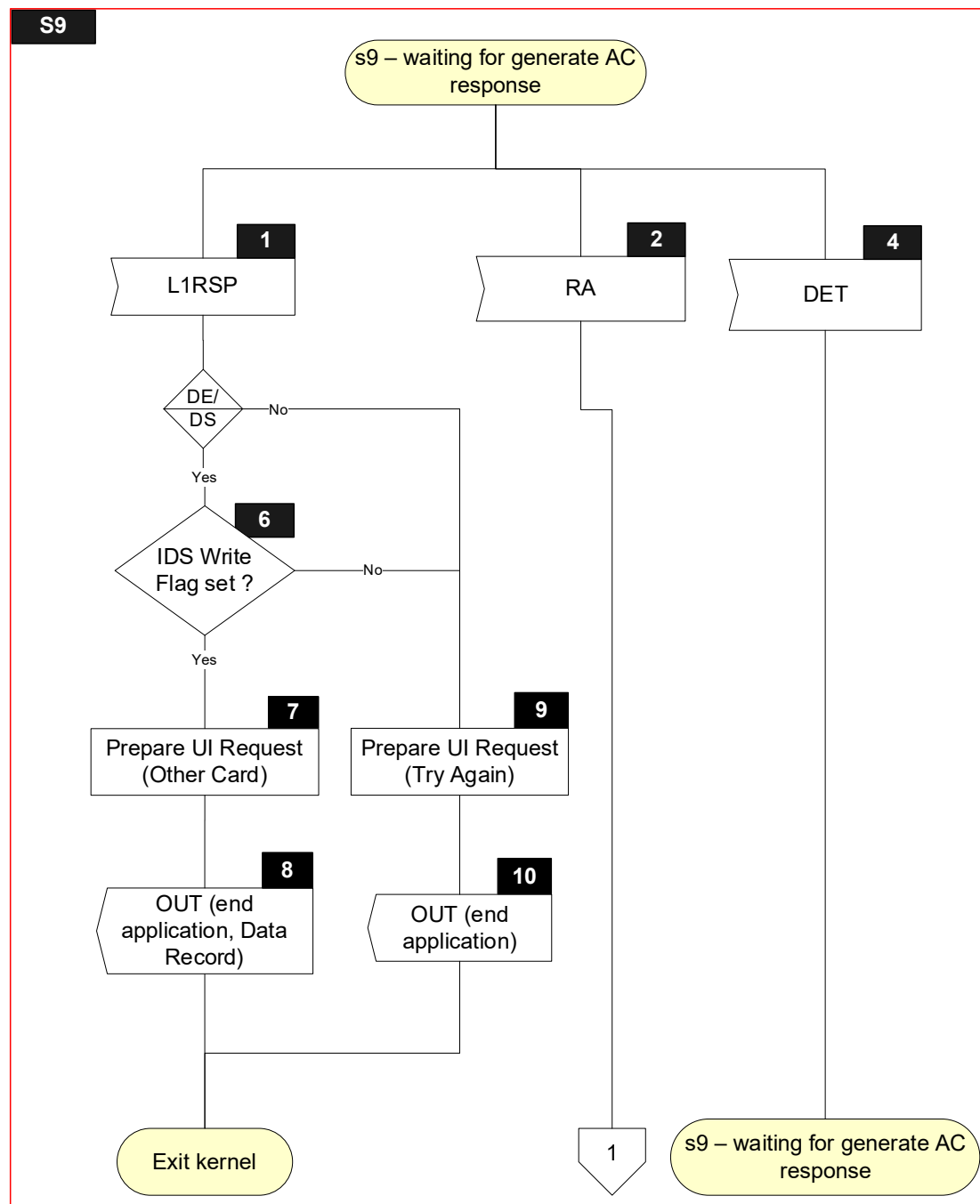
6.12.1 Local Variables

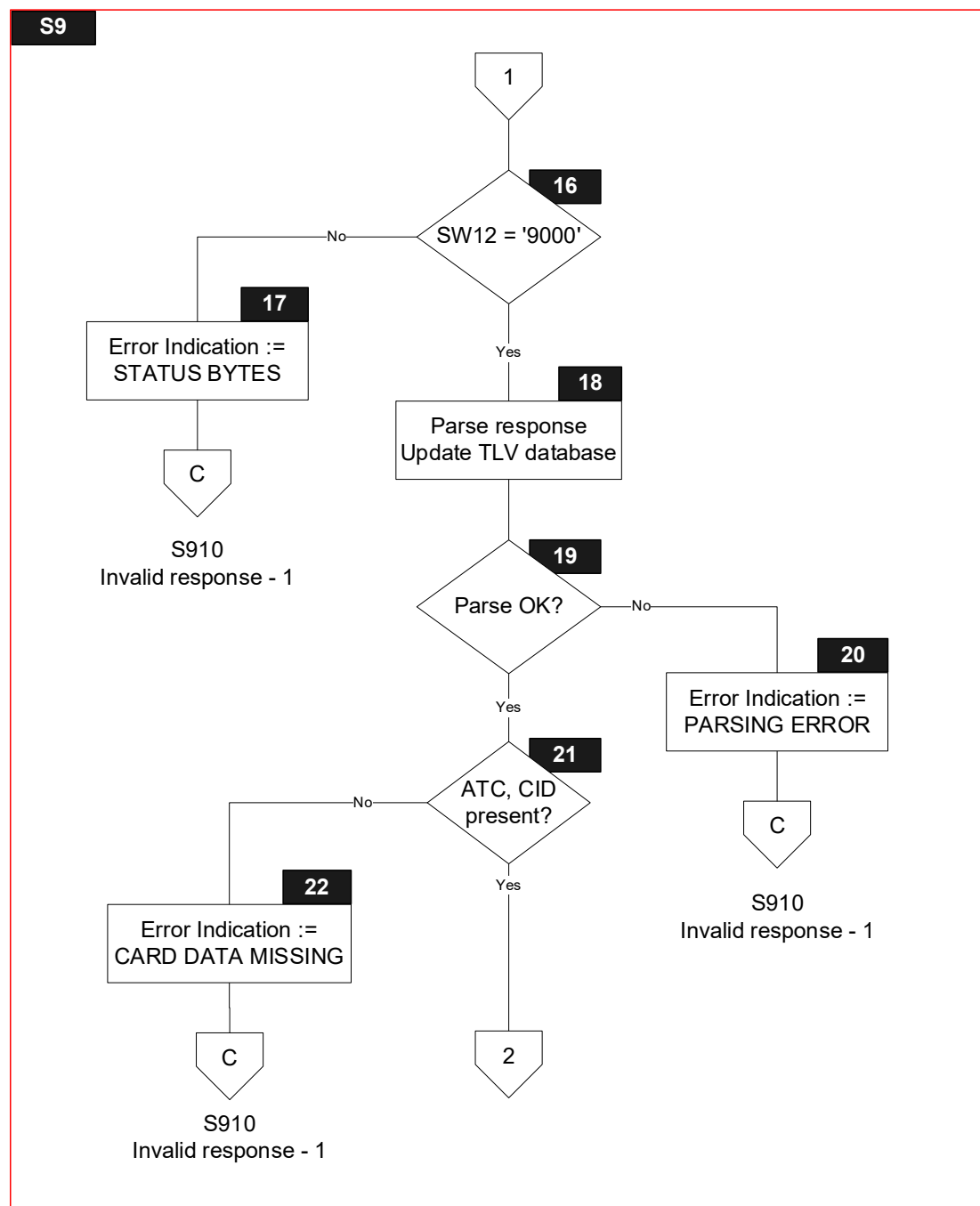
Name	Length	Format	Description
Return Code	1	b	Value returned with L1RSP Signal (TIME OUT ERROR, PROTOCOL ERROR, TRANSMISSION ERROR)
Parsing Result	1	b	Boolean used to store result of parsing a TLV string
SW12	2	b	Status bytes
Response Message Data Field	var. up to 256	b	TLV encoded string included in R-APDU of GENERATE AC
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 252	b	Value of TLV encoded string

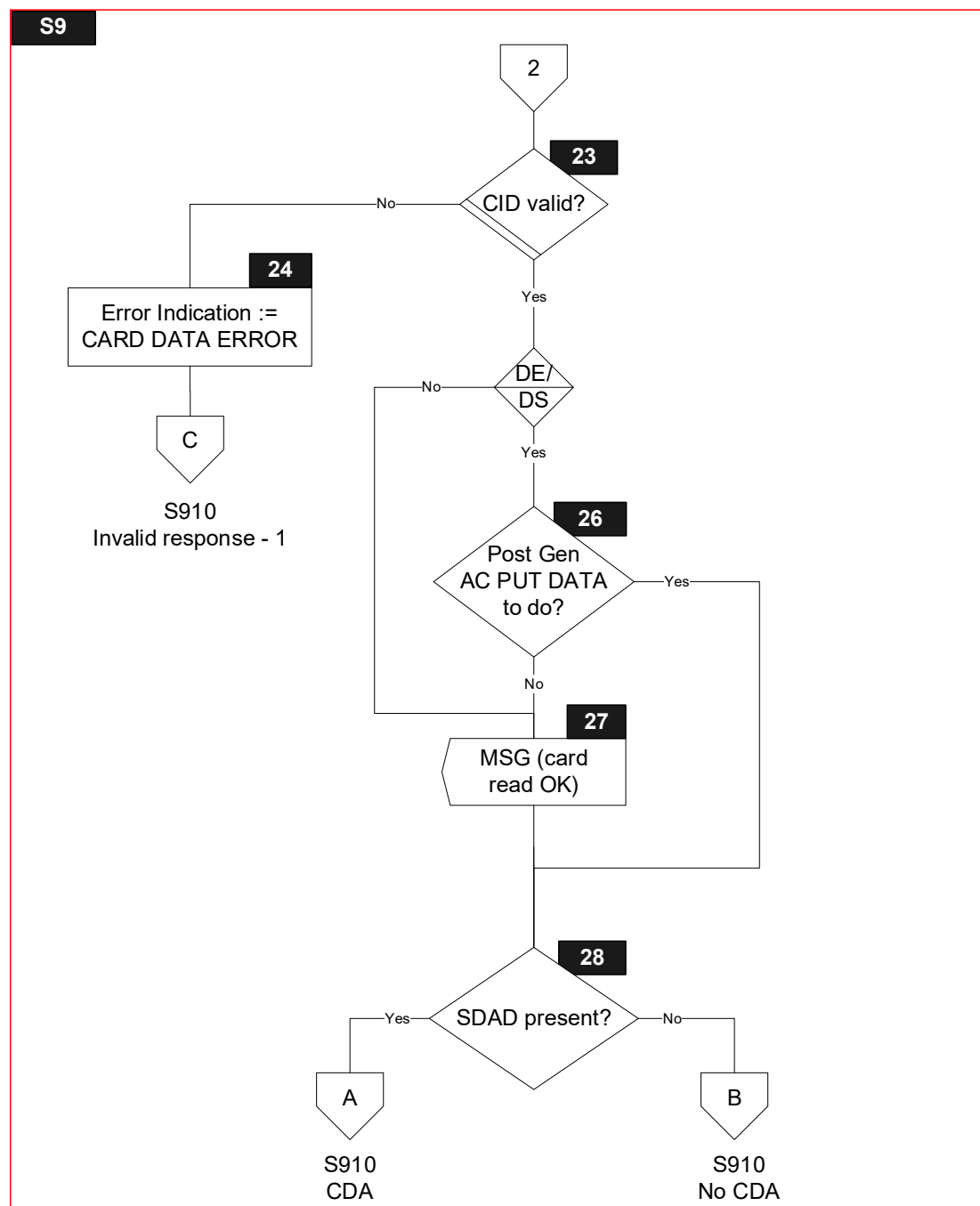
6.12.2 Flow Diagram

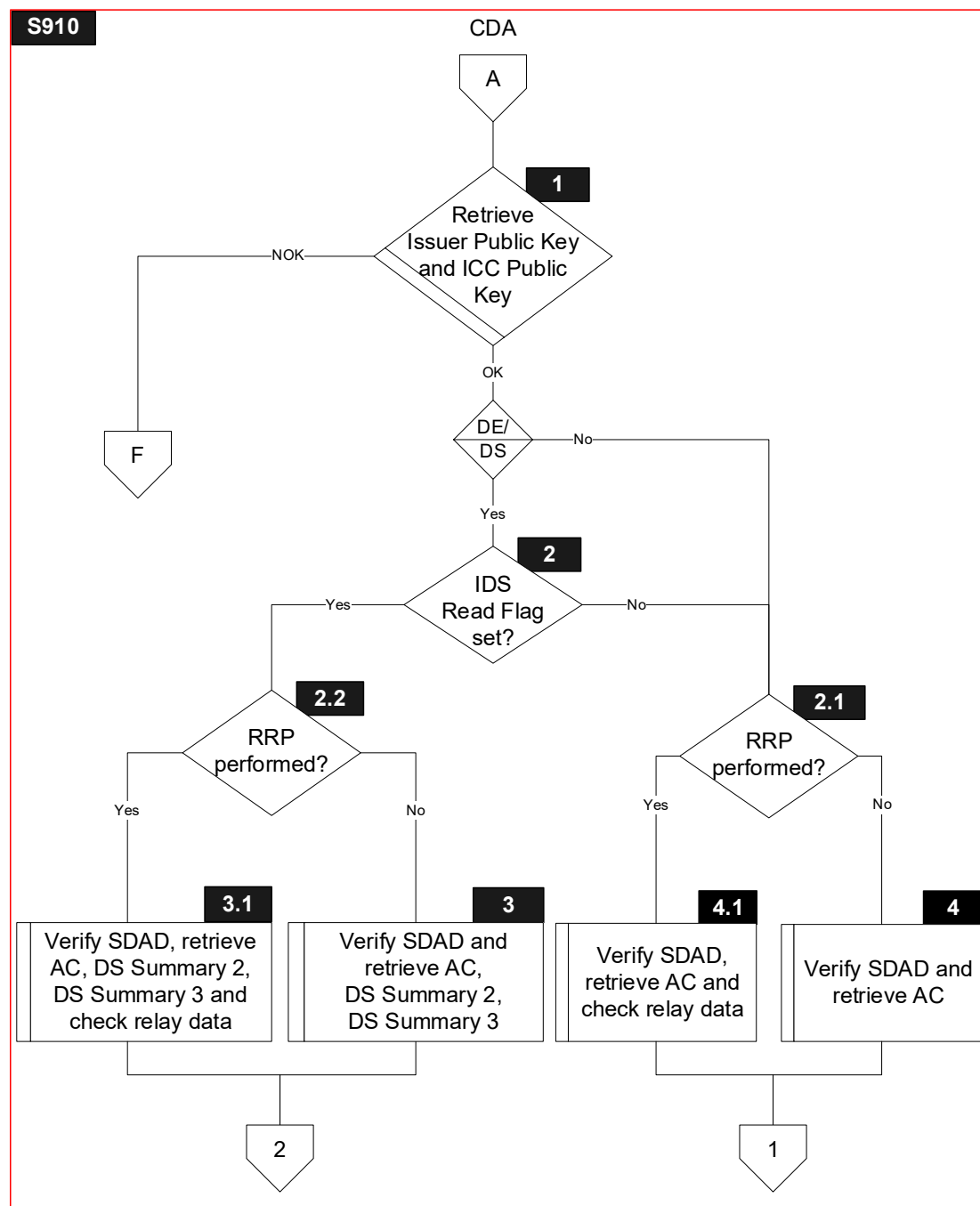
Figure 6.11 shows the flow diagram of s9 – waiting for generate AC response. Symbols in this diagram are labelled S9.X or S910.X.

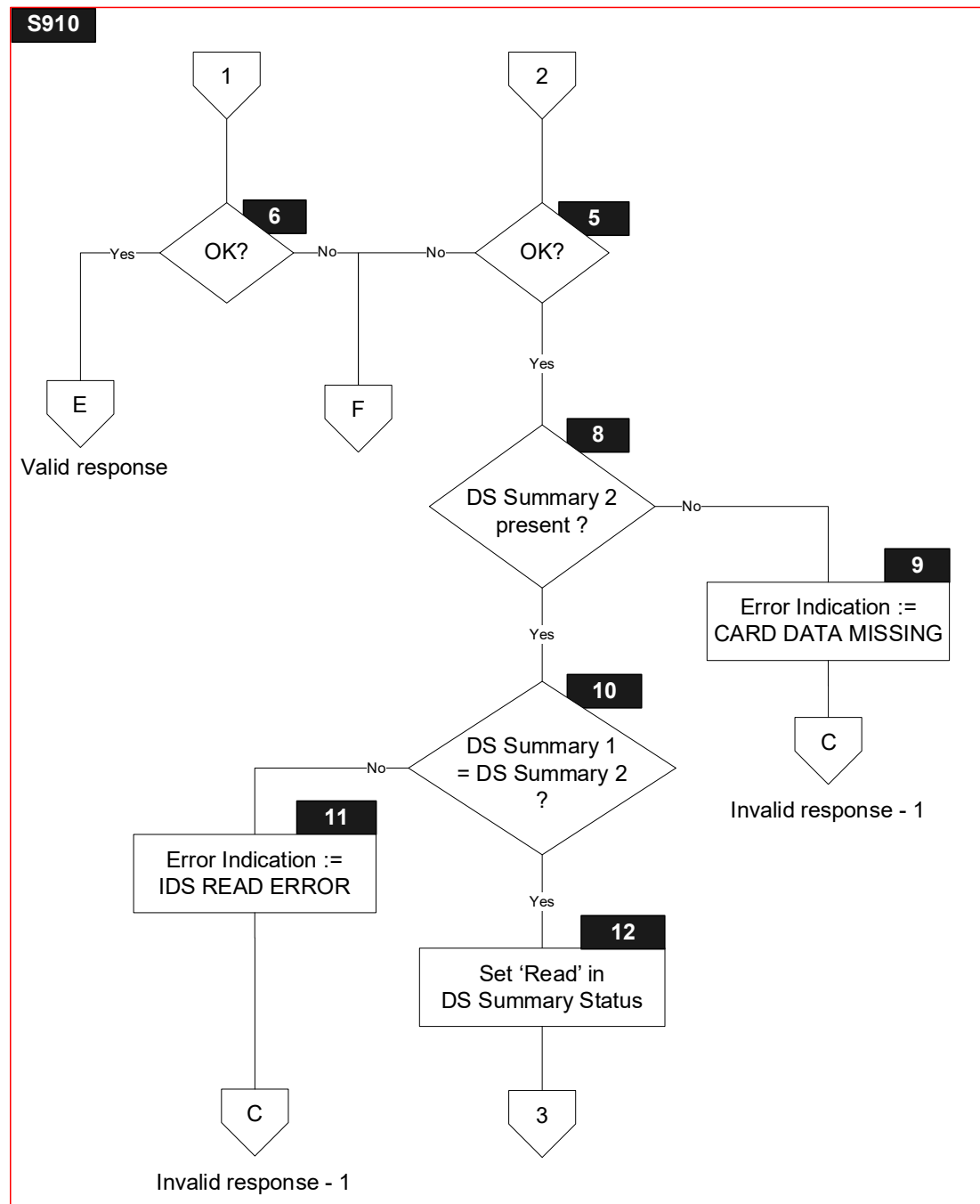
Figure 6.11: State 9 Flow Diagram

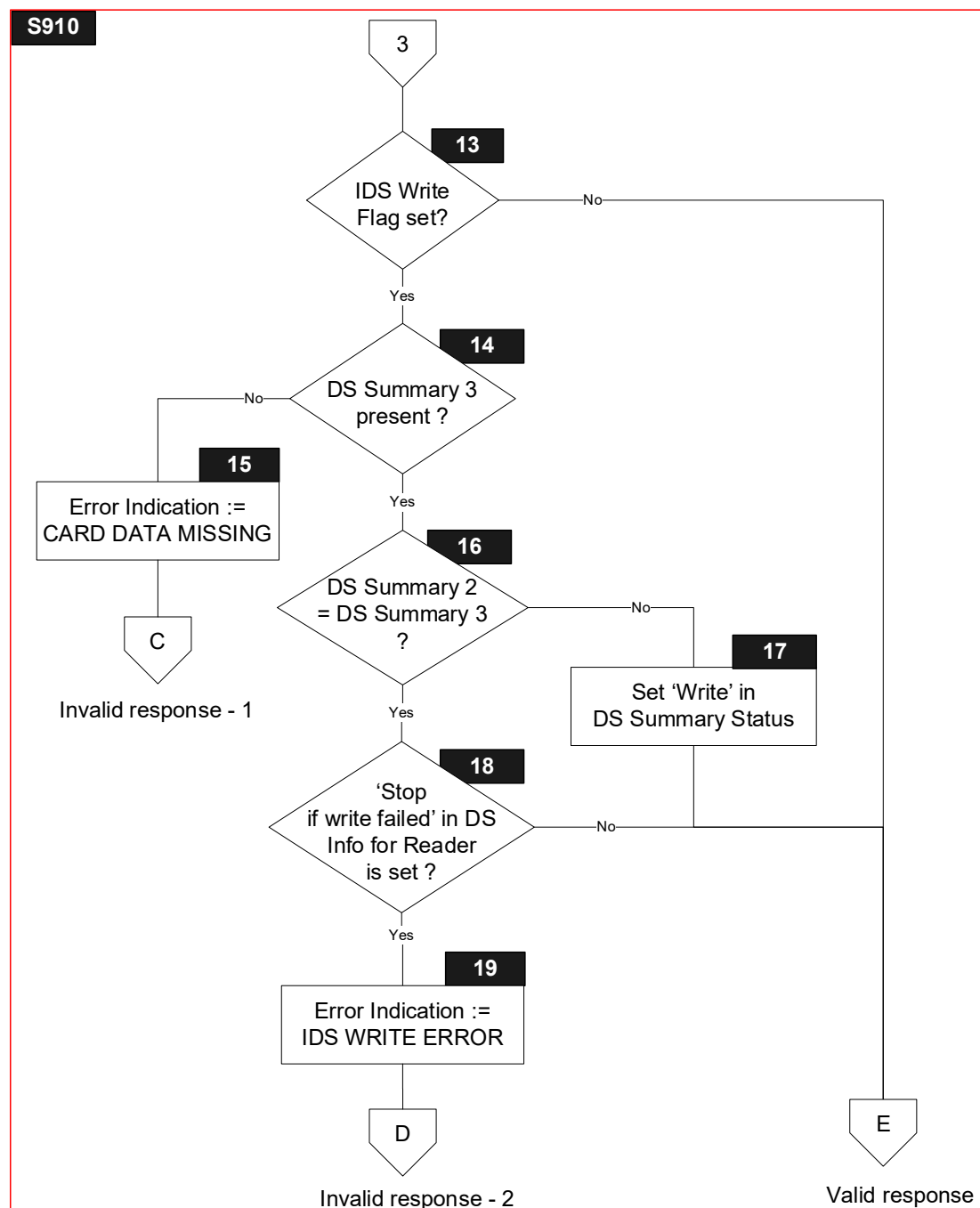




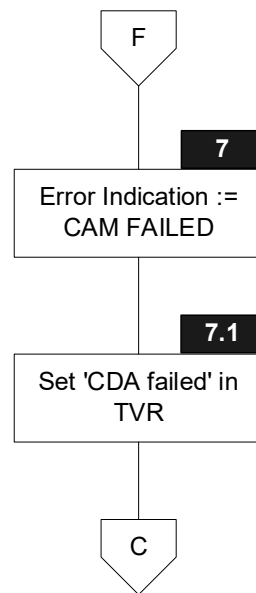




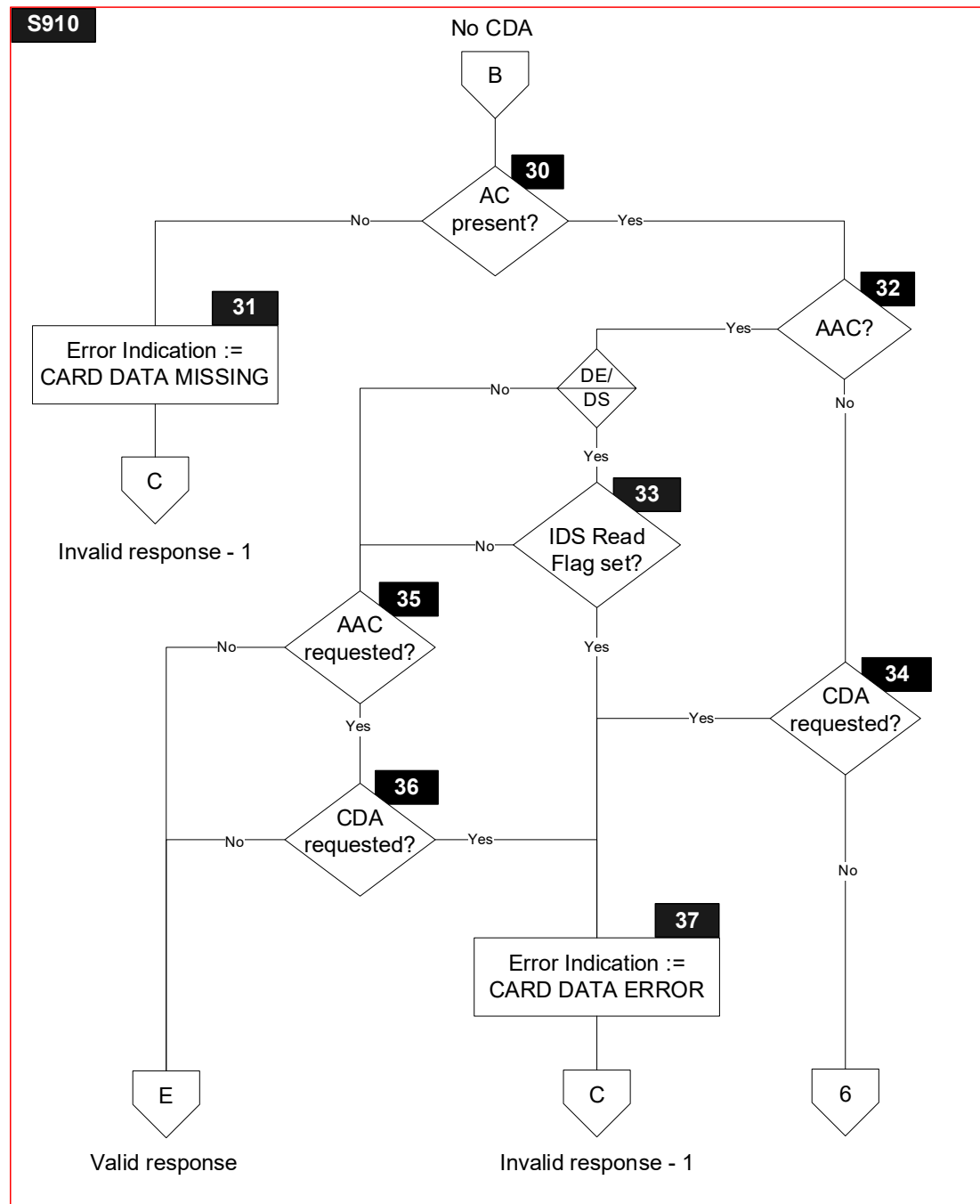


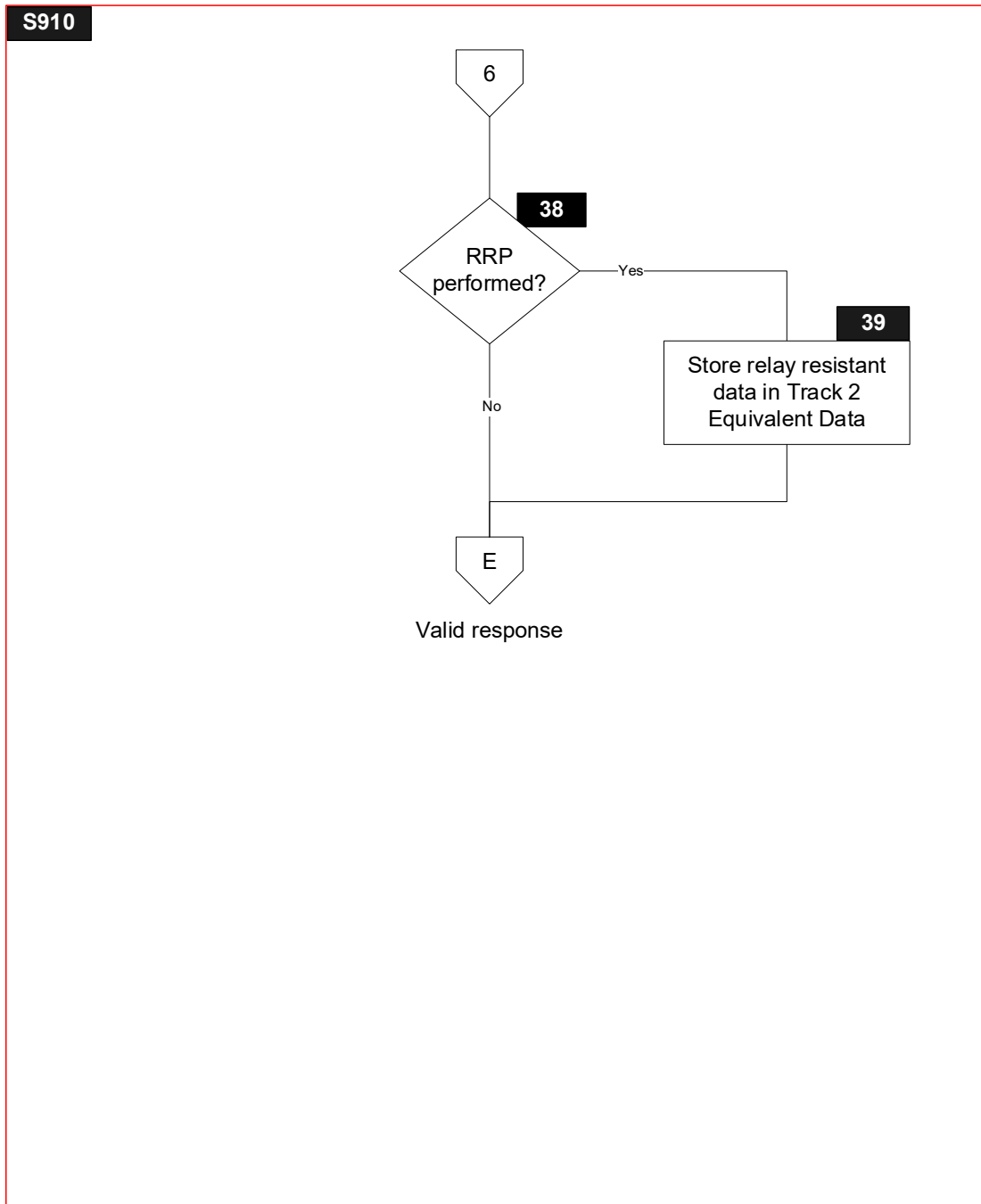


S910



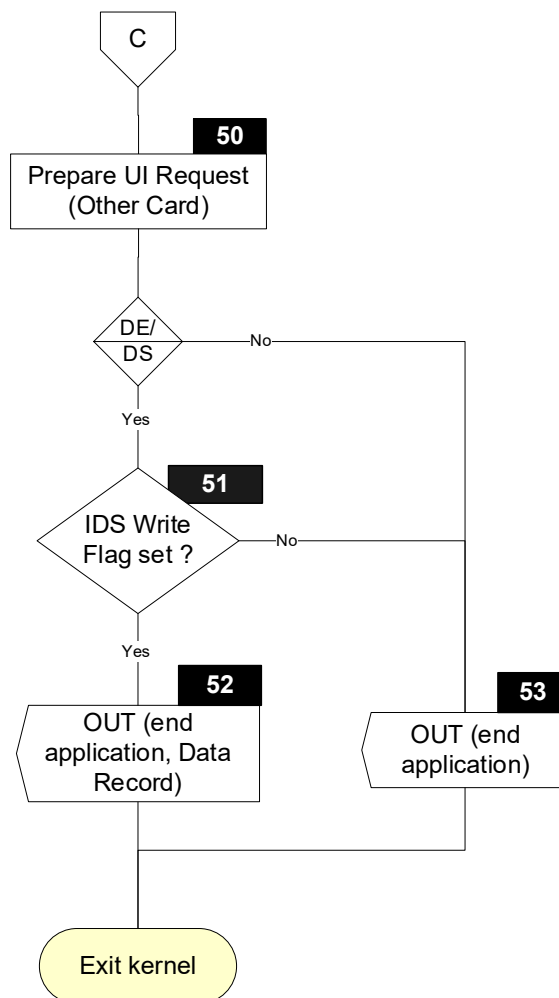
Invalid response - 1

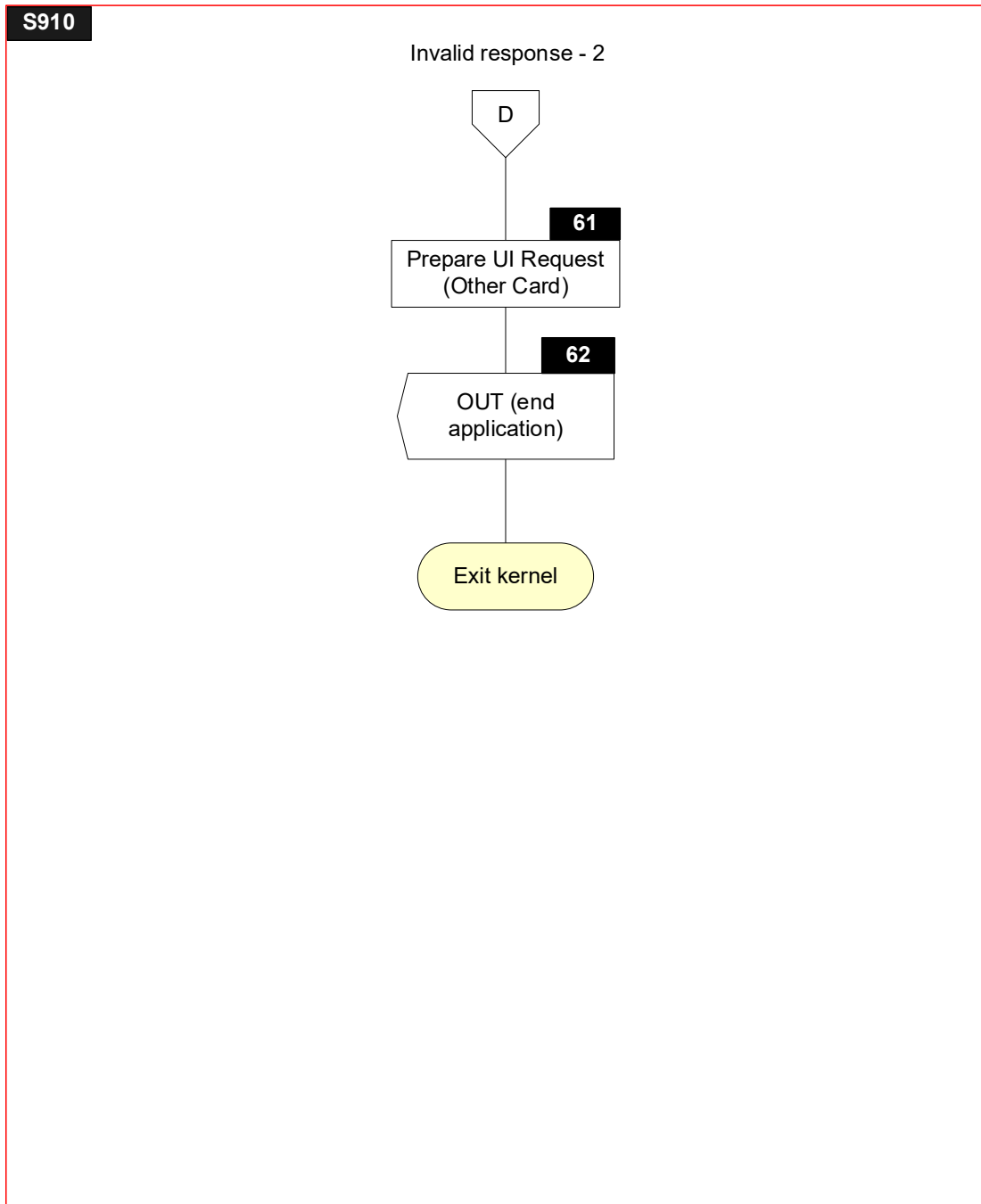


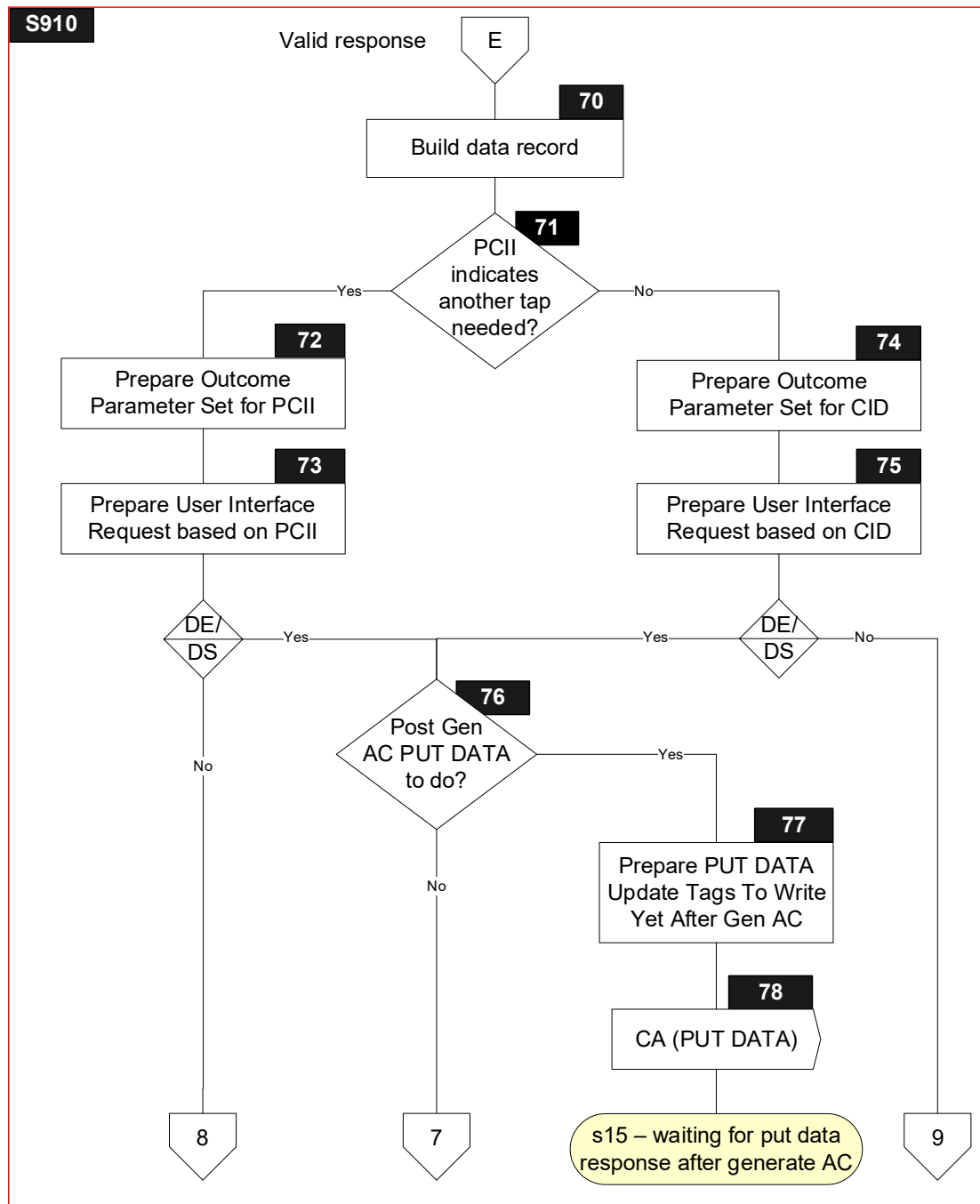


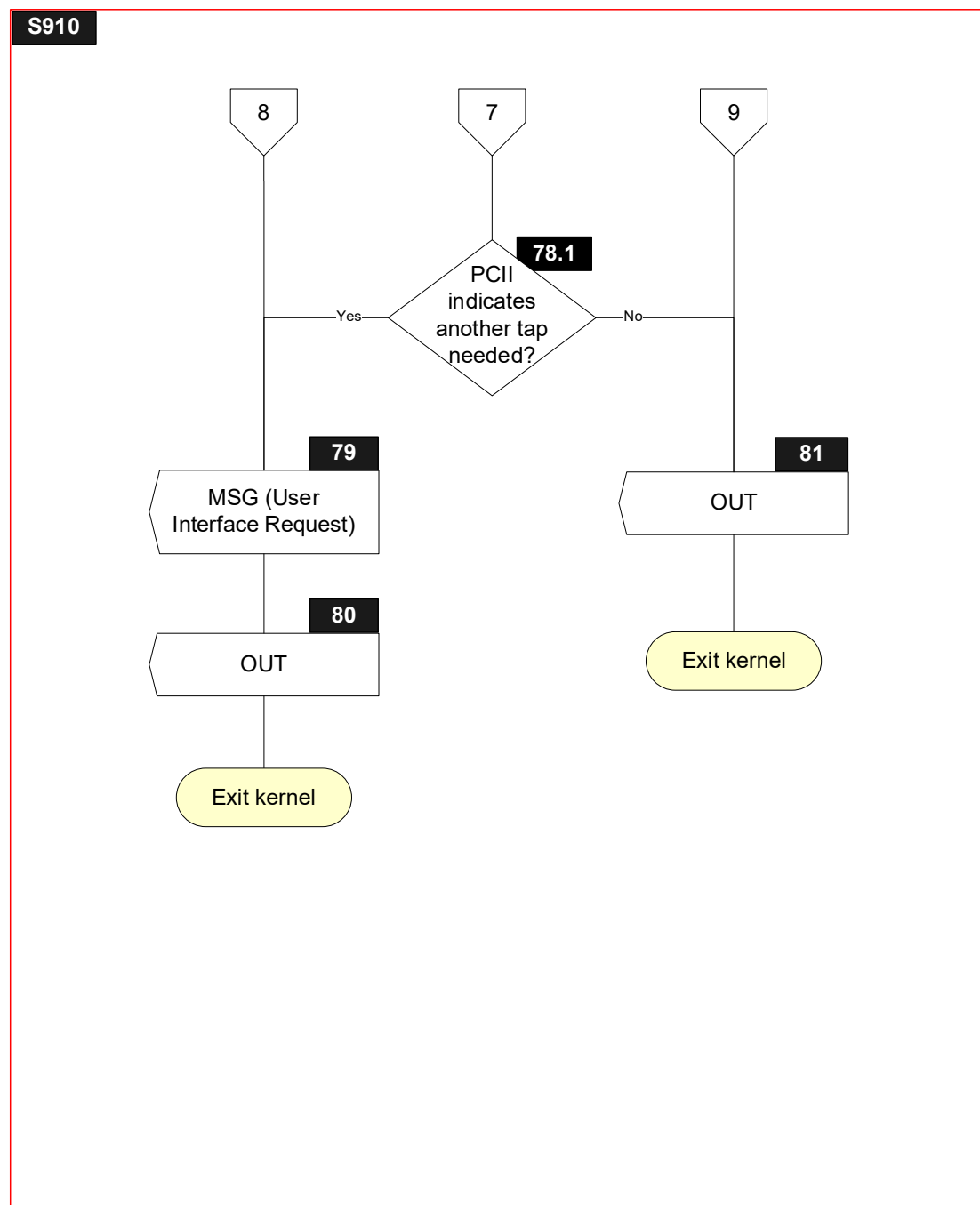
S910

Invalid response - 1









6.12.3 Processing

Note that symbols S9.6, S9.7, S9.8, S9.26, S910.2, S910.2.2, S910.3, S910.3.1, S910.5, S910.8, S910.9, S910.10, S910.11, S910.12, S910.13, S910.14, S910.15, S910.16, S910.17, S910.18, S910.19, S910.61, S910.62, S910.76, S910.77, S910.78 and S910.78.1 are only implemented for the DE/DS Implementation Option.

S9.1

Receive L1RSP Signal with Return Code

S9.2

Receive RA Signal with Response Message Data Field and SW12

S9.4

Receive DET Signal

S9.6

```
IF      ['Write' in IDS Status is set]
THEN
    GOTO S9.7
ELSE
    GOTO S9.9
ENDIF
```

S9.7

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD
'Status' in User Interface Request Data := NOT READY

S9.8

```
'Status' in Outcome Parameter Set := END APPLICATION
'Msg On Error' in Error Indication := ERROR – OTHER CARD
'L1' in Error Indication := Return Code
SET 'Data Record Present' in Outcome Parameter Set
CreateDataRecord ()
CreateDiscretionaryData ()
SET 'UI Request on Outcome Present' in Outcome Parameter Set
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
        GetTLV(TagOf(Data Record)),
        GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
        Data))) Signal
```

S9.9

'Message Identifier' in User Interface Request Data := TRY AGAIN
'Status' in User Interface Request Data := READY TO READ
'Hold Time' in User Interface Request Data := '000000'

S9.10

'Status' in Outcome Parameter Set := END APPLICATION
'Start' in Outcome Parameter Set := B
SET 'UI Request on Restart Present' in Outcome Parameter Set
'L1' in Error Indication := Return Code
'Msg On Error' in Error Indication:= TRY AGAIN
CreateDiscretionaryData ()
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

S9.16

IF [SW12 = '9000']
THEN
 GOTO S9.18
ELSE
 GOTO S9.17
ENDIF

S9.17

'L2' in Error Indication := STATUS BYTES
'SW12' in Error Indication := SW12

S9.18

Parsing Result := FALSE

IF [(Length of Response Message Data Field > 0) AND
(Response Message Data Field[1] = '77')]

THEN

Parsing Result := ParseAndStoreCardResponse(Response Message Data Field)

ELSE

IF [(Length of Response Message Data Field > 0) AND
(Response Message Data Field[1] = '80')]

THEN

Parse the Response Message Data Field according to section 5.3.3 to retrieve the value field

IF [Response Message Data Field does not parse correctly OR
The length of the value field of the Response Message Data Field is less than 11 OR
The length of the value field of the Response Message Data Field is greater than 43 OR
IsEmpty(TagOf(Cryptogram Information Data)) OR
IsEmpty(TagOf(Application Transaction Counter)) OR
IsEmpty(TagOf(Application Cryptogram)) OR
(The length of the value field of the Response Message Data Field is greater than 11 AND
IsEmpty(TagOf(Issuer Application Data)))]

THEN

Parsing Result := FALSE

ELSE

Store the first byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Cryptogram Information Data).

Store from the second up to the third byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Application Transaction Counter).

Store from the fourth up to the eleventh byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Application Cryptogram).

If the length of the value field of the Response Message Data Field is greater than 11, then store from the twelfth up to the last byte of the value field of Response Message Data Field in the TLV Database for tag TagOf(Issuer Application Data).

Parsing Result := TRUE

ENDIF

ENDIF

ENDIF

S9.19

```
IF      [Parsing Result]
THEN
    GOTO S9.21
ELSE
    GOTO S9.20
ENDIF
```

S9.20

'L2' in Error Indication := PARSING ERROR

S9.21

```
IF      [IsEmpty(TagOf(Application Transaction Counter))
        AND IsNotEmpty(TagOf(Cryptogram Information Data))]
THEN
    GOTO S9.23
ELSE
    GOTO S9.22
ENDIF
```

S9.22

'L2' in Error Indication := CARD DATA MISSING

S9.23

```
IF      [((Cryptogram Information Data AND 'C0' = '40') AND
        ('AC type' in Reference Control Parameter = TC))
        OR
        ((Cryptogram Information Data AND 'C0' = '80')
        AND
        (('AC type' in Reference Control Parameter = TC) OR
        ('AC type' in Reference Control Parameter = ARQC)))
        OR
        (Cryptogram Information Data AND 'C0' = '00')]
THEN
    GOTO S9.26 or S9.27
ELSE
    GOTO S9.24
ENDIF
```

S9.24

'L2' in Error Indication := CARD DATA ERROR

S9.26

```
IF      [IsEmptyList(Tags To Write Yet After Gen AC)]
THEN
    GOTO S9.28
ELSE
    GOTO S9.27
ENDIF
```

S9.27

'Message Identifier' in User Interface Request Data := CLEAR DISPLAY
'Status' in User Interface Request Data := CARD READ SUCCESSFULLY
'Hold Time' in User Interface Request Data := '000000'
Send MSG(User Interface Request Data) Signal

S9.28

```
IF      [IsEmpty(TagOf(Signed Dynamic Application Data))]
THEN
    GOTO S910.1
ELSE
    GOTO S910.30
ENDIF
```

CDA

S910.1

Retrieve with the CA Public Key Index (Card) the Certification Authority Public Key Modulus and Exponent and associated key related information, and the corresponding algorithm to be used from the CA Public Key Database (see section 4.4.2).

Retrieve the Issuer Public Key and ICC Public Key as described in section 6.3 and 6.4 of [EMV Book 2].

Check if the concatenation of the CA Public Key Index (Card) and the Certificate Serial Number recovered from the Issuer Public Key Certificate appears in the CRL. If this is the case, then ICC Public Key retrieval is not successful.

```
IF      [ICC Public Key retrieval was successful]
THEN
    GOTO S910.2 or S910.2.1
ELSE
    GOTO S910.7
ENDIF
```


S910.2

```
IF      ['Read' in IDS Status is set]
THEN
    GOTO S910.2.2
ELSE
    GOTO S910.2.1
ENDIF
```

S910.2.1

```
IF      ['Relay resistance performed' in Terminal Verification Results = RRP PERFORMED]
THEN
    GOTO S910.4.1
ELSE
    GOTO S910.4
ENDIF
```

S910.2.2

```
IF      ['Relay resistance performed' in Terminal Verification Results = RRP PERFORMED]
THEN
    GOTO S910.3.1
ELSE
    GOTO S910.3
ENDIF
```

S910.3

Verify Signed Dynamic Application Data as in section 6.6 of [EMV Book 2]. Use PDOL Values for the values of the data elements specified by, and in the order they appear in the PDOL.

If the length of ICC Dynamic Data is less than 30 + Length of ICC Dynamic Number bytes, then CDA fails.

Retrieve from the ICC Dynamic Data (see Table 6.4) the ICC Dynamic Number, Application Cryptogram, DS Summary 2 and DS Summary 3 and store in the TLV Database.

If the ICC Dynamic Data does not include DS Summary 3 (i.e. there are less than 16 bytes after Hash Result (if 'Data Storage Version Number' in Application Capabilities Information = VERSION 1) or less than 32 bytes after Hash Result (if 'Data Storage Version Number' in Application Capabilities Information = VERSION 2)), then do not store DS Summary 3. This is not a reason to fail CDA.

If the ICC Dynamic Data also does not include DS Summary 2 (i.e. there are less than 8 bytes after Hash Result (if 'Data Storage Version Number' in Application Capabilities Information = VERSION 1) or less than 16 bytes after Hash Result (if 'Data Storage Version Number' in Application Capabilities Information = VERSION 2)), then do not store DS Summary 2. This is not a reason to fail CDA.

Table 6.4: ICC Dynamic Data (IDS)

Value	Length
Length of ICC Dynamic Number	1
ICC Dynamic Number	2-8
Cryptogram Information Data	1
Application Cryptogram	8
Hash Result	20
DS Summary 2	8 or 16
DS Summary 3	8 or 16

S910.3.1

Verify Signed Dynamic Application Data as in section 6.6 of [EMV Book 2]. Use PDOL Values for the values of the data elements specified by, and in the order they appear in the PDOL.

If the length of ICC Dynamic Data is less than 60 + Length of ICC Dynamic Number bytes (if 'Data Storage Version Number' in Application Capabilities Information = VERSION 1) or less than 76 + Length of ICC Dynamic Number bytes (if 'Data Storage Version Number' in Application Capabilities Information = VERSION 2)), then CDA fails.

Retrieve from the ICC Dynamic Data (see Table 6.5) the ICC Dynamic Number, Application Cryptogram, DS Summary 2 and DS Summary 3 and store in the TLV Database.

Check if the relay resistance related data objects in the ICC Dynamic Data match the corresponding data objects in the TLV Database as follows:

Terminal Relay Resistance Entropy = Terminal Relay Resistance Entropy (CDA) and Device Relay Resistance Entropy = Device Relay Resistance Entropy (CDA) and

Min Time For Processing Relay Resistance APDU = Min Time For Processing Relay Resistance APDU (CDA) and

Max Time For Processing Relay Resistance APDU = Max Time For Processing Relay Resistance APDU (CDA) and

Device Estimated Transmission Time For Relay Resistance R-APDU = Device Estimated Transmission Time For Relay Resistance R-APDU (CDA).

If this is not the case, then CDA fails.

Table 6.5: ICC Dynamic Data (IDS + RRP)

Value	Length
Length of ICC Dynamic Number	1
ICC Dynamic Number	2-8
Cryptogram Information Data	1
Application Cryptogram	8
Hash Result	20
DS Summary 2	8 or 16
DS Summary 3	8 or 16
Terminal Relay Resistance Entropy (CDA)	4
Device Relay Resistance Entropy (CDA)	4
Min Time For Processing Relay Resistance APDU (CDA)	2
Max Time For Processing Relay Resistance APDU (CDA)	2
Device Estimated Transmission Time For Relay Resistance R-APDU (CDA)	2

S910.4

Verify Signed Dynamic Application Data as in section 6.6 of [EMV Book 2]. Use PDOL Values for the values of the data elements specified by, and in the order they appear in the PDOL.

If the length of ICC Dynamic Data is less than $30 + \text{Length of ICC Dynamic Number bytes}$, then CDA fails.

Retrieve from the ICC Dynamic Data (see Table 6.6) the ICC Dynamic Number and Application Cryptogram and store in the TLV Database.

Table 6.6: ICC Dynamic Data (No IDS)

Value	Length
Length of ICC Dynamic Number	1
ICC Dynamic Number	2-8
Cryptogram Information Data	1
Application Cryptogram	8
Hash Result	20

S910.4.1

Verify Signed Dynamic Application Data as in section 6.6 of [EMV Book 2]. Use PDOL Values for the values of the data elements specified by, and in the order they appear in the PDOL.

If the length of ICC Dynamic Data is less than $44 + \text{Length of ICC Dynamic Number bytes}$, then CDA fails.

Retrieve from the ICC Dynamic Data (see Table 6.7) the ICC Dynamic Number and Application Cryptogram and store in the TLV Database.

Check if the relay resistance related data objects in the ICC Dynamic Data match the corresponding data objects in the TLV Database as follows:

Terminal Relay Resistance Entropy = Terminal Relay Resistance Entropy (CDA) and Device Relay Resistance Entropy = Device Relay Resistance Entropy (CDA) and

Min Time For Processing Relay Resistance APDU = Min Time For Processing Relay Resistance APDU (CDA) and

Max Time For Processing Relay Resistance APDU = Max Time For Processing Relay Resistance APDU (CDA) and

Device Estimated Transmission Time For Relay Resistance R-APDU = Device Estimated Transmission Time For Relay Resistance R-APDU (CDA).

If this is not the case, then CDA fails.

Table 6.7: ICC Dynamic Data (RRP)

Value	Length
Length of ICC Dynamic Number	1
ICC Dynamic Number	2-8
Cryptogram Information Data	1
Application Cryptogram	8
Hash Result	20
Terminal Relay Resistance Entropy (CDA)	4
Device Relay Resistance Entropy (CDA)	4
Min Time For Processing Relay Resistance APDU (CDA)	2
Max Time For Processing Relay Resistance APDU (CDA)	2
Device Estimated Transmission Time For Relay Resistance R-APDU (CDA)	2

S910.5

```

IF      [Signed Dynamic Application Data verification is OK]
THEN
    GOTO S910.8
ELSE
    GOTO S910.7
ENDIF

```

S910.6

```

IF      [Signed Dynamic Application Data verification is OK]
THEN
    GOTO S910.70
ELSE
    GOTO S910.7
ENDIF

```

S910.7

'L2' in Error Indication := CAM FAILED

S910.7.1

SET 'CDA Failed' in Terminal Verification Results

S910.8

```
IF      [IsEmpty(TagOf(DS Summary 2))]  
THEN  
    GOTO S910.10  
ELSE  
    GOTO S910.9  
ENDIF
```

S910.9

'L2' in Error Indication := CARD DATA MISSING

S910.10

```
IF      [DS Summary 1 = DS Summary 2]  
THEN  
    GOTO S910.12  
ELSE  
    GOTO S910.11  
ENDIF
```

S910.11

'L2' in Error Indication := IDS READ ERROR

S910.12

SET 'Successful Read' in DS Summary Status

S910.13

```
IF      ['Write' in IDS Status is set]  
THEN  
    GOTO S910.14  
ELSE  
    GOTO S910.70  
ENDIF
```

S910.14

```
IF      [IsPresent(TagOf(DS Summary 3))]  
THEN  
    GOTO S910.16  
ELSE  
    GOTO S910.15  
ENDIF
```

S910.15

'L2' in Error Indication := CARD DATA MISSING

S910.16

```
IF      [DS Summary 2 = DS Summary 3]
THEN
    GOTO S910.18
ELSE
    GOTO S910.17
ENDIF
```

S910.17

SET 'Successful Write' in DS Summary Status

S910.18

```
IF      ['Stop if write failed' in DS ODS Info For Reader is set]
THEN
    GOTO S910.19
ELSE
    GOTO S910.70
ENDIF
```

S910.19

'L2' in Error Indication := IDS WRITE ERROR

No CDA

S910.30

```
IF      [IsEmpty(TagOf(Application Cryptogram))]  
THEN  
    GOTO S910.32  
ELSE  
    GOTO S910.31  
ENDIF
```

S910.31

'L2' in Error Indication := CARD DATA MISSING

S910.32

```
IF      [(Cryptogram Information Data AND 'C0') = '00']  
THEN  
    GOTO S910.33 or S910.35  
ELSE  
    GOTO S910.34  
ENDIF
```

S910.33

```
IF      ['Read' in IDS Status is set]  
THEN  
    GOTO S910.37  
ELSE  
    GOTO S910.35  
ENDIF
```

S910.34

```
IF      ['CDA signature requested' in Reference Control Parameter is set]  
THEN  
    GOTO S910.37  
ELSE  
    GOTO S910.38  
ENDIF
```

S910.35

```
IF      ['AC type' in Reference Control Parameter = AAC]  
THEN  
    GOTO S910.36  
ELSE  
    GOTO S910.70  
ENDIF
```


S910.36

```
IF      ['CDA signature requested' in Reference Control Parameter is set]
THEN
    GOTO S910.37
ELSE
    GOTO S910.70
ENDIF
```

S910.37

'L2' in Error Indication := CARD DATA ERROR

S910.38

```
IF      ['Relay resistance performed' in Terminal Verification Results = RRP PERFORMED]
THEN
    GOTO S910.39
ELSE
    GOTO S910.70
ENDIF
```

S910.39

```
IF      [IsEmpty(TagOf(Track 2 Equivalent Data))]
THEN
    IF      [Number of digits in 'Primary Account Number' in Track 2 Equivalent Data ≤
        16]
    THEN
        Replace 'Discretionary Data' in Track 2 Equivalent Data with
        '00000000000000' (13 hexadecimal zeroes). Pad with 'F' if needed to ensure
        whole bytes.
    ELSE
        Replace 'Discretionary Data' in Track 2 Equivalent Data with '0000000000'
        (10 hexadecimal zeroes). Pad with 'F' if needed to ensure whole bytes.
    ENDIF
    IF      [IsEmpty(TagOf(CA Public Key Index (Card))) AND
        CA Public Key Index (Card) < '0A']
    THEN
        Replace the most significant digit of the 'Discretionary Data' in Track 2
        Equivalent Data with a digit representing CA Public Key Index (Card).
    ENDIF
    Replace the second most significant digit of the 'Discretionary Data' in Track 2
    Equivalent Data with a digit representing RRP Counter.
    Convert the two least significant bytes of the Device Relay Resistance Entropy from 2
    byte binary to 5 digit decimal by considering the two bytes as an integer in the range 0
    to 65535. Replace the 5 digits of 'Discretionary Data' in Track 2 Equivalent Data that
    follow the RRP Counter digit with that value.
```

IF [Number of digits in 'Primary Account Number' in Track 2 Equivalent Data $\leq$ 16]

THEN

Convert the third least significant byte of Device Relay Resistance Entropy from binary to 3 digit decimal in the range 0 to 255. Replace the next 3 digits of 'Discretionary Data' in Track 2 Equivalent Data with that value.

ENDIF

Divide the Measured Relay Resistance Processing Time by 10 using the div operator to give a count in milliseconds. If the value exceeds '03E7' (999), then set the value to '03E7'. Convert this value from 2 byte binary to 3 digit decimal by considering the 2 bytes as an integer. Replace the 3 least significant digits of 'Discretionary Data' in Track 2 Equivalent Data with this 3 digit decimal value.

ENDIF

Invalid Response – 1

S910.50

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

'Hold Time' in User Interface Request Data := Message Hold Time

S910.51

IF ['Write' in IDS Status is set]

THEN

GOTO S910.52

ELSE

GOTO S910.53

ENDIF

S910.52

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

SET 'Data Record Present' in Outcome Parameter Set

CreateDataRecord ()

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Data Record)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

S910.53

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)), GetTLV(TagOf(Discretionary Data)),
GetTLV(TagOf(User Interface Request Data))) Signal

Invalid Response – 2

S910.61

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

'Hold Time' in User Interface Request Data := Message Hold Time

S910.62

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
 Data))) Signal

Valid Response

S910.70

SET 'Data Record Present' in Outcome Parameter Set

CreateDataRecord ()

S910.71

IF [IsEmpty(TagOf(POS Cardholder Interaction Information)) AND
(POS Cardholder Interaction Information AND '00030F' ≠ '000000')]

THEN

GOTO S910.72

ELSE

GOTO S910.74

ENDIF

S910.72

'Status' in Outcome Parameter Set := END APPLICATION

'Start' in Outcome Parameter Set := B

S910.73

FOR every entry in the Phone Message Table

{

IF [(PCII Mask AND POS Cardholder Interaction Information) =
PCII Value]

THEN

'Hold Time' in User Interface Request Data := Message Hold Time

'Message Identifier' in User Interface Request Data := Message Identifier

'Status' in User Interface Request Data := Status

EXIT loop

ENDIF

}

S910.74

```
IF      [(Cryptogram Information Data AND 'C0') = '40']
THEN
    'Status' in Outcome Parameter Set := APPROVED
ELSE
    IF [(Cryptogram Information Data AND 'C0') = '80']
    THEN
        'Status' in Outcome Parameter Set := ONLINE REQUEST
    ELSE
        Check if Transaction Type indicates a cash transaction (cash withdrawal or
        cash disbursement) or a purchase transaction (purchase or purchase with
        cashback).
        IF      [Transaction Type = '01' OR Transaction Type = '17' OR
        Transaction Type = '00' OR Transaction Type = '09']
        THEN
            IF      [(IsEmpty(TagOf(Third Party Data)) AND
            ('Unique Identifier' in Third Party Data AND '8000' = '0000')
            AND
            ('Device Type' in Third Party Data ≠ '3030'))
            OR
            ('IC with contacts' in Terminal Capabilities is not set)]
            THEN
                'Status' in Outcome Parameter Set := DECLINED
            ELSE
                'Status' in Outcome Parameter Set := TRY ANOTHER
                INTERFACE
            ENDIF
        ELSE
            'Status' in Outcome Parameter Set := END APPLICATION
        ENDIF
    ENDIF
ENDIF
```

S910.75

```
'Status' in User Interface Request Data := NOT READY
IF    [(Cryptogram Information Data AND 'C0') = '40']
THEN
    'Hold Time' in User Interface Request Data := Message Hold Time
    IF    ['CVM' in Outcome Parameter Set = OBTAIN SIGNATURE]
    THEN
        'Message Identifier' in User Interface Request Data := APPROVED – SIGN
    ELSE
        'Message Identifier' in User Interface Request Data := APPROVED
    ENDIF
ELSE
    IF    [(Cryptogram Information Data AND 'C0') = '80']
    THEN
        'Hold Time' in User Interface Request Data := '000000'
        'Message Identifier' in User Interface Request Data := AUTHORISING –
        PLEASE WAIT
    ELSE
        Check if Transaction Type indicates a cash transaction (cash withdrawal or
        cash disbursement) or a purchase transaction (purchase or purchase with
        cashback).
        IF    [Transaction Type = '01' OR Transaction Type = '17' OR
            Transaction Type = '00' OR Transaction Type = '09']
        THEN
            'Hold Time' in User Interface Request Data := Message Hold Time
            IF    [(IsEmpty(TagOf(Third Party Data)) AND
                ('Unique Identifier' in Third Party Data AND '8000' = '0000')
                AND
                ('Device Type' in Third Party Data ≠ '3030'))
                OR
                ('IC with contacts' in Terminal Capabilities is not set) ]
            THEN
                'Message Identifier' in User Interface Request Data :=
                DECLINED
            ELSE
                'Message Identifier' in User Interface Request Data :=
                INSERT CARD
            ENDIF
        ENDIF
    ENDIF
ENDIF
```

```
        ELSE
            'Hold Time' in User Interface Request Data := '000000'
            'Message Identifier' in User Interface Request Data :=
                CLEAR DISPLAY
        ENDIF
    ENDIF
ENDIF
```

S910.76

```
IF      [IsNotEmptyList(Tags To Write Yet After Gen AC)]
THEN
    GOTO S910.77
ELSE
    GOTO S910.78.1
ENDIF
```

S910.77

TLV = GetAndRemoveFromList(Tags To Write Yet After Gen AC)
Prepare the PUT DATA command with TLV as defined in section 5.6

S910.78

Send CA(PUT DATA command) Signal

S910.78.1

```
IF      [IsNotEmpty(TagOf(POS Cardholder Interaction Information)) AND
        (POS Cardholder Interaction Information AND '00030F' ≠ '000000')]
THEN
    GOTO S910.79
ELSE
    GOTO S910.81
ENDIF
```

S910.79

Send MSG(User Interface Request Data) Signal

S910.80

CreateDiscretionaryData ()

SET 'UI Request on Restart Present' in Outcome Parameter Set

'Status' in User Interface Request Data := READY TO READ

'Hold Time' in User Interface Request Data := '000000'

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Data Record)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

S910.81

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Data Record)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

6.13 State 12 – Waiting for Put Data Response Before Generate AC

State 12 is a state specific to SDS processing and is only implemented for the DE/DS Implementation Option.

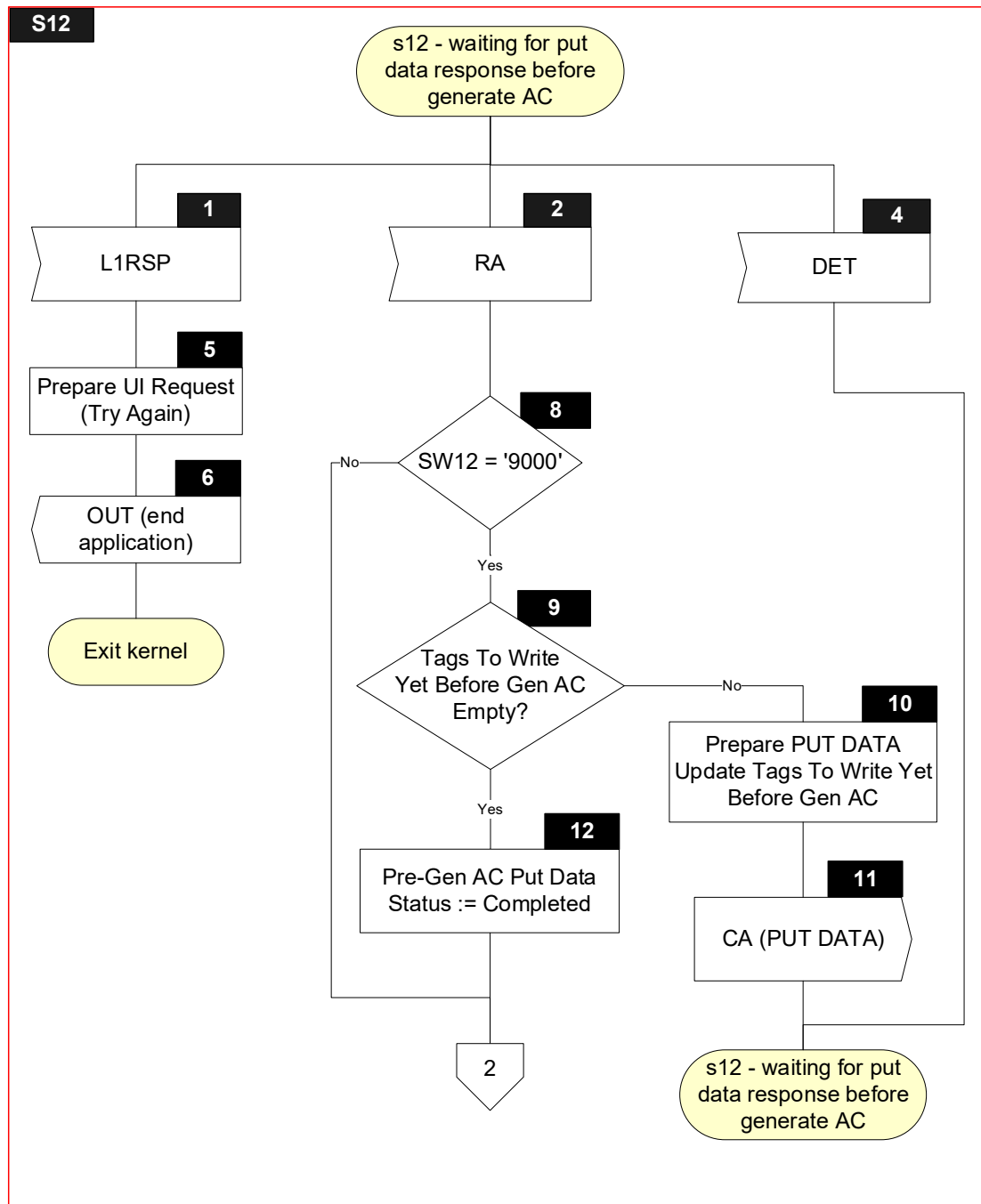
6.13.1 Local Variables

Name	Length	Format	Description
Return Code	1	b	Value returned with L1RSP Signal (TIME OUT ERROR, PROTOCOL ERROR, TRANSMISSION ERROR)
SW12	2	b	Status bytes
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 252	b	Value of TLV encoded string

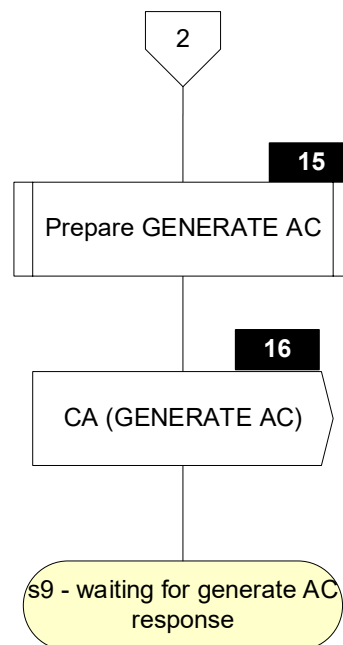
6.13.2 Flow Diagram

Figure 6.12 shows the flow diagram of s12 – waiting for put data response before generate AC. Symbols in this diagram are labelled S12.X.

Figure 6.12: State 12 Flow Diagram



S12



6.13.3 Processing

S12.1

Receive L1RSP Signal with Return Code

S12.2

Receive RA Signal with SW12

S12.4

Receive DET Signal

S12.5

'Message Identifier' in User Interface Request Data := TRY AGAIN

'Status' in User Interface Request Data := READY TO READ

'Hold Time' in User Interface Request Data := '000000'

S12.6

'Status' in Outcome Parameter Set := END APPLICATION

'Start' in Outcome Parameter Set := B

SET 'UI Request on Restart Present' in Outcome Parameter Set

'L1' in Error Indication := Return Code

'Msg On Error' in Error Indication:= TRY AGAIN

CreateDiscretionaryData ()

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

S12.7

'Status' in Outcome Parameter Set := END APPLICATION

'L3' in Error Indication := STOP

CreateDiscretionaryData ()

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Discretionary Data))) Signal

S12.8

IF [SW12 = '9000']

THEN

GOTO S12.9

ELSE

GOTO S12.15

ENDIF

S12.9

```
IF      [IsEmptyList(Tags To Write Yet Before Gen AC)]  
THEN  
    GOTO S12.12  
ELSE  
    GOTO S12.10  
ENDIF
```

S12.10

TLV := GetAndRemoveFromList(Tags To Write Yet Before Gen AC)
Prepare PUT DATA command for TLV as specified in section 5.6

S12.11

Send CA(PUT DATA command) Signal

S12.12

SET 'Completed' in Pre-Gen AC Put Data Status

S12.15

Prepare GENERATE AC command as specified in section 7.2

S12.16

Send CA(GENERATE AC) Signal

6.14 State 15 – Waiting for Put Data Response After Generate AC

State 15 is a state specific to SDS processing and is only implemented for the DE/DS Implementation Option.

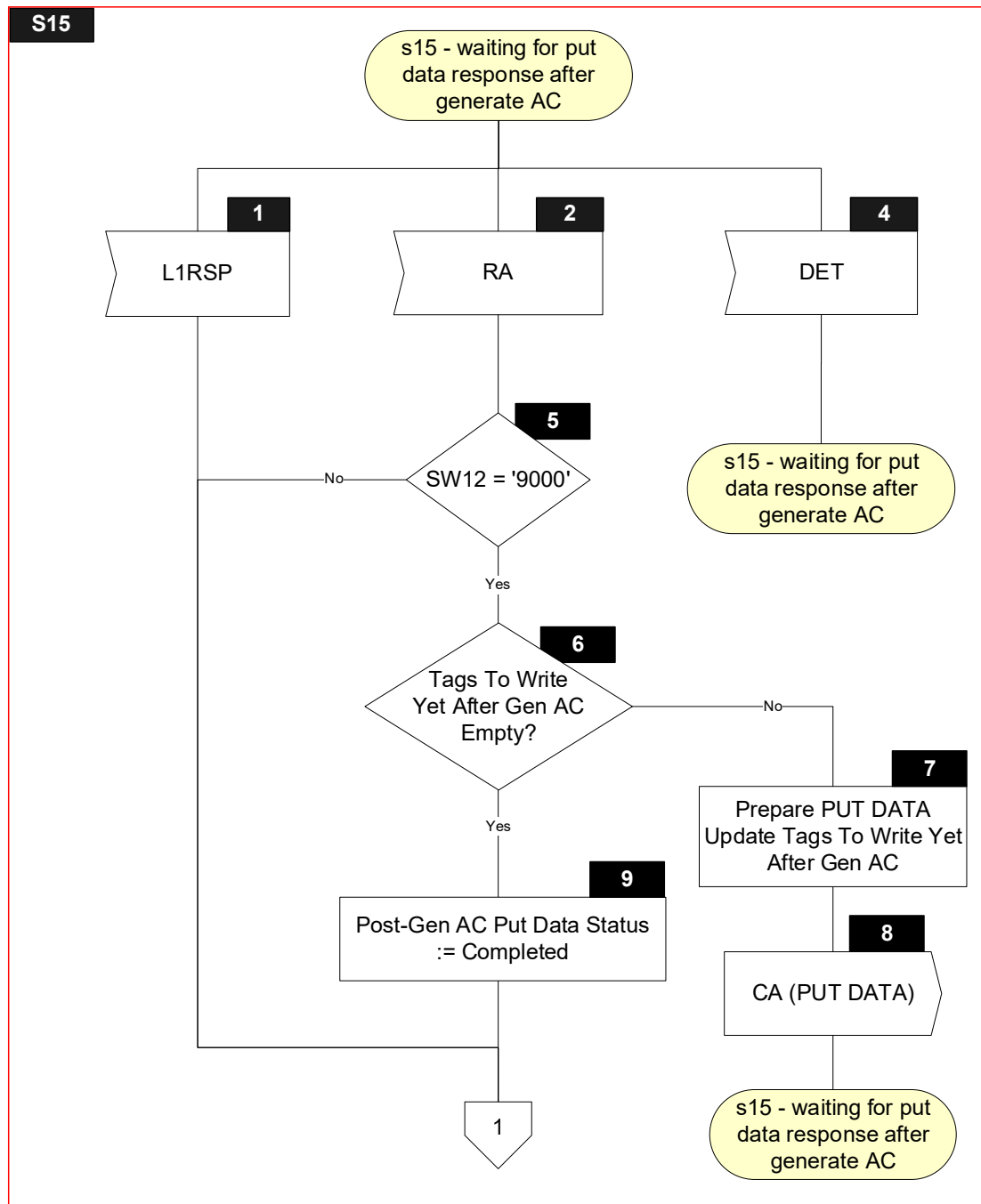
6.14.1 Local Variables

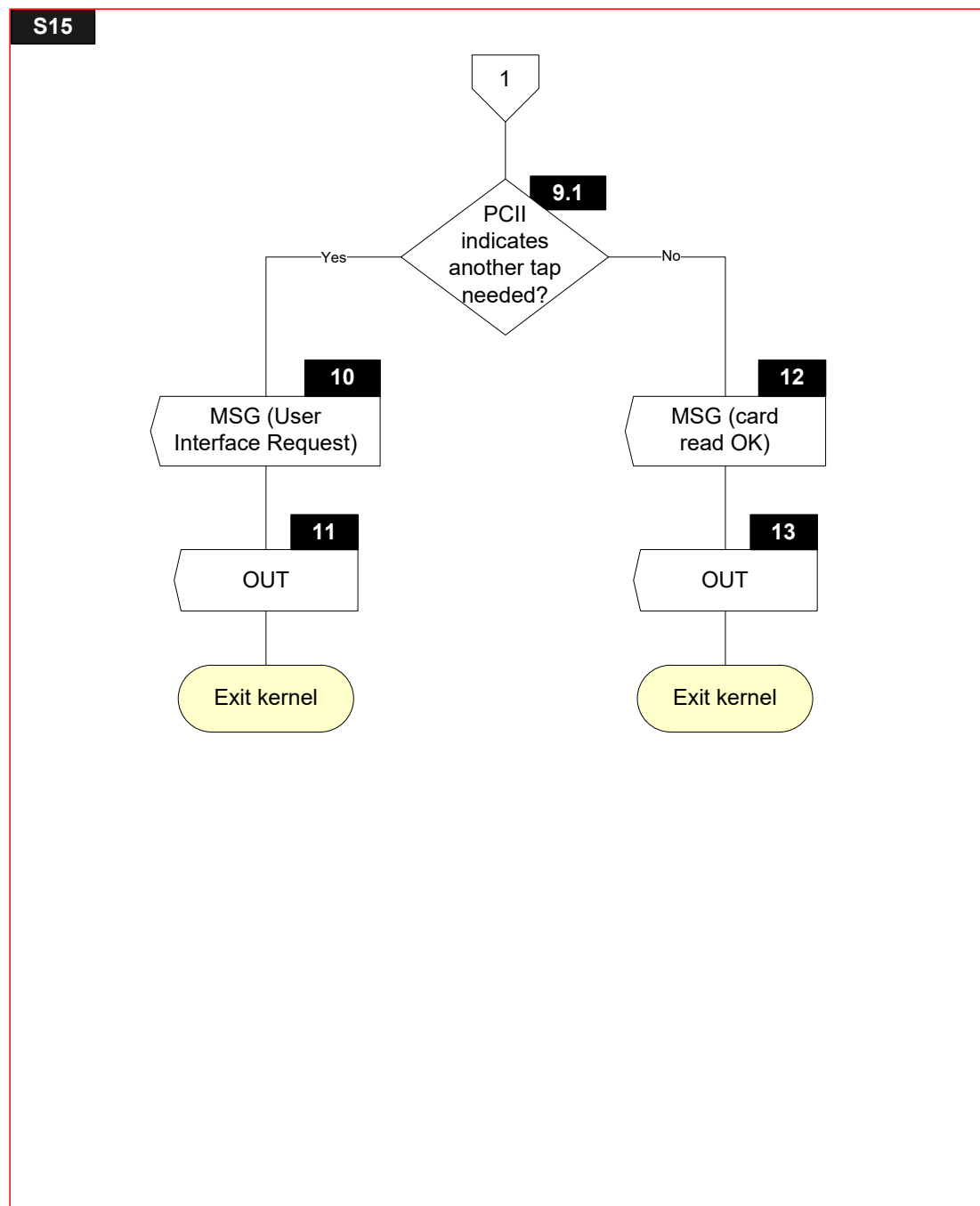
Name	Length	Format	Description
SW12	2	b	Status bytes
T	var.	b	Tag of TLV encoded string
L	var.	b	Length of TLV encoded string
V	var. up to 252	b	Value of TLV encoded string
Tmp UI Request Data	22	b	Local User Interface Request Data to clear display and change status to CARD READ SUCCESSFULLY

6.14.2 Flow Diagram

Figure 6.11 shows the flow diagram of s15 – waiting for put data response after generate AC. Symbols in this diagram are labelled S15.X.

Figure 6.13: State 15 Flow Diagram





6.14.3 Processing

S15.1

Receive L1RSP Signal

S15.2

Receive RA Signal with SW12

S15.4

Receive DET Signal

S15.5

IF [SW12 = '9000']

THEN

GOTO S15.6

ELSE

GOTO S15.10

ENDIF

S15.6

IF [IsEmptyList(Tags To Write Yet After Gen AC)]

THEN

GOTO S15.9

ELSE

GOTO S15.7

ENDIF

S15.7

TLV := GetAndRemoveFromList(Tags To Write Yet After Gen AC)

Prepare PUT DATA command for TLV as specified in section 5.6

S15.8

Send CA(PUT DATA command) Signal

S15.9

SET 'Completed' in Post-Gen AC Put Data Status

S15.9.1

IF [IsNotEmpty(TagOf(POS Cardholder Interaction Information)) AND
(POS Cardholder Interaction Information AND '00030F' ≠ '000000')]

THEN

GOTO S15.10

ELSE

GOTO S15.12

ENDIF

S15.10

'Status' in User Interface Request Data := CARD READ SUCCESSFULLY
Send MSG(User Interface Request Data) Signal

S15.11

CreateDiscretionaryData ()
SET 'UI Request on Restart Present' in Outcome Parameter Set
'Status' in User Interface Request Data := READY TO READ
'Hold Time' in User Interface Request Data := '000000'
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Data Record)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

S15.12

Tmp UI Request Data := '0000 ... 00'
'Message Identifier' in Tmp UI Request Data:= CLEAR DISPLAY
'Status' in Tmp UI Request Data:= CARD READ SUCCESSFULLY
'Hold Time' in Tmp UI Request Data:= '000000'
Send MSG(Tmp UI Request Data) Signal

S15.13

CreateDiscretionaryData ()
SET 'UI Request on Outcome Present' in Outcome Parameter Set
Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
 GetTLV(TagOf(Data Record)),
 GetTLV(TagOf(Discretionary Data)),
 GetTLV(TagOf(User Interface Request Data))) Signal

7 Procedures

7.1 Procedure – CVM Selection

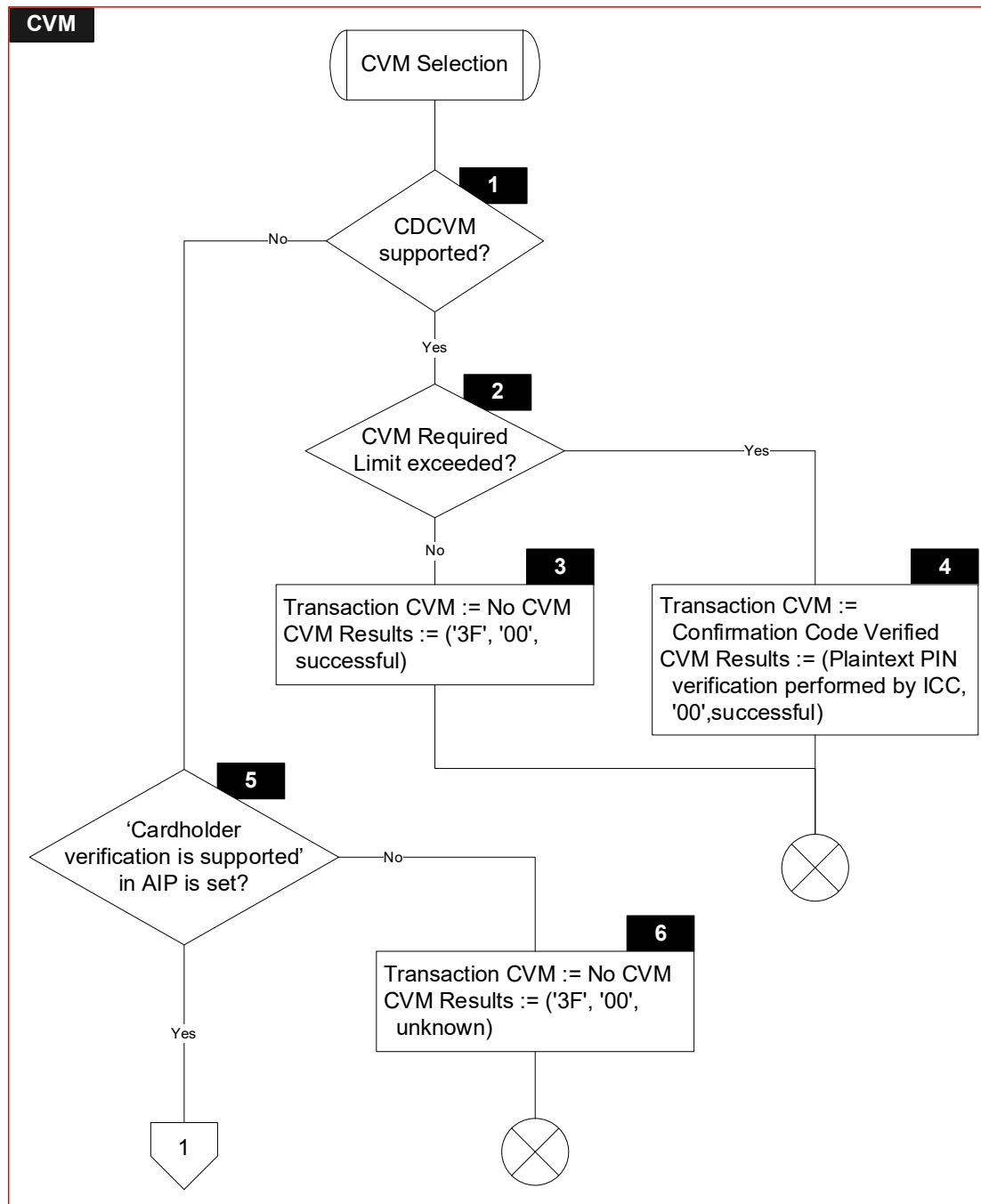
7.1.1 Local Variables

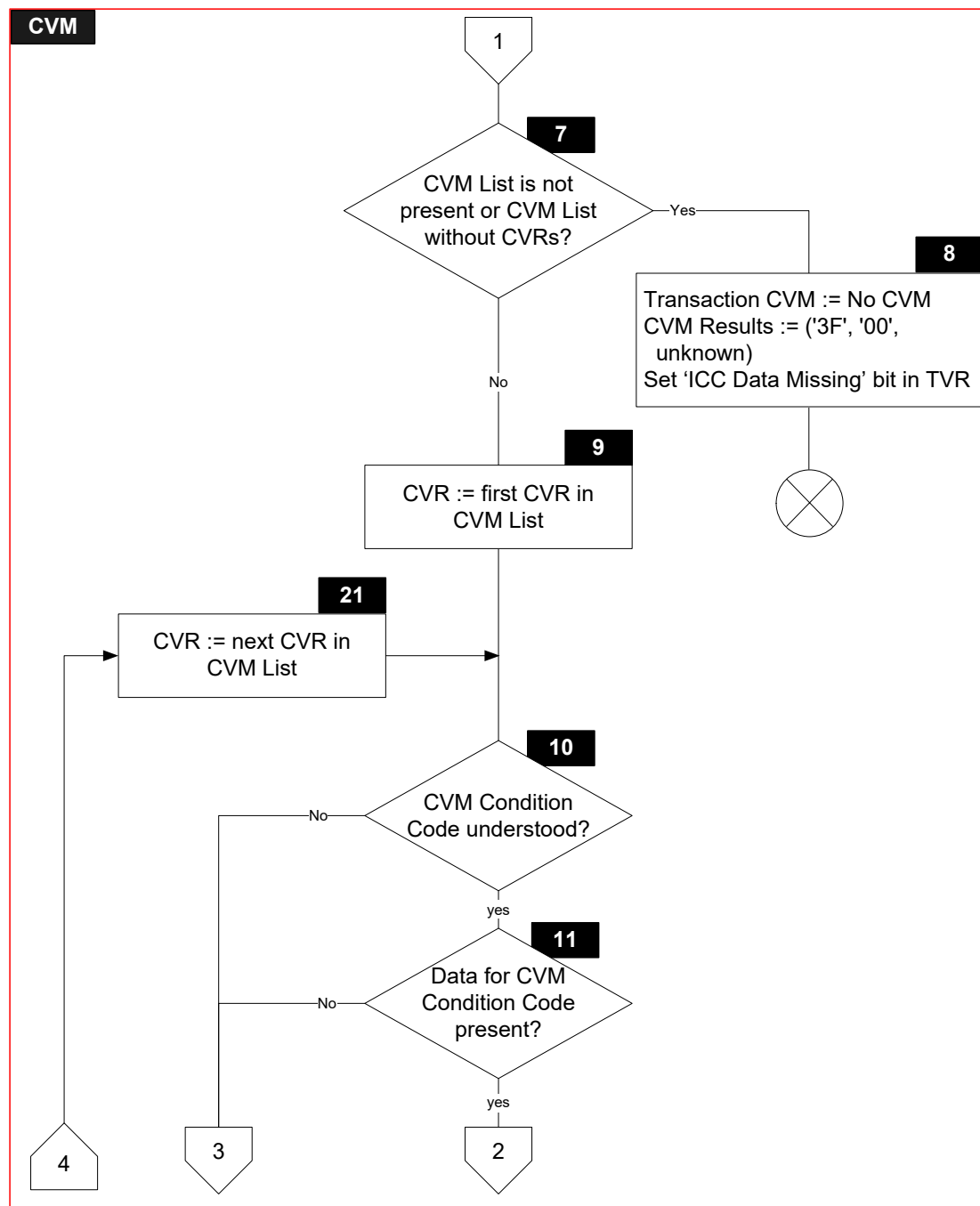
Name	Length	Format	Description
CVR	2	b	Cardholder Verification Rule
CVM Condition Code	1	b	Second byte of a CVR
CVM Code	1	b	First byte of a CVR

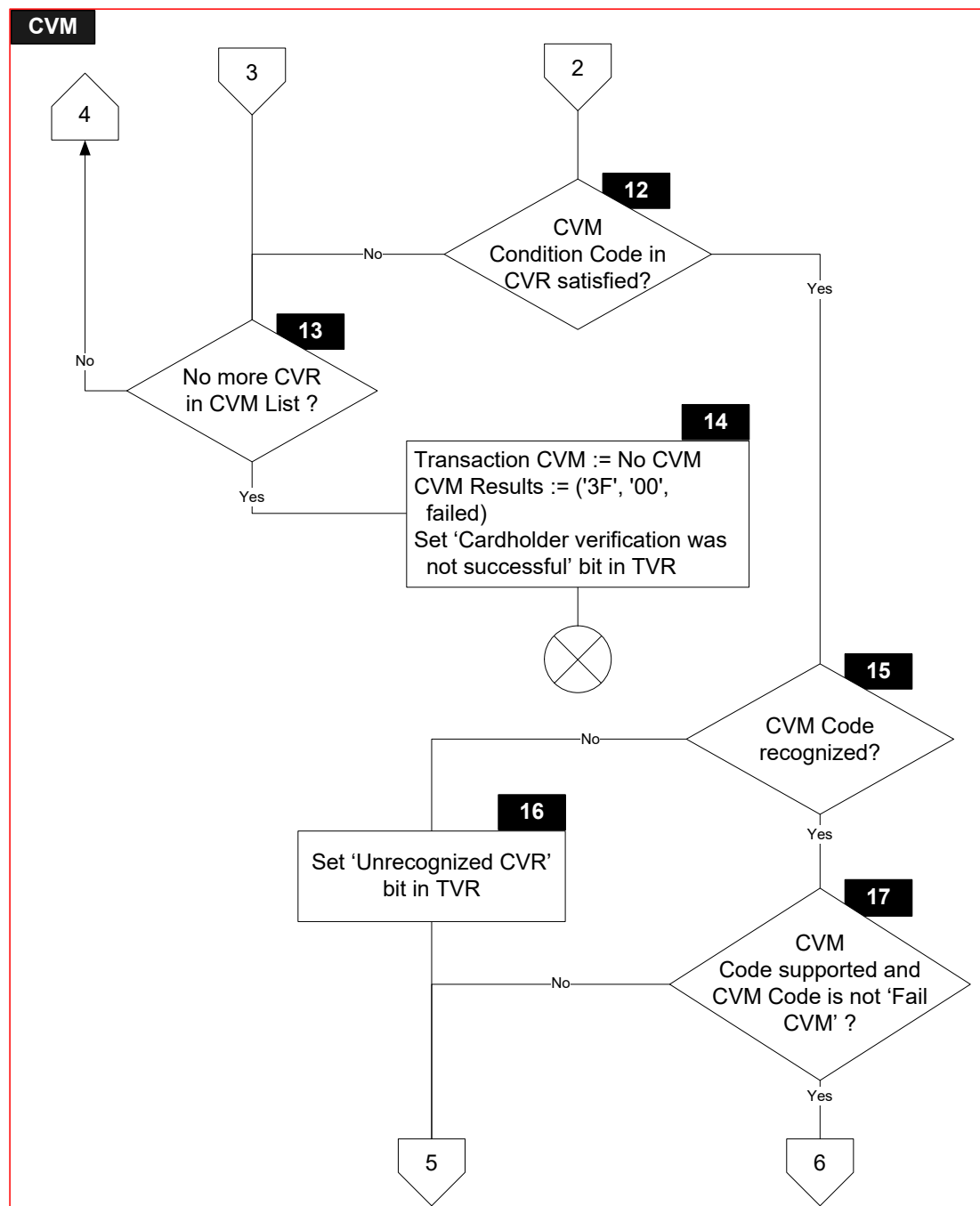
7.1.2 Flow Diagram

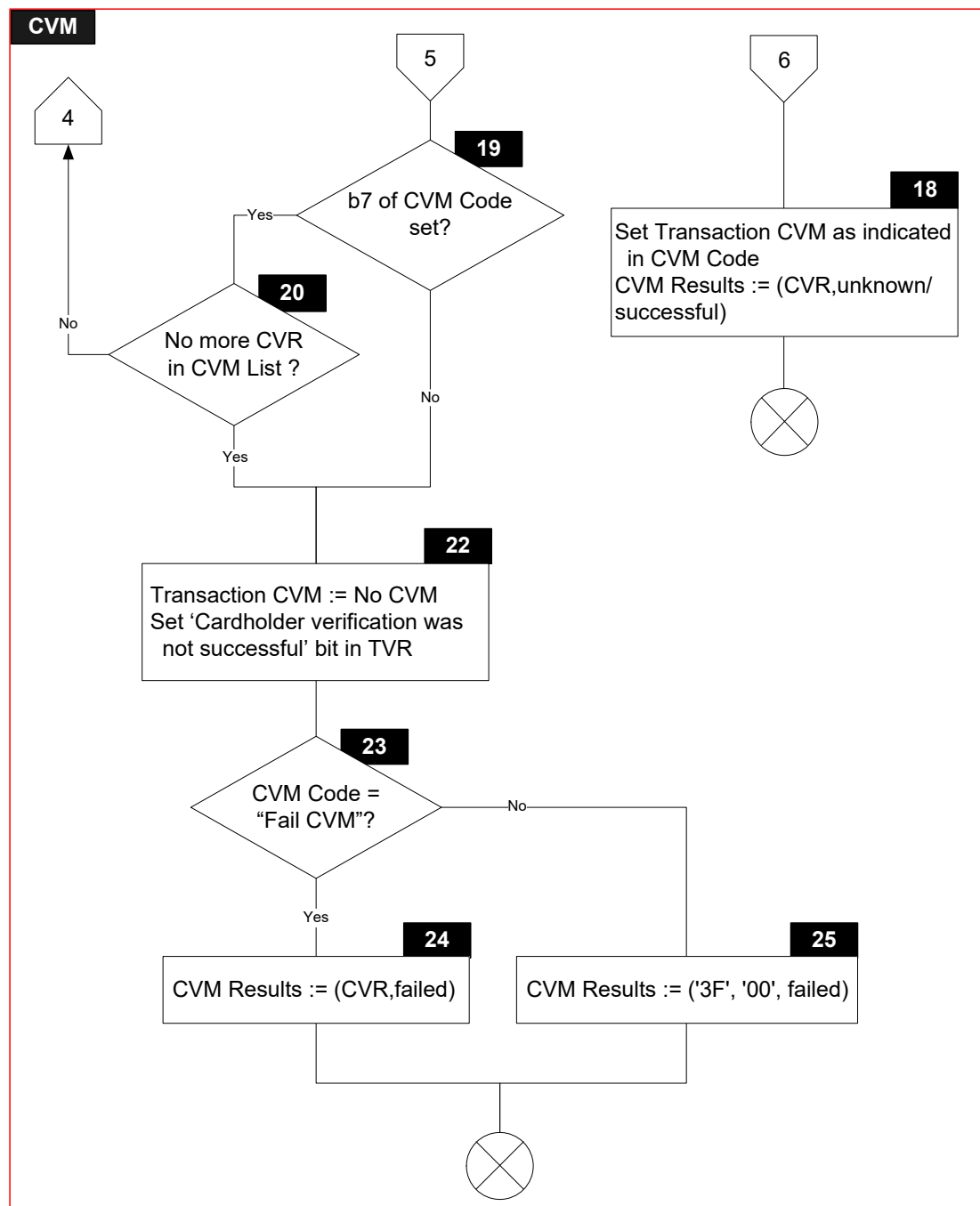
Figure 7.1 shows the flow diagram of the CVM Selection procedure. Symbols in this diagram are labelled CVM.X.

Figure 7.1: CVM Selection Flow Diagram









7.1.3 Processing

CVM.1

```
IF      ['CDCVM is supported' in Application Interchange Profile is set AND  
        'CDCVM supported' in Kernel Configuration is set]  
THEN  
    GOTO CVM.2  
ELSE  
    GOTO CVM.5  
ENDIF
```

CVM.2

```
IF      [Amount, Authorized (Numeric) > Reader CVM Required Limit]  
THEN  
    GOTO CVM.4  
ELSE  
    GOTO CVM.3  
ENDIF
```

CVM.3

```
'CVM' in Outcome Parameter Set := NO CVM  
'CVM Performed' in CVM Results := '3F' (No CVM performed)  
'CVM Condition' in CVM Results := '00'  
'CVM Result' in CVM Results := '02' (successful)
```

CVM.4

```
'CVM' in Outcome Parameter Set := CONFIRMATION CODE VERIFIED  
'CVM Performed' in CVM Results := '01' (CDCVM performed)  
'CVM Condition' in CVM Results := '00'  
'CVM Result' in CVM Results := '02' (successful)
```

CVM.5

```
IF      ['Cardholder verification is supported' in Application Interchange Profile is set]  
THEN  
    GOTO CVM.7  
ELSE  
    GOTO CVM.6  
ENDIF
```

CVM.6

```
'CVM' in Outcome Parameter Set := NO CVM  
'CVM Performed' in CVM Results := '3F' (No CVM performed)  
'CVM Condition' in CVM Results := '00'  
'CVM Result' in CVM Results := '00' (unknown)
```

CVM.7

```
IF      [IsNotPresent(TagOf(CVM List)) OR IsEmpty(TagOf(CVM List))]  
THEN  
    GOTO CVM.8  
ELSE  
    GOTO CVM.9  
ENDIF
```

CVM.8

```
'CVM' in Outcome Parameter Set := NO CVM  
'CVM Performed' in CVM Results := '3F' (No CVM performed)  
'CVM Condition' in CVM Results := '00'  
'CVM Result' in CVM Results := '00' (unknown)  
SET 'ICC data missing' in Terminal Verification Results
```

CVM.9

```
CVR := first CV Rule in CVM List  
CVM Code := CVR[1]  
CVM Condition Code := CVR[2]
```

CVM.10

```
IF      [CVM Condition Code is understood (i.e. the CVM Condition Code is included in  
        Table 40 of Annex C.3 of [EMV Book 3])]  
THEN  
    GOTO CVM.11  
ELSE  
    GOTO CVM.13  
ENDIF
```

Note that the Kernel may also understand proprietary CVM condition codes not defined in Annex C.3 of [EMV Book 3].

CVM.11

```
IF      [Data required by the conditions expressed by the CVM Condition Code are present  
        in the TLV Database]  
THEN  
    GOTO CVM.12  
ELSE  
    GOTO CVM.13  
ENDIF
```

CVM.12

```
IF      [Conditions expressed by the CVM Condition Code are satisfied]
THEN
    GOTO CVM.15
ELSE
    GOTO CVM.13
ENDIF
```

CVM.13

```
IF      [CVR is last CV Rule in CVM List]
THEN
    GOTO CVM.14
ELSE
    GOTO CVM.21
ENDIF
```

CVM.14

```
'CVM' in Outcome Parameter Set := NO CVM
'CVM Performed' in CVM Results := '3F' (No CVM performed)
'CVM Condition' in CVM Results := '00'
'CVM Result' in CVM Results := '01' (failed)
SET 'Cardholder verification was not successful' in Terminal Verification Results
```

CVM.15

```
IF      [CVM Code is recognized (i.e. the CVM Code is included in Table 39 of Annex C.3
        of [EMV Book 3])]
THEN
    GOTO CVM.17
ELSE
    GOTO CVM.16
ENDIF
```

Note that the Kernel may also recognize proprietary CVM codes not defined in Annex C.3 of [EMV Book 3].

CVM.16

```
SET 'Unrecognised CVM' in Terminal Verification Results
```

CVM.17

Verify if the CVM Code is supported:

- For CVM Codes defined in Annex C.3 of [EMV Book 3], support must be indicated in Terminal Capabilities.
- For CVM Codes not defined in Annex C.3 of [EMV Book 3], support may be known explicitly.
- For combination CVMs, both CVM codes must be supported.
- Fail CVM processing ('00' or '40') must always be supported.

IF [CVM Code is supported AND ((CVM Code AND '3F') ≠ '00')]

THEN

GOTO CVM.18

ELSE

GOTO CVM.19

ENDIF

CVM.18

```
IF      [(CVM Code AND '3F')= '02']
THEN
    'CVM' in Outcome Parameter Set := ONLINE PIN
    'CVM Result' in CVM Results := '00' (unknown)
    SET 'Online PIN entered' in Terminal Verification Results
ELSE
    IF      [(CVM Code AND '3F') = '1E']
    THEN
        'CVM' in Outcome Parameter Set := OBTAIN SIGNATURE
        'CVM Result' in CVM Results := '00' (unknown)
        'Receipt' in Outcome Parameter Set := YES
    ELSE
        IF      [(CVM Code AND '3F') = '1F']
        THEN
            'CVM' in Outcome Parameter Set := NO CVM
            'CVM Result' in CVM Results := '02' (successful)
        ELSE
            Set 'CVM' in Outcome Parameter Set to proprietary value
            'CVM Result' in CVM Results := '00' or '02'
        ENDIF
    ENDIF
ENDIF
```

'CVM Performed' in CVM Results := CVM Code
'CVM Condition' in CVM Results := CVM Condition Code

CVM.19

```
IF      [CVM Code[7] is set (i.e. apply succeeding CV Rule if this CVM is unsuccessful)]
THEN
    GOTO CVM.20
ELSE
    GOTO CVM.22
ENDIF
```

CVM.20

```
IF      [CVR is last CV Rule in CVM List]
THEN
    GOTO CVM.22
ELSE
    GOTO CVM.21
ENDIF
```

CVM.21

CVR := next CV Rule in CVM List

CVM Code := CVR[1]

CVM Condition Code := CVR[2]

CVM.22

'CVM' in Outcome Parameter Set := NO CVM

SET 'Cardholder verification was not successful' in Terminal Verification Results

CVM.23

IF [(CVM Code AND '3F') = '00']

THEN

GOTO CVM.24

ELSE

GOTO CVM.25

ENDIF

CVM.24

'CVM Performed' in CVM Results := CVM Code

'CVM Condition' in CVM Results := 'CVM Condition Code

'CVM Result' in CVM Results := '01' (failed)

CVM.25

'CVM Performed' in CVM Results := '3F'

'CVM Condition' in CVM Results := '00'

'CVM Result' in CVM Results := '01' (failed)

7.2 Procedure – Prepare Generate AC Command

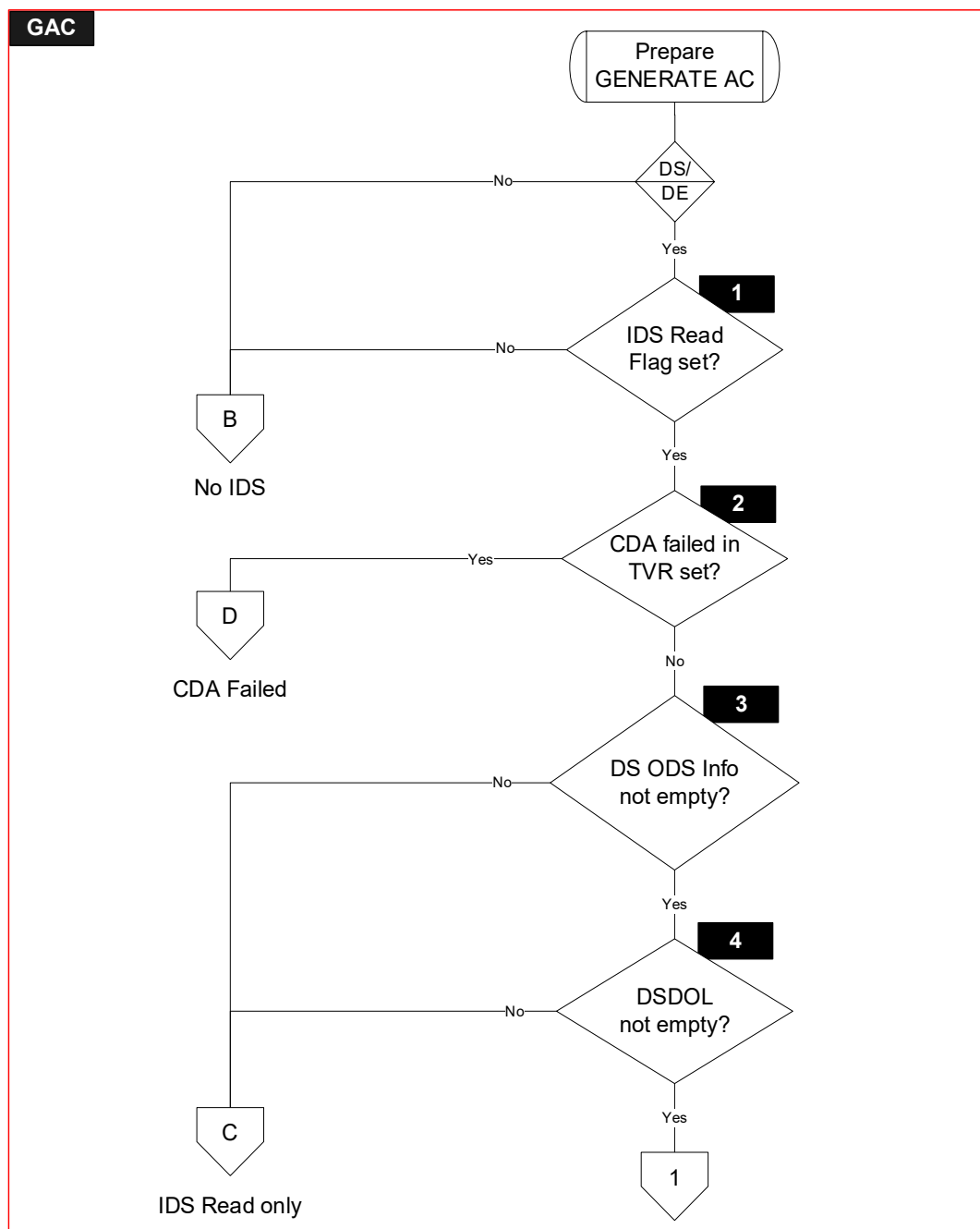
7.2.1 Local Variables

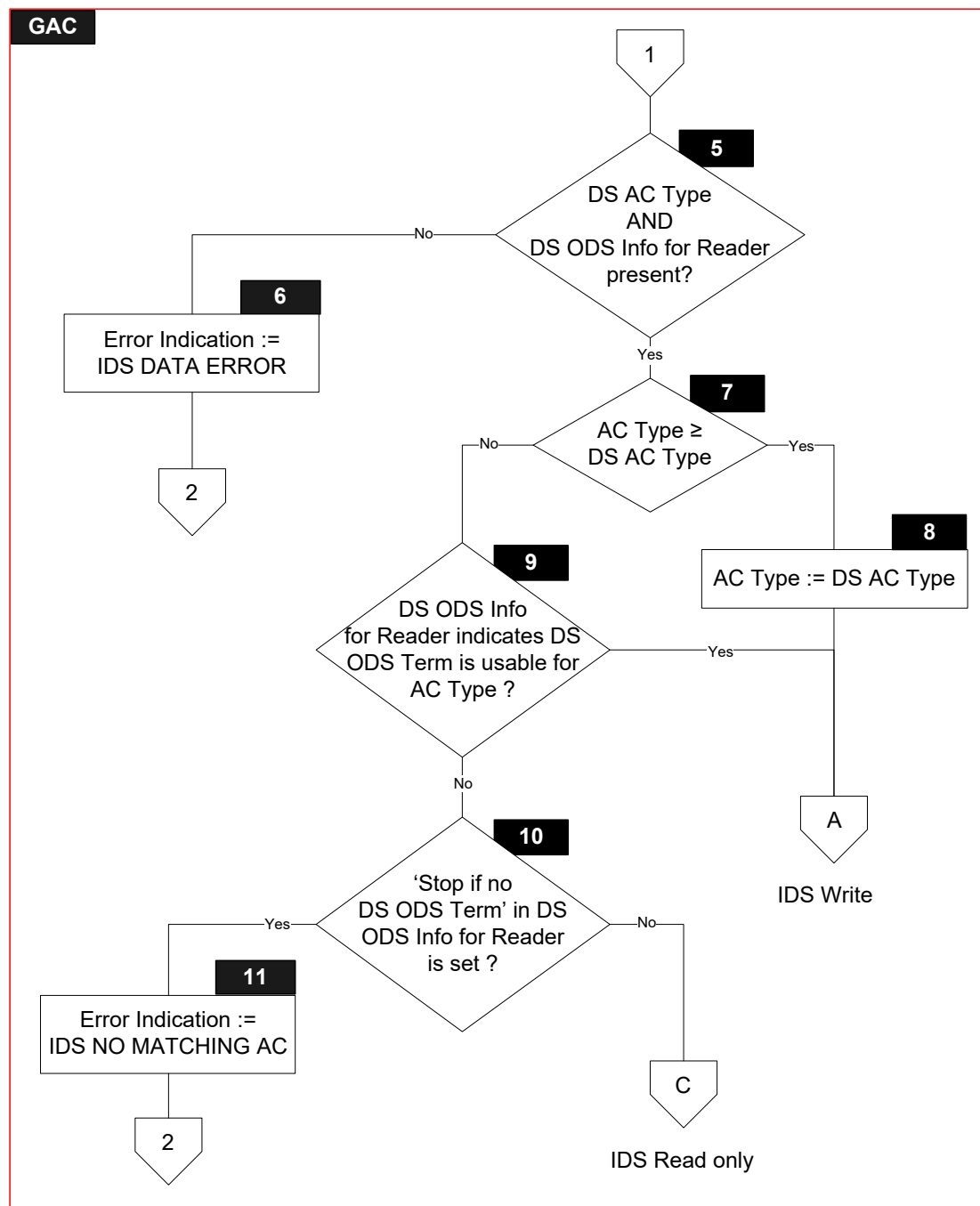
None

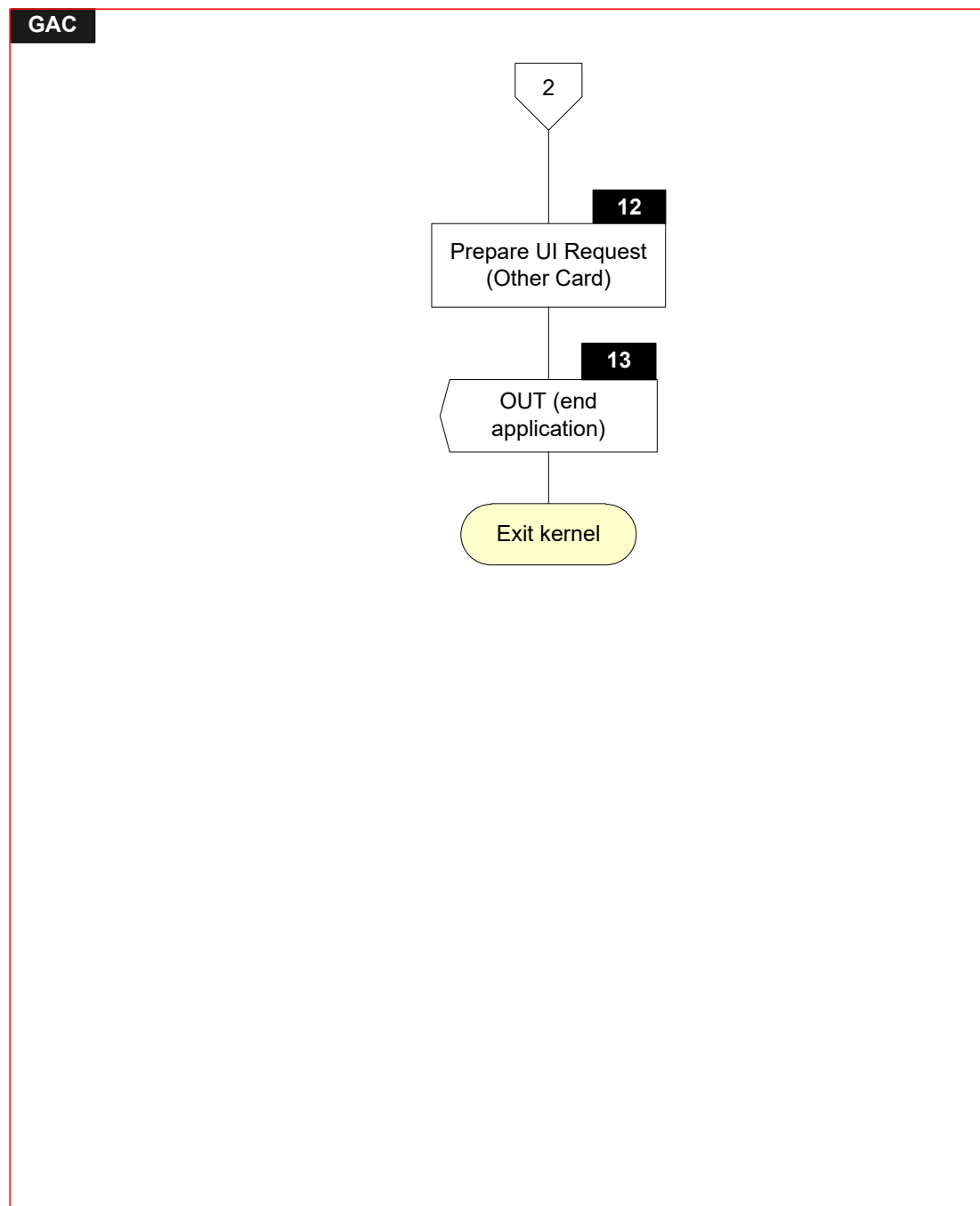
7.2.2 Flow Diagram

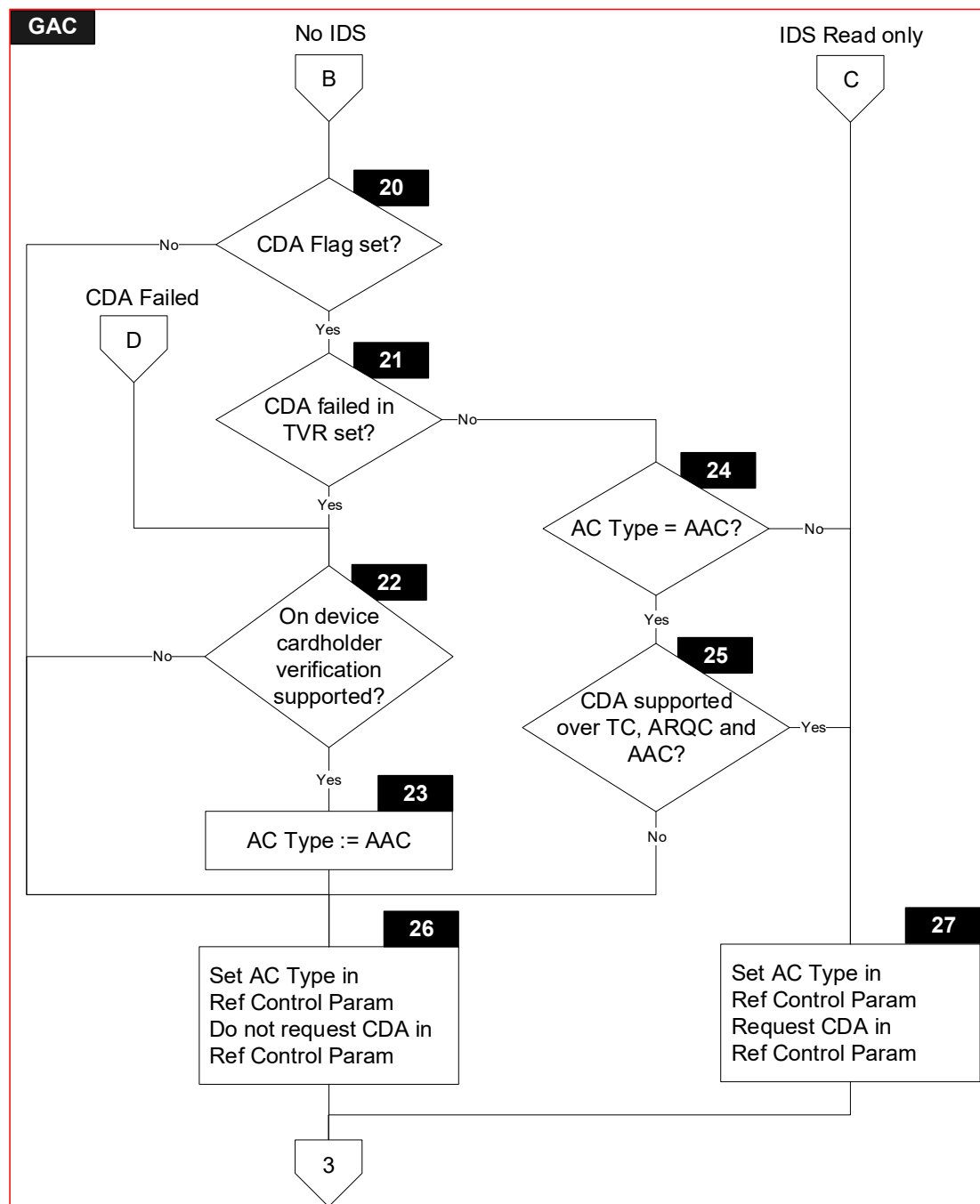
Figure 7.2 shows the flow diagram of the Prepare Generate AC Command procedure. Symbols in this diagram are labelled GAC.X.

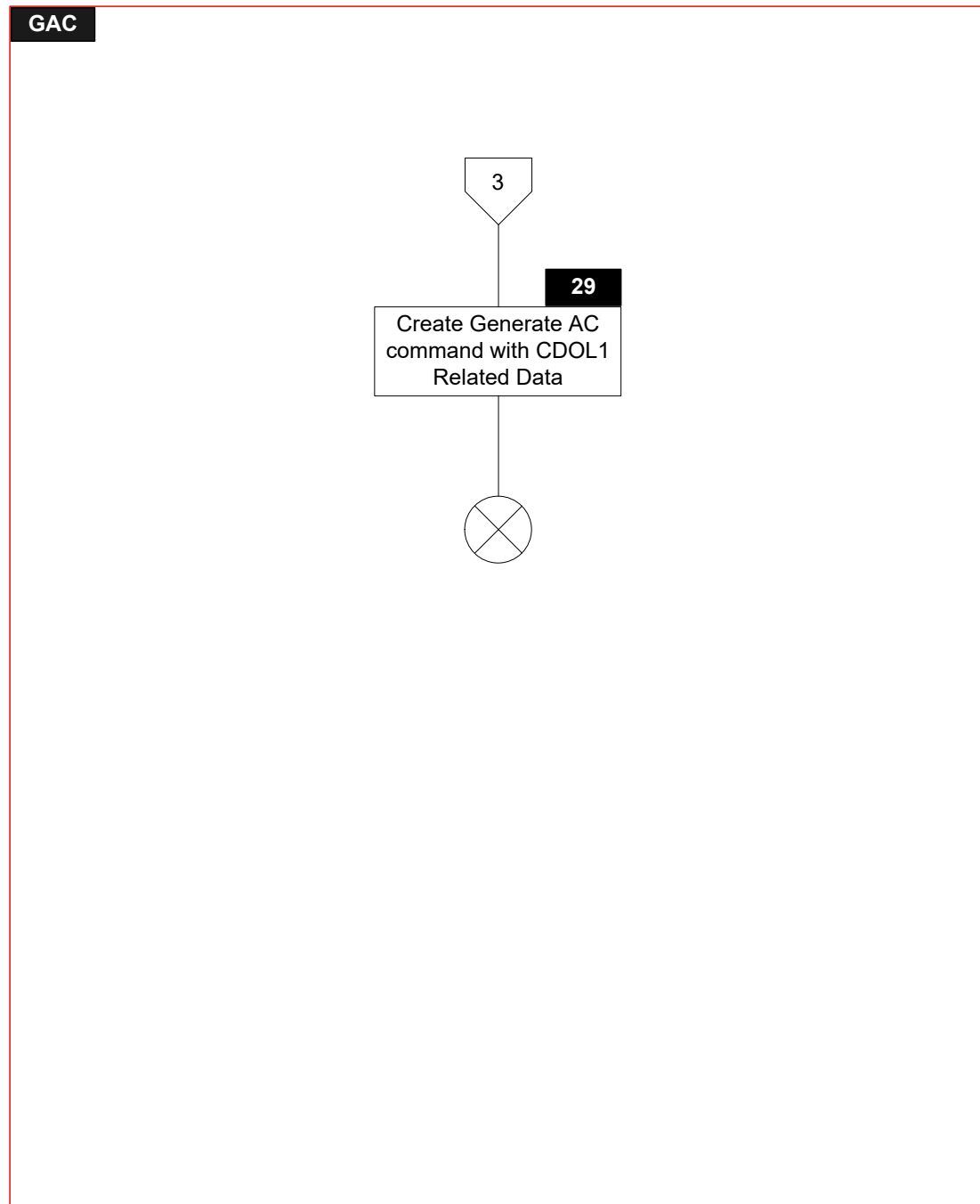
Figure 7.2: Prepare Generate AC Command Flow Diagram

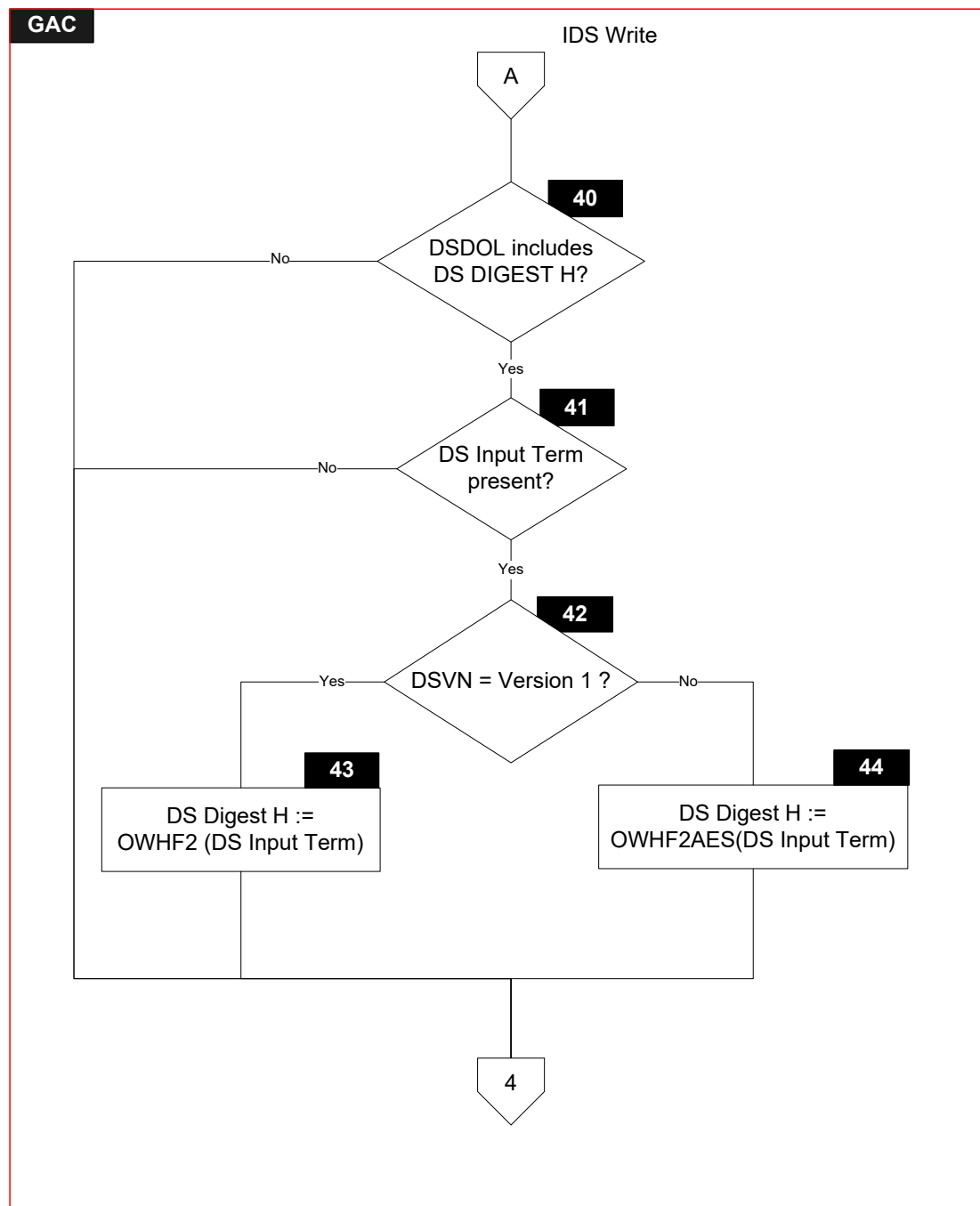


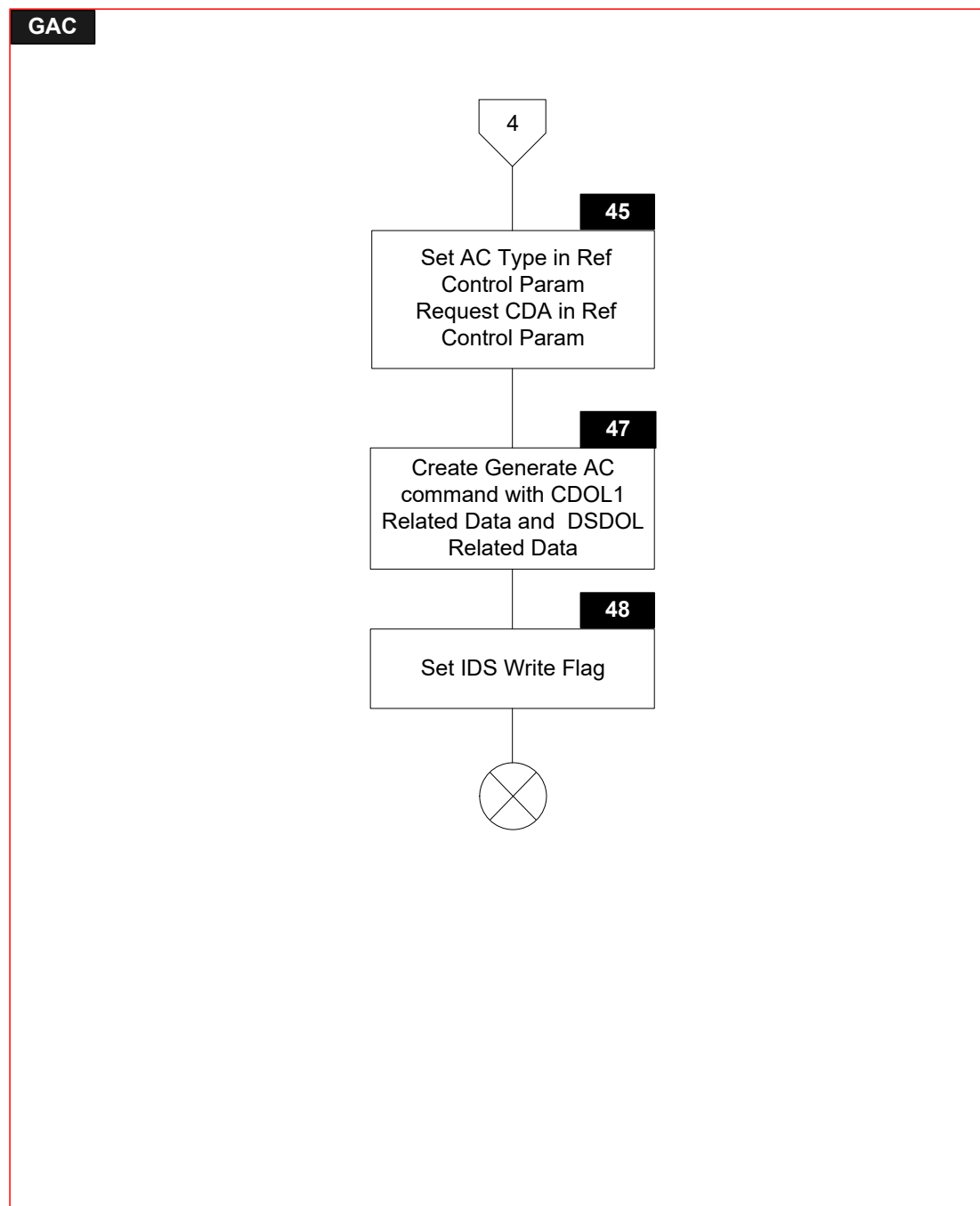












7.2.3 Processing

Note that symbols GAC.1, GAC.2, GAC.3, GAC.4, GAC.5, GAC.6, GAC.7, GAC.8, GAC.9, GAC.10, GAC.11, GAC.12, GAC.13, GAC.40, GAC.41, GAC.42, GAC.43, GAC.44, GAC.45, GAC.47 and GAC.48 only have to be implemented for the DE/DS Implementation Option.

GAC.1

```
IF      ['Read' in IDS Status is set]
THEN
    GOTO GAC.2
ELSE
    GOTO GAC.20
ENDIF
```

GAC.2

```
IF      ['CDA failed' in Terminal Verification Results is set]
THEN
    GOTO GAC.22
ELSE
    GOTO GAC.3
ENDIF
```

GAC.3

```
IF      [IsNotEmpty(TagOf(DS ODS Info))]
THEN
    GOTO GAC.4
ELSE
    GOTO GAC.27
ENDIF
```

GAC.4

```
IF      [IsNotEmpty(TagOf(DSDOL))]
THEN
    GOTO GAC.5
ELSE
    GOTO GAC.27
ENDIF
```

GAC.5

```
IF      [IsEmpty(TagOf(DS AC Type)) AND
        IsNotEmpty(TagOf(DS ODS Info For Reader))]
THEN
    GOTO GAC.7
ELSE
    GOTO GAC.6
ENDIF
```

GAC.6

'L2' in Error Indication := IDS DATA ERROR

GAC.7

```
IF      [('AC type' in DS AC Type = AAC)
        OR
        ('AC type' in AC Type = 'AC type' in DS AC Type)
        OR
        (('AC type' in DS AC Type = ARQC) AND ('AC type' in AC Type = TC))]
THEN
    GOTO GAC.8
ELSE
    GOTO GAC.9
ENDIF
```

GAC.8

'AC type' in AC Type := 'AC type' in DS AC Type

GAC.9

```
IF      [ (('AC type' in AC Type = AAC) AND 'Usable for AAC' in DS ODS Info For Reader
        is set)
        OR
        (('AC type' in AC Type = ARQC) AND 'Usable for ARQC' in DS ODS Info For
        Reader is set)]
THEN
    GOTO GAC.40
ELSE
    GOTO GAC.10
ENDIF
```

GAC.10

```
IF      ['Stop if no DS ODS Term' in DS ODS Info For Reader is set]
THEN
    GOTO GAC.11
ELSE
    GOTO GAC.27
ENDIF
```

GAC.11

'L2' in Error Indication := IDS NO MATCHING AC

GAC.12

'Message Identifier' in User Interface Request Data := ERROR – OTHER CARD

'Status' in User Interface Request Data := NOT READY

GAC.13

'Status' in Outcome Parameter Set := END APPLICATION

'Msg On Error' in Error Indication := ERROR – OTHER CARD

CreateDiscretionaryData ()

SET 'UI Request on Outcome Present' in Outcome Parameter Set

Send OUT(GetTLV(TagOf(Outcome Parameter Set)),
GetTLV(TagOf(Discretionary Data)), GetTLV(TagOf(User Interface Request
Data))) Signal

No IDS

GAC.20

IF ['CDA' in ODA Status is set]

THEN

GOTO GAC.21

ELSE

GOTO GAC.26

ENDIF

GAC.21

IF ['CDA failed' in Terminal Verification Results is set]

THEN

GOTO GAC.22

ELSE

GOTO GAC.24

ENDIF

GAC.22

IF ['CDCVM is supported' in Application Interchange Profile is set AND
'CDCVM supported' in Kernel Configuration is set]

THEN

GOTO GAC.23

ELSE

GOTO GAC.26

ENDIF

GAC.23

'AC type' in AC Type := AAC

GAC.24

IF ['AC type' in AC Type = AAC]

THEN

GOTO GAC.25

ELSE

GOTO GAC.27

ENDIF

GAC.25

IF [IsEmpty(TagOf(Application Capabilities Information)) AND
'CDA Indicator' in Application Capabilities Information = CDA
SUPPORTED OVER TC, ARQC AND AAC]

THEN

GOTO GAC.27

ELSE

GOTO GAC.26

ENDIF

GAC.26

Reference Control Parameter := '00'

'AC type' in Reference Control Parameter := 'AC type' in AC Type

GAC.27

Reference Control Parameter := '00'

'AC type' in Reference Control Parameter := 'AC type' in AC Type

SET 'CDA signature requested' in Reference Control Parameter

GAC.29

Prepare GENERATE AC command as specified in section 5.3.2. Use CDOL1 to create CDOL1 Related Data as a concatenated list of data objects without tags or lengths following the rules specified in section 4.1.4.

IDS Write

GAC.40

```
IF      [DSDOL includes TagOf(DS Digest H)]
THEN
    GOTO GAC.41
ELSE
    GOTO GAC.45
ENDIF
```

GAC.41

```
IF      [IsPresent(TagOf(DS Input (Term)))]
THEN
    GOTO GAC.42
ELSE
    GOTO GAC.45
ENDIF
```

GAC.42

```
IF      ['Data Storage Version Number' in Application Capabilities Information =
        VERSION 1]
THEN
    GOTO GAC.43
ELSE
    GOTO GAC.44
ENDIF
```

GAC.43

DS Digest H := OWHF2(DS Input (Term))
Refer to section 8.2 for the description of OWHF2

GAC.44

DS Digest H := OWHF2AES(DS Input (Term))
Refer to section 8.3 for the description of OWHF2AES

GAC.45

Reference Control Parameter := '00'
'AC type' in Reference Control Parameter := 'AC type' in AC Type
SET 'CDA signature requested' in Reference Control Parameter

GAC.47

Prepare GENERATE AC command as specified in section 5.3.2. Use CDOL1 and DSDOL to create CDOL1 Related Data and DSDOL related data as concatenated lists of data objects without tags or lengths following the rules specified in section 4.1.4.

GAC.48

SET 'Write' in IDS Status

7.3 Procedure – Processing Restrictions

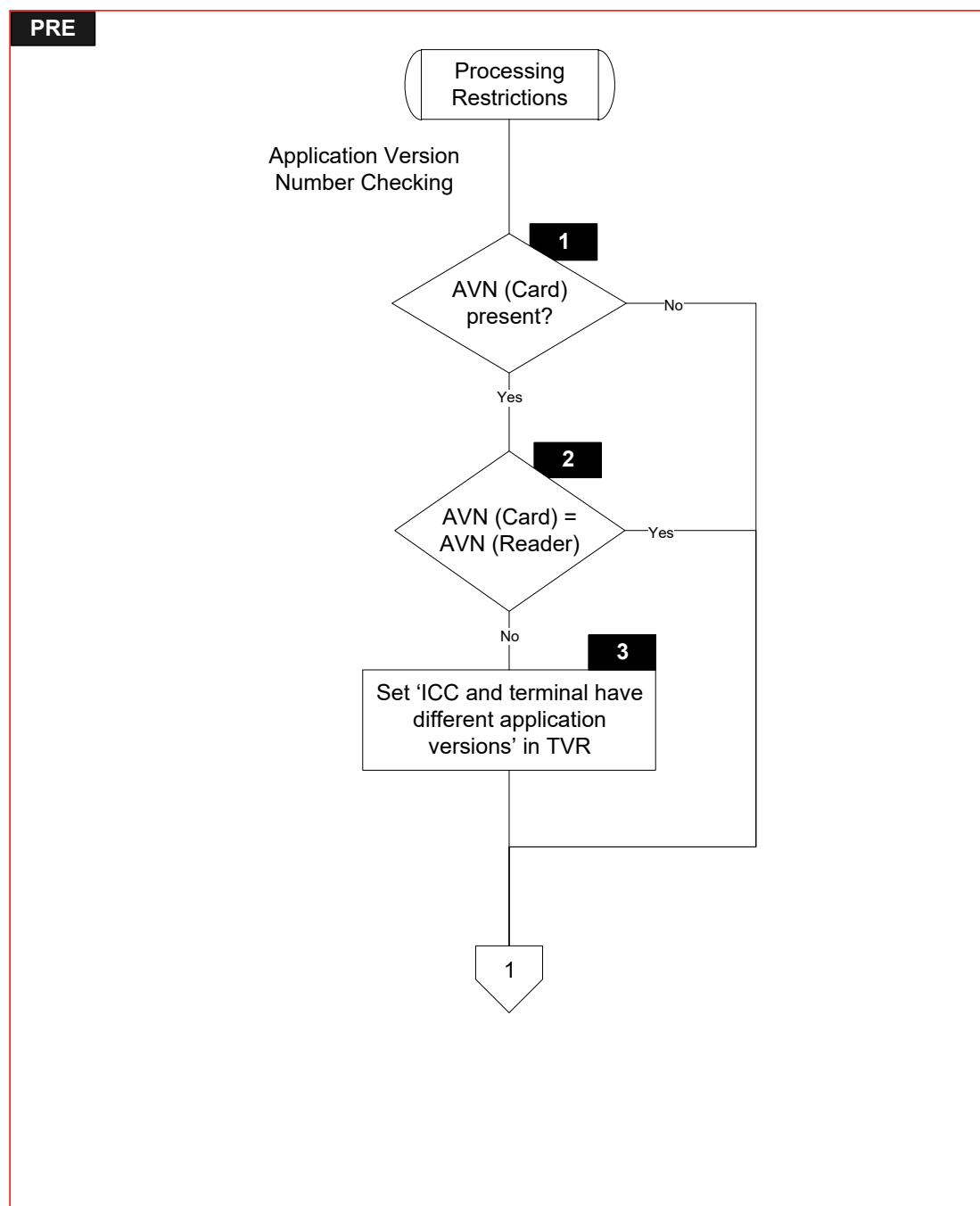
7.3.1 Local Variables

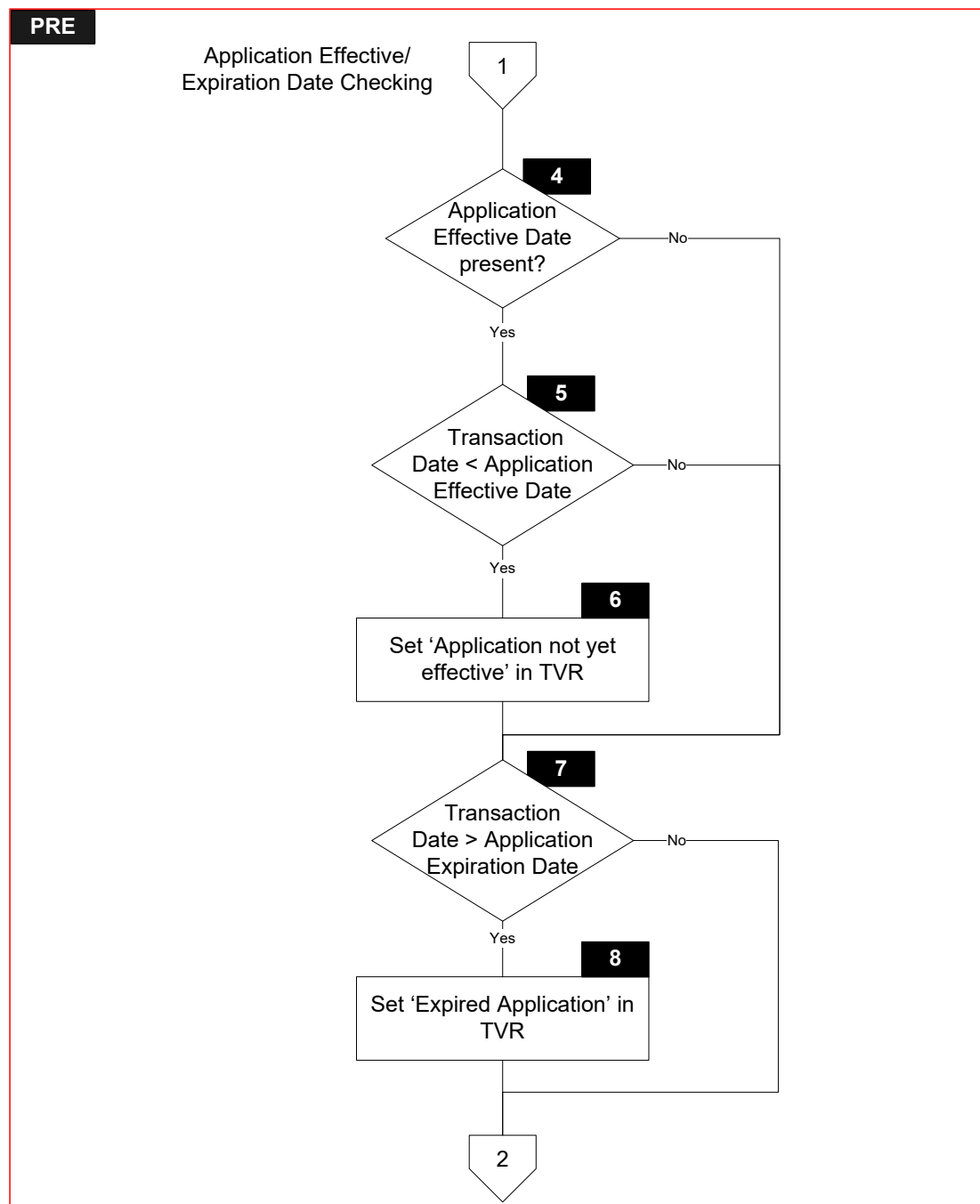
None

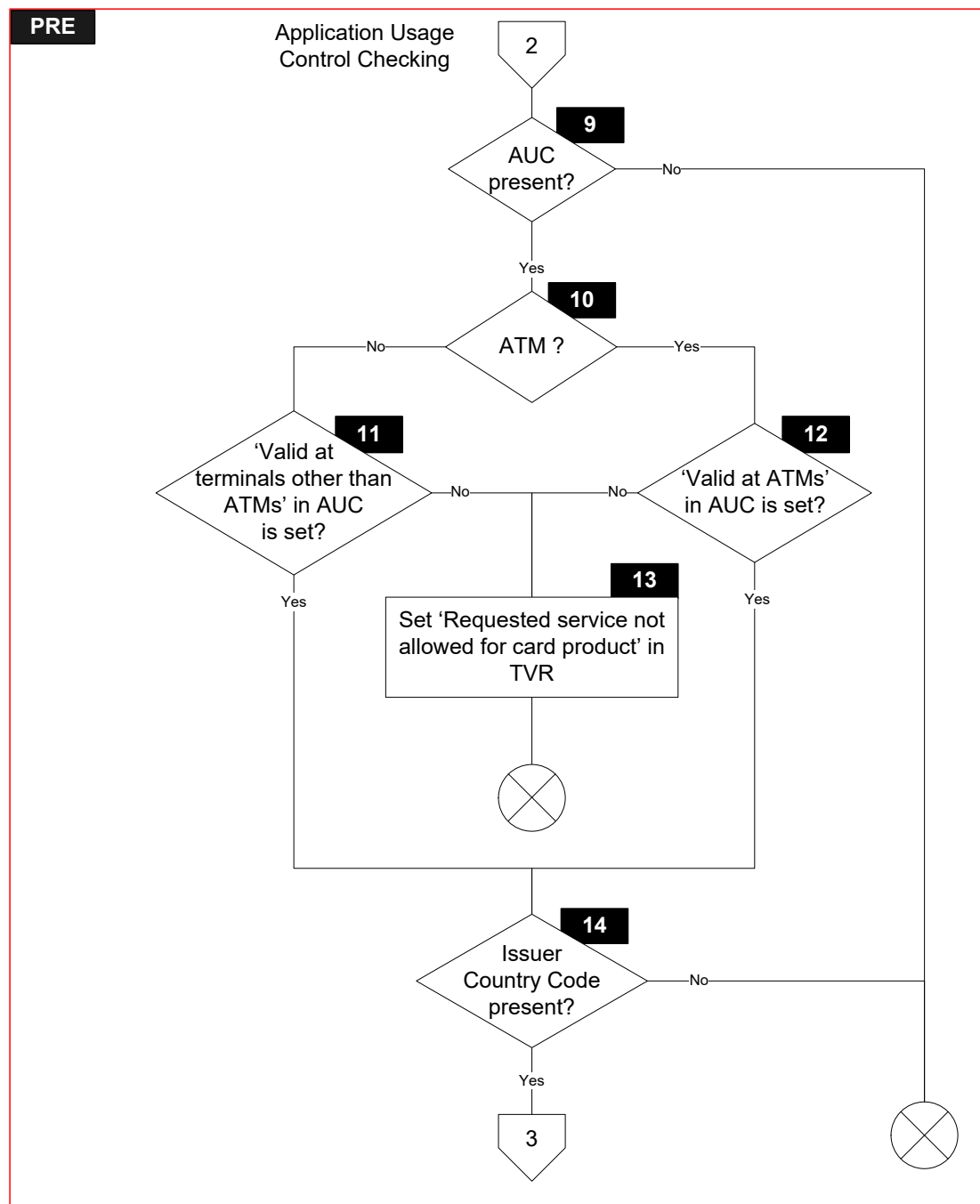
7.3.2 Flow Diagram

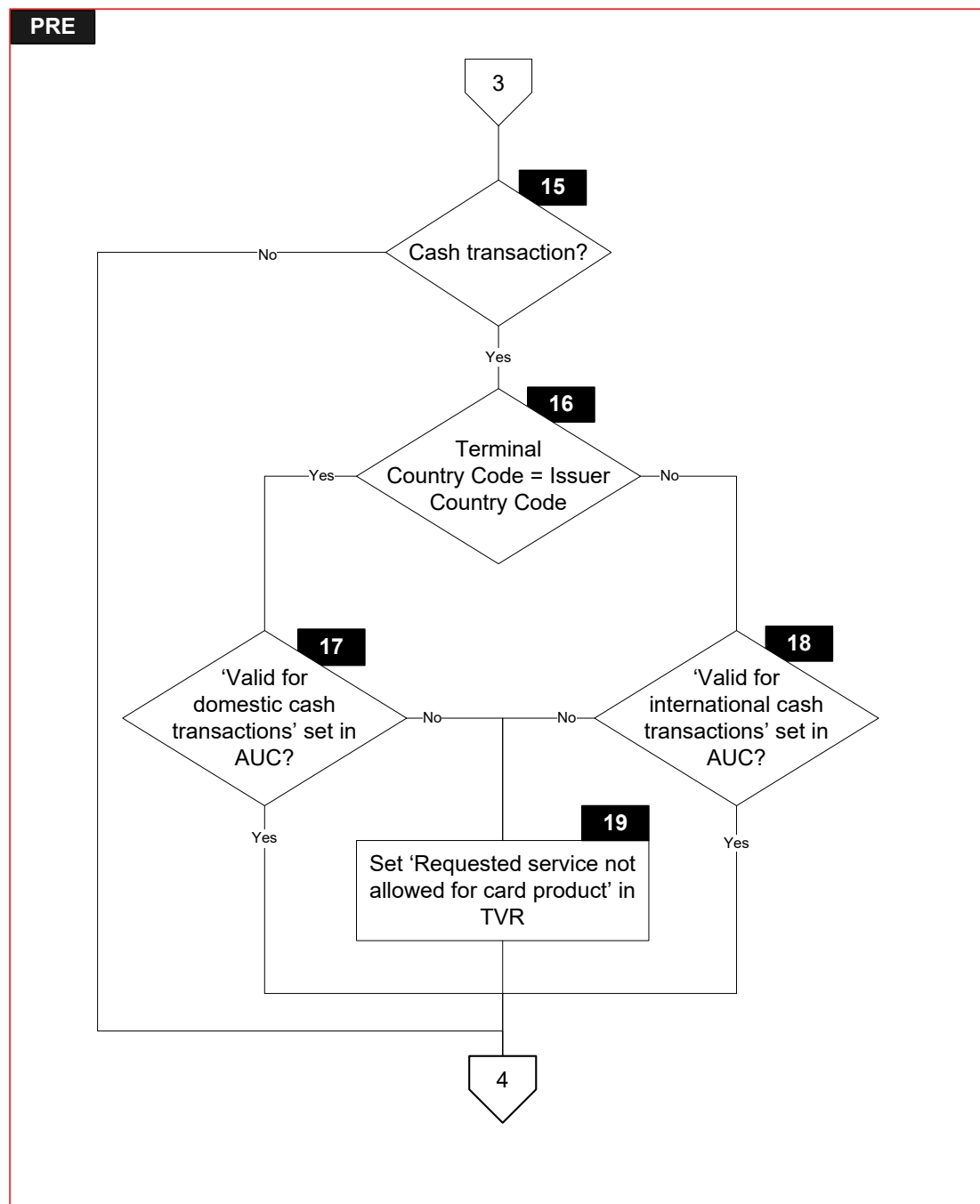
Figure 7.3 shows the flow diagram of the Processing Restrictions procedure. Symbols in this diagram are labelled PRE.X.

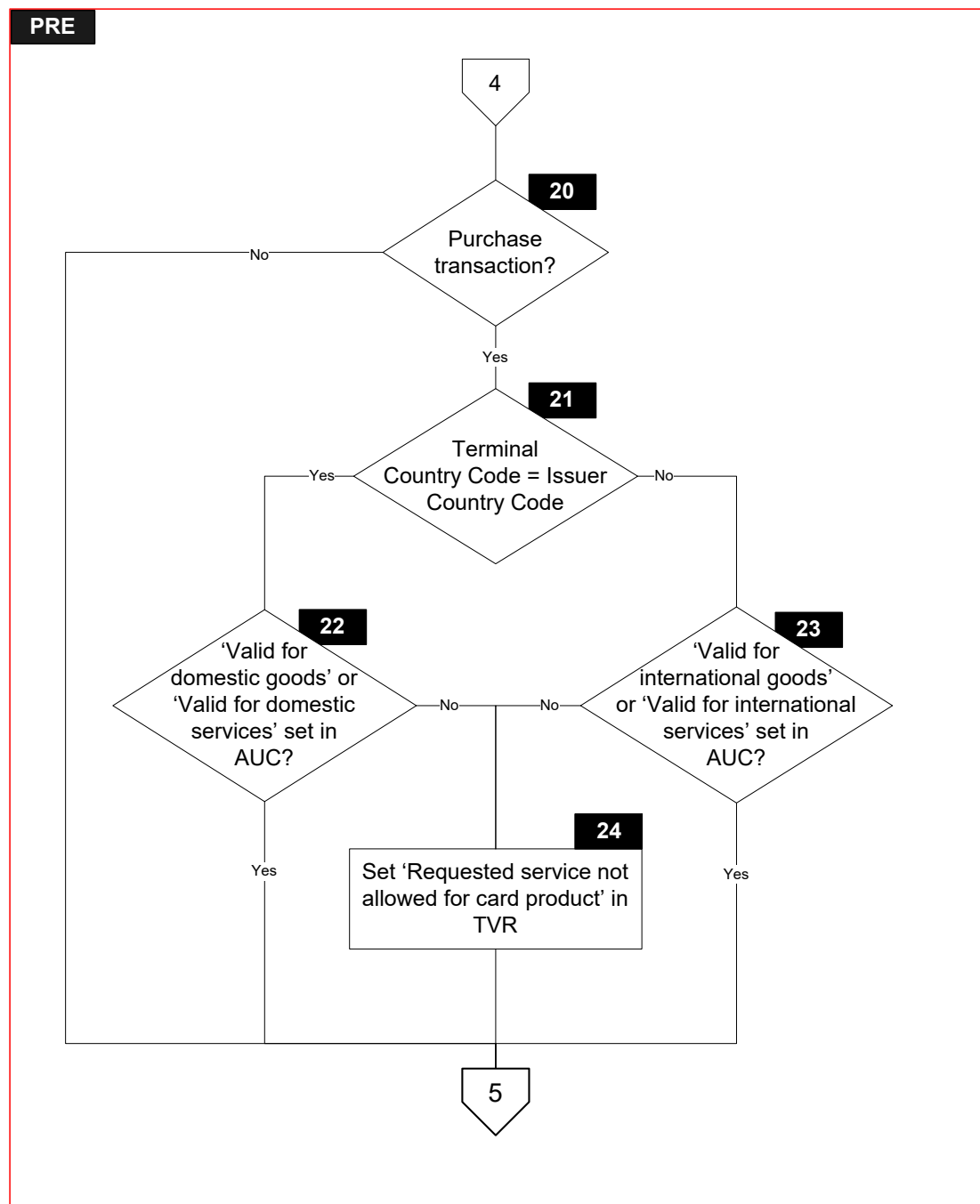
Figure 7.3: Processing Restrictions Flow Diagram

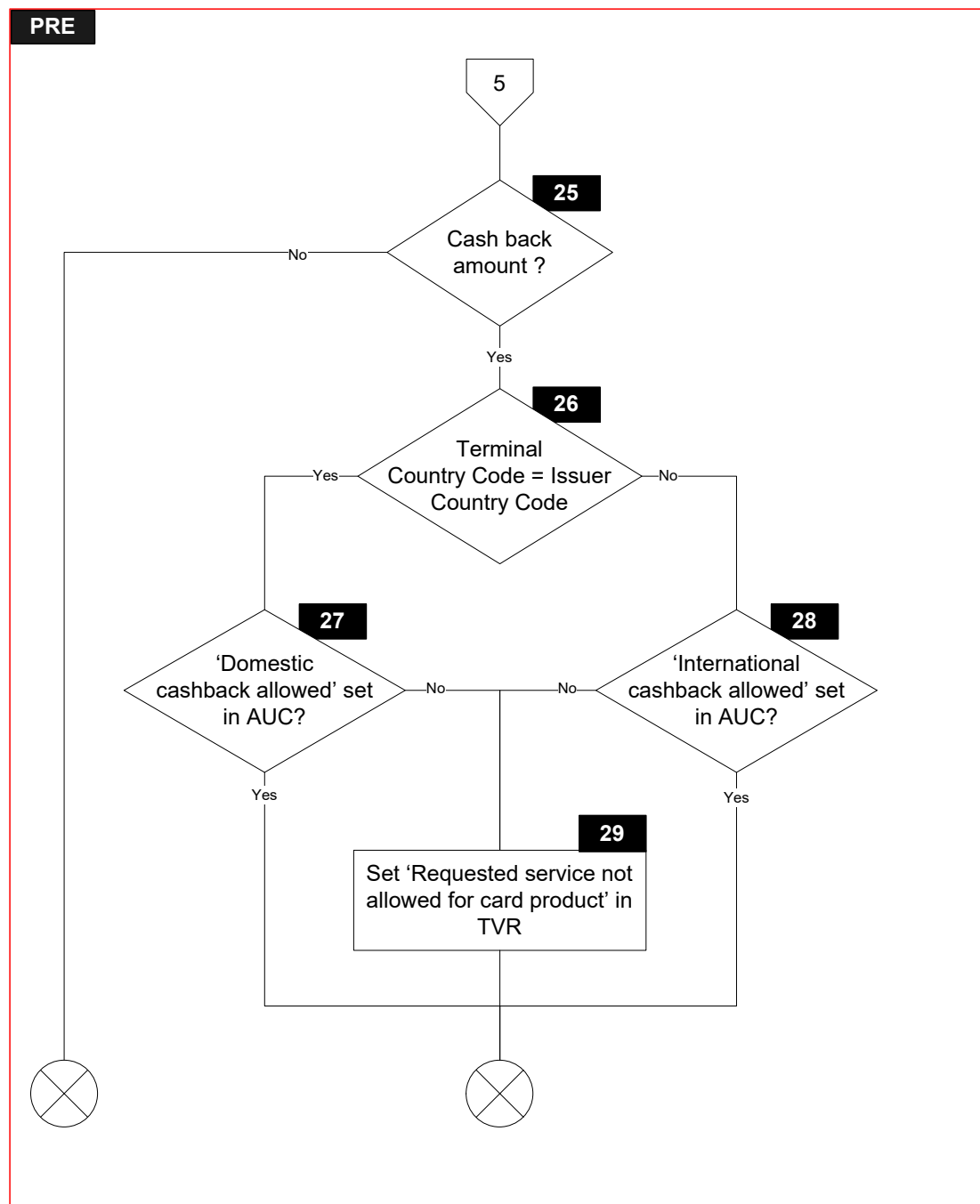












7.3.3 Processing

Application Version Number Checking

PRE.1

```
IF      [IsNotEmpty(TagOf(Application Version Number (Card)))]  
THEN  
    GOTO PRE.2  
ELSE  
    GOTO PRE.4  
ENDIF
```

PRE.2

```
IF      [Application Version Number (Card) = Application Version Number (Reader)]  
THEN  
    GOTO PRE.4  
ELSE  
    GOTO PRE.3  
ENDIF
```

PRE.3

SET 'ICC and terminal have different application versions' in Terminal Verification Results

Application Effective/Expiration Date Checking

PRE.4

```
IF      [IsEmpty(TagOf(Application Effective Date))]  
THEN  
    GOTO PRE.5  
ELSE  
    GOTO PRE.7  
ENDIF
```

PRE.5

```
IF      [Transaction Date is before Application Effective Date]  
THEN  
    GOTO PRE.6  
ELSE  
    GOTO PRE.7  
ENDIF
```

PRE.6

SET 'Application not yet effective' in Terminal Verification Results

PRE.7

```
IF      [Transaction Date is after Application Expiration Date]  
THEN  
    GOTO PRE.8  
ELSE  
    GOTO PRE.9  
ENDIF
```

PRE.8

SET 'Expired application' in Terminal Verification Results

Application Usage Control Checking

PRE.9

```
IF      [IsEmpty(TagOf(Application Usage Control))]  
THEN  
    GOTO PRE.10  
ELSE  
    EXIT Processing Restrictions  
ENDIF
```

PRE.10

```
IF      [((Terminal Type = '14') OR  
          (Terminal Type = '15') OR  
          (Terminal Type = '16'))  
        AND  
        'Cash' in Additional Terminal Capabilities is set]  
THEN  
    GOTO PRE.12  
ELSE  
    GOTO PRE.11  
ENDIF
```

PRE.11

```
IF      ['Valid at terminals other than ATMs' in Application Usage Control is set]  
THEN  
    GOTO PRE.14  
ELSE  
    GOTO PRE.13  
ENDIF
```

PRE.12

```
IF      ['Valid at ATMs' in Application Usage Control is set]  
THEN  
    GOTO PRE.14  
ELSE  
    GOTO PRE.13  
ENDIF
```

PRE.13

```
SET 'Requested service not allowed for card product' in Terminal Verification Results
```

PRE.14

```
IF      [IsEmpty(TagOf(Issuer Country Code))]  
THEN  
    GOTO PRE.15  
ELSE  
    EXIT Processing Restrictions  
ENDIF
```

PRE.15

Check if Transaction Type indicates a cash transaction (cash withdrawal or cash disbursement).

```
IF      [Transaction Type = '01' OR Transaction Type = '17']  
THEN  
    GOTO PRE.16  
ELSE  
    GOTO PRE.20  
ENDIF
```

PRE.16

```
IF      [Terminal Country Code = Issuer Country Code]  
THEN  
    GOTO PRE.17  
ELSE  
    GOTO PRE.18  
ENDIF
```

PRE.17

```
IF      ['Valid for domestic cash transactions' in Application Usage Control is set]  
THEN  
    GOTO PRE.20  
ELSE  
    GOTO PRE.19  
ENDIF
```

PRE.18

```
IF      ['Valid for international cash transactions' in Application Usage Control is set]  
THEN  
    GOTO PRE.20  
ELSE  
    GOTO PRE.19  
ENDIF
```

PRE.19

SET 'Requested service not allowed for card product' in Terminal Verification Results

PRE.20

Check if Transaction Type indicates a purchase transaction (purchase or purchase with cashback).

IF [Transaction Type = '00' OR Transaction Type = '09']

THEN

GOTO PRE.21

ELSE

GOTO PRE.25

ENDIF

PRE.21

IF [Terminal Country Code = Issuer Country Code]

THEN

GOTO PRE.22

ELSE

GOTO PRE.23

ENDIF

PRE.22

IF ['Valid for domestic goods' in Application Usage Control is set OR
'Valid for domestic services' in Application Usage Control is set]

THEN

GOTO PRE.25

ELSE

GOTO PRE.24

ENDIF

PRE.23

IF ['Valid for international goods' in Application Usage Control is set OR
'Valid for international services' in Application Usage Control is set]

THEN

GOTO PRE.25

ELSE

GOTO PRE.24

ENDIF

PRE.24

SET 'Requested service not allowed for card product' in Terminal Verification Results

PRE.25

```
IF      [IsPresent(TagOf(Amount, Other (Numeric))) AND  
        (Amount, Other (Numeric) ≠ '000000000000')]  
THEN  
    GOTO PRE.26  
ELSE  
    EXIT Processing Restrictions  
ENDIF
```

PRE.26

```
IF      [Terminal Country Code = Issuer Country Code]  
THEN  
    GOTO PRE.27  
ELSE  
    GOTO PRE.28  
ENDIF
```

PRE.27

```
IF      ['Domestic cashback allowed' in Application Usage Control is set]  
THEN  
    EXIT Processing Restrictions  
ELSE  
    GOTO PRE.29  
ENDIF
```

PRE.28

```
IF      ['International cashback allowed' in Application Usage Control is set]  
THEN  
    EXIT Processing Restrictions  
ELSE  
    GOTO PRE.29  
ENDIF
```

PRE.29

```
SET 'Requested service not allowed for card product' in Terminal Verification Results
```

7.4 Procedure – Terminal Action Analysis

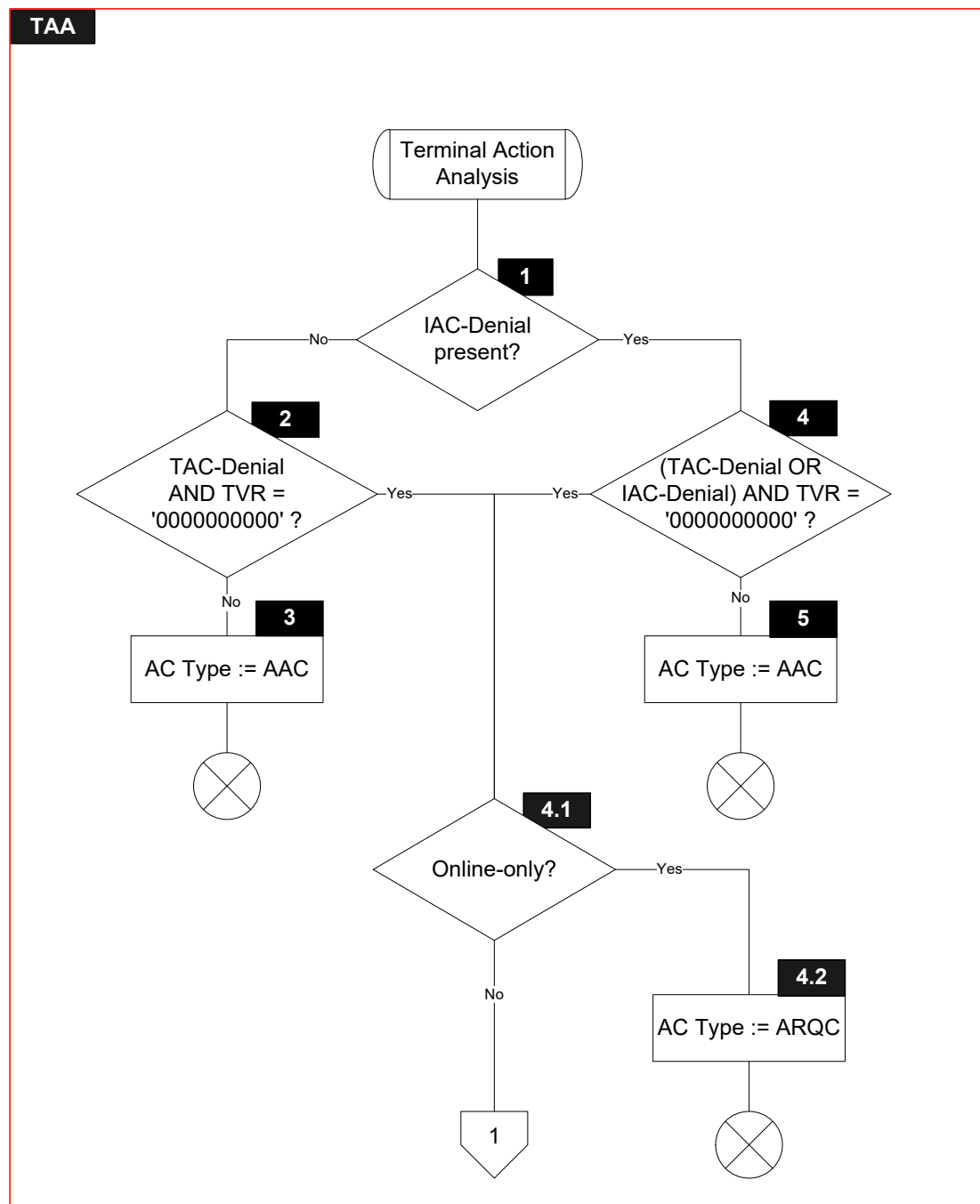
7.4.1 Local Variables

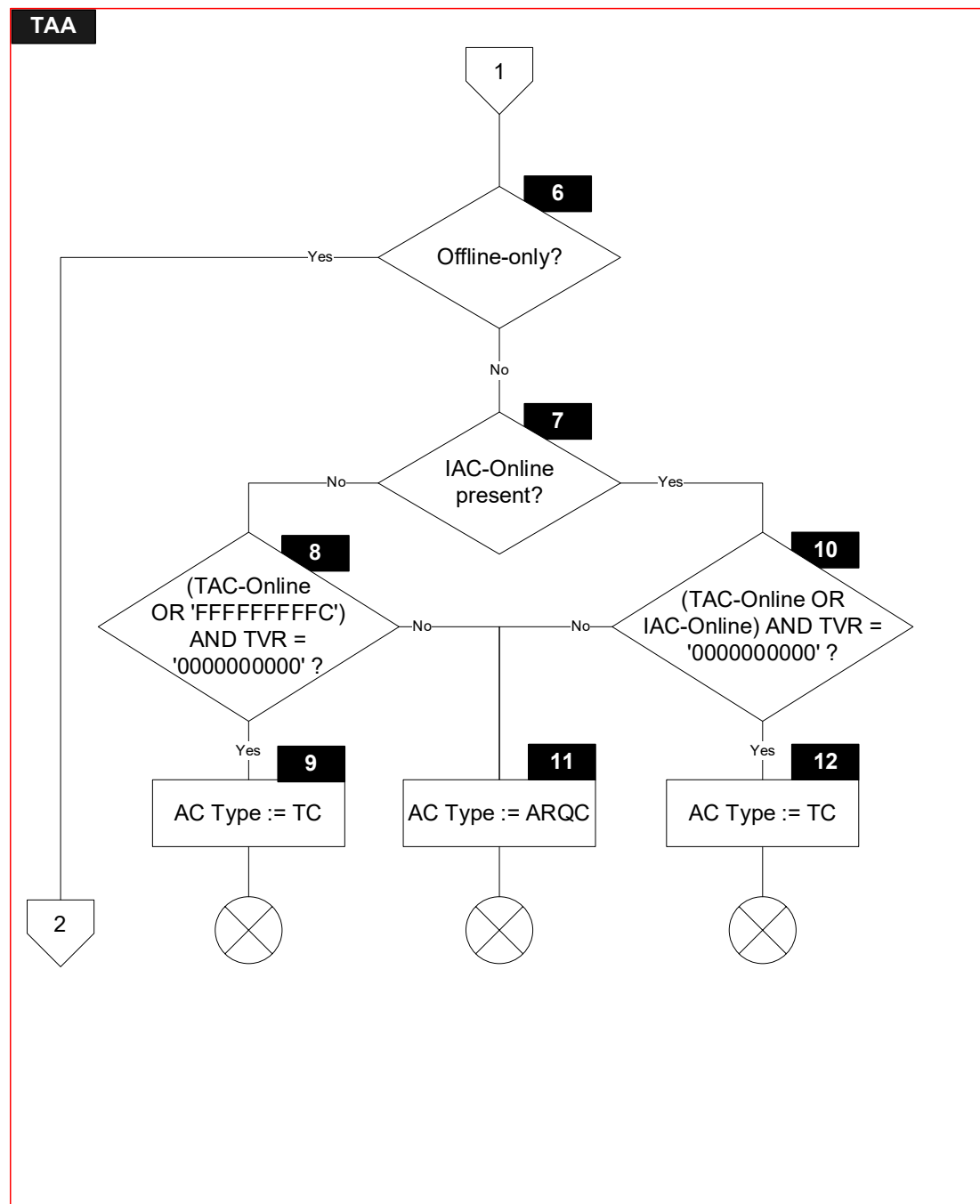
None

7.4.2 Flow Diagram

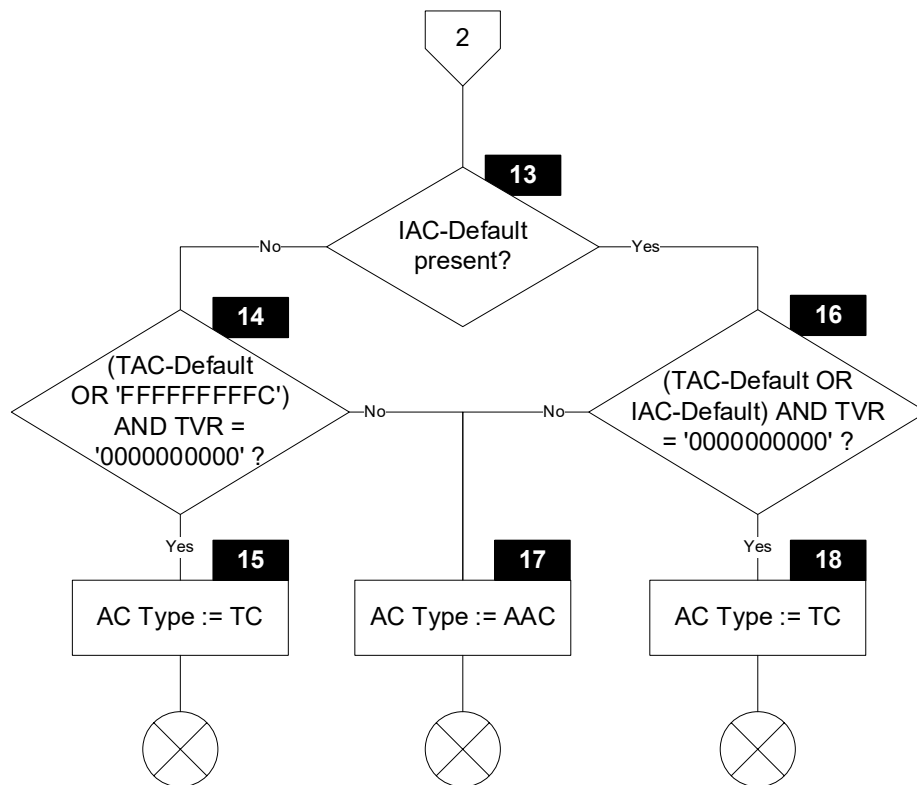
Figure 7.4 shows the flow diagram of the Terminal Action Analysis procedure. Symbols in this diagram are labelled TAA.X.

Figure 7.4: Terminal Action Analysis Flow Diagram





TAA



7.4.3 Processing

TAA.1

```
IF      [IsNotEmpty(TagOf(Issuer Action Code – Denial))]  
THEN  
    GOTO TAA.4  
ELSE  
    GOTO TAA.2  
ENDIF
```

TAA.2

```
IF      [(Terminal Action Code – Denial AND Terminal Verification Results) =  
        '0000000000']  
THEN  
    GOTO TAA.4.1  
ELSE  
    GOTO TAA.3  
ENDIF
```

TAA.3

'AC type' in AC Type := AAC

TAA.4

```
IF      [((Terminal Action Code – Denial OR Issuer Action Code – Denial) AND Terminal  
        Verification Results) = '0000000000']  
THEN  
    GOTO TAA.4.1  
ELSE  
    GOTO TAA.5  
ENDIF
```

TAA.4.1

```
IF      [(Terminal Type = '11') OR (Terminal Type = '21') OR (Terminal Type = '14') OR  
        (Terminal Type = '24') OR (Terminal Type = '34')]  
THEN  
    GOTO TAA.4.2  
ELSE  
    GOTO TAA.6  
ENDIF
```

TAA.4.2

'AC type' in AC Type := ARQC

TAA.5

'AC type' in AC Type := AAC

TAA.6

```
IF      [(Terminal Type = '23') OR (Terminal Type = '26') OR (Terminal Type = '36') OR  
        (Terminal Type = '13') OR (Terminal Type = '16')]  
THEN  
    GOTO TAA.13  
ELSE  
    GOTO TAA.7  
ENDIF
```

TAA.7

```
IF      [IsEmpty(TagOf(Issuer Action Code – Online))]  
THEN  
    GOTO TAA.10  
ELSE  
    GOTO TAA.8  
ENDIF
```

TAA.8

```
IF      [((Terminal Action Code – Online OR 'FFFFFFFFFC') AND Terminal Verification  
        Results) = '0000000000']  
THEN  
    GOTO TAA.9  
ELSE  
    GOTO TAA.11  
ENDIF
```

TAA.9

'AC type' in AC Type := TC

TAA.10

```
IF      [((Terminal Action Code – Online OR Issuer Action Code – Online) AND Terminal  
        Verification Results) = '0000000000']  
THEN  
    GOTO TAA.12  
ELSE  
    GOTO TAA.11  
ENDIF
```

TAA.11

'AC type' in AC Type := ARQC

TAA.12

'AC type' in AC Type := TC

TAA.13

```
IF      [IsNotEmpty(TagOf(Issuer Action Code – Default))]  
THEN  
        GOTO TAA.16  
ELSE  
        GOTO TAA.14  
ENDIF
```

TAA.14

```
IF      [((Terminal Action Code – Default OR 'FFFFFFFFFC') AND Terminal Verification  
Results) = '0000000000']  
THEN  
        GOTO TAA.15  
ELSE  
        GOTO TAA.17  
ENDIF
```

TAA.15

'AC type' in AC Type := TC

TAA.16

```
IF      [((Terminal Action Code – Default OR Issuer Action Code – Default) AND Terminal  
Verification Results) = '0000000000']  
THEN  
        GOTO TAA.18  
ELSE  
        GOTO TAA.17  
ENDIF
```

TAA.17

'AC type' in AC Type := AAC

TAA.18

'AC type' in AC Type := TC

8 Security Algorithms

8.1 Unpredictable Number Generation

Random numbers needed by the Kernel (for example for the Unpredictable Number and Unpredictable Number (Numeric)) should be generated in a hardware Random Number Generator. Any hardware random number generator must be tested in operation according to [NIST SP 800-22A]. A software random number generator must be seeded from an unpredictable source of data. A software whitening process may be applied to a hardware Number Generator if required. Regardless of the method used, there must be no observable correlation from one set of random data to a preceding set of random data and the Terminal must raise a suitable alarm in the event of a random number generator failure. A software Number Generator may be temporarily used until a hardware Number Generator is reinstated.

All values of random number (for example when used as the 4 byte Unpredictable Number) must be equally likely to occur, and the value of the random numbers must be unpredictable from the perspective of an attacker (even given knowledge of previous values). This may be achieved using a Random Number Generator compliant with [ISO 18031:2005] and tested using [NIST SP800-22A].

As generation of random data can be a slow process and transaction performance is important, an implementation may consider generating random data before it is needed, for example in a frequently refreshed buffer of random data. If random data is generated ahead of its use it must not be possible to observe this externally and thus to predict all or part of a number that may be used for a specific transaction.

The random number generator must not be susceptible to external perturbation that might reduce its quality, for example EM fields, glitch or other attacks. It must also not be possible for an attacker to deliberately cause fallback from the hardware RNG to a software one.

8.2 OWHF2¹¹

OWHF2 is the DES-based variant of the one-way function for computing the digest. OWHF2 computes an 8-byte output R based on an 8-byte input PD.

Let PL be the length in bytes of DS ID.

Compute two 6-byte values DSPKL and DSPKR as follows:

$\text{DSPKL}[i] := ((\text{DS ID}[i] \text{ div } 16) \times 10 + (\text{DS ID}[i] \text{ mod } 16)) \times 2$, for $i = 1, 2, \dots, 6$

$\text{DSPKR}[i] := ((\text{DS ID}[PL - 6 + i] \text{ div } 16) \times 10 + (\text{DS ID}[PL - 6 + i] \text{ mod } 16)) \times 2$, for $i = 1, 2, \dots, 6$

Compute an 8 byte value OID as follows:

IF [IsEmpty(TagOf(DS Slot Management Control)) AND
'Permanent slot type' in DS Slot Management Control is set AND
'Volatile slot type' in DS ODS Info is set]

THEN

OID := '0000000000000000'

ELSE

OID := DS Requested Operator ID

ENDIF

Generate two DES keys KL and KR as follows:

$\text{KL}[i] := \text{DSPKL}[i]$, for $i = 1, 2, \dots, 6$

$\text{KL}[i] := \text{OID}[i - 2]$, for $i = 7, \dots, 8$

$\text{KR}[i] := \text{DSPKR}[i]$, for $i = 1, 2, \dots, 6$

$\text{KR}[i] := \text{OID}[i]$, for $i = 7, \dots, 8$

Compute R as follows:

$R := \text{DES}(\text{KL})[\text{DES}^{-1}(\text{KR})[\text{DES}(\text{KL})[\text{OID} \oplus \text{PD}]]] \oplus \text{PD}$

¹¹ OWHF2 is a function specific to IDS and is only implemented for the DE/DS Implementation Option.

8.3 OWHF2AES¹²

OWHF2AES is the AES-based variant of the one-way function for computing the digest. OWHF2AES computes an 8-byte output R based on an 8-byte input C.

Compute an 8 byte value OID as follows:

```
IF      [IsNotEmpty(TagOf(DS Slot Management Control)) AND  
        'Permanent slot type' in DS Slot Management Control is set AND  
        'Volatile slot type' in DS ODS Info is set]  
THEN  
    OID := '0000000000000000'  
ELSE  
    OID := DS Requested Operator ID  
ENDIF
```

Compute R as follows:

Create a 16-byte message M by concatenating the following data:

M := C || OID

Create an 11-byte value Y by padding DS ID to the left with zeroes up to 11 bytes

Create a 16-byte AES key K by concatenating the following data:

K := Y || OID[5..8] || '3F'

Compute a 16-byte value T as follows:

T := AES(K)[M] ⊕ M

Compute R as the 8 leftmost bytes from T

¹² OWHF2AES is a function specific to IDS and is only implemented for the DE/DS Implementation Option.

Annex A Data Dictionary

This section contains the data dictionary of the Kernel. It lists all the data objects known to the Kernel other than local working variables.

A.1 Data Objects by Name

A.1.1 Account Type

Tag:	'5F57'
Template:	—
Length:	1
Format:	n 2
Update:	K/ACT/DET
Description:	Indicates the type of account selected on the Terminal, coded as specified in Annex G of [EMV Book 3].

A.1.2 Acquirer Identifier

Tag:	'9F01'
Template:	—
Length:	6
Format:	n 6-11
Update:	K
Description:	Uniquely identifies the acquirer within each payment system.

A.1.3 Active AFL

Tag:	—
Template:	—
Length:	var. up to 248
Format:	b
Update:	K
Description:	Contains the AFL indicating the (remaining) terminal file records to be read from the Card. The Active AFL is updated after each successful READ RECORD.

A.1.4 Active Tag

Tag: —
 Template: —
 Length: var. up to 2
 Format: b
 Update: K
 Description: Contains the tag requested by the GET DATA command.

A.1.5 AC Type

Tag: —
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Contains the AC type to be requested from the Card with the GENERATE AC command. This is the outcome of Terminal Action Analysis.

AC Type			
Byte 1	b8-7	AC type	
			00: AAC
			01: TC
			10: ARQC
			11: RFU
	b6-1	Each bit RFU	

A.1.6 Additional Terminal Capabilities

Tag: '9F40'
 Template: —
 Length: 5
 Format: b
 Update: K
 Description: Indicates the data input and output capabilities of the Terminal and Reader.

Additional Terminal Capabilities		
Byte 1	b8	Cash
	b7	Goods
	b6	Services
	b5	Cashback
	b4	Inquiry
	b3	Transfer
	b2	Payment
	b1	Administrative
Byte 2	b8	Cash Deposit
	b7-1	Each bit RFU
Byte 3	b8	Numeric keys
	b7	Alphabetical and special characters keys
	b6	Command keys
	b5	Function keys
	b4-1	Each bit RFU
Byte 4	b8	Print, attendant
	b7	Print, cardholder
	b6	Display, attendant
	b5	Display, cardholder
	b4-3	Each bit RFU
	b2	Code table 10
	b1	Code table 9
Byte 5	b8	Code table 8
	b7	Code table 7
	b6	Code table 6
	b5	Code table 5
	b4	Code table 4
	b3	Code table 3
	b2	Code table 2
	b1	Code table 1

A.1.7 Amount, Authorized (Numeric)

Tag: '9F02'

Template: —

Length: 6

Format: n 12

Update: K/ACT/DET

Description: Authorized amount of the transaction (excluding adjustments).

This amount is expressed with implicit decimal point corresponding to the minor unit of currency as defined by [ISO 4217] (for example the six bytes '00 00 00 00 01 23' represent USD 1.23 when the currency code is '840').

If the initial transaction amount needs to be replaced with a revised transaction amount, the Terminal must provide it before the chokepoint.

A.1.8 Amount, Other (Numeric)

Tag: '9F03'

Template: —

Length: 6

Format: n 12

Update: K/ACT/DET

Description: Secondary amount associated with the transaction representing a cash back amount.

This amount is expressed with implicit decimal point corresponding to the minor unit of currency as defined by [ISO 4217] (for example the 6 bytes '00 00 00 00 01 23' represent GBP 1.23 when the currency code is '826').

A.1.9 Application Capabilities Information

Tag: '9F5D'
 Template: 'BF0C'
 Length: 3
 Format: b
 Update: K/RA
 Description: Lists a number of card features beyond regular payment.

Application Capabilities Information			
Byte 1	b8-5	ACI Version number	
			0000: VERSION 0
			Other values: RFU
	b4-1	Data Storage Version Number	
			0000: DATA STORAGE NOT SUPPORTED
			0001: VERSION 1
			0010: VERSION 2
			Other values: RFU
Byte 2	b8-4	Each bit RFU	
	b3	Support for field off detection	
	b2	RFU	
	b1	CDA Indicator	
			0: CDA SUPPORTED AS IN EMV
			1: CDA SUPPORTED OVER TC, ARQC AND AAC

Application Capabilities Information		
Byte 3	b8-1	SDS Scheme Indicator
		00000000: Undefined SDS configuration
		00000001: All 10 tags 32 bytes
		00000010: All 10 tags 48 bytes
		00000011: All 10 tags 64 bytes
		00000100: All 10 tags 96 bytes
		00000101: All 10 tags 128 bytes
		00000110: All 10 tags 160 bytes
		00000111: All 10 tags 192 bytes
		00001000: All SDS tags 32 bytes except '9F78' which is 64 bytes
		Other values: RFU

A.1.10 Application Cryptogram

Tag: '9F26'
 Template: '77'
 Length: 8
 Format: b
 Update: K/RA
 Description: Cryptogram returned by the Card in response to the GENERATE AC command.

A.1.11 Application Currency Code

Tag: '9F42'
 Template: '70' or '77'
 Length: 2
 Format: n 3
 Update: K/RA
 Description: Indicates the currency in which the account is managed in accordance with [ISO 4217].

A.1.12 Application Currency Exponent

Tag:	'9F44'
Template:	'70' or '77'
Length:	1
Format:	n 1
Update:	K/RA
Description:	Indicates the implied position of the decimal point from the right of the amount represented in accordance with [ISO 4217].

A.1.13 Application Effective Date

Tag:	'5F25'
Template:	'70' or '77'
Length:	3
Format:	n 6 (YYMMDD)
Update:	K/RA
Description:	Date from which the application may be used. The date is expressed in the YYMMDD format.

A.1.14 Application Expiration Date

Tag:	'5F24'
Template:	'70' or '77'
Length:	3
Format:	n 6 (YYMMDD)
Update:	K/RA
Description:	Date after which application expires. The date is expressed in the YYMMDD format.

A.1.15 Application File Locator

Tag: '94'

Template: '77'

Length: var. multiple of 4 between 4 and 248

Format: b

Update: K/RA

Description: Indicates the location (SFI range of records) of the Application Elementary Files associated with a particular AID, and read by the Kernel during a transaction.

The Application File Locator is a list of entries of 4 bytes each. Each entry codes an SFI and a range of records as follows:

- The five most significant bits of the first byte indicate the SFI.
- The second byte indicates the first (or only) record number to be read for that SFI.
- The third byte indicates the last record number to be read for that SFI. When the third byte is greater than the second byte, all the records ranging from the record number in the second byte to and including the record number in the third byte must be read for that SFI. When the third byte is equal to the second byte, only the record number coded in the second byte must be read for that SFI.
- The fourth byte indicates the number of records involved in offline data authentication starting with the record number coded in the second byte. The fourth byte may range from zero to the value of the third byte less the value of the second byte plus 1.

A.1.16 Application Interchange Profile

Tag: '82'
 Template: '77'
 Length: 2
 Format: b
 Update: K/RA
 Description: Indicates the capabilities of the Card to support specific functions in the application.
 The Application Interchange Profile is returned in the response message of the GET PROCESSING OPTIONS command.

Application Interchange Profile		
Byte 1	b8-6	Each bit RFU
	b5	Cardholder verification is supported
	b4	Terminal risk management is to be performed
	b3	Issuer Authentication is supported
	b2	CDCVM is supported
	b1	CDA supported
Byte 2	b8	EMV mode is supported
	b7-2	Each bit RFU
	b1	Relay resistance protocol is supported

A.1.17 Application Label

Tag: '50'
 Template: 'A5'
 Length: var. up to 16
 Format: ans
 Update: K/RA
 Description: Name associated with the AID, in accordance with [ISO/IEC 7816-5].

A.1.18 Application Preferred Name

Tag: '9F12'
Template: 'A5'
Length: var. up to 16
Format: ans
Update: K/RA
Description: Preferred name associated with the AID.

A.1.19 Application PAN

Tag: '5A'
Template: '70' or '77'
Length: var. up to 10
Format: cn var. up to 19
Update: K/RA
Description: Valid cardholder account number.

A.1.20 Application PAN Sequence Number

Tag: '5F34'
Template: '70' or '77'
Length: 1
Format: n 2
Update: K/RA
Description: Identifies and differentiates cards with the same Application PAN.

A.1.21 Application Priority Indicator

Tag: '87'
Template: 'A5'
Length: 1
Format: b
Update: K/RA
Description: Indicates the priority of a given application or group of applications in a directory.

A.1.22 Application Transaction Counter

Tag: '9F36'
 Template: '77'
 Length: 2
 Format: b
 Update: K/RA
 Description: Counter maintained by the application in the Card (incrementing the Application Transaction Counter is managed by the Card).

A.1.23 Application Usage Control

Tag: '9F07'
 Template: '70' or '77'
 Length: 2
 Format: b
 Update: K/RA
 Description: Indicates the issuer's specified restrictions on the geographic use and services allowed for the application.

Application Usage Control		
Byte 1	b8	Valid for domestic cash transactions
	b7	Valid for international cash transactions
	b6	Valid for domestic goods
	b5	Valid for international goods
	b4	Valid for domestic services
	b3	Valid for international services
	b2	Valid at ATMs
	b1	Valid at terminals other than ATMs
Byte 2	b8	Domestic cashback allowed
	b7	International cashback allowed
	b6-1	Each bit RFU

A.1.24 Application Version Number (Card)

Tag:	'9F08'
Template:	'70' or '77'
Length:	2
Format:	b
Update:	K/RA
Description:	Version number assigned by the payment system for the application in the Card.

A.1.25 Application Version Number (Reader)

Tag:	'9F09'
Template:	—
Length:	2
Format:	b
Update:	K
Description:	Version number assigned by the payment system for the Kernel application.

A.1.26 CA Public Key Index (Card)

Tag:	'8F'
Template:	'70' or '77'
Length:	1
Format:	b
Update:	K/RA
Description:	Identifies the CA public key in conjunction with the RID.

A.1.27 Card Data Input Capability

Tag: 'DF8117'
Template: —
Length: 1
Format: b
Update: K
Description: Indicates the card data input capability of the Terminal and Reader.

Card Data Input Capability		
Byte 1	b8	Manual key entry
	b7	Magnetic stripe
	b6	IC with contacts
	b5-1	Each bit RFU

A.1.28 CDOL1

Tag: '8C'
Template: '70' or '77'
Length: var. up to 250
Format: b
Update: K/RA
Description: A data object in the Card that provides the Kernel with a list of data objects that must be passed to the Card in the data field of the GENERATE AC command.

A.1.29 CDOL1 Related Data

Tag: 'DF8107'
Template: —
Length: var.
Format: b
Update: K
Description: Command data field of the GENERATE AC command, coded according to CDOL1.

A.1.30 Cryptogram Information Data

Tag: '9F27'
Template: '77'
Length: 1
Format: b
Update: K/RA
Description: Indicates the type of cryptogram and the actions to be performed by the Kernel. The Cryptogram Information Data is coded according to Table 14 of [EMV Book 3].

A.1.31 CVM Capability – CVM Required

Tag: 'DF8118'
Template: —
Length: 1
Format: b
Update: K
Description: Indicates the CVM capability of the Terminal and Reader when the transaction amount is greater than the Reader CVM Required Limit.

CVM Capability – CVM Required		
Byte 1	b8	Plaintext PIN for ICC verification
	b7	Enciphered PIN for online verification
	b6	Signature
	b5	Enciphered PIN for offline verification
	b4	No CVM required
	b3-1	Each bit RFU

A.1.32 CVM Capability – No CVM Required

Tag: 'DF8119'
Template: —
Length: 1
Format: b
Update: K
Description: Indicates the CVM capability of the Terminal and Reader when the transaction amount is less than or equal to the Reader CVM Required Limit.

CVM Capability – No CVM Required		
Byte 1	b8	Plaintext PIN for ICC verification
	b7	Enciphered PIN for online verification
	b6	Signature
	b5	Enciphered PIN for offline verification
	b4	No CVM required
	b3-1	Each bit RFU

A.1.33 CVM List

Tag: '8E'
Template: '70' or '77'
Length: var. 10 to 250
Format: b
Update: K/RA
Description: Identifies the methods of verification of the cardholder supported by the application.
The CVM List is coded as specified in section 10.5 of [EMV Book 3].

A.1.34 CVM Results

Tag: '9F34'
Template: —
Length: 3
Format: b
Update: K
Description: Indicates the results of the last CVM performed.
The CVM Results are coded as specified in Annex A.4 of [EMV Book 4].

CVM Results		
Byte 1	b8-1	CVM Performed
Byte 2	b8-1	CVM Condition
Byte 3	b8-1	CVM Result

A.1.35 Data Needed

Tag: 'DF8106'
Template: —
Length: var.
Format: b
Update: K
Description: List of tags included in the DEK Signal to request information from the Terminal.

A.1.36 Data Record

Tag: 'FF8105'
Template: —
Length: var.
Format: b
Update: K
Description: The Data Record is a list of TLV encoded data objects returned with the Outcome Parameter Set on the completion of transaction processing.

A.1.37 Data To Send

Tag:	'FF8104'
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	List of data objects that contains the accumulated data sent by the Kernel to the Terminal in a DEK Signal. These data may correspond to Terminal reading requests, obtained from the Card by means of GET DATA or READ RECORD commands, or may correspond to data that the Kernel posts to the Terminal as part of its own processing.

A.1.38 Device Estimated Transmission Time For Relay Resistance R-APDU

Tag:	'DF8305'
Template:	—
Length:	2
Format:	b
Update:	K/RA
Description:	Indicates the time the Card expects to need for transmitting the EXCHANGE RELAY RESISTANCE DATA R-APDU. The Device Estimated Transmission Time For Relay Resistance R-APDU is expressed in units of hundreds of microseconds.

A.1.39 Device Relay Resistance Entropy

Tag:	'DF8302'
Template:	—
Length:	4
Format:	b
Update:	K/RA
Description:	Random number returned by the Card in the response to the EXCHANGE RELAY RESISTANCE DATA command.

A.1.40 DF Name

Tag: '84'
 Template: '6F'
 Length: 5-16
 Format: b
 Update: K/RA
 Description: Identifies the name of the DF, as described in [ISO/IEC 7816-4].

A.1.41 Discretionary Data

Tag: 'FF8106'
 Template: —
 Length: var.
 Format: b
 Update: K
 Description: The Discretionary Data is a list of Kernel-specific data objects sent to the Terminal as a separate field in the OUT Signal.

A.1.42 DS AC Type

Tag: 'DF8108'
 Template: —
 Length: 1
 Format: b
 Update: K/ACT/DET
 Description: Contains the AC type indicated by the Terminal for which IDS data must be stored in the Card.

DS AC Type			
Byte 1	b8-7	AC type	
			00: AAC
			01: TC
			10: ARQC
			11: RFU
	b6-1	Each bit RFU	

A.1.43 DS Digest H

Tag:	'DF61'
Template:	—
Length:	8
Format:	b
Update:	K
Description:	Contains the result of OWHF2(DS Input (Term)) or OWHF2AES(DS Input (Term)), if DS Input (Term) is provided by the Terminal. This data object is to be supplied to the Card with the GENERATE AC command, as per DSDOL formatting.

A.1.44 DSDOL

Tag:	'9F5B'
Template:	'70'
Length:	var. up to 250
Format:	b
Update:	K/RA
Description:	A data object in the Card that provides the Kernel with a list of data objects that must be passed to the Card in the data field of the GENERATE AC command after the CDOL1 Related Data. An example of value for DSDOL is 'DF6008DF6108DF6201DF63A0', representing TLDS Input (Card) TLDS Digest H TLDS ODS Info TLDS ODS Term. The Kernel must not presume that this is a given though, as the sequence and presence of data objects can vary. The presence of TL DS ODS Info is mandated and the processing of the last TL entry in DSDOL is different from normal TL processing as described in section 4.1.4.

A.1.45 DS ID

Tag:	'9F5E'
Template:	'BF0C'
Length:	var. 8 to 11
Format:	n, 16 to 22
Update:	K/RA
Description:	Data Storage Identifier constructed as follows: Application PAN (without any 'F' padding) Application PAN Sequence Number

If necessary, it is padded to the left with one hexadecimal zero to ensure whole bytes.

If necessary, it is padded to the left with hexadecimal zeroes to ensure a minimum length of 8 bytes.

A.1.46 DS Input (Card)

Tag:	'DF60'
Template:	—
Length:	8
Format:	b
Update:	K/ACT/DET
Description:	Contains Terminal provided data if permanent data storage in the Card was applicable (DS Slot Management Control[8]=1b), remains applicable, or becomes applicable (DS ODS Info[8]=1b). Otherwise this data item is a filler to be supplied by the Kernel. The data is forwarded to the Card with the GENERATE AC command, as per DSDOL formatting.

A.1.47 DS Input (Term)

Tag:	'DF8109'
Template:	—
Length:	8
Format:	b
Update:	K/ACT/DET
Description:	Contains Terminal provided data if permanent data storage in the Card was applicable (DS Slot Management Control[8]=1b), remains applicable or becomes applicable (DS ODS Info[8]=1b). DS Input (Term) is used by the Kernel as input to calculate DS Digest H.

A.1.48 DS ODS Card

Tag:	'9F54'
Template:	'77'
Length:	var. up to 160
Format:	b
Update:	K/RA
Description:	Contains the Card stored operator proprietary data obtained in the response to the GET PROCESSING OPTIONS command.

A.1.49 DS ODS Info

Tag: 'DF62'
Template: —
Length: 1
Format: b
Update: K/ACT/DET
Description: Contains Terminal provided data to be forwarded to the Card with the GENERATE AC command, as per DSDOL formatting.

DS ODS Info		
Byte 1	b8	Permanent slot type
	b7	Volatile slot type
	b6	Low volatility
	b5	RFU
	b4	Decline payment transaction in case of data storage error
	b3-1	Each bit RFU

A.1.50 DS ODS Info For Reader

Tag: 'DF810A'
Template: —
Length: 1
Format: b
Update: K/ACT/DET
Description: Contains instructions from the Terminal on how to proceed with the transaction if:

- The AC requested by the Terminal does not match the AC proposed by the Kernel
- The update of the slot data has failed

DS ODS Info For Reader		
Byte 1	b8	Usable for TC
	b7	Usable for ARQC
	b6	Usable for AAC
	b5-4	Each bit RFU
	b3	Stop if no DS ODS Term
	b2	Stop if write failed
	b1	RFU

A.1.51 DS ODS Term

Tag: 'DF63'
 Template: —
 Length: var. up to 160
 Format: b
 Update: K/ACT/DET
 Description: Contains Terminal provided data to be forwarded to the Card with the GENERATE AC command, as per DSDOL formatting.

A.1.52 DS Requested Operator ID

Tag: '9F5C'
 Template: —
 Length: 8
 Format: b
 Update: K/ACT/DET
 Description: Contains the Terminal determined operator identifier for data storage. It is sent to the Card in the GET PROCESSING OPTIONS command.

A.1.53 DS Slot Availability

Tag:	'9F5F'
Template:	'77'
Length:	1
Format:	b
Update:	K/RA
Description:	Contains the Card indication, obtained in the response to the GET PROCESSING OPTIONS command, about the slot type(s) available for data storage.

DS Slot Availability		
Byte 1	b8	Permanent slot type
	b7	Volatile slot type
	b6-1	Each bit RFU

A.1.54 DS Slot Management Control

Tag:	'9F6F'
Template:	'77'
Length:	1
Format:	b
Update:	K/RA
Description:	Contains the Card indication, obtained in the response to the GET PROCESSING OPTIONS command, about the status of the slot containing data associated to the DS Requested Operator ID.

DS Slot Management Control		
Byte 1	b8	Permanent slot type
	b7	Volatile slot type
	b6	Low volatility
	b5	Locked slot
	b4-2	Each bit RFU
	b1	Deactivated slot

A.1.55 DS Summary 1

Tag:	'9F7D'
Template:	'77'
Length:	8 or 16
Format:	b
Update:	K/RA
Description:	Contains the Card indication, obtained in the response to the GET PROCESSING OPTIONS command, about either the stored summary associated with DS ODS Card if present, or about a default zero-filled summary if DS ODS Card is not present and DS Unpredictable Number is present.

A.1.56 DS Summary 2

Tag:	'DF8101'
Template:	—
Length:	8 or 16
Format:	b
Update:	K/RA
Description:	This data allows the Kernel to check the consistency between DS Summary 1 and DS Summary 2, and so to ensure that DS ODS Card is provided by a genuine Card. It is located in the ICC Dynamic Data recovered from the Signed Dynamic Application Data.

A.1.57 DS Summary 3

Tag:	'DF8102'
Template:	—
Length:	8 or 16
Format:	b
Update:	K/RA
Description:	This data allows the Kernel to check whether the Card has seen the same transaction data as were sent by the Terminal/Kernel. It is located in the ICC Dynamic Data recovered from the Signed Dynamic Application Data.

A.1.58 DS Summary Status

Tag: 'DF810B'

Template: —

Length: 1

Format: b

Update: K

Description: Information reported by the Kernel to the Terminal about:

- The consistency between DS Summary 1 and DS Summary 2 (successful read)
- The difference between DS Summary 2 and DS Summary 3 (successful write)

This data object is part of the Discretionary Data.

DS Summary Status		
Byte 1	b8	Successful Read
	b7	Successful Write
	b6-1	Each bit RFU

A.1.59 DS Unpredictable Number

Tag: '9F7F'

Template: '77'

Length: 4

Format: b

Update: K/RA

Description: Contains the Card challenge (random), obtained in the response to the GET PROCESSING OPTIONS command, to be used by the Terminal in the summary calculation when providing DS ODS Term.

A.1.60 DSVN Term

Tag:	'DF810D'
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	Integrated data storage support by the Kernel depends on the presence of this data object. If it is absent, or is present with a length of zero, integrated data storage is not supported. Its value is '02' for this version of data storage functionality. This variable length data item has an initial byte that defines the maximum version number supported by the Terminal and a variable number of subsequent bytes that define how the Terminal supports earlier versions of the specification. As this is the first version, no legacy support is described and no additional bytes are present.

A.1.61 Error Indication

Tag:	'DF8115'
Template:	—
Length:	6
Format:	b
Update:	K
Description:	Contains information regarding the nature of the error that has been encountered during the transaction processing. This data object is part of the Discretionary Data.

Data Field	Length	Format
L1	1	b (see below)
L2	1	b (see below)
L3	1	b (see below)
SW12	2	b
Msg On Error	1	b (see Message Identifier as defined in A.1.164)

L1		
Byte 1	b8-1	L1
		00000000: OK
		00000001: TIME OUT ERROR
		00000010: TRANSMISSION ERROR
		00000011: PROTOCOL ERROR
		Other values: RFU

L2		
Byte 1	b8-1	L2
		00000000: OK
		00000001: CARD DATA MISSING
		00000010: CAM FAILED
		00000011: STATUS BYTES
		00000100: PARSING ERROR
		00000101: MAX LIMIT EXCEEDED
		00000110: CARD DATA ERROR
		00000111: MAGSTRIPE NOT SUPPORTED
		00001000: NO PPSE
		00001001: PPSE FAULT
		00001010: EMPTY CANDIDATE LIST
		00001011: IDS READ ERROR
		00001100: IDS WRITE ERROR
		00001101: IDS DATA ERROR
		00001110: IDS NO MATCHING AC
		00001111: TERMINAL DATA ERROR
		Other values: RFU

L3		
Byte 1	b8-1	L3
		00000000: OK
		00000001: TIME OUT
		00000010: STOP
		00000011: AMOUNT NOT PRESENT
		Other values: RFU

A.1.62 File Control Information Issuer Discretionary Data

Tag: 'BF0C'
 Template: 'A5'
 Length: var. up to 220
 Format: b
 Update: K/RA
 Description: Issuer discretionary part of the File Control Information Proprietary Template.

A.1.63 File Control Information Proprietary Template

Tag: 'A5'
 Template: '6F'
 Length: var. up to 240
 Format: b
 Update: K/RA
 Description: Identifies the data object proprietary to this specification in the File Control Information Template, in accordance with [ISO/IEC 7816-4].

A.1.64 File Control Information Template

Tag: '6F'

Template: —

Length: var. up to 250

Format: b

Update: K/RA

Description: Identifies the File Control Information Template, in accordance with [ISO/IEC 7816-4].

The table below gives the File Control Information Template expected in response to a successful selection of a Card application matching Kernel 2.

Tag	Value		Presence
'6F'	File Control Information Template		M
	'84'	DF Name (AID)	M
	'A5'	File Control Information Proprietary Template	M ¹³
		'50' Application Label	O
		'87' Application Priority Indicator	O
		'5F2D' Language Preference	O
		'9F38' PDOL	O
		'9F11' Issuer Code Table Index	O
		'9F12' Application Preferred Name	O
		'BF0C' File Control Information Issuer Discretionary Data	O
		'9F6E' Third Party Data	O
		'9F5D' Application Capabilities Information	O
		'XXXX' One or more additional data objects from application provider, Issuer, or ICC supplier	O

¹³The *File Control Information Proprietary Template* may be empty. In this case the length must be set to zero.

A.1.65 Hold Time Value

Tag:	'DF8130'
Template:	—
Length:	1
Format:	b
Update:	K
Description:	Indicates the time that the field is to be turned off after the transaction is completed if requested to do so by the cardholder device. The Hold Time Value is in units of 100ms.

A.1.66 ICC Dynamic Number

Tag:	'9F4C'
Template:	—
Length:	var. 2 to 8
Format:	b
Update:	K/RA
Description:	Time-variant number generated by the Card, to be captured by the Kernel.

A.1.67 ICC Public Key Certificate

Tag:	'9F46'
Template:	'70' or '77'
Length:	N _I (var. up to 248)
Format:	b
Update:	K/RA
Description:	ICC public key certified by the issuer.

A.1.68 ICC Public Key Exponent

Tag:	'9F47'
Template:	'70' or '77'
Length:	1 or 3
Format:	b
Update:	K/RA
Description:	Exponent used for the verification of the Signed Dynamic Application Data.

A.1.69 ICC Public Key Remainder

Tag: '9F48'
Template: '70' or '77'
Length: var.
Format: b
Update: K/RA
Description: Remaining digits of the modulus of the ICC public key.

A.1.70 IDS Status

Tag: 'DF8128'
Template: —
Length: 1
Format: b
Update: K
Description: Indicates if the transaction performs an IDS read and/or write.

IDS Status		
Byte 1	b8	Read
	b7	Write
	b6-1	Each bit RFU

A.1.71 Interface Device Serial Number

Tag: '9F1E'
Template: —
Length: 8
Format: an
Update: K
Description: Unique and permanent serial number assigned to the IFD by the manufacturer.

A.1.72 Issuer Action Code – Default

Tag:	'9F0D'
Template:	'70' or '77'
Length:	5
Format:	b
Update:	K/RA
Description:	Specifies the issuer's conditions that cause a transaction to be rejected on an offline only Terminal.

A.1.73 Issuer Action Code – Denial

Tag:	'9F0E'
Template:	'70' or '77'
Length:	5
Format:	b
Update:	K/RA
Description:	Specifies the issuer's conditions that cause the denial of a transaction without any attempt to go online.

A.1.74 Issuer Action Code – Online

Tag:	'9F0F'
Template:	'70' or '77'
Length:	5
Format:	b
Update:	K/RA
Description:	Specifies the issuer's conditions that cause a transaction to be transmitted online on an online capable Terminal.

A.1.75 Issuer Application Data

Tag:	'9F10'
Template:	'77'
Length:	var. up to 32
Format:	b
Update:	K/RA
Description:	Contains proprietary application data for transmission to the issuer in an online transaction.

A.1.76 Issuer Code Table Index

Tag:	'9F11'
Template:	'A5'
Length:	1
Format:	n 2
Update:	K/RA
Description:	Indicates the code table, in accordance with [ISO/IEC 8859], for displaying the Application Preferred Name. The Issuer Code Table Index is coded as specified in Annex C.4 of [EMV Book 3].

A.1.77 Issuer Country Code

Tag:	'5F28'
Template:	'70' or '77'
Length:	2
Format:	n 3
Update:	K/RA
Description:	Indicates the country of the issuer, in accordance with [ISO 3166-1].

A.1.78 Issuer Public Key Certificate

Tag:	'90'
Template:	'70' or '77'
Length:	N _{CA} (var. up to 248)
Format:	b
Update:	K/RA
Description:	Issuer public key certified by a certification authority.

A.1.79 Issuer Public Key Exponent

Tag:	'9F32'
Template:	'70' or '77'
Length:	1 or 3
Format:	b
Update:	K/RA
Description:	Exponent used for the recovery and verification of the ICC Public Key Certificate.

A.1.80 Issuer Public Key Remainder

Tag: '92'
 Template: '70' or '77'
 Length: $N_I - N_{CA} + 36$
 Format: b
 Update: K/RA
 Description: Remaining digits of the modulus of the Issuer public key.

A.1.81 Kernel Configuration

Tag: 'DF811B'
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Indicates the Kernel configuration options.

Kernel Configuration		
Byte 1	b8-7	Each bit RFU
	b6	CDCVM supported
	b5	Relay resistance protocol supported
	b4	Reserved for Payment system
	b3	Read all records even when no CDA
	b2-1	Each bit RFU

A.1.82 Kernel ID

Tag: 'DF810C'
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Contains a value that uniquely identifies each Kernel. There is one occurrence of this data object for each Kernel in the Reader.

A.1.83 Language Preference

Tag:	'5F2D'
Template:	'A5'
Length:	2-8
Format:	an
Update:	K/RA
Description:	1-4 languages stored in order of preference, each represented by two alphabetical characters, in accordance with [ISO 639-1].

A.1.84 Log Entry

Tag:	'9F4D'
Template:	'BF0C'
Length:	2
Format:	b
Update:	K/RA
Description:	Provides the SFI of the Transaction Log file and its number of records.

A.1.85 Maximum Relay Resistance Grace Period

Tag:	'DF8133'
Template:	—
Length:	2
Format:	b
Update:	K
Description:	The Minimum Relay Resistance Grace Period and Maximum Relay Resistance Grace Period represent how far outside the window defined by the Card that the measured time may be and yet still be considered acceptable. The Maximum Relay Resistance Grace Period is expressed in units of hundreds of microseconds.

A.1.86 Max Time For Processing Relay Resistance APDU

Tag:	'DF8304'
Template:	—
Length:	2
Format:	b
Update:	K/RA
Description:	Indicates the maximum estimated time the Card requires for processing the EXCHANGE RELAY RESISTANCE DATA command. The Max Time For Processing Relay Resistance APDU is expressed in units of hundreds of microseconds.

A.1.87 Measured Relay Resistance Processing Time

Tag:	'DF8306'
Template:	—
Length:	2
Format:	b
Update:	K
Description:	Contains the time measured by the Kernel for processing the EXCHANGE RELAY RESISTANCE DATA command. The Measured Relay Resistance Processing Time is expressed in units of hundreds of microseconds.

A.1.88 Merchant Category Code

Tag:	'9F15'
Template:	—
Length:	2
Format:	n 4
Update:	K
Description:	Classifies the type of business being done by the merchant, represented in accordance with [ISO 8583:1993] for Card Acceptor Business Code.

A.1.89 Merchant Custom Data

Tag:	'9F7C'
Template:	—
Length:	20
Format:	b
Update:	K/ACT/DET
Description:	Proprietary merchant data that may be requested by the Card.

A.1.90 Merchant Identifier

Tag:	'9F16'
Template:	—
Length:	15
Format:	ans 15
Update:	K
Description:	When concatenated with the Acquirer Identifier, uniquely identifies a given merchant.

A.1.91 Merchant Name and Location

Tag:	'9F4E'
Template:	—
Length:	var.
Format:	ans
Update:	K
Description:	Indicates the name and location of the merchant.

A.1.92 Message Hold Time

Tag:	'DF812D'
Template:	—
Length:	3
Format:	n 6
Update:	K
Description:	Indicates the default delay for the processing of the next MSG Signal. The Message Hold Time is an integer in units of 100ms.

A.1.93 Minimum Relay Resistance Grace Period

Tag:	'DF8132'
Template:	—
Length:	2
Format:	b
Update:	K
Description:	The Minimum Relay Resistance Grace Period and Maximum Relay Resistance Grace Period represent how far outside the window defined by the Card that the measured time may be and yet still be considered acceptable. The Minimum Relay Resistance Grace Period is expressed in units of hundreds of microseconds.

A.1.94 Min Time For Processing Relay Resistance APDU

Tag:	'DF8303'
Template:	—
Length:	2
Format:	b
Update:	K/RA
Description:	Indicates the minimum estimated time the Card requires for processing the EXCHANGE RELAY RESISTANCE DATA command. The Min Time For Processing Relay Resistance APDU is expressed in units of hundreds of microseconds.

A.1.95 Next Cmd

Tag: —
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: An internal working variable used to indicate the C-APDU that is currently being processed by the Card.

Next Cmd			
Byte 1	b8-7	Next Cmd	
			00: READ RECORD
			01: GET DATA
			10: NONE
			11: RFU
	b6-1	Each bit RFU	

A.1.96 ODA Status

Tag: —
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Indicates if CDA is to be performed for the transaction in progress.

ODA Status		
Byte 1	b8	CDA
	b7-1	Each bit RFU

A.1.97 Outcome Parameter Set

Tag: 'DF8129'

Template: —

Length: 8

Format: b

Update: K

Description: This data object is used to indicate to the Terminal the outcome of the transaction processing by the Kernel. Its value is an accumulation of results about applicable parts of the transaction.

Outcome Parameter Set			
Byte 1	b8-5	Status	
			0001: APPROVED
			0010: DECLINED
			0011: ONLINE REQUEST
			0100: END APPLICATION
			0101: SELECT NEXT
			0110: TRY ANOTHER INTERFACE
			0111: TRY AGAIN
			1111: N/A
			Other values: RFU
	b4-1	Each bit RFU	
Byte 2	b8-5	Start	
			0000: A
			0001: B
			0010: C
			0011: D
			1111: N/A
			Other values: RFU
	b4-1	Each bit RFU	

Outcome Parameter Set			
Byte 3	b8-5	Online Response Data	
			1111: N/A
			Other values: RFU
	b4-1	Each bit RFU	
Byte 4	b8-5	CVM	
			0000: NO CVM
			0001: OBTAIN SIGNATURE
			0010: ONLINE PIN
			0011: CONFIRMATION CODE VERIFIED
			1111: N/A
			Other values: RFU
	b4-1	Each bit RFU	
Byte 5	b8	UI Request on Outcome Present	
	b7	UI Request on Restart Present	
	b6	Data Record Present	
	b5	Discretionary Data Present	
	b4	Receipt	
			0: N/A
			1: YES
	b3-1	Each bit RFU	
Byte 6	b8-5	Alternate Interface Preference	
			1111: N/A
			Other values: RFU
	b4-1	Each bit RFU	
Byte 7	b8-1	Field Off Request	
			11111111: N/A
			Other values: Hold time in units of 100 ms
Byte 8	b8-1	Removal Timeout in units of 100 ms	

A.1.98 Payment Account Reference

Tag:	'9F24'
Template:	'70' or '77'
Length:	29
Format:	an
Update:	K/RA
Description:	The Payment Account Reference is a data object associated with an Application PAN. It allows acquirers and merchants to link transactions, whether tokenised or not, that are associated to the same underlying Application PAN. Lower case alphabetic characters are not permitted for the Payment Account Reference, however the Kernel is not expected to check this.

A.1.99 PDOL

Tag:	'9F38'
Template:	'A5'
Length:	var. up to 240
Format:	b
Update:	K/RA
Description:	A data object in the Card that provides the Kernel with a list of data objects that must be passed to the Card in the GET PROCESSING OPTIONS command.

A.1.100 PDOL Related Data

Tag:	'DF8111'
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	Command data field of the GET PROCESSING OPTIONS command, coded according to PDOL.

A.1.101 PDOL Values

Tag:	—
Length:	var.
Format:	b
Update:	K
Description:	PDOL Values is the value field of the Command Template (tag '83') stored in PDOL Related Data. PDOL Values is created by the Kernel according to PDOL as a concatenated list of data objects without tags or lengths following the rules specified in section 4.1.4. If PDOL is not returned by the Card, then PDOL Values is an empty byte string.

A.1.102 Phone Message Table

Tag:	'DF8131'
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	The Phone Message Table is a variable length list of entries of eight bytes each, and defines for the selected AID the message and status identifiers as a function of the POS Cardholder Interaction Information. Each entry in the Phone Message Table contains the fields shown in the table below.

Data Field	Length	Format
PCII Mask	3	b
PCII Value	3	b
Message Identifier	1	b
Status	1	b

Note that the last entry in the Phone Message Table must always have PCII Mask and PCII Value set to '000000'.

A.1.103 POS Cardholder Interaction Information

Tag: 'DF4B'

Template: '77'

Length: 3

Format: b

Update: K/RA

Description: The POS Cardholder Interaction Information informs the Kernel about the indicators set in the mobile phone that may influence the action flow of the merchant and cardholder.

POS Cardholder Interaction Information		
Byte 1	b8-1	Version Number
Byte 2	b8-6	Each bit RFU
	b5	CDCVM successful
	b4	Context is conflicting
	b3	Offline change PIN required
	b2	ACK required
	b1	CDCVM required
Byte 3	b8-2	Each bit RFU
	b1	Wallet requires second tap

A.1.104 Post-Gen AC Put Data Status

Tag:	'DF810E'
Template:	—
Length:	1
Format:	b
Update:	K
Description:	Information reported by the Kernel to the Terminal, about the processing of PUT DATA commands after processing the GENERATE AC command. Possible values are 'completed' or 'not completed'. In the latter case, this status is not specific about which of the PUT DATA commands failed, or about how many of these commands have failed or succeeded. This data object is part of the Discretionary Data provided by the Kernel to the Terminal.

Post-Gen AC Put Data Status		
Byte 1	b8	Completed
	b7-1	Each bit RFU

A.1.105 Pre-Gen AC Put Data Status

Tag:	'DF810F'
Template:	—
Length:	1
Format:	b
Update:	K
Description:	Information reported by the Kernel to the Terminal, about the processing of PUT DATA commands before sending the GENERATE AC command. Possible values are 'completed' or 'not completed'. In the latter case, this status is not specific about which of the PUT DATA commands failed, or about how many of these commands have failed or succeeded. This data object is part of the Discretionary Data provided by the Kernel to the Terminal.

Pre-Gen AC Put Data Status		
Byte 1	b8	Completed
	b7-1	Each bit RFU

A.1.106 Proceed To First Write Flag

Tag:	'DF8110'
Template:	—
Length:	1
Format:	b
Update:	K/ACT/DET
Description:	Indicates that the Terminal will send no more requests to read data other than as indicated in Tags To Read. This data item indicates the point at which the Kernel shifts from the Card reading phase to the Card writing phase. If Proceed To First Write Flag is not present or is present with non zero length and value different from zero, then the Kernel proceeds without waiting. If Proceed To First Write Flag is present with zero length, then the Kernel sends a DEK Signal to the Terminal and waits for the DET Signal. If Proceed To First Write Flag is present with non zero length and value equal to zero, then the Kernel waits for a DET Signal from the Terminal without sending a DEK Signal.

A.1.107 Protected Data Envelope 1

Tag:	'9F70'
Template:	—
Length:	var. up to 192
Format:	b
Update:	K/RA/ACT/DET
Description:	The Protected Data Envelopes contain proprietary information from the issuer, payment system or third party. The Protected Data Envelope can be retrieved with the GET DATA command. Updating the Protected Data Envelope with the PUT DATA command requires secure messaging and is outside the scope of this specification.

A.1.108 Protected Data Envelope 2

Tag:	'9F71'
Template:	—
Length:	var. up to 192
Format:	b
Update:	K/RA/ACT/DET
Description:	Same as Protected Data Envelope 1.

A.1.109 Protected Data Envelope 3

Tag: '9F72'
Template: —
Length: var. up to 192
Format: b
Update: K/RA/ACT/DET
Description: Same as Protected Data Envelope 1.

A.1.110 Protected Data Envelope 4

Tag: '9F73'
Template: —
Length: var. up to 192
Format: b
Update: K/RA/ACT/DET
Description: Same as Protected Data Envelope 1.

A.1.111 Protected Data Envelope 5

Tag: '9F74'
Template: —
Length: var. up to 192
Format: b
Update: K/RA/ACT/DET
Description: Same as Protected Data Envelope 1.

A.1.112 Reader Contactless Floor Limit

Tag: 'DF8123'
Template: —
Length: 6
Format: n 12
Update: K
Description: Indicates the transaction amount above which transactions must be authorized online.

A.1.113 Reader Contactless Transaction Limit

Tag:	—
Template:	—
Length:	6
Format:	n 12
Update:	K
Description:	Indicates the transaction amount above which the transaction is not allowed. This data object is instantiated with Reader Contactless Transaction Limit (CDCVM) if CDCVM is supported by the Card and with Reader Contactless Transaction Limit (No CDCVM) otherwise.

A.1.114 Reader Contactless Transaction Limit (No CDCVM)

Tag:	'DF8124'
Template:	—
Length:	6
Format:	n 12
Update:	K
Description:	Indicates the transaction amount above which the transaction is not allowed, when CDCVM is not supported.

A.1.115 Reader Contactless Transaction Limit (CDCVM)

Tag:	'DF8125'
Template:	—
Length:	6
Format:	n 12
Update:	K
Description:	Indicates the transaction amount above which the transaction is not allowed, when CDCVM is supported.

A.1.116 Reader CVM Required Limit

Tag:	'DF8126'
Template:	—
Length:	6
Format:	n 12
Update:	K
Description:	Indicates the transaction amount above which the Kernel instantiates the CVM capabilities field in Terminal Capabilities with CVM Capability – CVM Required.

A.1.117 Read Record Response Message Template

Tag:	'70'
Template:	—
Length:	var. up to 253
Format:	b
Update:	K/RA
Description:	Template containing the data objects returned by the Card in response to a READ RECORD command.

A.1.118 Reference Control Parameter

Tag: 'DF8114'
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Working variable to store the reference control parameter of the GENERATE AC command.

Reference Control Parameter			
Byte 1	b8-7	AC type	
			00: AAC
			01: TC
			10: ARQC
			11: RFU
	b6	RFU	
	b5	CDA signature requested	
	b4-1	Each bit RFU	

A.1.119 Relay Resistance Accuracy Threshold

Tag: 'DF8136'
 Template: —
 Length: 2
 Format: b
 Update: K
 Description: Represents the threshold above which the Kernel considers the variation between Measured Relay Resistance Processing Time and Min Time For Processing Relay Resistance APDU no longer acceptable. The Relay Resistance Accuracy Threshold is expressed in units of hundreds of microseconds.

A.1.120 Relay Resistance Transmission Time Mismatch Threshold

Tag:	'DF8137'
Template:	—
Length:	1
Format:	b
Update:	K
Description:	Represents the threshold above which the Kernel considers the variation between Device Estimated Transmission Time For Relay Resistance R-APDU and Terminal Expected Transmission Time For Relay Resistance R-APDU no longer acceptable. The Relay Resistance Transmission Time Mismatch Threshold is a percentage and expressed as an integer.

A.1.121 Response Message Template Format 1

Tag:	'80'
Template:	—
Length:	var. up to 253
Format:	b
Update:	K/RA
Description:	Contains the data objects (without tags and lengths) returned by the Card in response to a command.

A.1.122 Response Message Template Format 2

Tag:	'77'
Template:	—
Length:	var. up to 253
Format:	b
Update:	K/RA
Description:	Contains the data objects (with tags and lengths) returned by the Card in response to a command.

A.1.123 RRP Counter

Tag: 'DF8307'
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Represents the number of retry attempts to send the EXCHANGE RELAY RESISTANCE DATA command to the Card within one transaction.

A.1.124 Security Capability

Tag: 'DF811F'
 Template: —
 Length: 1
 Format: b
 Update: K
 Description: Indicates the security capability of the Kernel.

Security Capability		
Byte 1	b8-7	Each bit RFU
	b6	Card capture
	b5	RFU
	b4	CDA
	b3-1	Each bit RFU

A.1.125 Signed Dynamic Application Data

Tag: '9F4B'
 Template: '77'
 Length: N_{IC}
 Format: b
 Update: K/RA
 Description: Digital signature on critical application parameters for CDA.

A.1.126 Static Data Authentication Tag List

Tag:	'9F4A'
Template:	'70' or '77'
Length:	var. up to 250
Format:	b
Update:	K/RA
Description:	List of tags of primitive data objects defined in this specification for which the value fields must be included in the static data to be signed.

A.1.127 Static Data To Be Authenticated

Tag:	—
Template:	—
Length:	var. up to 2048
Format:	b
Update:	K
Description:	Buffer used to concatenate records that are involved in offline data authentication.

A.1.128 Tags To Read

Tag:	'DF8112'
Template:	—
Length:	var.
Format:	b
Update:	K/ACT/DET
Description:	List of tags indicating the data the Terminal has requested to be read. This data item is present if the Terminal wants any data back from the Card before the Data Record. This could be in the context of SDS, or for non data storage usage reasons, for example the PAN. This data item may contain configured data. This data object may be provided several times by the Terminal. Therefore, the values of each of these tags must be accumulated in the Tags To Read Yet buffer.

A.1.129 Tags To Read Yet

Tag:	—
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	List of tags that contains the accumulated Terminal data reading requests received in Tags To Read. Requested data objects that are sent to the Terminal are spooled from this buffer. Tags To Read Yet is initiated when the Kernel is started with Tags To Read if present in the ACT Signal. This list can be augmented with Terminal requested data items provided during Kernel processing in DET Signals. The Kernel sends the requested data objects to the Terminal with the DEK Signal in Data To Send.

A.1.130 Tags To Write After Gen AC

Tag:	'FF8103'
Template:	—
Length:	var.
Format:	b
Update:	K/ACT/DET
Description:	Contains the Terminal data writing requests to be sent to the Card after processing the GENERATE AC command. The value of this data object is composed of a series of TLVs. This data object may be provided several times by the Terminal in a DET Signal. Therefore, these values must be accumulated in Tags To Write Yet After Gen AC.

A.1.131 Tags To Write Before Gen AC

Tag:	'FF8102'
Template:	—
Length:	var.
Format:	b
Update:	K/ACT/DET
Description:	List of data objects indicating the Terminal data writing requests to be sent to the Card before processing the GENERATE AC command. This data object may be provided several times by the Terminal in a DET Signal. Therefore, these values must be accumulated in Tags To Write Yet Before Gen AC buffer.

A.1.132 Tags To Write Yet After Gen AC

Tag:	—
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	List of data objects that contains the accumulated Terminal data writing requests received in Tags To Write After Gen AC.

A.1.133 Tags To Write Yet Before Gen AC

Tag:	—
Template:	—
Length:	var.
Format:	b
Update:	K
Description:	List of data objects that contains the accumulated Terminal data writing requests received in Tags To Write Before Gen AC.

A.1.134 Terminal Action Code – Default

Tag: 'DF8120'
Template: —
Length: 5
Format: b
Update: K
Description: Specifies the acquirer's conditions that cause a transaction to be rejected on an offline only Terminal.

A.1.135 Terminal Action Code – Denial

Tag: 'DF8121'
Template: —
Length: 5
Format: b
Update: K
Description: Specifies the acquirer's conditions that cause the denial of a transaction without attempting to go online.

A.1.136 Terminal Action Code – Online

Tag: 'DF8122'
Template: —
Length: 5
Format: b
Update: K
Description: Specifies the acquirer's conditions that cause a transaction to be transmitted online on an online capable Terminal.

A.1.137 Terminal Capabilities

Tag:	'9F33'
Template:	—
Length:	3
Format:	b
Update:	K
Description:	Indicates the card data input, CVM, and security capabilities of the Terminal and Reader. The CVM capability (Byte 2) is instantiated with values depending on the transaction amount.

Terminal Capabilities		
Byte 1	b8	Manual key entry
	b7	Magnetic stripe
	b6	IC with contacts
	b5-1	Each bit RFU
Byte 2	b8	Plaintext PIN for ICC verification
	b7	Enciphered PIN for online verification
	b6	Signature
	b5	Enciphered PIN for offline verification
	b4	No CVM required
	b3-1	Each bit RFU
Byte 3	b8-7	Each bit RFU
	b6	Card capture
	b5	RFU
	b4	CDA
	b3-1	Each bit RFU

A.1.138 Terminal Country Code

Tag:	'9F1A'
Template:	—
Length:	2
Format:	n 3
Update:	K
Description:	Indicates the country of the Terminal, represented in accordance with [ISO 3166-1].

A.1.139 Terminal Expected Transmission Time For Relay Resistance C-APDU

Tag: 'DF8134'
Template: —
Length: 2
Format: b
Update: K
Description: Represents the time that the Kernel expects to need for transmitting the EXCHANGE RELAY RESISTANCE DATA command to the Card. The Terminal Expected Transmission Time For Relay Resistance C-APDU is expressed in units of hundreds of microseconds.

A.1.140 Terminal Expected Transmission Time For Relay Resistance R-APDU

Tag: 'DF8135'
Template: —
Length: 2
Format: b
Update: K
Description: Represents the time that the Kernel expects that the Card will need for transmitting the EXCHANGE RELAY RESISTANCE DATA R-APDU. The Terminal Expected Transmission Time For Relay Resistance R-APDU is expressed in units of hundreds of microseconds.

A.1.141 Terminal Identification

Tag: '9F1C'
Template: —
Length: 8
Format: an 8
Update: K
Description: Designates the unique location of the Terminal.

A.1.142 Terminal Relay Resistance Entropy

Tag:	'DF8301'
Template:	—
Length:	4
Format:	b
Update:	K
Description:	Contains a Kernel challenge (random) to be used in the value field of the EXCHANGE RELAY RESISTANCE DATA command.

A.1.143 Terminal Risk Management Data

Tag:	'9F1D'
Template:	—
Length:	8
Format:	b
Update:	K
Description:	Terminal Risk Management Data indicates reader capabilities, requirements, and preferences to the Card. Bits 3, 4, 6, and 7 in byte 1 and bit 8 in byte 2 in Terminal Risk Management Data are dynamically set by the Kernel based on the CVM characteristics of the transaction. All other Terminal Risk Management Data bits are static configuration values, and not modified based on transaction conditions.

Terminal Risk Management Data		
Byte 1	8	Restart supported
	7	Enciphered PIN verified online (Contactless)
	6	Signature (Contactless)
	5	Enciphered PIN verification performed by ICC (Contactless)
	4	No CVM required (Contactless)
	3	CDCVM (Contactless)
	2	Plaintext PIN verification performed by ICC (Contactless)
	1	Present and Hold supported
Byte 2	8	CVM Limit exceeded
	7	Enciphered PIN verified online (Contact)
	6	Signature (Contact)
	5	Enciphered PIN verification performed by ICC (Contact)
	4	No CVM required (Contact)
	3	CDCVM (Contact)
	2	Plaintext PIN verification performed by ICC (Contact)
	1	RFU
Byte 3	8	Mag-stripe mode contactless transactions not supported
	7	EMV mode contactless transactions not supported
	6	CDCVM without CDA supported
	5-1	RFU
Byte 4	8	CDCVM bypass requested
	7	SCA exempt
	6	Deferred Authorisation
	5	Transit Gate
	4-1	RFU
Byte 5	8-1	RFU
Byte 6	8-1	RFU
Byte 7	8-1	RFU
Byte 8	8-1	RFU

A.1.144 Terminal Type

Tag:	'9F35'
Template:	—
Length:	1
Format:	n 2
Update:	K
Description:	Indicates the environment of the Terminal, its communications capability, and its operational control. The Terminal Type is coded according to Annex A.1 of [EMV Book 4].

A.1.145 Terminal Verification Results

Tag:	'95'
Template:	—
Length:	5
Format:	b
Update:	K
Description:	Status of the different functions from the Terminal perspective. The Terminal Verification Results is coded according to Annex C.5 of [EMV Book 3]. Bits that have been reserved for use by contactless specifications are defined as shown.

Terminal Verification Results		
Byte 1	b8	Offline data authentication was not performed
	b7	RFU
	b6	ICC data missing
	b5	Card appears on terminal exception file
	b4	RFU
	b3	CDA failed
	b2-1	Each bit RFU
Byte 2	b8	ICC and terminal have different application versions
	b7	Expired application
	b6	Application not yet effective
	b5	Requested service not allowed for card product
	b4-1	Each bit RFU

Terminal Verification Results		
Byte 3	b8	Cardholder verification was not successful
	b7	Unrecognised CVM
	b6-4	Each bit RFU
	b3	Online PIN entered
	b2-1	Each bit RFU
Byte 4	b8	Transaction exceeds floor limit
	b7-5	Each bit RFU
	b4	Merchant forced transaction online
	b3-1	Each bit RFU
Byte 5	b8-5	Each bit RFU
	b4	Relay resistance threshold exceeded
	b3	Relay resistance time limits exceeded
	b2-1	Relay resistance performed
		00: Relay resistance protocol not supported (not used by this version of the specification)
		01: RRP NOT PERFORMED
		10: RRP PERFORMED
		11: RFU

A.1.146 Token Requestor ID

Tag: '9F19'

Template: '70' or '77'

Length: 6

Format: n 11

Update: K/RA

Description: The Token Requestor ID uniquely identifies the pairing of the Token Requestor with the Token Domain, as defined in [EMV Token].

A.1.147 Third Party Data

Tag: '9F6E'
 Template: 'BF0C' or '70'
 Length: var. 5 to 32
 Format: b
 Update: K/RA
 Description: The Third Party Data contains various information, possibly including information from a third party. If present in the Card, the Third Party Data must be returned in a file read using the READ RECORD command or in the File Control Information Template.
 'Device Type' is present when the most significant bit of byte 1 of 'Unique Identifier' is set to 0b. In this case, the maximum length of 'Proprietary Data' is 26 bytes. Otherwise it is 28 bytes.

Data Field	Length	Format
Country Code	2	Country Code according to [ISO 3166-1]
Unique Identifier	2	b (value assigned by MasterCard)
Device Type	0 or 2	an
Proprietary Data	1-26 or 28	b

A.1.148 Time Out Value

Tag: 'DF8127'
 Template: —
 Length: 2
 Format: b
 Update: K
 Description: Defines the time in ms before the timer generates a TIMEOUT Signal.

A.1.149 Track 1 Discretionary Data

Tag: '9F1F'
 Template: '70' or '77'
 Length: var. up to 54
 Format: ans
 Update: K/RA
 Description: Discretionary part of track 1 according to [ISO/IEC 7813].

A.1.150 Track 2 Discretionary Data

Tag: '9F20'
Template: '70' or '77'
Length: var. up to 16
Format: cn
Update: K/RA
Description: Discretionary part of track 2 according to [ISO/IEC 7813].

A.1.151 Track 2 Equivalent Data

Tag: '57'
Template: '70' or '77'
Length: var. up to 19
Format: b
Update: K/RA
Description: Contains the data objects of the track 2, in accordance with [ISO/IEC 7813], excluding start sentinel, end sentinel, and LRC. The Track 2 Equivalent Data has a maximum length of 37 positions and is present in the file read using the READ RECORD command. It is made up of the following sub-fields:

Data Field	Length	Format
Primary Account Number	var. up to 19 nibbles	n
Field Separator	1 nibble	b ('D')
Expiration Date (YYMM)	2	n (YYMM)
Service Code	3 nibbles	n
Discretionary Data	var.	n
Padded with 'F' if needed to ensure whole bytes	1 nibble	b

A.1.152 Transaction Category Code

Tag:	'9F53'
Template:	—
Length:	1
Format:	an
Update:	K/ACT/DET
Description:	This is a data object defined by MasterCard which indicates the type of transaction being performed, and which may be used in card risk management.

A.1.153 Transaction Currency Code

Tag:	'5F2A'
Template:	—
Length:	2
Format:	n 3
Update:	K/ACT/DET
Description:	Indicates the currency code of the transaction, in accordance with [ISO 4217].

A.1.154 Transaction Currency Exponent

Tag:	'5F36'
Template:	—
Length:	1
Format:	n 1
Update:	K/ACT/DET
Description:	Indicates the implied position of the decimal point from the right of the transaction amount represented, in accordance with [ISO 4217].

A.1.155 Transaction Date

Tag:	'9A'
Template:	—
Length:	3
Format:	n 6 (YYMMDD)
Update:	K/ACT/DET
Description:	Local date that the transaction was performed.

A.1.156 Transaction Time

Tag:	'9F21'
Template:	—
Length:	3
Format:	n 6 (HHMMSS)
Update:	K/ACT/DET
Description:	Local time at which the transaction was performed.

A.1.157 Transaction Type

Tag:	'9C'
Template:	—
Length:	1
Format:	n 2
Update:	K/ACT/DET
Description:	Indicates the type of financial transaction, represented by the first two digits of [ISO 8583:1987] Processing Code.

A.1.158 Unpredictable Number

Tag:	'9F37'
Template:	—
Length:	4
Format:	b
Update:	K
Description:	Contains a Kernel challenge (random) to be used by the Card to ensure the variability and uniqueness to the generation of a cryptogram.

A.1.159 Unprotected Data Envelope 1

Tag:	'9F75'
Template:	—
Length:	var. up to 192
Format:	b
Update:	K/RA/ACT/DET
Description:	The Unprotected Data Envelopes contain proprietary information from the issuer, payment system or third party. Unprotected Data Envelopes can be retrieved with the GET DATA command and can be updated with the PUT DATA (CLA='80') command without secure messaging.

A.1.160 Unprotected Data Envelope 2

Tag:	'9F76'
Template:	—
Length:	var. up to 192
Format:	b
Update:	K/RA/ACT/DET
Description:	Same as Unprotected Data Envelope 1.

A.1.161 Unprotected Data Envelope 3

Tag:	'9F77'
Template:	—
Length:	var. up to 192
Format:	b
Update:	K/RA/ACT/DET
Description:	Same as Unprotected Data Envelope 1.

A.1.162 Unprotected Data Envelope 4

Tag:	'9F78'
Template:	—
Length:	var. up to 192
Format:	b
Update:	K/RA/ACT/DET
Description:	Same as Unprotected Data Envelope 1.

A.1.163 Unprotected Data Envelope 5

Tag: '9F79'
Template: —
Length: var. up to 192
Format: b
Update: K/RA/ACT/DET
Description: Same as Unprotected Data Envelope 1.

A.1.164 User Interface Request Data

Tag: 'DF8116'
Template: —
Length: 13
Format: b
Update: K
Description: Combines all parameters to be sent with the OUT Signal or MSG Signal.

Data Field	Length	Format
Message Identifier	1	b (see below)
Status	1	b (see below)
Hold Time	3	n 6
Language Preference	8	an (padded with hexadecimal zeroes if length of tag '5F2D' is less than 8 bytes)

Message Identifier		
Byte 1	b8-1	Message Identifier
		00010111: CARD READ OK
		00100001: TRY AGAIN
		00000011: APPROVED
		00011010: APPROVED – SIGN
		00000111: DECLINED
		00011100: ERROR – OTHER CARD
		00011101: INSERT CARD
		00100000: SEE PHONE
		00011011: AUTHORISING – PLEASE WAIT
		00011110: CLEAR DISPLAY
		11111111: N/A
		Other values: RFU

Status		
Byte 1	b8-1	Status
		00000000: NOT READY
		00000001: IDLE
		00000010: READY TO READ
		00000011: PROCESSING
		00000100: CARD READ SUCCESSFULLY
		00000101: PROCESSING ERROR
		11111111: N/A
		Other values: RFU

A.2 Data Objects by Tag

Table A.1 lists all data objects of Kernel 2 ordered by tag number.

Table A.1—Data Objects by Tag

Tag	Data Object	Implementation Option
'50'	Application Label	Always
'57'	Track 2 Equivalent Data	Always
'5A'	Application PAN	Always
'5F24'	Application Expiration Date	Always
'5F25'	Application Effective Date	Always
'5F28'	Issuer Country Code	Always
'5F2A'	Transaction Currency Code	Always
'5F2D'	Language Preference	Always
'5F34'	Application PAN Sequence Number	Always
'5F36'	Transaction Currency Exponent	Always
'5F57'	Account Type	Always
'6F'	File Control Information Template	Always
'70'	Read Record Template	Always
'77'	Response Message Template Format 2	Always
'80'	Response Message Template Format 1	Always
'82'	Application Interchange Profile	Always
'84'	DF Name	Always
'87'	Application Priority Indicator	Always
'8C'	CDOL1	Always
'8E'	CVM List	Always
'8F'	CA Public Key Index (Card)	Always
'90'	Issuer Public Key Certificate	Always
'92'	Issuer Public Key Remainder	Always
'94'	Application File Locator	Always
'95'	Terminal Verification Results	Always

Tag	Data Object	Implementation Option
'9A'	Transaction Date	Always
'9C'	Transaction Type	Always
'9F01'	Acquirer Identifier	Always
'9F02'	Amount, Authorized (Numeric)	Always
'9F03'	Amount, Other (Numeric)	Always
'9F07'	Application Usage Control	Always
'9F08'	Application Version Number (Card)	Always
'9F09'	Application Version Number (Reader)	Always
'9F0D'	Issuer Action Code – Default	Always
'9F0E'	Issuer Action Code – Denial	Always
'9F0F'	Issuer Action Code – Online	Always
'9F10'	Issuer Application Data	Always
'9F11'	Issuer Code Table Index	Always
'9F12'	Application Preferred Name	Always
'9F15'	Merchant Category Code	Always
'9F16'	Merchant Identifier	Always
'9F19'	Token Requestor ID	Always
'9F1A'	Terminal Country Code	Always
'9F1C'	Terminal Identification	Always
'9F1D'	Terminal Risk Management Data	Always
'9F1E'	Interface Device Serial Number	Always
'9F1F'	Track 1 Discretionary Data	Always
'9F20'	Track 2 Discretionary Data	Always
'9F21'	Transaction Time	Always
'9F24'	Payment Account Reference	Always
'9F26'	Application Cryptogram	Always
'9F27'	Cryptogram Information Data	Always
'9F32'	Issuer Public Key Exponent	Always
'9F33'	Terminal Capabilities	Always
'9F34'	CVM Results	Always

Tag	Data Object	Implementation Option
'9F35'	Terminal Type	Always
'9F36'	Application Transaction Counter	Always
'9F37'	Unpredictable Number	Always
'9F38'	PDOL	Always
'9F40'	Additional Terminal Capabilities	Always
'9F42'	Application Currency Code	Always
'9F44'	Application Currency Exponent	Always
'9F46'	ICC Public Key Certificate	Always
'9F47'	ICC Public Key Exponent	Always
'9F48'	ICC Public Key Remainder	Always
'9F4A'	Static Data Authentication Tag List	Always
'9F4B'	Signed Dynamic Application Data	Always
'9F4C'	ICC Dynamic Number	Always
'9F4D'	Log Entry	Always
'9F4E'	Merchant Name and Location	Always
'9F53'	Transaction Category Code	Always
'9F54'	DS ODS Card	DE/DS
'9F5B'	DSDOL	DE/DS
'9F5C'	DS Requested Operator ID	DE/DS
'9F5D'	Application Capabilities Information	Always
'9F5E'	DS ID	DE/DS
'9F5F'	DS Slot Availability	DE/DS
'9F6E'	Third Party Data	Always
'9F6F'	DS Slot Management Control	DE/DS
'9F70'	Protected Data Envelope 1	DE/DS
'9F71'	Protected Data Envelope 2	DE/DS
'9F72'	Protected Data Envelope 3	DE/DS
'9F73'	Protected Data Envelope 4	DE/DS
'9F74'	Protected Data Envelope 5	DE/DS
'9F75'	Unprotected Data Envelope 1	DE/DS

Tag	Data Object	Implementation Option
'9F76'	Unprotected Data Envelope 2	DE/DS
'9F77'	Unprotected Data Envelope 3	DE/DS
'9F78'	Unprotected Data Envelope 4	DE/DS
'9F79'	Unprotected Data Envelope 5	DE/DS
'9F7C'	Merchant Custom Data	Always
'9F7D'	DS Summary 1	DE/DS
'9F7F'	DS Unpredictable Number	DE/DS
'A5'	File Control Information Proprietary Template	Always
'BF0C'	File Control Information Issuer Discretionary Data	Always
'DF4B'	POS Cardholder Interaction Information	Always
'DF60'	DS Input (Card)	DE/DS
'DF61'	DS Digest H	DE/DS
'DF62'	DS ODS Info	DE/DS
'DF63'	DS ODS Term	DE/DS
'DF8101'	DS Summary 2	DE/DS
'DF8102'	DS Summary 3	DE/DS
'DF8106'	Data Needed	DE/DS
'DF8107'	CDOL1 Related Data	Always
'DF8108'	DS AC Type	DE/DS
'DF8109'	DS Input (Term)	DE/DS
'DF810A'	DS ODS Info For Reader	DE/DS
'DF810B'	DS Summary Status	DE/DS
'DF810C'	Kernel ID	Always
'DF810D'	DSVN Term	DE/DS
'DF810E'	Post-Gen AC Put Data Status	DE/DS
'DF810F'	Pre-Gen AC Put Data Status	DE/DS
'DF8110'	Proceed To First Write Flag	DE/DS
'DF8111'	PDOL Related Data	Always
'DF8112'	Tags To Read	DE/DS
'DF8114'	Reference Control Parameter	Always

Tag	Data Object	Implementation Option
'DF8115'	Error Indication	Always
'DF8116'	User Interface Request Data	Always
'DF8117'	Card Data Input Capability	Always
'DF8118'	CVM Capability – CVM Required	Always
'DF8119'	CVM Capability – No CVM Required	Always
'DF811B'	Kernel Configuration	Always
'DF811F'	Security Capability	Always
'DF8120'	Terminal Action Code – Default	Always
'DF8121'	Terminal Action Code – Denial	Always
'DF8122'	Terminal Action Code – Online	Always
'DF8123'	Reader Contactless Floor Limit	Always
'DF8124'	Reader Contactless Transaction Limit (No On-device CVM)	Always
'DF8125'	Reader Contactless Transaction Limit (On-device CVM)	Always
'DF8126'	Reader CVM Required Limit	Always
'DF8127'	Time Out Value	DE/DS
'DF8128'	IDS Status	DE/DS
'DF8129'	Outcome Parameter Set	Always
'DF812D'	Message Hold Time	Always
'DF8130'	Hold Time Value	Always
'DF8131'	Phone Message Table	Always
'DF8132'	Minimum Relay Resistance Grace Period	Always
'DF8133'	Maximum Relay Resistance Grace Period	Always
'DF8134'	Terminal Expected Transmission Time For Relay Resistance C-APDU	Always
'DF8135'	Terminal Expected Transmission Time For Relay Resistance R-APDU	Always
'DF8136'	Relay Resistance Accuracy Threshold	Always
'DF8137'	Relay Resistance Transmission Time Mismatch Threshold	Always
'DF8301'	Terminal Relay Resistance Entropy	Always
'DF8302'	Device Relay Resistance Entropy	Always

Tag	Data Object	Implementation Option
'DF8303'	Min Time For Processing Relay Resistance APDU	Always
'DF8304'	Max Time For Processing Relay Resistance APDU	Always
'DF8305'	Device Estimated Transmission Time For Relay Resistance R-APDU	Always
'DF8306'	Measured Relay Resistance Processing Time	Always
'DF8307'	RRP Counter	Always
'FF8102'	Tags To Write Before Gen AC	DE/DS
'FF8103'	Tags To Write After Gen AC	DE/DS
'FF8104'	Data To Send	DE/DS
'FF8105'	Data Record	Always
'FF8106'	Discretionary Data	Always

Annex B Kernel State Machine

B.1 Application States

The Kernel defined in this document is specified as an abstract state machine using states, transitions and Signals to model its behavior.

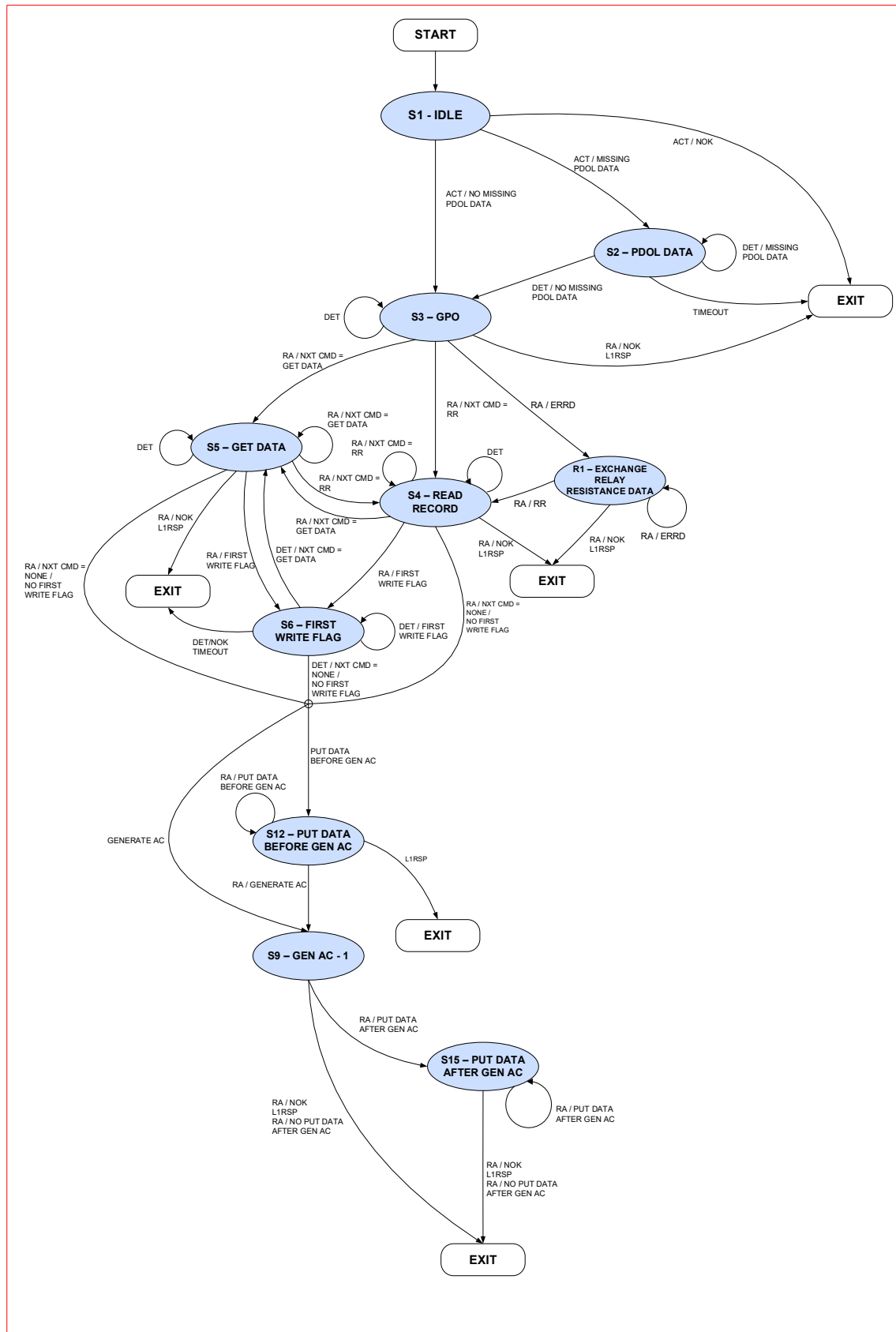
The application states are listed in Table B.1.

Table B.1—Kernel Application States

State	Description	Implementation Option
1 – Idle	Process has been created	Always
2 – Waiting for PDOL Data	Waiting for data to be provided by Terminal	DE/DS
3 – Waiting for GPO Response	Waiting for Card response to GET PROCESSING OPTIONS command	Always
R1 – Waiting for Exchange Relay Resistance Data Response	Waiting for Card response to EXCHANGE RELAY RESISTANCE DATA command	Always
4 – Waiting for Read Record Response	Waiting for Card response to READ RECORD command	Always
5 – Waiting for Get Data Response	Waiting for Card response to GET DATA command	DE/DS
6 – Waiting for First Write Flag	Waiting for Terminal to indicate that transaction can be completed	DE/DS
9 – Waiting for Generate AC Response	Waiting for Card response to GENERATE AC command	Always
12 – Waiting for Put Data Response Before Generate AC	Waiting for Card response to PUT DATA command sent before GENERATE AC	DE/DS
15 – Waiting for Put Data Response After Generate AC	Waiting for Card response to PUT DATA command sent after GENERATE AC	DE/DS

B.2 State Machine

Figure B.1—Kernel State Machine



Annex C Glossary

The following terms are used in this document.

Table C.1—Terms

Term	Description
Card	Card, as used in these specifications, is a consumer device supporting contactless transactions.
Combination	Combination is the combination of an AID and a Kernel ID.
Configuration Option	A Configuration Option allows activation or deactivation of the Kernel software behind this option. The Configuration Option may change the execution path of the software but it does not change the software itself. A Configuration Option is set in the Kernel database per AID and transaction type.
EMV Mode	Operating mode that indicates that the contactless payment transaction is validated based on EMV minimum data.
EMVCo	The organization that manages the EMV Specifications and their related testing processes.
EMV®	A trademark owned by EMVCo, referring to the technical specifications published by EMVCo.
Implementation Option	An Implementation Option allows the vendor to select whether the functionality behind the option will be implemented in a particular installation.
Kernel	The Kernel contains the interface routines, security and control functions, and logic to manage a set of commands and responses to retrieve all the necessary data from the Card to complete a transaction. The Kernel processing covers the interaction with the Card between the selection of the card application (excluded) and the processing of the transaction's outcome (excluded).
Mag-stripe Mode	Mag-stripe Mode describes an operating mode of the POS System that indicates that this particular acceptance environment and acceptance rules support magnetic stripe infrastructure. Mag-stripe Mode transactions are no longer supported by this version of the specification.

Term	Description
POS System	The POS System is the collective term given to the payment infrastructure present at the merchant. It is made up of the Terminal and Reader.
Process	A Process is a logical component within a Reader that has one or more Queues to receive Signals. The processing of Signals, in combination with the carried data, may then generate other Signals to be sent. Processing can continue until all the Queues of a Process are empty, or until the Process terminates.
Queue	A Queue is a buffer that stores events to be processed. The events are stored in the order received.
Reader	The Reader is the device that supports the Kernel(s) and provides the contactless interface used by the Card. In this specification, the Reader is considered as a separate logical entity, although it can be an integral part of the POS System.
Signal	A Signal is an asynchronous event that is placed in a Queue. A Signal can convey data as parameters, and the data provided in this way is used in the processing of the Signal.
Terminal	The Terminal is the device that connects to the authorization and/or clearing network and that together with the Reader makes up the POS System. The Terminal and the Reader may exist in a single integrated device. However, in this specification, they are considered separate logical entities.

The following abbreviations are used in this document.

Table C.2—Abbreviations

Abbreviation	Description
AAC	Application Authentication Cryptogram
AC	Application Cryptogram
ADF	Application Definition File
AES	Advanced Encryption Standard
AFL	Application File Locator
AID	Application Identifier
AIP	Application Interchange Profile
an	Alphanumeric characters
ans	Alphanumeric and Special characters
APDU	Application Protocol Data Unit
ARQC	Authorization Request Cryptogram
ATC	Application Transaction Counter
b	Binary
BCD	Binary Coded Decimal
BER	Basic Encoding Rules
C	Conditional
CA	Certification Authority
C-APDU	Command APDU
CDA	Combined DDA/AC Generation
CDOL	Card Risk Management Data Object List
CDCVM	Consumer Device Cardholder Verification Method
CID	Cryptogram Information Data
CLA	Class byte of command message
cn	Compressed Numeric
CRL	Certification Revocation List
CVC	Card Verification Code
CVM	Cardholder Verification Method
DDA	Dynamic Data Authentication
DE	Data Exchange
DEK	Data Exchange Kernel
DES	Data Encryption Standard

Abbreviation	Description
DET	Data Exchange Terminal
DF	Dedicated File
DOL	Data Object List
DSDOL	Data Storage Data Object List
ERRD	Exchange Relay Resistance Data
FCI	File Control Information
FIFO	First In First Out
IAD	Issuer Application Data
ICC	Integrated Circuit Card
IDS	Integrated Data Storage
INS	Instruction byte of command message
ISO	International Organization for Standardization
M	Mandatory
n	Numeric
N/A	Not applicable
N _{CA}	Length of CA Public Key Modulus
N _I	Length of Issuer Public Key Modulus
N _{IC}	Length of ICC Public Key Modulus
O	Optional
ODA	Offline Data Authentication
ODS	Operator Data Set
OWF	One Way Function
PAN	Primary Account Number
PCII	POS Cardholder Interaction Information
PDOL	Processing Options Data Object List
POS	Point of Sale
PPSE	Proximity Payment System Environment
PIN	Personal Identification Number
R/CNS	Rejected, Conditions Not Satisfied
RFU	Reserved for Future Use
R-APDU	Response APDU
RID	Registered Application Provider Identifier
RRP	Relay Resistance Protocol
SDAD	Signed Dynamic Application Data
SDS	Standalone Data Storage

Abbreviation	Description
SFI	Short File Identifier
SHA	Secure Hash Algorithm
SW12	Status bytes 1-2
TC	Transaction Certificate
TL	Tag Length
TLV	Tag Length Value
TVR	Terminal Verification Results
UDOL	Unpredictable Number Data Object List
UN	Unpredictable Number
var.	Variable

*** END OF DOCUMENT ***